



Introduction to Pygame-ce

2D Game Programming

Ralf Adams
Version 1.3 (June 9, 2026)

Contents

1	Goals	9
2	Basics	13
2.0	Installation and Organization	13
2.0.1	Installing Python and Pygame	13
2.0.1.1	Python	13
2.0.1.2	Pygame Community Edition	14
2.0.2	A Recommended Project Structure	15
2.1	Kind of <i>Hello World!</i>	17
2.1.1	The Very First Steps	17
2.1.2	More Input	21
2.1.2.1	Multiple Windows	21
2.1.2.2	Information About the Graphics Environment	22
2.1.2.3	Using <code>pygame.display</code> instead of <code>pygame.Window</code>	23
2.1.3	What was new?	24
2.1.4	Homework	26
2.2	Graphic Primitives	28
2.2.1	Introduction	28
2.2.2	More Input	30
2.2.2.1	Example: Particle Swarm	30
2.2.2.2	Example: Landscape	36
2.2.3	What was new?	43
2.2.4	Homework	44
2.3	Load and Blit Bitmaps	45
2.3.1	Introduction	45
2.3.2	More Input	52
2.3.2.1	Blitting Parts of a Bitmap	52
2.3.2.2	Message Box	53
2.3.2.3	Creating Bitmaps	54
2.3.3	What was new?	57
2.3.4	Homework	58
2.4	Moving Bitmaps	59
2.4.1	Class <code>Rect/FRect</code>	59
2.4.2	Introduction	61
2.4.3	More Input	64
2.4.3.1	Normalizing Speeds (<i>delta time</i>)	64
2.4.3.2	Optimizing Normalized Speed	69
2.4.4	What was new?	76
2.4.5	Homework	77

2.5	Class Sprite	78
2.5.1	Introduction	78
2.5.2	More Input	82
2.5.2.1	OO Issues	82
2.5.2.2	Add Sprite Objects to a Group Right Away	85
2.5.2.3	Delete Sprites from Groups	86
2.5.3	What was new?	87
2.5.4	Homework	88
2.6	Handling Keyboard Input	89
2.6.1	Introduction	89
2.6.2	More Input	91
2.6.2.1	Example: Shift and Related Keys	91
2.6.2.2	In Which Window was the Key Pressed?	93
2.6.2.3	Example: Visualizing the Keyboard	95
2.6.2.4	Not by Event, but by Function	99
2.6.3	Homework	105
2.7	Text output using fonts	106
2.7.1	Introduction	106
2.7.2	More Input	107
2.7.2.1	A More Sophisticated Approach	107
2.7.2.2	List of all Installed Fonts	110
2.7.2.3	Using Locally Installed Fonts	113
2.7.2.4	Text output	115
2.7.3	What was new?	121
2.7.4	Homework	122
2.8	Collision Detection	123
2.8.1	Introduction	123
2.8.2	More Input	126
2.8.2.1	Three Types of Collision Detection (of a Bullet)	126
2.8.2.2	Checking all Sprites in a List	131
2.8.2.3	Using Function Pointer/Collision Callback	132
2.8.3	What was new?	133
2.8.4	Homework	134
2.9	Time-based Actions	136
2.9.1	Introduction	136
2.9.2	More Input	142
2.9.2.1	The Class Timer	142
2.9.2.2	Accumulated Time	144
2.9.2.3	Cool Down	145
2.9.2.4	Start Delay	145
2.9.3	What was new?	145
2.10	Mouse	146
2.10.1	Introduction	146

2.10.2	More Input	151
2.10.2.1	In Which Window Took the Mouse Action Place?	151
2.10.2.2	Surprise: No Double Click	152
2.10.2.3	Not by Event, but by Function	153
2.10.3	What was new?	153
2.10.4	Homework	154
2.11	Sound	156
2.11.1	Introduction	156
2.11.1.1	Sound: Music	156
2.11.1.2	Sound: Events	159
2.11.2	More Input	161
2.11.2.1	Stereo	161
2.11.2.2	Sound Formats and Technical Basics	166
2.11.2.3	Volume Hierarchies and Sound Mixing	167
2.11.2.4	Mono Sounds and Stereo Panning	167
2.11.2.5	Sound Lifetime and Resource Management	168
2.11.2.6	Event-driven Sound Output	168
2.11.2.7	Looping and Transitions	168
2.11.2.8	Muting and Pausing	168
2.11.2.9	Typical Errors and Debugging	169
2.11.3	What was new?	169
2.12	Events	171
2.12.1	Introduction	171
2.12.1.1	What Information is Contained in an Event?	171
2.12.1.2	How can I Create and Use User-defined Events?	172
2.12.2	More Input	178
2.12.2.1	How can periodic events be generated?	178
2.12.2.2	Structuring the Event Loop Correctly	179
2.12.2.3	Choosing the Right Event Retrieval Method	180
2.12.2.4	Avoid Generating Events Every Frame	180
2.12.2.5	Defining Event Data Clearly and Consistently	180
2.12.2.6	Managing User-Defined Event IDs	181
2.12.2.7	Avoid Manually Set Event IDs	181
2.12.2.8	Use <code>set_timer()</code> Correctly!	181
2.12.2.9	Filtering Events for Performance	181
2.12.2.10	Event-Based Input vs. State-Based Input	182
2.12.2.11	Window Focus and Application State	182
2.12.2.12	Debugging Events Effectively	182
2.12.2.13	Structuring Event Handling Code	183
2.12.3	What was new?	183
3	Techniques	185
3.1	Animation	185
3.1.1	The Running Cat	185

3.1.2	The Class Animation	188
3.1.3	The Exploding Rock	190
3.2	Tiles Are Beautiful	193
3.2.1	Our Example	193
3.2.2	A Green Meadow	194
3.2.3	Tile Numbers and Two-Dimensional Arrays	198
3.3	Very Large Worlds	202
3.3.1	A Large Example World	202
3.3.2	Top-Down View / Bird's-Eye View	209
3.3.3	Player Centered Camera	213
3.3.4	Page Scrolling/Edge Scrolling	219
3.3.5	Auto Scrolling/Endless Scrolling	221
3.3.6	As a Strategy Pattern	222
3.4	State Machine and Scenes	224
3.4.1	A Simple Example: Tic-Tac-Toe	224
3.4.1.1	Start Screen	224
3.4.1.2	The Playground	226
3.4.1.3	Do we have a winner?	230
3.4.1.4	Restart	232
3.4.1.5	Separating Strings from the Code	235
3.4.2	Introduction of State Machines	237
3.4.2.1	Refactoring Scenes	237
3.4.2.2	Using Events	244
3.4.3	Stacked States / Pushdown Automaton	248
3.4.4	I'm confused!	252
4	Examples	253
4.1	Pong	253
4.1.1	Requirement 1: Standards	253
4.1.2	Requirement 2: The Paddles	255
4.1.3	Requirement 3: The Ball	258
4.1.4	Requirement 4: Scoring	261
4.1.5	Requirement 5: Paddle hit	264
4.1.6	Requirement 6: Computer-controlled player	265
4.1.7	Requirement 7: Sound	268
4.1.8	Requirement 8: Pause and Help Screen	270
4.2	Bubbles	274
4.2.1	Requirement 1: Standards	274
4.2.2	Requirement 2: Bubbles appear	277
4.2.3	Requirement 3: Number of bubbles	280
4.2.4	Requirement 4: Bubble growth	281
4.2.5	Requirement 5: Mouse cursor	284
4.2.6	Requirement 6: Bubbles burst	286
4.2.7	Requirement 7: Score	287

4.2.8	Requirement 8: Game over	289
4.2.9	Requirement 9: Time-based adjustments	291
4.2.10	Requirement 10: Display collision	293
4.2.11	Requirement 11: Pause	297
4.2.12	Requirement 12: Restart	299
4.2.13	Requirement 13: Sound	301
4.2.14	Or maybe not?	303
4.3	Moonlander	307
4.3.1	Requirement 1: Standards	307
4.3.2	Requirement 2: Lunar surface	311
4.3.3	Requirement 3: Earth	315
4.3.4	Requirement 4: Stars	317
4.3.5	Requirement 5: Lander	319
4.3.6	Requirement 6: Gravitation and landing	323
4.3.7	Requirement 7: Counter-thrust	324
4.3.8	Requirement 8: Fuel	325
4.3.9	Requirement 9: Status display	326
4.3.10	Requirement 10: Game over and restart	330
4.3.11	Requirement 11: Autopilot	335

1 Goals

In this script, you will learn how to program simple 2D games using the programming language [Python](#) and the game library [Pygame-ce](#).

The main goal is not to create a perfect or finished game. Instead, this script focuses on helping you understand the basic ideas and principles behind game programming.

You will learn, step by step,

- 👉 how a simple game is structured,
- 👉 how graphics are drawn on the screen,
- 👉 how bitmaps are drawn on the screen,
- 👉 how to move game elements,
- 👉 how to use the classes `Sprite` and `Group`,
- 👉 how keyboard and mouse input work,
- 👉 how to produce text outputs by fonts and bitmaps,
- 👉 how sounds and music can be used,
- 👉 how system events and user defined events work,
- 👉 how game objects like figures or obstacles interact i.e. collision detection,
- 👉 how to implement time based logic,
- 👉 and many small details about pygame-ce.

I will also present some programming techniques in the chapter *Techniques* that you may find useful. This chapter is still fairly thin, but it already contains an introduction to the topics: animation, tile-based graphics, and how to handle very large game worlds. Additional techniques – such as a 3D-style visual effect for passing landscapes – are currently being developed.

In the final chapter, I introduce a few smaller games in order to demonstrate the concrete application of these techniques: the classic example *Pong*, a bubble sticking one, and the *Moonlander*.

You can find all source code and resources on github.com/adamsralf/pygame_book and will be updated regularly. Each section kicks off with a link to the relevant GitHub subdirectory.

What is *not* part of this script:

- ⚠ camera, controller, touch pad, joystick as input devices
- ⚠ clipboard support
- ⚠ test module
- ⚠ freetype font
- ⚠ interacting with other languages like C/C++
- ⚠ other platforms like phone, web browser, etc.
- ⚠ client-server communication
- ⚠ midi sound
- ⚠ direct usage of SDL

One thing I'm really not good at is creating visually appealing game worlds. And if I'm being honest, I've always cared far more about programming than about game design. So if you're looking for a deep dive into everything from sketchbooks and graphics tools to a polished final game, you'll be better off turning to other authors.

This script is especially designed for beginners. You do not need any previous experience with game programming. Basic knowledge of Python is required.

Many examples are kept short and simple. You are encouraged to **try things out, experiment, and change the code**. Making mistakes is part of learning — and often the best way to understand how things work. At the end of this script, you should be able to create your own small 2D games and continue learning on your own.

It is up to you which development environment you use; in this script, I use [VS Code](#).

This script is based on the Pygame fork *Pygame Community Edition* ([Pygame-ce](#)). The source code examples are **not** checked for compatibility with the original Pygame. To keep things simple and easier to read, I will usually just say *Pygame* and will not make a distinction between the two versions.

Support my work

If you enjoyed this book and found it helpful, you're welcome to support my work with a small voluntary contribution. Writing, testing, and explaining things takes time – and occasionally coffee.

If you feel like buying me one (or helping fund the next version of this script), you can do so via PayPal: adamsralf@outlook.de.

Of course, this is entirely optional – but very much appreciated. Thank you for reading.

If you have any suggestions or feedback, feel free to get in touch: adamsralf@outlook.de

Have fun programming and creating your first games!

Ralf Adams

2 Basics

2.0 Installation and Organization

2.0.1 Installing Python and Pygame

2.0.1.1 Python



So ... where are the files?

<https://www.python.org/downloads/>

I will not provide a detailed step-by-step installation guide for Python here. For that, it is best to consult the up-to-date instructions on the official Python website (see above). So, if you do not yet have Python installed, visit the homepage, download the current installation files, and run the installer.

The exact procedure differs depending on whether you are working on Windows, Linux, macOS, Android, or even with Docker images.

For the current edition of this book, I used Python 3.12.150 on Windows 10.0.26200. Python was installed using the official Python stand alone installer.

Python options

- ⚠ During installation, you can choose whether Python should be added to the system search path (environment variable) or not. It is highly recommended to do so, as it allows you to start a Python program simply with `python program.py`. Otherwise, you would always have to specify the full path to the Python executable.
- ⚠ You will also be asked whether Python should be installed just for you or for all users. My recommendation is to always install it for yourself only. This allows other users to use different Python versions, and any customization remain local. It also makes it much harder to break things ;-)
- ⚠ If you do not need multiple Python versions, consider removing older installations – this avoids a lot of unnecessary confusion.

2.0.1.2 Pygame Community Edition

 So ... where are the files?

- <https://pyga.me/>
- https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/00%20Setup/start.py

The installation itself is very simple. Before we continue, however, we should make sure that the original Pygame (not Pygame-ce) is not already installed:

```
1 pip uninstall pygame
2 WARNING: Skipping pygame as it is not installed.
```

If you receive this response, everything is fine. Otherwise, you will be asked whether all Pygame files should be removed, which you can safely confirm with Yes.

After that, install Pygame-ce using:

```
1 pip install pygame-ce
2 Collecting pygame-ce
3 Using cached pygame_ce-2.5.6-cp312-cp312-win_amd64.whl.metadata (12 kB)
4 Using cached pygame_ce-2.5.6-cp312-cp312-win_amd64.whl (10.4 MB)
5 Installing collected packages: pygame-ce
6 Successfully installed pygame-ce-2.5.6
```

And that's it – everything is in place :-)

You might also see the following output:

```
1 pip install pygame-ce
2 Requirement already satisfied: pygame-ce in
  .\AppData\Local\Programs\Python\Python312\Lib\site-packages (2.5.6)
```

In this case, everything is fine as well. You should only check the version number. If it is not the desired or the most recent one, simply uninstall Pygame-ce and then install it again.

After Python and Pygame have been installed, you should run a quick installation check. To do so, download a minimal program and try to start it.

Listing 2.1: Pygame installation test

```
1 import pygame # Pygame-Modul (also for pygame-ce!)
2
3
4 def main():
5     pygame.init()
6     window = pygame.Window(size=(400, 100))
7     window.title = "Pygame test"
8     screen = window.get_surface()
9
10    running = True
```

```

11     while running:
12         for event in pygame.event.get():
13             if event.type == pygame.QUIT:
14                 running = False
15                 screen.fill("white")
16                 window.flip()
17
18     pygame.quit()
19
20
21 if __name__ == "__main__":
22     main()

```

If everything is configured correctly, you should see a window like the one in figure 2.1.

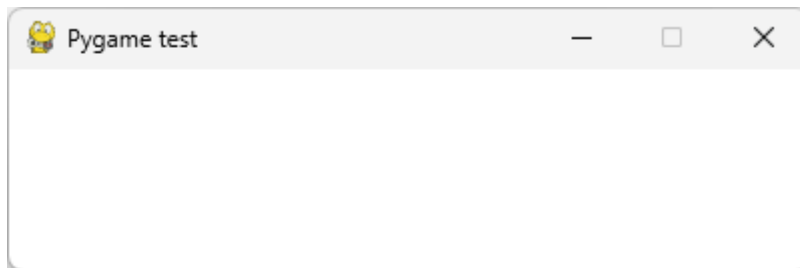


Figure 2.1: Pygame installation test

Error reason number 1

This is by far the most common problem I see with my students: The editor in Visual Studio Code, the word `pygame` is underlined in the line `import pygame`, accompanied by an error message claiming that the module cannot be found.

In the vast majority of cases, this has nothing to do with Pygame itself. The real reason is almost always that multiple Python installations exist on the system.

Pygame is installed into one specific Python installation, usually in the subdirectory

`.\AppData\Local\Programs\Python\Pythonxyz\Lib\site-packages`

If the development environment happens to use a different Python interpreter, it will simply not find the Pygame files – even though they are actually installed.

2.0.2 A Recommended Project Structure

At the beginning, it is tempting to put everything into a single file called `main.py`. This works – for about five minutes.

As soon as your game grows beyond a few hundred lines, structure becomes important. A good directory layout makes your code easier to understand, easier to extend, and much easier to debug.

Here is a simple and proven project structure that we will *loosely* follow throughout this book.

Do not worry if this feels like overkill right now. It will make sense very soon. But what goes where? Let us briefly go through the most important parts.

main.py This is the entry point of your game. It starts Pygame, creates the main window, and runs the main loop. Ideally, **main.py** does not contain a lot of game logic. Think of it as the conductor, not the orchestra.

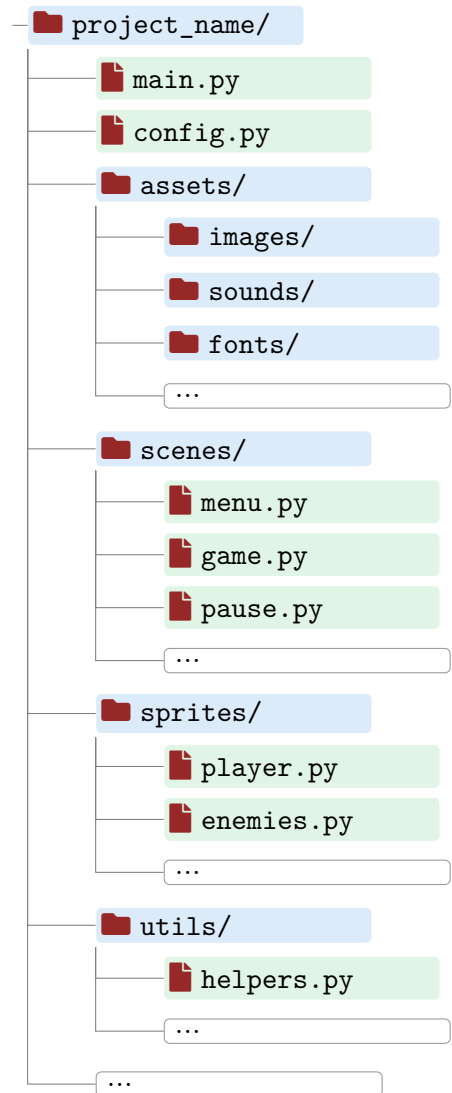
assets/ description Everything that is not Python code goes here: images, sounds, fonts. Keeping assets separate from code avoids clutter and makes it easier to move or reuse them later.

scenes/ Scenes (or states) represent different phases of the game: main menu, actual gameplay, pause screen, game over screen, etc. Separating scenes into individual files keeps each part manageable and avoids huge, unreadable files.

sprites/ This directory contains sprite classes such as the player, enemies, or other game objects. If an object moves, animates, or collides with something, it usually belongs here.

utils/ Helper functions, small tools, or reusable code that does not belong to a specific scene or sprite can live here.

Not every project needs this directory – but many projects end up needing it sooner or later. In this book i. e. I skip the level **assets/**.



A note on file paths

One important habit: Always use relative paths, and always load assets relative to your project directory. Hard-coded absolute paths like `C:\Users\...` or `/home/...` will break as soon as you move the project to another computer. We will look at clean and safe ways to handle paths later in the book (see source code 2.85 on page 117).

If you have reached this point and Pygame starts without errors, the test window opens and your project directory looks reasonably clean, then you are perfectly prepared for the chapters ahead.

From here on, we can finally focus on what this book is really about: writing games instead of fighting your setup.

2.1 Kind of *Hello World!*

 So ... where are the files?

- https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/01%20TheBeginning/v01
- https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/01%20TheBeginning/v02

2.1.1 The Very First Steps

Listing 2.2: My first *Game*, Version 1.0

```

1 import pygame # Pygame-Modul (also for pygame-ce!)
2
3
4 def main():
5     pygame.init()                                # Start subsystem
6     window = pygame.Window(size=(600, 400))      # Create Window
7     window.title = "My first Pygame program"      # Set window title
8     window.position = (10, 50)                   # Set window position
9     screen = window.get_surface()                 # Get the window's bitmap surface
10
11     running = True
12     while running:                               # Main program loop: start
13         for event in pygame.event.get():         # Retrieve events
14             if event.type == pygame.QUIT:        # Window X clicked?
15                 running = False
16             screen.fill((0, 255, 0))             # Fill playground
17             window.flip()                         # Swap double buffer
18
19     pygame.quit()                                # Shut down subsystem
20
21
22 if __name__ == "__main__":
23     main()

```

When you start the application now, you will see a nicely designed window with a green background (figure 2.2). At the moment, however, not much is happening. The only thing you can do is close the window by clicking on the *X* button in the upper right corner of the window frame.

In order to use Pygame, the module `pygame` must be imported into the program (line 1). This makes the `constants`, `functions`, `events`, and `classes` of the `namespace` available.

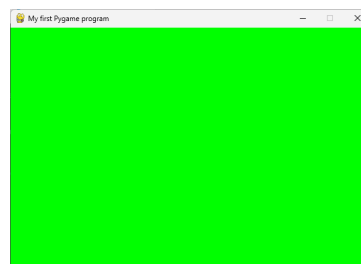


Fig. 2.2: Playground

Pygame is not just about calling functions or creating objects; a whole subsystems must be initialized explicitly. In this example, this is done using the static function

`pygame.init()`. Pygame is now connected to parts of the [Operating System \(OS\)](#) system, to deliver and receive required information and actions. In line 5 the Pygame engine is started by calling `init()`. It is also possible to start only parts of the engine e.g. the sound subsystem by `pygame.mixer.init()`.

init()

For our games, we need a *playfield*/a window in which everything takes place. The class `pygame.Window` represents such a playfield. In line 6, the constructor receives one argument – namely the width and the height of the window as the 2-tuple `size`. Our window is therefore 600px wide and 400px high (see [Pixel \(px\)](#)). The method `get_surface()` in line 9 returns a `pygame.Surface` object, which is roughly something like a [bitmap](#).

Window

get_surface()

In line 9, I store this return value in the [variable](#) named `screen`. I can then assign a title to the window using the attribute `Window.title` (see line 7) and set the position of the window relative to the desktop using the attribute `Window.position` (see line 8).

screen

title

The game itself – just like all future games – runs inside a [main loop](#). The loop starts in line 12 and ends in line 19.

main loop

Inside this loop, three things will always happen in the future:

- 👉 Reading and processing events: As shown in line 13f., mouse, keyboard, or game events are detected and passed on to the game elements. In our case, only clicking the X in the upper right corner of the window is registered.
- 👉 Updating the state of the game elements: Based on the events detected above and the current states of the game elements, the new states are determined (the player moves, a projectile bounces off, points increase, etc.). In our case, only the [flag running](#) of the main program loop is set to `False`.
- 👉 Drawing the bitmaps of the game elements: The game elements have a new position or a new appearance and must therefore be redrawn. In this minimal example, only the background of the playfield is colored in line 16, and afterwards the [double buffer](#) is swapped using `Window.flip()` in line 17.

By calling `pygame.init()`, Pygame places a kind of listener inside the operating system. More precisely, Pygame listens to the [message queue](#). This is where the operating system collects all messages that are triggered by events. These can include [USB](#) connection messages, [SSD](#) error messages, mouse actions, program starts or crashes, and many others.

event.get()

Pygame now retrieves from the message queue, using `pygame.event.get()`, all events that could be relevant to the game. Using a `for`-loop, I then iterate over these events starting at line 13 and pick out the ones that are relevant to me.

event.type

First, I check which type of event (`pygame.event.type`) is being offered. At the moment, only the type `pygame.QUIT` is important to me. This type is triggered when the operating system sends a *quit* message to the application. If I receive such a message,

QUIT

I set the flag `running` to `False`, so that the main program loop is terminated.

If I do not receive this signal, the main program loop continues to run happily and fills the entire playfield with a color in line 16 using `screen.fill()` – in this case, green. Please note that, similar to line 6, the function expects one argument – namely a 3-tuple. This 3-tuple encodes the color using **RGB** values between 0 and 255. Predefined color names such as *green* can also be used here.

RGB

color
names

quit()

What remains is line 17: Here, the function `pygame.quit()` is called. This function is essentially the opposite of `pygame.init()` in line 5. All reserved resources are released again, and the Pygame listeners are removed from the system. You should always make sure to call this function at the end of your application; do not simply terminate the game. The difference is similar to just running out of your apartment versus properly turning off the lights and locking the door when leaving.

If we take a look at the game in the task manager (see figure 2.3), we might be a bit surprised: around 30 % of the CPU time is being used by this *IAmActuallyDoingNothing* game.

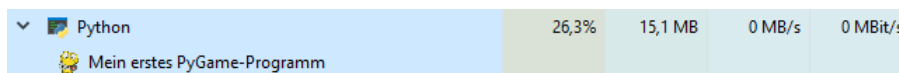


Figure 2.3: Resource usage without timing control

However, if we take a closer look at the main program loop, this should not really be surprising. A bitmap is being drawn onto the screen without any limitation and without interruption. It would be better to allow enough time in each loop iteration to collect events, calculate the new states, and only then generate the screen output. The screen output itself should also not happen arbitrarily fast or too often; in general, about 60 **Frames Per Second (FPS)** are sufficient for motion to be perceived as smooth.

fps

Listing 2.3: My first *Game*, Version 1.1

```

1 import pygame
2
3
4 def main():
5     pygame.init()
6     window = pygame.Window(size=(600, 400),
7                             title="My first Pygame program",
8                             position=(10, 50))
9     screen = window.get_surface()
10    clock = pygame.time.Clock()
11
12    running = True
13    while running:
14        for event in pygame.event.get():
15            if event.type == pygame.QUIT:
16                running = False
17        screen.fill((0, 255, 0))
18        window.flip()
19        clock.tick(60)
20

```

```

21     pygame.quit()
22
23
24 if __name__ == "__main__":
25     main()

```

Clock

In line 10, a `pygame.time.Clock` object is created for timing control. With the help of this object, various time-related tasks can be handled; for the moment, however, we only need it for timing in line 19. There, `pygame.time.Clock.tick()` is called with a frame rate measured in *fps*. This function ensures that the application now runs at a maximum of 60 fps. This can be seen in the significantly reduced CPU usage shown in figure 2.4.

tick()

Hint

The Pygame documentation points out that the function `tick()` is very resource-efficient, but somewhat imprecise. If accuracy is important for timing, the function `tick_loop()` is recommended instead. Its disadvantage, however, is that it consumes significantly more processing time than `tick()`.

Python	0,3%	15,3 MB	0 MB/s	0 MBit/s
Mein erstes PyGame-Programm				

Figure 2.4: Resource usage with timing control

2.1.2 More Input

2.1.2.1 Multiple Windows

 So ... where are the files?

https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/01%20TheBeginning/v03/start.py

You can also create multiple windows for a game (see <https://pyga.me/docs/ref/window.html>).

Listing 2.4: Multiple Windows

```

4 def main():
5     pygame.init()
6     window_first = pygame.Window(size=(300, 50),
7         title="Main Window",
8         position=(500, 50))
9     window_second = pygame.Window(size=(300, 50),
10         title="Side Window",
11         position=(820, 50))
12     screen_first = window_first.get_surface()
13     screen_second = window_second.get_surface()
14     clock = pygame.time.Clock()
15
16
17     running = True
18     while running:
19         for event in pygame.event.get():
20             if event.type == pygame.QUIT:
21                 running = False
22             elif event.type == pygame.WINDOWCLOSE:
23                 running = False
24                 event.window.destroy()
25         if running:
26             screen_first.fill((0, 255, 0))
27             window_first.flip()
28             screen_second.fill((255, 0, 0))
29             window_second.flip()
30             clock.tick(60)
31
32     pygame.quit()

```



Figure 2.5: Multiple Windows

2.1.2.2 Information About the Graphics Environment

 So ... where are the files?

https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/01%20TheBeginning/v04/start.py

Info()

Sometimes it is necessary to know information about the graphics environment: perhaps to identify performance problems, or perhaps to find out which display features are available. Using the function `pygame.display.Info()`, various parameters can be queried. On my system, calling this function produced the output shown in figure 2.6 on the next page.

Please refer to table 2.1 for the meaning of the values (source: <https://pyga.me/docs/ref/display.html#pygame.display.Info>).

Table 2.1: Fields of `pygame.display.Info()`

Field	Description
<code>hw</code>	1 if the display is hardware accelerated.
<code>wm</code>	1 if windowed display modes can be used.
<code>video_mem</code>	The amount of video memory on the display in megabytes. This value is 0 if the amount is unknown.
<code>bitsize</code>	Number of bits used to store each pixel.
<code>bytesize</code>	Number of bytes used to store each pixel.
<code>masks</code>	Four values used to pack RGBA values into pixels.
<code>shifts</code>	Four values used to pack RGBA values into pixels.
<code>losses</code>	Four values used to pack RGBA values into pixels.
<code>blit_hw</code>	1 if hardware Surface blitting is accelerated.
<code>blit_hw_CC</code>	1 if hardware Surface colorkey blitting is accelerated.
<code>blit_hw_A</code>	1 if hardware Surface pixel alpha blitting is accelerated.
<code>blit_sw</code>	1 if software Surface blitting is accelerated.
<code>blit_sw_CC</code>	1 if software Surface colorkey blitting is accelerated.
<code>blit_sw_A</code>	1 if software Surface pixel alpha blitting is accelerated.
<code>current_w,</code> <code>current_h</code>	Width and height of the current video mode, or of the desktop mode if called before <code>display.set_mode()</code> . The values are -1 on error.
<code>pixel_format</code>	The pixel format of the display surface as a string, for example <code>PIXELFORMAT_RGB888</code> .

Listing 2.5: `pygame.display.Info()`

```

4 def main():
5     pygame.init()
6     screen=pygame.display.set_mode([200, 200])
7     info = pygame.display.Info() #pygame.display.get_wm_info()
8     pygame.display.message_box("Window System Information", repr(info), "info")
9     pygame.quit()

```

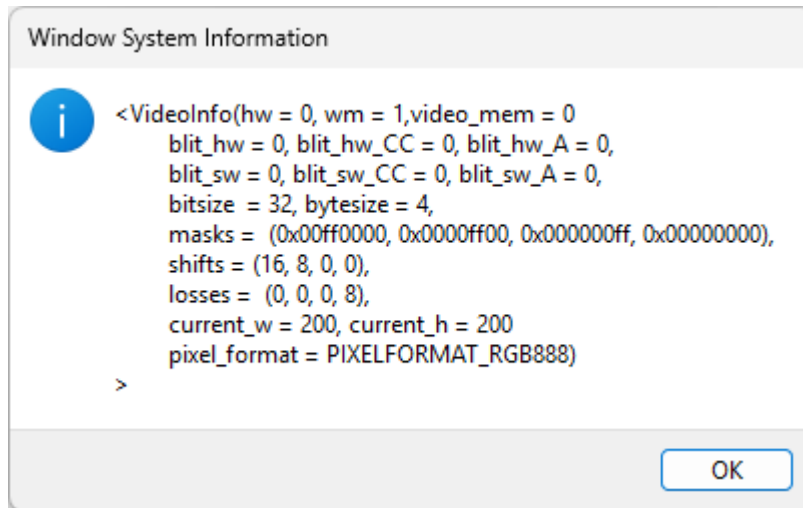


Figure 2.6: Infos about the graphical environment

2.1.2.3 Using `pygame.display` instead of `pygame.Window`

So ... where are the files?

https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/01%20TheBeginning/v04/start.py

Based on feedback from the community, it became clear that there is a need to also show how to create a window using `pygame.display`. This seems to cause issues especially on Linux systems running [Wayland](#).

Creating the window happens like this (or something very close to it) in line 6. Using `pygame.display.set_mode()`, you pass the window size in pixels as a pair of numbers – here 400 px × 600 px. The function returns a `Surface` object, which we can then use to draw everything we need (or want) onto the screen.

`set_mode()`

Number pair!

A very common beginner mistake is passing the size as two separate numbers instead of a number pair. So don't forget the parentheses – even if they look a bit odd at first!

Listing 2.6: Window by `pygame.display`

```

1 import pygame
2
3
4 def main():
5     pygame.init()
6     screen = pygame.display.set_mode((400,600))      # Create window
7     pygame.display.set_caption("Window by Display Module") # Set window title
8
9     running = True
10    while running:
11        for event in pygame.event.get():
12            if event.type == pygame.QUIT:
13                running = False
14
15        screen.fill("white")                          # Fill background with white
16        pygame.display.flip()                          # Update display
17
18    pygame.quit()
19
20
21 if __name__ == "__main__":
22     main()

```

The function `set_mode()` can take a couple of additional parameters. For example, you can decide whether the window should be resizable (see table 2.2 on page 26) or specify which color depth to use.

`set_caption()`

The window title is set in line 7 using `pygame.display.set_caption()`. Finally, `pygame.display.flip()` pushes the complete contents of `screen` to the display in one go.

`flip()`

2.1.3 What was new?

To start a minimal Pygame application

- 👉 Import the Pygame library.
- 👉 Initialize the Pygame system.
- 👉 Create a window / a playfield.
- 👉 Set up a main program loop:
 1. Poll events.

2. Update game objects.
3. Render the screen contents.
4. Control the timing of the loop iterations.

👉 Shut down the Pygame system when exiting.



The following Pygame elements were introduced



```
+ import pygame:
  https://pyga.me/docs/tutorials/en/import-init.html

+ pygame.display.flip():
  https://pyga.me/docs/ref/display.html#pygame.display.flip

+ pygame.display.set_caption():
  https://pyga.me/docs/ref/display.html#pygame.display.set_caption

+ pygame.display.set_mode():
  https://pyga.me/docs/ref/display.html#pygame.display.set_mode

+ pygame.init():
  https://pyga.me/docs/ref/pygame.html#pygame.init

+ pygame.quit():
  https://pyga.me/docs/ref/pygame.html#pygame.quit

+ pygame.QUIT:
  https://pyga.me/docs/ref/event.html#pygame.event.EventType.type

+ pygame.WINDOWCLOSE:
  https://pyga.me/docs/ref/event.html#pygame.event.EventType.type

+ pygame.event.get():
  https://pyga.me/docs/ref/event.html#pygame.event.get

+ pygame.event.type:
  https://pyga.me/docs/ref/event.html#pygame.event.EventType.type

+ pygame.time.Clock:
  https://pyga.me/docs/ref/time.html#pygame.time.Clock

+ pygame.time.Clock.tick():
  https://pyga.me/docs/ref/time.html#pygame.time.Clock.tick
```

	<code>pygame.time.Clock.tick_busy_loop():</code> https://pyga.me/docs/ref/time.html#pygame.time.Clock.tick_busy_loop
	<code>pygame.Surface.fill():</code> https://pyga.me/docs/ref/surface.html#pygame.Surface.fill
	<code>pygame.Window:</code> https://pyga.me/docs/ref/window.html
	<code>pygame.Window.destroy():</code> https://pyga.me/docs/ref/window.html#pygame.Window.destroy
	<code>pygame.Window.flip():</code> https://pyga.me/docs/ref/window.html#pygame.Window.flip
	<code>pygame.Window.get_surface():</code> https://pyga.me/docs/ref/window.html#pygame.Window.get_surface
	<code>pygame.Window.title:</code> https://pyga.me/docs/ref/window.html#pygame.Window.title
	<code>pygame.Window.position:</code> https://pyga.me/docs/ref/window.html#pygame.Window.position

Table 2.2: Predefined Display Modes

Constant	Description
FULLSCREEN	create a fullscreen display
DOUBLEBUF	only applicable with OPENGL
HWSURFACE	(obsolete in pygame 2) hardware accelerated, only in FULLSCREEN
OPENGL	create an OpenGL-renderable display
RESIZABLE	display window should be sizeable
NOFRAME	display window will have no border or controls
SCALED	resolution depends on desktop size and scale graphics
SHOWN	window is opened in visible mode (default)
HIDDEN	window is opened in hidden mode

2.1.4 Homework

Please have a look at <https://pyga.me/docs/ref/window.html> and then try to solve the following small exercises:

1. Set up a working environment for your game programming. Install Python, your

preferred editor, and the latest Pygame-ce version. Open the directory containing your Pygame source code and try to run `start01.py`.

2. Change the background color of the window. Use RGB values as well as named colors.
3. Change the size of the window.
4. Change the position of the window. Use position values and also try `WINDOWPOS_CENTERED` and `WINDOWPOS_UNDEFINED`.
5. Create the window as a resizable window and try to resize it.
6. Define a minimum and maximum window size, show the actual size in the title bar, and try to resize the window to its limits.
7. Show the actual window position in the title bar.
8. Play a little bit with the `opacity` attribute of the window.
9. Change the title of the window during runtime according to a counter. Shut down the program if counter is greater 600.
10. Create a borderless window.
11. Try a fullscreen window.
12. Try to arrange three windows in a row. Compute the x-position of the second and third window based on the window size and a useful gap between them.

WINDOW-
POS_CEN-
TERED

WINDOW-
POS_UN-
DEFINED

2.2 Graphic Primitives

2.2.1 Introduction

 So ... where are the files?

https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/02%20Primitives/primitives00.py

Graphic primitives are simple graphical shapes that are drawn, such as lines, points, circles, and so on. They do not play a very important role in game programming, because most graphics are predefined bitmaps, but they can be quite useful. For this reason, I will only introduce a few of them here.

Listing 2.7: Graphic Primitives

```

1  import pygame
2  import pygame.gfxdraw  # !
3
4
5  def main():
6      pygame.init()
7      window = pygame.Window(size=(530, 530), title = "Grafic Primitives", position = (10, 50))
8      screen = window.get_surface()
9      clock = pygame.time.Clock()
10
11     mygrey = pygame.Color(200, 200, 200)                # Custom color
12     myrectangle1 = pygame.Rect(10, 10, 20, 30)          # Rectangle object
13     myrectangle2 = pygame.Rect(60, 10, 20, 30)
14     points1 = ((120, 10), (160, 10), (140, 90))        # List of points
15     points2 = ((180, 10), (220, 10), (200, 90))
16
17     running = True
18     while running:
19         for event in pygame.event.get():
20             if event.type == pygame.QUIT:
21                 running = False
22             screen.fill(mygrey)
23             pygame.draw.rect(screen, "red", myrectangle1)    # Filled rectangle
24             pygame.draw.rect(screen, "red", myrectangle2, 3, 5) # Rectangle outline
25             pygame.draw.polygon(screen, "green", points1)    # Filled polygon
26             pygame.draw.polygon(screen, "green", points2, 1) # Polygon outline
27             pygame.draw.line(screen, "red", (5, 230), (240, 230), 3) # Line
28             pygame.draw.circle(screen, "blue", (40, 150), 30) # Filled circle
29             pygame.draw.circle(screen, "blue", (110, 150), 30, 2) # Circle outline
30             pygame.draw.circle(screen, "blue", (180, 150), 30, 5, True) # Arc segment
31             for i in range(255):
32                 for j in range(255):
33                     screen.set_at((265+i, 10+j), (255, i, j)) # Points variant 1
34                     screen.fill((i, j, 255), ((10+i, 265+j), (1, 1)) ) # Variant 2
35                     pygame.gfxdraw.pixel(screen, 265+i, 265+j, (i, 255, j)) # Variant 3
36
37             window.flip()
38             clock.tick(60)
39
40     pygame.quit()

```


The basic structure is the same as in source code 2.3 on page 19. The differences begin in line 11. The class `pygame.Color` can encode color information in various formats, including an [alpha channel](#) ([alpha blending](#), transparency); more about this will follow later in section 2.3 on page 45. Here, I use RGB encoding with color channel values between 0 and 255.

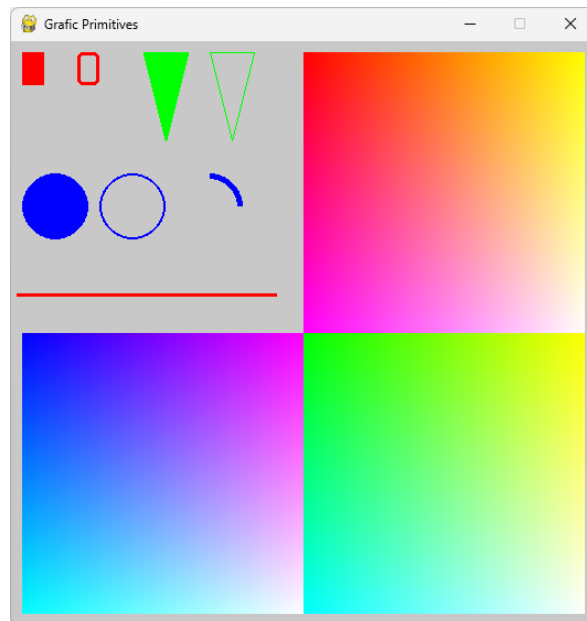
Color

In most cases, however, I do not need to define my own colors. Pygame provides a really extensive list of 664 predefined color names. Wherever color values are expected, I can pass either a `Color` object, a numeric color code, or a color name as a string.

Color names

Let us go through the individual shapes one by one and start with the rectangle. There are several ways to define a rectangle in Pygame. Since we will need it very often later on, I would like to introduce the class `pygame.Rect` here. It is defined by four parameters: the upper-left corner, its width, and its height. In line 12, a rectangle is therefore defined at the position (10, 10) with a width of 20 px and a height of 30 px.

In line 23, `pygame.draw.rect()` draws a filled rectangle. The [Semantics](#) of the parameters should be self-explanatory. The call in line 24, however, is different. The first parameter after the rectangle – here 3 – specifies the thickness of the line. If this parameter is given and greater than 0, the rectangle is no longer filled. The value 10 specifies the rounding of the corners. Here, a value between 0 and $\min(\text{width}, \text{height})/2$ can be used, as this value corresponds to the radius of the corner rounding.



Rect

rect()

Fig. 2.7: Some graphic primitives

Hint

The class `Rect` is not a drawn rectangle, but merely a container for information that is relevant for a rectangle. More in chapter 2.4.1 on page 59.

More general than a rectangle is a [Polygon](#). A polygon is a closed chain of lines that is defined in Pygame by its points. Similar to rectangles, there are filled (line 25) and unfilled (line 26) variants. Both are drawn using `pygame.draw.polygon()`.

polygon()

Warning

Be careful with the line thickness: the lines grow outward, which can quickly lead to ugly offsets at the corners. Try it out by changing the value 2 to 5.

line()

For individual lines, there is `pygame.draw.line()`, and for a [polyline](#) – without an example here – there is `pygame.draw.lines()`. An example can be found in [line 27](#).

lines()

circle()

A circle is defined by two values: its center point and its radius. In [line 28](#), a filled circle with the center at (40, 150) and a radius of 30 px is drawn using `pygame.draw.circle()`. As with rectangles and polygons, there are also unfilled variants ([line 29](#)). Of particular interest is the circular arc segment in [line 30](#). Here, Boolean variables are used to control which section of the circular arc is drawn (for more details, see the Pygame reference).

Finally, one small color experiment. Strangely enough, Pygame does not provide a dedicated function for drawing a single point or pixel. Here, I have implemented three workarounds that I found. One could think of additional ones as well: a line with *start = end*, a circle with a radius of 1 px, and so on.

set_at()

In [line 33](#), a point is drawn by setting a single color value at a specific position using `pygame.Surface.set_at()`. Alternatively, the `fill()` surface function used earlier can be applied with an area of only one pixel in width and height ([line 34](#)). Another way to set a pixel using a graphics library is the experimental `gfxdraw` module. In [line 35](#), a single pixel is set using `pygame.gfxdraw.pixel()`. The `gfxdraw` module is not imported automatically by `import pygame` (see [line 2](#)).

pixel()

2.2.2 More Input

2.2.2.1 Example: Particle Swarm

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/02%20Primitives/circles

Using graphic primitives, it is possible to create dynamic effects, such as particle swarms. Here, I would like to present a very simple example of a mouse-controlled fountain made of circles.

Let us first build a small program that draws a circle at the mouse position. The class `Circle` (see [line 4](#)) contains all the information I need to draw circles: position, radius, and color. The position is defined via a constructor argument. In the method `draw()`, the screen output is encapsulated.

The function `main()` now contains a lot of familiar elements, but also a few new ones. In [line 4](#), the screen

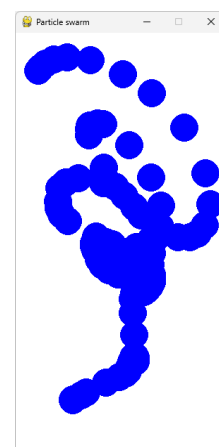


Fig. 2.8: Not a particle swarm

size is stored in a list, because we still need this information at another place, namely in line 18. Below that, in line 24, a list for storing the circles is defined.

Inside the main loop, line 32 checks whether the left mouse button has been pressed. If so, a circle is drawn at the mouse position. After that, the screen is filled with white color and the circles stored in the container are drawn.

get_pressed()

get_pos()

The result is not very impressive yet (see figure 2.8 on the preceding page) and is more reminiscent of a drawing program like Paint.

Listing 2.8: Particle swarm, Version 1.0

```

1 import pygame
2
3
4 class Circle:                                # Very helpful
5     def __init__(self, pos) -> None:
6         self.posx = pos[0]
7         self.posy = pos[1]
8         self.radius = 20
9         self.color = "blue"
10
11     def draw(self, screen: pygame.Surface) -> None:
12         pygame.draw.circle(screen, self.color, (self.posx, self.posy), self.radius)
13
14
15 def main():
16     size = (300, 600)                        # Screen size
17     pygame.init()
18     window = pygame.Window( size=size,        # Create window
19                             title = "Particle swarm",
20                             position = (10, 50))
21     screen = window.get_surface()
22
23     clock = pygame.time.Clock()
24     circles = []                             # Container for circles
25
26     running = True
27     while running:
28         for event in pygame.event.get():
29             if event.type == pygame.QUIT:
30                 running = False
31
32         if pygame.mouse.get_pressed()[0]:      # Left mouse button?
33             circles.append(Circle(pygame.mouse.get_pos()))
34
35         screen.fill("white")
36         for p in circles:
37             p.draw(screen)
38
39         window.flip()
40         clock.tick(60)
41
42     pygame.quit()
43
44
45 if __name__ == '__main__':
46     main()

```

In the next step, we want to turn the bulky circles into colorful particles. These particles should also no longer appear exactly at the mouse position, but be scattered around it. To achieve this, only minimal changes need to be made to the `Circle` class.

The two position values are now extended by a random value between -2 and $+2$. The radius is also reduced to 2px. The color is likewise varied using random values. I tried out several combinations here, and I quite like this color variation. Feel free to experiment with the color channels and the random values yourself. The result shown in figure 2.9 already looks much better.

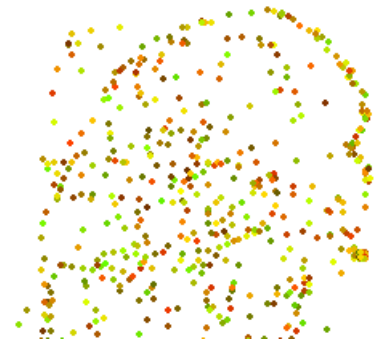


Fig. 2.9: Particle swarm
Version 2

Listing 2.9: Particle swarm, Version 2.0

```

7  def __init__(self, pos) -> None:
8      self.posx = pos[0] + randint(-2, 2)
9      self.posy = pos[1] + randint(-2, 2)
10     self.radius = 2
11     self.color = [randint(100, 255), randint(50, 255), 0]

```

Now we want to add a bit of dynamics to the game. The particles should first rise upward and then fall down again. To achieve this, I added the vertical velocity `speedy` to the `Circle` class and assigned it a random initial value (line 14). The division by 10.1 ensures that no smooth, rounded values are created. Here as well, feel free to experiment with the values to see the different effects.

The class also needs to be extended by the method `update()`. In this method, the new vertical position `posy` is calculated based on the vertical velocity `speedy`, and the velocity is in turn modified with respect to `gravity`. In order for all particles to be subject to the same gravitational force, I defined `GRAVITY` as a static attribute (line 7).

Gravity

Listing 2.10: Particle swarm, Version 3.0, Class `Circle`

```

6  class Circle:
7      GRAVITY = 0.3                                     # Gravity as a static attribute
8
9      def __init__(self, pos) -> None:
10         self.posx = pos[0] + randint(-2, 2)
11         self.posy = pos[1] + randint(-2, 2)
12         self.radius = 2
13         self.color = [randint(100, 255), randint(50, 255), 0]
14         self.speedy = randint(-100, 0) / 10.01         # Initial vertical speed
15
16     def update(self) -> None:
17         self.speedy += Circle.GRAVITY
18         self.posy += self.speedy

```

All that remains is the call of `update()` inside the main program loop.

Listing 2.11: Particle swarm, Version 3.0, Call of `update()`

```

38     if pygame.mouse.get_pressed()[0]:
39         circles.append(Circle(pygame.mouse.get_pos()))
40
41     for p in circles:
42         p.update()

```

The fountain is still not really lively yet. So let us also scatter the particles horizontally. For this purpose, the attribute `speedx` is added in the constructor. The upper and lower bounds of the random number generator determine the width of the particle fountain. Try out values here that match your own sense of aesthetics. In `update()`, the new horizontal position `posx` then has to be calculated.

The horizontal velocity does not need to be adjusted, since `GRAVITY` is only supposed to act downward.

Listing 2.12: Particle swarm, Version 4.0, `Circle.update()`

```

9     def __init__(self, pos) -> None:
10         self.posx = pos[0] + randint(-2, 2)
11         self.posy = pos[1] + randint(-2, 2)
12         self.radius = 2
13         self.color = [randint(100, 255), randint(50, 255), 0]
14         self.speedx = randint(-10, 10) / 10.01
15         self.speedy = randint(-100, 0) / 10.01
16
17     def update(self) -> None:
18         self.speedy += Circle.GRAVITY
19         self.posx += self.speedx
20         self.posy += self.speedy

```

After some time, the list `circles` contains many particles that are no longer displayed at all. We want to remove these particles. To do this, the `Circle` class needs to determine whether the object can be deleted.

As a first step, we add the deletion flag `todelete` to the class (see line 16), which is initialized to `False`; a new particle should of course not be deleted immediately.

In line 22, it is checked whether the right edge of the particle (center point plus radius) lies outside the screen on the left. If this is the case, the deletion flag must be set to `True`. Analogously, the right and the bottom edges of the screen are checked in line 24 and line 26.

For this purpose, the attribute `pygame.Window.size` is used to determine the width and height of the screen. This attribute returns the screen size as a 2-tuple. The zeroth value represents the width, and the first value represents the height. A check to see whether the



Fig. 2.10: Particle swarm,
Version 5: nearly finished

size

particle has disappeared upward is not necessary, since it will eventually fall down again and thus become visible once more.

Listing 2.13: Particle swarm, Version 5.0, Class Circle

```

6 class Circle:
7     GRAVITY = 0.3
8
9     def __init__(self, pos) -> None:
10         self.posx = pos[0] + randint(-2, 2)
11         self.posy = pos[1] + randint(-2, 2)
12         self.radius = 2
13         self.color = [randint(100, 255), randint(50, 255), 0]
14         self.speedx = randint(-10, 10) / 10.01
15         self.speedy = randint(-100, 0) / 10.01
16         self.todelete = False                                # Delete flag
17
18     def update(self, window: pygame.Window) -> None:
19         self.speedy += Circle.GRAVITY
20         self.posx += self.speedx
21         self.posy += self.speedy
22         if self.posx - self.radius < 0:                        # Left side out
23             self.todelete = True
24         elif self.posx + self.radius > window.size[0]:        # Right side out
25             self.todelete = True
26         elif self.posy - self.radius > window.size[1]:        # Bottom out
27             self.todelete = True
28
29     def draw(self, screen: pygame.Surface) -> None:
30         pygame.draw.circle(screen, self.color, (self.posx, self.posy), self.radius)

```

In the main program, I now need to implement suitable deletion logic. But first, I want my fountain to have a bit more *oomph*: In line 48, not just one particle is created, but always five at once.

In line 51, an empty list is created that will contain the particles to be deleted. Inside the update loop, it is now additionally checked whether a particle should be deleted (line 54). If so, this particle is added to the list `todelete`. After the update loop has finished, the particles to be deleted are removed from the list `circles` starting at line 56.

In figure 2.10 on the previous page, you can see a fountain. It really starts to look cool only when you move the mouse while it is running.

Listing 2.14: Particle swarm, Version 5.0, Main loop

```

41 running = True
42 while running:
43     for event in pygame.event.get():
44         if event.type == pygame.QUIT:
45             running = False
46
47     if pygame.mouse.get_pressed()[0]:
48         for i in range(5):                                    # 5 particles at once
49             circles.append(Circle(pygame.mouse.get_pos()))
50
51     todelete = []                                            # Temporary storag
52     for p in circles:
53         p.update(window)
54         if p.todelete:                                       # Marked for deletion?

```

```

55         todelete.append(p)
56     for p in todelete:                                # Delete
57         circles.remove(p)
58
59     screen.fill("white")
60     for p in circles:
61         p.draw(screen)
62
63     window.flip()
64     clock.tick(60)

```

Why do I not call `remove()` already inside the update loop?

Because: *Never increase or decrease the length of a list while you are iterating over it.* Very strange effects can occur.

Try to guess the number of loop iterations of the following program:

```

1     values = [1, 2, 3]
2     for a in values:
3         values.append(a*10)
4     print(vlues)

```

Even small changes to the parameters can already produce interesting visual effects. Unfortunately, these cannot be shown very well using images here, so: program it yourself and try it out.

Listing 2.15: Particle swarm, Version 6.0

```

6 class Circle:
7     GRAVITY = 0.3
8     RADIUS_INC = -0.1                                # Radius increment
9
10    def __init__(self, pos) -> None:
11        self.posx = pos[0] + randint(-4, 4)            # Changed spread
12        self.posy = pos[1] + randint(-4, 4)
13        self.radius = 8
14        self.color = [randint(100, 255), randint(50, 255), 0]
15        self.speedx = randint(-15, 15) / 10.01        # Fountain wider
16        self.speedy = randint(-100, 0) / 10.01
17        self.todelete = False
18
19    def update(self, window: pygame.Window) -> None:
20        self.speedy -= Circle.GRAVITY
21        self.posx += self.speedx
22        self.posy += self.speedy
23        self.radius += Circle.RADIUS_INC
24        if self.posx - self.radius < 0:
25            self.todelete = True
26        elif self.posx + self.radius > window.size[0]:
27            self.todelete = True
28        elif self.posy - self.radius > window.size[1]:
29            self.todelete = True
30        elif self.radius <= 0.0:                        # Can be removed
31            self.todelete = True
32
33    def draw(self, screen: pygame.Surface) -> None:
34        pygame.draw.circle(screen, self.color, (self.posx, self.posy), self.radius)
35

```

```

36
37 def main():
38     size = (300, 600)
39     pygame.init()
40     window = pygame.Window( size=size, title = "Particle swarm", position = (10, 50))
41     screen = window.get_surface()
42     clock = pygame.time.Clock()
43     circles = []
44
45     running = True
46     while running:
47         for event in pygame.event.get():
48             if event.type == pygame.QUIT:
49                 running = False
50
51             if pygame.mouse.get_pressed()[0]:
52                 for i in range(5):
53                     circles.append(Circle(pygame.mouse.get_pos()))
54
55             todelete = []
56             for p in circles:
57                 p.update(window)
58                 if p.todelete:
59                     todelete.append(p)
60             for p in todelete:
61                 circles.remove(p)
62
63             screen.fill("white")
64             for p in circles:
65                 p.draw(screen)
66
67             window.flip()
68             clock.tick(60)
69
70     pygame.quit()

```

2.2.2.2 Example: Landscape

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/02%20Primitives/landscape

In this example, we combine graphic primitives, object-oriented design, and simple mathematics to create a small animated scene.

I would like to create a small animated landscape that should look like in figure 2.11 on the facing page. The sun should rise on the left, move across the sky, and set on the right. The blue color of the sky should also change depending on the time of day.

Let us start with a basic framework that I want to expand step by step. The elements of the source code 2.16 on the next page should be self-explanatory. The variable `horizon` is meant to control the boundary between the sky and the meadow – in other words, it forms the horizon. The sky, sun, tree, house, and meadow should all align with this boundary.

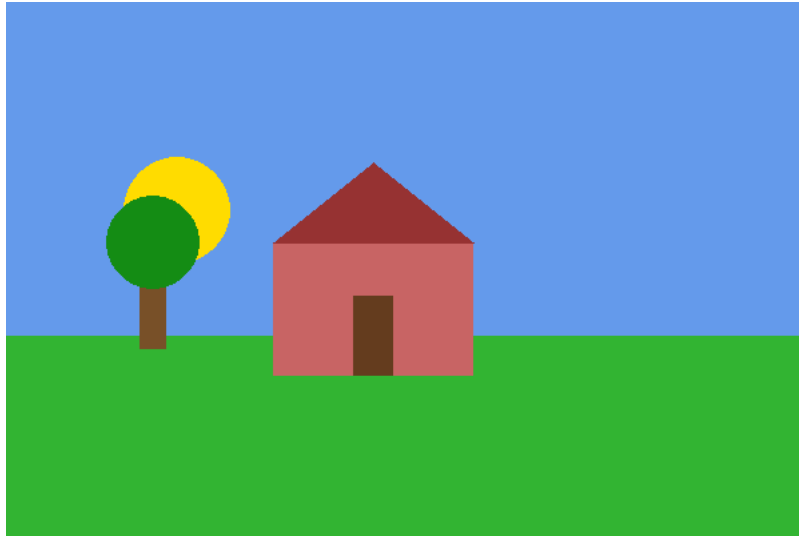


Figure 2.11: Example: Drawing a Landscape

Listing 2.16: Landscape, Version 1.0

```

1 import pygame
2
3
4 def main():
5     size = (600, 400)
6     pygame.init()
7     window = pygame.Window( size=size, title = "A Peaceful Day")
8     clock = pygame.time.Clock()
9     horizon = 250
10
11
12     running = True
13     while running:
14         # Watch for events
15         for event in pygame.event.get():
16             if event.type == pygame.QUIT:
17                 running = False
18
19         # Updates
20
21         # Draw
22         window.flip()
23         clock.tick(60)
24
25     pygame.quit()
26
27
28 if __name__ == "__main__":
29     main()

```

I extend the program with the very simple class **Meadow**. In the constructor, a reference to the window and the horizon are stored, and the color is defined — in this case, a custom shade of green. After that, the upper-left corner and the size of the meadow are calculated. Both values take the horizon into account. The method `draw()` then draws

the meadow as a green rectangle into the window.

Listing 2.17: Landscape, Version 2.0, Class Meadow

```

4 class Meadow:
5     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
6         self.window = window
7         self.horizon = horizon
8         self.color = (50, 180, 50)
9         self.lefttop = 0, self.horizon
10        self.widthheight= self.window.size[0], self.window.size[1] - self.horizon
11
12    def draw(self) -> None:
13        screen = self.window.get_surface()
14        pygame.draw.rect(screen, self.color, (self.lefttop, self.widthheight))

```

In `main()`, an object of the class `Meadow` is now created in line 23, and in line 36 the meadow is drawn using the `draw()` method. The result looks like the one shown in figure 2.12 on the facing page.

Listing 2.18: Landscape, Version 2.0, `main()`

```

22 horizon = 250
23 meadow = Meadow(window, horizon) #
24
25
26 running = True
27 while running:
28     # Watch for events
29     for event in pygame.event.get():
30         if event.type == pygame.QUIT:
31             running = False
32
33     # Updates
34
35     # Draw
36     meadow.draw() #
37     window.flip()

```

The class `Sky` is similarly simple. Here as well, reference data is stored in the constructor, and in `draw()` a blue rectangle is drawn whose size depends on `horizon`.

Listing 2.19: Landscape, Version 3.0, Class `Sky`

```

16 class Sky:
17     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
18         self.window = window
19         self.horizon = horizon
20         self.color = (100, 150, 255)
21
22     def draw(self) -> None:
23         screen = self.window.get_surface()
24         pygame.draw.rect(screen, self.color, (0, 0, self.window.size[0], self.horizon))

```

All that remains is to integrate it into `main()` in the same way as `Meadow` (see figure 2.13 on the next page). Play around a bit with the variable `horizon` to see the effect.

Listing 2.20: Landscape, Version 3.0, `main()`

```

31 horizon = 250
32 meadow = Meadow(window, horizon)
33 sky = Sky(window, horizon)                                #
34
35
36 running = True
37 while running:
38     # Watch for events
39     for event in pygame.event.get():
40         if event.type == pygame.QUIT:
41             running = False
42
43     # Updates
44
45     # Draw
46     meadow.draw()
47     sky.draw()                                           #
48     window.flip()

```

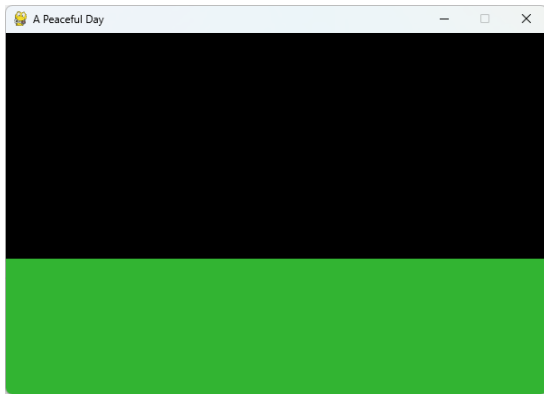


Figure 2.12: Drawing a Landscape (2)

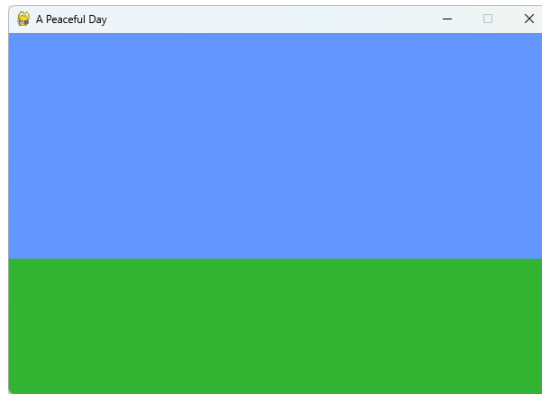


Figure 2.13: Drawing a Landscape (3)

The class `Tree` consists of two parts: a tree trunk and a leafy crown. The tree trunk is created in `draw()` using a rectangle, and the leafy crown is created using a circle. I will not show the integration into `main()` here, since it is completely analogous to the integration of `Meadow` and `Sky`. Only the order needs to be considered, because the tree is supposed to appear in the foreground. The tree should look like the one shown in figure 2.14 on the following page.

Order matters

One of the most common beginner mistakes is drawing graphic objects in the wrong order. When that happens, objects often end up being partly or completely covered, and you start looking for the bug in the wrong place.

Listing 2.21: Landscape, Version 4.0, Class Tree

```

28 class Tree:
29     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
30         self.window = window
31         self.colors = [(120, 80, 40), (20, 140, 20)]
32         self.start = (100, horizon - 50)
33
34     def draw(self) -> None:
35         screen = self.window.get_surface()
36         pygame.draw.rect(screen, self.colors[0], (self.start, (20, 60)))
37         pygame.draw.circle(screen, self.colors[1], (self.start[0]+10, self.start[1]-20), 35)

```

The basic principle of the class `House` is the same as for the other classes. It is just a bit more complex, since it consists of two rectangles and a triangle. Here as well, the integration into `main()` is trivial and is left to you. In figure 2.15, only the sun is missing now.

Listing 2.22: Landscape, Version 5.0, Class House

```

40 class House:
41     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
42         self.window = window
43         self.colors = [(200, 100, 100), (150, 50, 50), (100, 60, 30)]
44         self.start = (200, horizon - 70)
45
46     def draw(self) -> None:
47         screen = self.window.get_surface()
48         pygame.draw.rect(screen, self.colors[0], (self.start, (150, 100)))
49         pygame.draw.polygon(
50             screen,
51             self.colors[1],
52             [self.start, (self.start[0]+75, self.start[1]-60), (self.start[0]+150,
53                 self.start[1])]
54         )
55         pygame.draw.rect(screen, self.colors[2], (self.start[0]+60, self.start[1]+40, 30,
56             60))

```

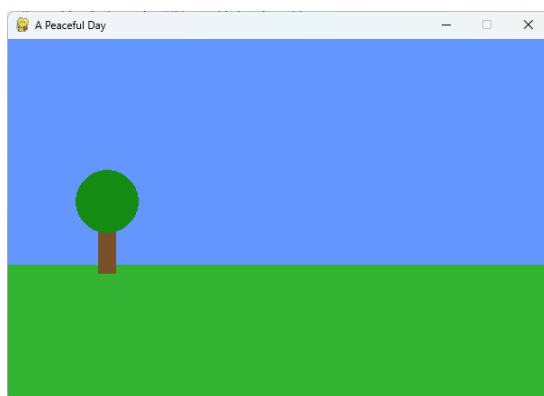


Figure 2.14: Drawing a Landscape (4)

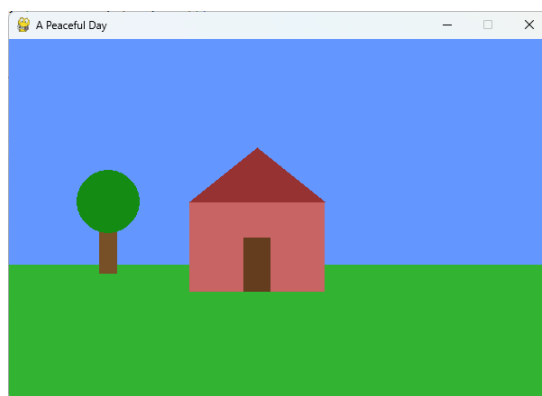


Figure 2.15: Drawing a Landscape (5)

In its basic shape, the sun is a simple yellow filled circle. However, we want it to move across the sky. Therefore, we need a start position below the horizon (see line 63) and a method `update()` that calculates the new position of the sun in each frame.

In `update()`, the new horizontal position is first calculated based on `speed`. After that, I calculate how far the sun has already progressed along its path. This value is relative and has a range of $[0, 1]$. If the sun has covered a quarter of the distance, the value of `progress` is 0.25, at halfway it is 0.5, and so on.

How do I calculate the height now? For the sake of simplicity, I let the sun follow the first half of the sine function. For this, the domain must be $[0, \pi]$; this is the hump of the [sine function](#) that lies above the x-axis. The range of the sine function from 0 to π is $[0, 1]$. If I multiply this value by the horizon, I obtain values from 0 to `horizon`. Finally, I add the radius so that the sun just touches the upper edge at its highest point.

The function `update()` returns the value of `progress` so that I can reuse this value to calculate the color of the sky, which still needs to be implemented. Everything clear? By the way, do not forget to add `import math` at the beginning because of the sine function!

Listing 2.23: Landscape, Version 6.0, Class Sun

```

59 class Sun:
60     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
61         self.speed = 1
62         self.radius = 40
63         self.pos = [-self.radius, 0] #
64         self.color = (255, 220, 0)
65         self.horizon = horizon
66         self.window = window
67
68     def update(self) -> float:
69         self.pos[0] += self.speed
70         progress = self.pos[0] / self.window.size[0] # 0.0 -> 1.0
71         self.pos[1] = self.horizon * (1 - math.sin(progress * math.pi)) + self.radius
72         return progress
73
74     def draw(self) -> None:
75         screen = self.window.get_surface()
76         pygame.draw.circle(screen, self.color, self.pos, self.radius)

```

Like the other classes, the sun is integrated into `main()`: the object is created in line 89 and drawn using `draw()` in line 104. Only the call to `update()` in line 100 is new. Important: The order of the `draw()` calls must be observed! The sun should be drawn after the sky, but before the meadow and the tree.

Listing 2.24: Landscape, Version 6.0, `main()`

```

85 meadow = Meadow(window, horizon)
86 sky = Sky(window, horizon)
87 tree = Tree(window, horizon)
88 house = House(window, horizon)
89 sun = Sun(window, horizon) #
90
91
92 running = True
93 while running:
94     # Watch for events
95     for event in pygame.event.get():
96         if event.type == pygame.QUIT:

```

```

97         running = False
98
99     # Updates
100    sun.update()
101
102    # Draw
103    sky.draw()
104    sun.draw()
105    meadow.draw()
106    tree.draw()
107    house.draw()
108    window.flip()

```

The final stage of the extension concerns the color of the sky. Depending on the position of the sun – more precisely, on the progress of the sun – the blue color of the sky should change. This is done in the new method `update()` of the class `Sky` (see source code [2.25](#)). Here as well, the green component of the color is calculated using the sine function; a linear approach would also have been possible, but the sine function produces smoother transitions near sunrise and sunset.

Listing 2.25: Landscape, Version 7.0, `Sky.update()`

```

25    def update(self, progress: float) -> None:
26        brightness = max(0, min(1, math.sin(progress * math.pi)))
27        blue = int(80 + brightness * 120)
28        self.color = (100, blue, 235)

```

In `main()`, only the relative progress of the sun is now taken and passed to the `update()` method of `Sky`. Done :-)

Listing 2.26: Landscape, Version 7.0, `main()`

```

104    # Updates
105    progress = sun.update()
106    sky.update(progress)

```

2.2.3 What was new?

Using graphic primitives, you can create and use your own drawings. They are usually available in both filled and unfilled variants. Colors can either be defined manually or selected from a list of predefined colors. Objects drawn later appear in front of earlier ones.



The following Pygame elements were introduced



-  Named colors:
https://pyga.me/docs/ref/color_list.html
-  `import pygame.gfxdraw:`
<https://pyga.me/docs/ref/gfxdraw.html>
-  `pygame.Color:`
<https://pyga.me/docs/ref/color.html>
-  `pygame.draw.circle():`
<https://pyga.me/docs/ref/draw.html#pygame.draw.circle>
-  `pygame.draw.line():`
<https://pyga.me/docs/ref/draw.html#pygame.draw.line>
-  `pygame.draw.lines():`
<https://pyga.me/docs/ref/draw.html#pygame.draw.lines>
-  `pygame.draw.polygon():`
<https://pyga.me/docs/ref/draw.html#pygame.draw.polygon>
-  `pygame.draw.rect():`
<https://pyga.me/docs/ref/draw.html#pygame.draw.rect>
-  `pygame.gfxdraw.pixel():`
<https://pyga.me/docs/ref/gfxdraw.html#pygame.gfxdraw.pixel>
-  `pygame.mouse.get_pos():`
https://pyga.me/docs/ref/mouse.html#pygame.mouse.get_pos
-  `pygame.mouse.get_pressed():`
https://pyga.me/docs/ref/mouse.html#pygame.mouse.get_pressed
-  `pygame.Rect:`
<https://pyga.me/docs/ref/rect.html>

➤ `pygame.Surface.set_at()`:
https://pyga.me/docs/ref/surface.html#pygame.Surface.set_at

➤ `pygame.Window.size`:
<https://pyga.me/docs/ref/window.html#pygame.Window.size>

2.2.4 Homework

Please have a look at <https://pyga.me/docs/ref/draw.html> and then try to solve the following small exercises:

1. Program the following: Randomly choose a point and a radius. Using these values, draw a circle with a random, semi-transparent color. As an additional challenge, make sure that the circle may touch the edge of the window at most, but must not go beyond it.
2. Create a window with a color gradient from blue in the upper-left corner to red in the lower-left corner. Then draw two white filled circles with the same radius. One circle should be created using `texttdraw.circle()` and the other using `draw.aacircle()`. Compare the results.
3. Draw 10 random lines in a window using `draw.aaline()`. Each line should start at the edge of the window and end at the edge. Then apply `draw.flood_fill()` to the center of the window and observe the effect.
4. Try to draw the Moonlander like in figure 2.16 using only the functions in `pygame.draw`. Of course, you may also choose any other non-trivial object instead.

`aacircle()`

`aaline()`

`flood_fill()`

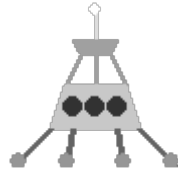


Figure 2.16: Drawing a Moonlander

2.3 Load and Blit Bitmaps

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/03%20Bitmaps

2.3.1 Introduction

config.py

In Python, it is common practice to move program settings, global variables, and similar configuration data into a file named `config.py`.

Listing 2.27: Load and blit bitmaps: config.py (1)

```
1 WINDOW_WIDTH = 600
2 WINDOW_HEIGHT = 400
3 FPS = 60
```

The file is then imported and usually given a shorter name, so its contents can be used across the whole project.

Listing 2.28: Load and blit bitmaps, Version 1.0

```
3 import config as cfg
4
5
6 def main():
7     pygame.init()
8     window = pygame.Window(
9         size=(cfg.WINDOW_WIDTH, cfg.WINDOW_HEIGHT),
10        title="Load and Draw of Bitmaps",
11        position=(10, 50))
12    screen = window.get_surface()
13    clock = pygame.time.Clock()
14
15    defender_image = pygame.image.load("images/defender01.png") # Load bitmap
16    enemy_image = pygame.image.load("images/alienbig0101.png")
17
18    running = True
19    while running:
20        for event in pygame.event.get():
21            if event.type == pygame.QUIT:
22                running = False
23
24        screen.fill("white")
25        screen.blit(enemy_image, (10, 10)) # Draw bitmap
26        screen.blit(defender_image, (10, 80))
27        window.flip()
```

```

28         clock.tick(cfg.FPS)
29
30     pygame.quit()

```

In source code 2.28, two bitmaps – in this case two [Portable Network Graphics \(PNG\)](#) files – are loaded and displayed on the screen.

load()

Loading is done using the function `pygame.image.load()`. In line 15f., the bitmaps – also called [sprites](#) – are loaded and converted into a `Surface` object. In line 25 the two bitmaps are then printed onto the `screen` surface without any further processing using `pygame.Surface.blit()`. The first parameter of `blit()` is the `Surface` object that is to be drawn, followed by the position. Here, the horizontal (x) coordinate is specified first, and then the vertical (y) coordinate. You can *admire* the result in figure 2.17.

blit()

The coordinate system's origin

Unlike in school mathematics, the origin is not at the lower left, but at the upper left.

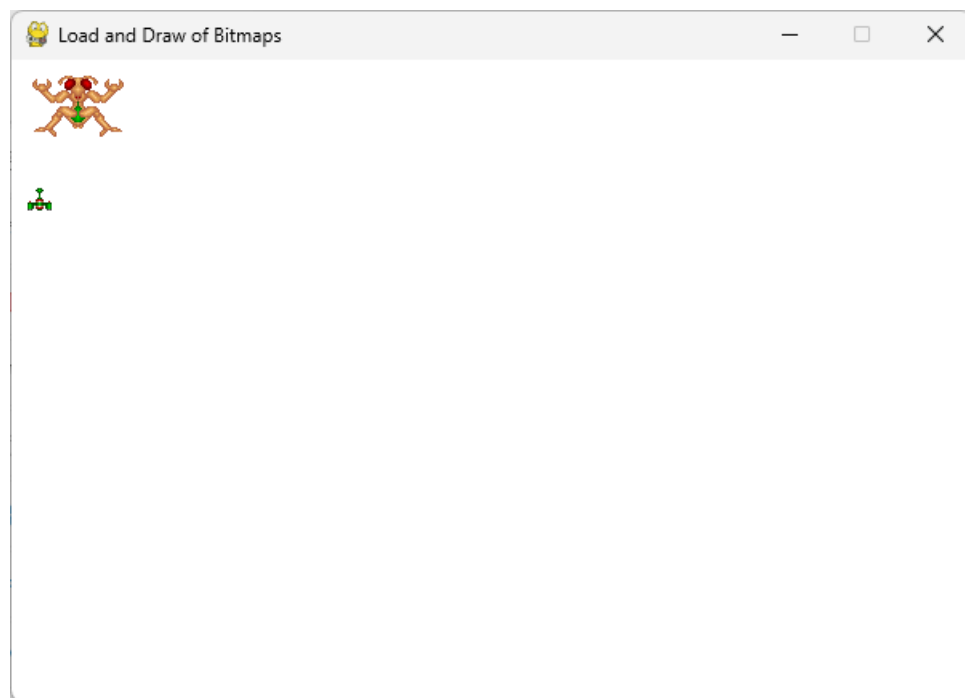


Figure 2.17: Load and blit bitmaps, Version 1.0

We now want to adapt the bitmaps a bit to better suit our needs. First, the documentation recommends converting the bitmap into a format that is easier for Pygame to process after loading. In addition, I want to adjust the size ratios of the two bitmaps, because the enemy appears too large compared to the defender.

Listing 2.29: Load and blit bitmaps, Version 1.1

```

15 defender_image = pygame.image.load("images/defender01.png").convert() # to display format
16 defender_image = pygame.transform.scale(defender_image, (30, 30))      # Scale surface
17
18 enemy_image = pygame.image.load("images/alienbig0101.png").convert()
19 enemy_image = pygame.transform.scale(enemy_image, (50, 45))

```

The function `pygame.Surface.load()` returned a `Surface` object. The `Surface` class now provides a method that performs the desired conversion: `pygame.Surface.convert()`. As an example, please refer to line 15.

Resizing is done using `pygame.transform.scale()`. In line 16, the image is scaled to the specified (*width,height*) in the unit of pixels. The result shown in figure 2.18 does not quite meet my expectations.

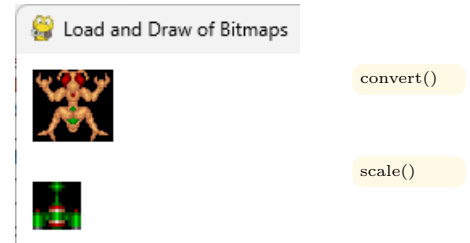


Fig. 2.18: Sizes OK

I do like the size ratios now, but why does a black background suddenly appear? The reason is that the conversion using `convert()` caused the transparency information to be lost. Transparency controls how *see-through* a pixel is. This is achieved by storing not only the three RGB values for each pixel, but also an opacity value. This additional piece of information is called the *alpha channel*.

I now have two options to make this transparency available again:

Transparency of loaded bitmaps

- 👉 `pygame.Surface.convert_alpha()`: Put very simply, the alpha channel is preserved during the conversion. If possible, this should be your method of choice.
- 👉 `pygame.Surface.set_colorkey()`: Here, you pass the color that Pygame should skip when drawing onto the target surface. This can lead to two disadvantages. First, transparency levels between fully visible and fully invisible cannot be represented. It would therefore not be possible to make a pixel *semi-transparent*. Second, parts of the figure that have the same color as the background will also appear transparent. If our alien had a black eye in the middle, it would disappear and the alien would have a hole in the center.

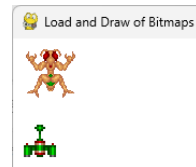
Listing 2.30: Load and blit bitmaps, Version 1.2

```

15 defender_image = pygame.image.load("images/defender01.png").convert_alpha() #
16 defender_image = pygame.transform.scale(defender_image, (30, 30))
17
18 enemy_image = pygame.image.load("images/alienbig0101.png").convert()
19 enemy_image.set_colorkey("black")      # Set colorkey for transparency
20 enemy_image = pygame.transform.scale(enemy_image, (50, 45))

```

In source code 2.30 on the preceding page, I tried out both variants, and you can see the result in figure 2.19. Now both bitmaps are visible without a black background; the white background shows through again.

Fig. 2.19: α OK

What I still do not like is the position and the number of attackers. I want to place the defender centered at the bottom and the attackers along the top edge of the screen, arranged so that they are horizontally [equidistant](#). There are two ways to do this: I can specify a minimum spacing and compute the number of attackers, or I can specify the maximum number of attackers and compute the spacing. Which approach I choose depends on my game logic; in most cases, the number is fixed in advance.

equidistant

Listing 2.31: Load and blit bitmaps: config.py (2)

```
1 WINDOW_WIDTH = 600
2 WINDOW_HEIGHT = 400
3 FPS = 60
4 ALIENS_NOF = 7
```

Listing 2.32: Bitmap: positioning, Version 1.4

```
15 defender_image = pygame.image.load("images/defender01.png").convert_alpha()
16 defender_image = pygame.transform.scale(defender_image, (30, 30))
17 defender_pos_left = (cfg.WINDOW_WIDTH - 30) // 2 # Left coordinate
18 defender_pos_top = cfg.WINDOW_HEIGHT - 30 - 5 # Top coordinate
19 defender_pos = (defender_pos_left, defender_pos_top) # Create a 2-tuple
20
21 alien_image = pygame.image.load("images/alienbig0101.png").convert_alpha()
22 alien_image = pygame.transform.scale(alien_image, (50, 45))
23 space_for.aliens = cfg.ALIENS_NOF * 50 # Space occupied by aliens
24 space_available = cfg.WINDOW_WIDTH - space_for.aliens # Remaining available space
25 space_nof = cfg.ALIENS_NOF + 1 # Number of gaps
26 space_between.aliens = space_available // space_nof # Space per gap
27
28 running = True
29 while running:
30     for event in pygame.event.get():
31         if event.type == pygame.QUIT:
32             running = False
33
34     screen.fill("white")
35     alien_top = 10 # Distance from top
36     for i in range(cfg.ALIENS_NOF): # Compute and draw positions
37         alien_left = (i + 1) * space_between.aliens + i * 50
38         alien_pos = (alien_left, alien_top)
39         screen.blit(alien_image, alien_pos)
40     screen.blit(defender_image, defender_pos) # Draw defender at its position
41     window.flip()
42     clock.tick(cfg.FPS)
```

In source code 2.32, the requirements above have been implemented. Let us take a closer look at the individual aspects.

The defender should be positioned centered at the bottom. We remember that the function `blit()` also expects the coordinates of the upper-left corner. So this position

has to be calculated first. For the sake of clarity – in a normal source code I would not write the calculation in such a fine-grained way – I calculate the coordinates separately here.

The top edge is fairly easy to determine. If we set `defender_top` to the full height of the screen, `cfg.WINDOWS_HEIGHT`, we would not see the defender because it would stick out below the screen completely. So by how many pixels do we need to move the top edge upward? Exactly by the height of the spaceship, 30 px:

```
18     defender_pos_top = cfg.WINDOWS_HEIGHT - 30
```

However, I do not like how the defender looks glued to the edge this way. So I give it an additional 5 px of space, making it look more as if it were floating in space:

```
18     defender_pos_top = cfg.WINDOWS_HEIGHT - 30 - 5
```

In line 17, the distance of the left edge of the bitmap from the edge of the playfield is calculated. Using

```
17     defender_pos_left = cfg.WINDOWS_WIDTH // 2
```

we would calculate the horizontal center of the screen. However, we cannot use this value, because it would place the left edge of the defender at the horizontal center – that is, too far to the right (see figure 2.20).

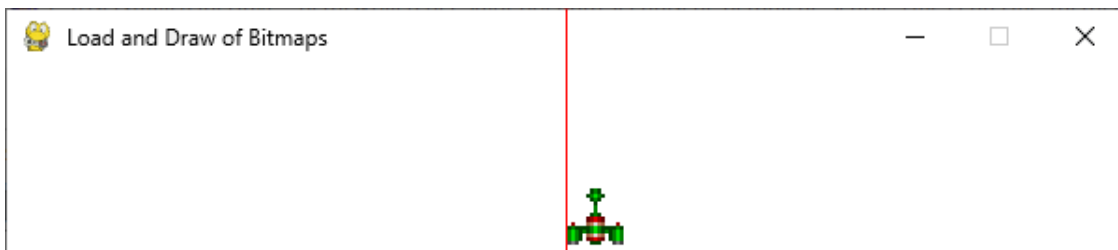


Figure 2.20: Bitmaps positioning defender

However, we can determine exactly how many pixels we have shifted too far to the right and then subtract this value: it is exactly half of the width of the defender (here 30 px):

```
17     defender_pos_left = cfg.WINDOWS_WIDTH // 2 - 30 // 2
```

With the help of a little fraction arithmetic, the expression can be simplified:

```
17     defender_pos_left = (cfg.WINDOWS_WIDTH - 30) // 2
```

Now we move on to the aliens. In the first approach, we want to display them one after another at the top without any overlap. The top edge `alien_top` can be set to a constant value with a pleasant distance of 10 px from the upper edge:

```
35     alien_top = 10
```

The left position `alien_left` has to be determined individually for each alien. Since they are placed directly next to each other at first, the left edge of one alien is exactly one alien width away from the left edge of the next one. So if I am at the 0th alien, its horizontal coordinate is directly at the left edge of the screen. For the 1st alien it is exactly 1×50 px, for the 2nd exactly 2×50 px, and so on, since the alien is 50 px wide. Written as a `for`-loop, it looks like this:

```

36     for i in range(cfg.ALIENS_NOF):
37         alien_left = i * 50
38         alien_pos = (alien_left, alien_top)
39         screen.blit(alien_image, alien_pos)

```

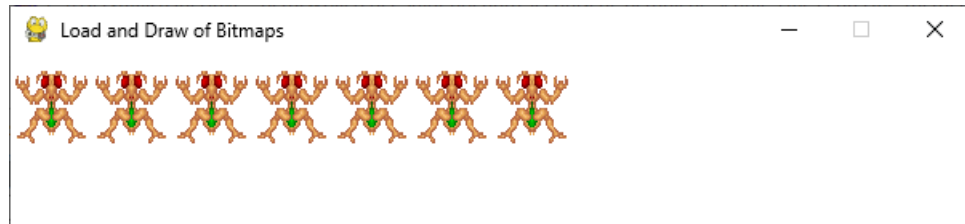


Figure 2.21: Bitmaps positioning alien, Version 1

The entire remaining space after the last alien can now be distributed before, between, and after the aliens in such a way that the spacing is the same between the aliens, between the leftmost alien and the left edge of the screen, and between the rightmost alien and the right edge of the screen. So how many gaps are there? First of all, the two outer gaps on the far left and far right – that makes 2:

```

25     space_nof = 2

```

Then there are the gaps between the aliens. This is always one less than the number of aliens (count it to check!):

```

25     space_nof = cfg.ALIENS_NOF - 1 + 2

```

thus:

```

25     space_nof = cfg.ALIENS_NOF + 1

```

Now the available space `space_available` behind the aliens still has to be calculated. I do this by first calculating the space occupied by the aliens, `space_for_alien`

```

28     space_for_alien = cfg.ALIENS_NOF * 50

```

and subtract this value from the screen width.

```

24     space_available = cfg.WINDOWS_WIDTH - space_for_alien

```

So I now have the available space stored in `space_available` and the number of gaps that need to be filled stored in `space_nof`. If I now want to determine the width of the gaps, `space_between_alien`, I simply have to divide these two values:

```

26     space_between.aliens = space_available // space_nof

```

Now we only need to adjust the calculation of `alien_left`. First, we shift the starting position by one such gap (see figure 2.22):

```

36     for i in range(cfg.ALIENS_NOF):
37         alien_left = space_between.aliens + i * 50
38         alien_pos = (alien_left, alien_top)
39         screen.blit(alien_image, alien_pos)

```

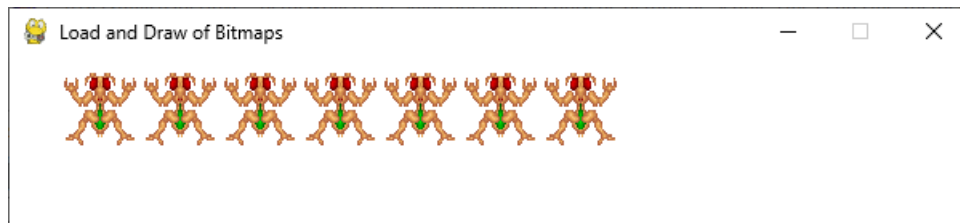


Figure 2.22: Bitmaps positioning alien, Version 2

Now the distance from one left edge to the next, which previously consisted only of the width of the alien, must be extended by the spacing `space_between.aliens`:

```

36     for i in range(cfg.ALIENS_NOF):
37         alien_left = (i + 1) * space_between.aliens + i * 50
38         alien_pos = (alien_left, alien_top)
39         screen.blit(alien_image, alien_pos)

```

And just like that, everything fits (see figure 2.23).

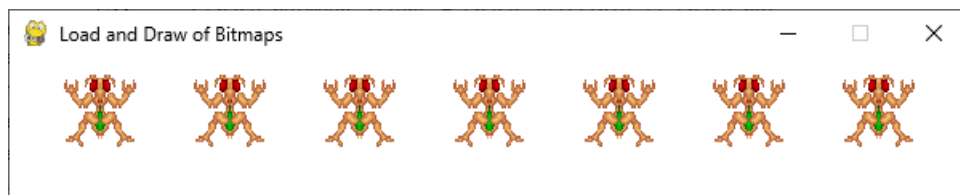


Figure 2.23: Bitmaps positioning alien, Version 3

Why am I explaining this in so much detail?

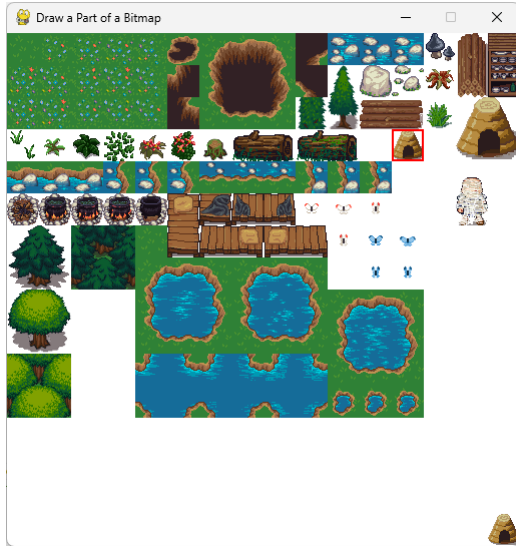
Not because it's particularly difficult, but because when you're just starting out with programming, it's easy to feel overwhelmed when trying to understand other people's source code. Where do numbers like `-1` even come from? This step-by-step approach is meant to reassure you. Complex solutions are often built from many small, simple pieces put together.

2.3.2 More Input

2.3.2.1 Blitting Parts of a Bitmap

Very often, only parts of a bitmap need to be blitted. For this purpose, the function `Surface.blit()` provides the parameter `area`.

`area`



`clamp()`

Fig. 2.24: Tiles to build a forest

As an example, I use a bitmap that consists of tiles of size 32 px × 32 px. From these tiles, I could build a forest and lake landscape for a game. The logic of this small application is that, using the arrow keys, I jump 32 px to the right, left, up, or down, and in this way move from tile to tile. The tile currently selected is marked with a red rectangle (line 34) and drawn as a sub-image into the lower right corner of the window – in this example, a tent or a small hut.

Using `clamp()` I make sure that I cannot wander outside the image.

In line 35, this approach is applied as shown in figure 2.24. The first parameter of `blit()` is the bitmap – here referenced as `image` – that is

to be drawn. The second parameter (512-32, 512-32) specifies the position within the window where the image should be drawn. How do we arrive at these values? The entire image has a width of 512 px and a height of 512 px. Each tile has a size of 32 px × 32 px, and therefore the upper-left corner of the tile must be positioned 32 px away from the edges. The third parameter – the `area` – is a 4-tuple. Its values represent *left*, *top*, *width*, and *height*. The variables `x` and `y` are determined by movement using the arrow keys, which is a preview of later sections of this book (see section 2.4 on page 59). The width and height of the tiles are fixed at 32 px.

Listing 2.33: Blit a part of a bitmap

```

4 def main():
5     pygame.init()
6     window = pygame.Window(size=(512, 512), title="Draw a Part of a Bitmap")
7     screen = window.get_surface()
8     clock = pygame.time.Clock()
9
10    image = pygame.image.load("images/forest_tiles.png")
11    x, y = 0, 0
12
13    running = True
14    while running:
15        for event in pygame.event.get():
16            if event.type == pygame.QUIT:
```



```

17     running = False
18     if event.type == pygame.KEYDOWN:
19         if event.key == pygame.K_ESCAPE:
20             running = False
21         elif event.key == pygame.K_RIGHT:
22             x += 32
23         elif event.key == pygame.K_LEFT:
24             x -= 32
25         if event.key == pygame.K_DOWN:
26             y += 32
27         if event.key == pygame.K_UP:
28             y -= 32
29     x = pygame.math.clamp(x, 0, 512-32)
30     y = pygame.math.clamp(y, 0, 512-32)
31
32     screen.fill("white")
33     screen.blit(image, (0, 0))
34     pygame.draw.rect(screen, "red", (x, y, 32, 32), 2) # Draw rectangle around the part
35     screen.blit(image, (512-32, 512-32), (x, y, 32, 32)) # Blit a part of the image
36     window.flip()
37     clock.tick(60)
38
39     pygame.quit()

```

2.3.2.2 Message Box

A message box is a simple way to communicate an [information](#), a [warning](#), or an [error](#) to the player. Its appearance can only be customized to a very limited extent and it usually does not fit into the visual design concept of a game. For this reason, message boxes are almost never used for in-game interactions. However, they are well suited for use during installation or configuration, or when real errors occur that require quick and clear interaction.

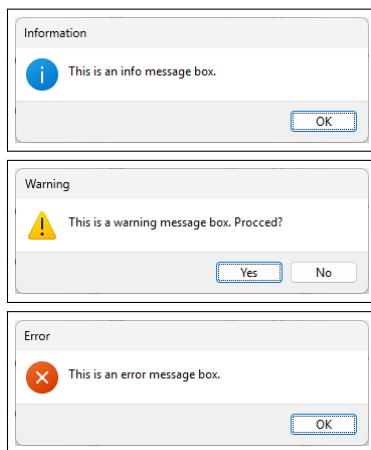


Fig. 2.25: Messageboxes

Here, I have only shown the crucial part of the program in source code [2.34](#) on the following page; the rest is not important. The call is made using `pygame.display.message_box()`.

message-
_box()

In line [20](#), an information message is displayed. The first parameter is the text of the window's title bar. The second parameter is the message text, which can also be much longer and formatted. After that, the type of the message box is specified. There are three – self-explanatory – types available: **info**, **warn**, and **error**. This setting determines the icon that is shown (see figure [2.25](#)). The call for an error message works in exactly the same way, as shown in line [30](#).

A bit more is demonstrated with the warning call starting at line [24](#). First of all, you can see that the named parameter **buttons** is passed. It contains a list of strings. Each string is the label text of one displayed button (see

Button

figure 2.25 on the preceding page). So how do we find out which button was pressed? By using the return value. Internally, the list of button labels is numbered, and each button is assigned an index. The index value of the pressed button is then returned and – as in this example with `a` – stored in a variable. Which button has which index, and how to proceed afterwards, is something you need to keep in mind. In this example, the value is simply printed to demonstrate the effect.

Listing 2.34: Types of Messageboxes

```

19         elif event.key == pygame.K_1:
20             pygame.display.message_box("Information", #
21                                     "This is an info message box.",
22                                     "info")
23
24         elif event.key == pygame.K_2:
25             a = pygame.display.message_box("Warning", #
26                                     "This is a warning message box. Procced?",
27                                     "warn",
28                                     buttons=["Yes", "No"])
29
30             print("User selected:", a)
31         elif event.key == pygame.K_3:
32             pygame.display.message_box("Error", #
33                                     "This is an error message box.",
34                                     "error")

```

2.3.2.3 Creating Bitmaps

Bitmaps do not necessarily have to be loaded from disk using `pygame.image.load()`. It is also possible to create a bitmap at runtime using `pygame.Surface()`. Let us take another look at section 2.2.2.2 on page 36. In that section, an animated landscape was created using drawing primitives. These primitives are drawn in every frame – for example 60 times per second. In the end, this is an enormous waste of computing time.

Creating bitmaps during runtime

A much more efficient approach is to draw the graphics once onto a **Surface** object and then only blit this bitmap to the correct position. Blitting **Surface** objects is much faster than drawing the shapes again and again.

Let us look at this starting with the **Meadow** class. In the constructor, a **Surface** object is created in line 11. For that, we need a width and a height. Both values are computed beforehand and stored in the local variable `widthheight`; this does not need to be a class attribute anymore, because the information is only required to create the bitmap and is not needed later.

After that, the surface is filled completely with a green color. Now we have a finished meadow graphic, and in `draw()` it only needs to be blitted.

Listing 2.35: Creating Bitmaps: Class Meadow

```

6 class Meadow:
7     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
8         self.window = window
9         self.pos = 0, horizon
10        widthheight = self.window.size[0], self.window.size[1] - horizon
11        self.surface = pygame.Surface(widthheight) # Meadow surface
12        self.surface.fill((50, 180, 50))
13
14    def draw(self) -> None:
15        screen = self.window.get_surface()
16        screen.blit(self.surface, self.pos)

```

The `Sky` class follows a similar approach. In the constructor, a `Surface` object is created and filled with a shade of blue, and in `draw()` the bitmap is simply blitted. Only `update()` remains computationally more expensive, since the color of the sky changes depending on the position of the sun.

Listing 2.36: Creating Bitmaps: Class Sky

```

18 class Sky:
19     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
20         self.window = window
21         self.pos = 0, 0
22         self.color = (100, 150, 255)
23         self.surface = pygame.Surface((self.window.size[0], horizon)) # Sky surface
24         self.surface.fill(self.color)
25
26     def update(self, progress: float) -> None:
27         brightness = max(0, min(1, math.sin(progress * math.pi)))
28         blue = int(80 + brightness * 120)
29         self.color = (100, blue, 235)
30         self.surface.fill(self.color)
31
32     def draw(self) -> None:
33         screen = self.window.get_surface()
34         screen.blit(self.surface, self.pos)

```

The `Tree` class is more similar to `Meadow`. In line 40, the bitmap is created and its contents are drawn using drawing primitives. However, the `Surface` object is created with the additional parameter `pygame.SRCALPHA`. This parameter ensures that the unpainted background of the `Surface` object remains transparent. Otherwise, a black background with the size of the `Surface` object would appear around the tree; with transparency enabled, the sky, sun, and meadow can be seen through it.

SRCALPHA

Listing 2.37: Creating Bitmaps: Class Tree

```

36 class Tree:
37     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
38         self.window = window
39         self.pos = (65, horizon - 80)
40         self.surface = pygame.Surface((90, 120), pygame.SRCALPHA) # Tree surface
41         pygame.draw.rect(self.surface, (120, 80, 40), (35, 60, 20, 60))
42         pygame.draw.circle(self.surface, (20, 140, 20), (45, 35), 35)
43
44     def draw(self) -> None:
45         screen = self.window.get_surface()

```

```
46         screen.blit(self.surface, self.pos)
```

The same approach is used in the House class in line 53.

Listing 2.38: Creating Bitmaps: Class House

```
49 class House:
50     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
51         self.window = window
52         self.pos = (200, horizon - 70)
53         self.surface = pygame.Surface((150, 160), pygame.SRCALPHA)      # House surface
54         pygame.draw.rect(self.surface, (200, 100, 100), ((0,60), (150, 100)))
55         pygame.draw.polygon(
56             self.surface,
57             (150, 50, 50),
58             [(0,60), (75, 0), (150, 60)])
59         )
60         pygame.draw.rect(self.surface, (100, 60, 30), (60, 100, 30, 60))
61
62     def draw(self) -> None:
63         screen = self.window.get_surface()
64         screen.blit(self.surface, self.pos)
```

And, for the sake of completeness, the Sun class as well.

Listing 2.39: Creating Bitmaps: Class Sun

```
67 class Sun:
68     def __init__(self, window: pygame.window.Window, horizon: int) -> None:
69         self.speed = 1
70         radius = 40
71         self.pos = [-radius, 0]
72         self.horizon = horizon
73         self.window = window
74         self.surface = pygame.Surface((radius*2, radius*2), pygame.SRCALPHA)  # Sun surface
75         pygame.draw.circle(self.surface, (255, 220, 0), (radius, radius), radius)
76
77     def update(self) -> float:
78         self.pos[0] += self.speed
79         progress = self.pos[0] / self.window.size[0]  # 0.0 -> 1.0
80         self.pos[1] = round(self.horizon * (1 - math.sin(progress * math.pi)))
81         return progress
82
83     def draw(self) -> None:
84         screen = self.window.get_surface()
85         screen.blit(self.surface, (self.pos[0], self.pos[1]))
```

2.3.3 What was new?

- 👉 The position values are needed when drawing on the screen. Later, we will see that we also need these position values for other questions, such as [collision detection](#). The position always refers to the upper-left corner of the bitmap, or in other words: *The coordinate system has its origin in the upper left, not in the lower left.*
- 👉 We often have to perform basic geometry calculations, and it is best to do them step by step.
- 👉 For such geometry calculations, the following information is needed: the position of the bitmap, its width, and its height. So far, we have treated width and height as constants, but that is not a good long-term solution.



The following Pygame elements were introduced



- 👉 `pygame.display.Info()` :
<https://pyga.me/docs/ref/display.html#pygame.display.Info>
- 👉 `pygame.display.message_box()` :
https://pyga.me/docs/ref/display.html#pygame.display.message_box
- 👉 `pygame.image` :
<https://pyga.me/docs/ref/image.html>
- 👉 `pygame.image.load()` :
<https://pyga.me/docs/ref/image.html#pygame.image.load>
- 👉 `pygame.Surface()`:
<https://pyga.me/docs/ref/surface.html>
- 👉 `pygame.Surface.blit()`:
<https://pyga.me/docs/ref/surface.html#pygame.Surface.blit>
- 👉 `pygame.Surface.convert()`:
<https://pyga.me/docs/ref/surface.html#pygame.Surface.convert>
- 👉 `pygame.Surface.convert_alpha()`:
https://pyga.me/docs/ref/surface.html#pygame.Surface.convert_alpha
- 👉 `pygame.Surface.set_colorkey()`:
https://pyga.me/docs/ref/surface.html#pygame.Surface.set_colorkey

🔗 `pygame.SRCALPHA:`
<https://pyga.me/docs/ref/surface.html>

🔗 `pygame.transform.scale():`
<https://pyga.me/docs/ref/transform.html#pygame.transform.scale>

2.3.4 Homework

1. Look for freely available sources of game graphics (sprites). If you know that you want to work with this more intensively, also look for sources that are behind a paywall.
2. Blit your own graphics to sensible positions within your window.
3. Try to build a realistic background for a simple game using graphics. If needed, make use of the option to blit sub-images from a larger bitmap.

2.4 Moving Bitmaps

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/04%20Moving

2.4.1 Class Rect/FRect

In the summary of the previous chapter, we noted that when displaying bitmaps we need the *upper-left corner* as the position value, and we need the *height* and *width*, for example for distance calculations. These values can be conveniently encoded in a rectangle. For this purpose, Pygame provides the classes `pygame.rect.Rect` (integers only) and `pygame.rect.FRect` (floating-point numbers). In figure 2.26, you can find what I consider to be the most important attributes of this class.

Rect

FRect

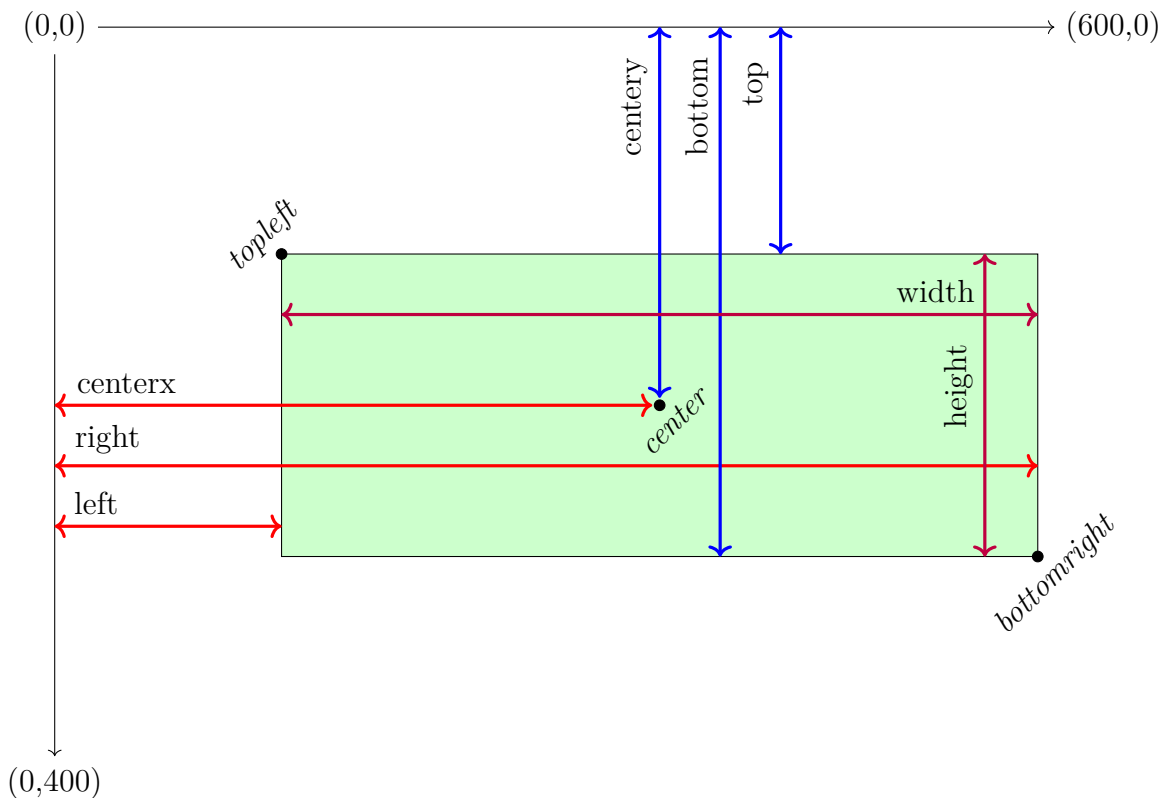


Figure 2.26: Elements of a Rect-/FRect-Object

In the figure, line segments are shown in normal font, while points are shown in *italic font*. The segments are one-dimensional, and the points are two-dimensional (x, y). The coordinate x represents the horizontal distance from the origin of the coordinate system or the upper-left corner of the rectangle, and y represents the vertical distance. The meaning of the individual labels should be self-explanatory.

The nice advantage is that all these values are computed from each other. For example, if I set `opleft = (10, 10)` and `width, height = 30, 40`, all other values are calculated automatically. I no longer need to compute the right edge manually using `left + width`; instead, I can directly use `right`.

It is also often useful to work with the center position `center` or the corresponding coordinates `centerx` and `centery`. If I change the center to `center = (100, 10)`, all other values are updated accordingly and do not need to be recalculated by me – very convenient.

Let us take a look at a reduced version of the last source code. In source code 2.41, the `Rect` class is already being used; i.e., in line 3 the window dimensions are stored in a `Rect` object.

Listing 2.40: Moving Bitmaps: `config.py`

```

1 import pygame
2
3 WINDOW = pygame.Rect((0, 0), (600, 100))          # Rect object
4 FPS = 60

```

As a result, the screen information can be accessed conveniently and without performing manual calculations in line 9, line 18, and line 19.

Listing 2.41: Moving Bitmaps, Version 1.0

```

6 def main():
7     pygame.init()
8     window = pygame.Window(
9         size=config.WINDOW.size,                # Accessing a Rect attribute
10        title="Movement",
11        position=(10, 50))
12    screen = window.get_surface()
13    clock = pygame.time.Clock()
14
15    defender_image = pygame.image.load("images/defender01.png").convert_alpha()
16    defender_image = pygame.transform.scale(defender_image, (30, 30))
17    defender_rect = defender_image.get_rect()      # Rect-object
18    defender_rect.centerx = config.WINDOW.centerx # Not only using left
19    defender_rect.bottom = config.WINDOW.height - 5 # Not only using top
20
21    running = True
22    while running:
23        # Events
24        for event in pygame.event.get():
25            if event.type == pygame.QUIT:
26                running = False
27
28        # Update
29

```

```

30     # Draw
31     screen.fill("white")
32     screen.blit(defender_image, defender_rect)      # blit can also take a Rect
33     window.flip()
34     clock.tick(cfg.FPS)
35
36     pygame.quit()

```

For `Surface` objects, we can conveniently create a `Rect` object using `pygame.Surface.get_rect()` (line 17). Positioning can now be handled much more easily via the attributes. For example, the center no longer needs to be part of a calculation; instead, I can directly set the horizontal center to half the window width (line 18). Likewise, the vertical coordinate no longer has to be considered from the top edge; instead, I can specify the distance of the bottom edge from the screen edge in a much more intuitive way (line 19). And as a final bonus, the `Rect` object can even be passed directly as a parameter to the `blit()` function (line 32).

get_rect()

blit()



Figure 2.27: Moving Bitmaps, Version 1.0

The result is unspectacular (see figure 2.27) and has nothing to do with movement yet.

2.4.2 Introduction

Movement in games is animated by changing positions. If the spaceship is supposed to move to the right, the horizontal coordinate of the ship therefore has to increase. Which horizontal coordinate you use for this – `left`, `right`, or `centerx` – can be chosen depending on your game logic. In our example, this does not matter, so I will use `left`.

```

26     defender_rect.left += 1

```

This small addition alone now causes our spaceship to move to the right.

The `+1` encodes two pieces of information:

- 👉 **Direction:** Here, the sign is `+`. This increases the value of `left` in each loop iteration; as a result, the left edge of the graphic moves to the right. If you wanted to move to the left, the sign would have to be `-`. In that case, the horizontal coordinate would become smaller and approach 0. Completely

analogously, the sign also controls the direction in the vertical axis. A + moves the graphic downward, and a - moves it upward. Try it out!

 **Speed:** The 1 specifies by how much the value of `left` changes. The larger this value is, the larger the jumps between frames; the movement appears faster.

Listing 2.42: Moving Bitmaps, Version 1.2

```

17     defender_speed = 2          #
18     defender_direction_h = +1  #
19
20     running = True
21     while running:
22         # Events
23         for event in pygame.event.get():
24             if event.type == pygame.QUIT:
25                 running = False
26
27         # Update
28         defender_rect.left += defender_direction_h * defender_speed # more flexible

```

These two pieces of information are now used in source code 2.42 to make movement much more flexible. In line 17, the speed is now represented by the variable `defender_speed`. This would allow us to change the speed dynamically during the game, for example when accelerating by firing rocket thrusters.

The direction is also stored in a variable in line 18: `defender_direction`. At the moment it is positive, but we will soon see that we can also use it for changing direction. Both values can now be used in line 28 to calculate the new horizontal position.

If you run the program, the defender will leave the screen after a while and disappear beyond the right edge, never to be seen again. Let us now use our rectangle for a first simple collision check. I want the spaceship to *bounce* off the edges and reverse its direction.

Listing 2.43: Move Bitmaps, Version 1.3

```

27         # Update
28         defender_rect.left += defender_direction_h * defender_speed
29         if defender_rect.right >= cfg.WINDOW.right:      # Right edge reached
30             defender_direction_h *= -1                  # Change direction
31         elif defender_rect.left <= cfg.WINDOW.left:      # Left edge reached
32             defender_direction_h *= -1

```

I hope you can recognize the idea behind the code. After calculating the new horizontal position, line 29 checks whether the new right edge of the bitmap has reached or exceeded the right edge of the screen. If this is the case, the sign of the direction variable is simply reversed! The same logic works analogously when the left edge of the screen is reached.

Try combining this with vertical movement as well.

Direction
change

There is still one problem: In line 28, the new position is assigned to the `Rect` object even though it may already extend beyond the edge. With a speed of 1 or 2, this may not be very noticeable, but if we set the speed to the width of the spaceship, the problem becomes obvious (temporarily set `cfg.FPS = 5` so that you can see it clearly). The spaceship ends up leaving the screen halfway.

So we should check the new position first and only then assign it to the `Rect` object `defender_rect`. In this context, let us introduce a very useful method of the `Rect` class: `pygame.rect.Rect.move()`.

move()

Listing 2.44: Move Bitmaps, Version 1.4

```

27     # Update
28     newpos = defender_rect.move(defender_direction_h * defender_speed, 0) # New position
29     if newpos.right >= cfg.WINDOW.right:
30         defender_direction_h *= -1
31         newpos.right = cfg.WINDOW.right      # Align to right edge
32     elif newpos.left <= cfg.WINDOW.left:
33         defender_direction_h *= -1
34         newpos.left = cfg.WINDOW.left       # Align to left edge
35     defender_rect = newpos                  # Accept new position

```

The new function appears for the first time in line 28. It takes two parameters. The first one specifies the horizontal displacement, and the second one specifies the vertical displacement. Since we do not want to change the vertical position, this parameter is constant 0 in our example. The function returns a new `Rect` object containing the updated position values. We store this temporarily in `newpos`.

The subsequent collision checks are then performed using the `newpos` rectangle. If a collision occurs, the direction values are changed as before. Likewise, the left edge of the bitmap is aligned with the left edge of the screen, and the right edge is aligned with the right edge. After that, `newpos` becomes the new rectangle for the defender (line 35).



Figure 2.28: The Defender moves and bounces

2.4.3 More Input

2.4.3.1 Normalizing Speeds (*delta time*)

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/04%20Moving/dt

At the moment, the movement does not depend only on `defender_speed`, but also on the frame rate `cfg.FPS`. To illustrate this dependency, I modified the previous source code for a small experiment (see source code 2.45 and source code 2.46 on the facing page).

Listing 2.45: Movement without normalization: `config.py`

```

1 import pygame
2
3 WINDOW = pygame.Rect((0, 0), (120, 650))
4 FPS = 60 # 10 30 60 120 240 300 600
5 LIMIT = 500

```

In line 4, you can see the different frame rates with which the experiment was carried out. In the line above, the window dimensions are set so that the window is tall and narrow, and in the line below the absolute number of milliseconds is specified during which the spaceship moves upward.

Line 20 stores the time at which the spaceship's ascent began. For this purpose, the function `pygame.time.get_ticks()` returns the number of milliseconds since the call to `pygame.init()`; for example, 5 ms.

`get_ticks()`

Inside the main loop, the spaceship now moves upward by a certain number of pixels per frame. The new position is calculated by adding the product of direction and speed to the `top` coordinate of the old position (line 31) – so there is nothing new at this point.

After a fixed time interval (`cfg.LIMIT`, here 500 ms), the direction stored in `defender_direction` is set to 0, causing the movement to stop. To do this, line 23 checks whether the current number of milliseconds since the start of the program is greater than `start_time` plus `cfg.LIMIT`.

In numerical terms: during the first loop iteration (frame 1), the condition would be, for example,

Is 17ms greater than 5ms + 500ms?

The answer is *No*, so the spaceship continues to move upward. At frame 61, the condition would be

Is 508ms greater than 5ms + 500ms?

Now the answer is *Yes*, and the direction variable is therefore set to 0, stopping the movement.

Listing 2.46: Movement without normalization

```

6 def main():
7     pygame.init()
8     window = pygame.Window(size=cfg.WINDOW.size, title="Movement", position=(10, 50))
9     screen = window.get_surface()
10    clock = pygame.time.Clock()
11
12    defender_image = pygame.image.load("images/defender01.png").convert_alpha()
13    defender_image = pygame.transform.scale(defender_image, (30, 30))
14    defender_rect = defender_image.get_rect()
15    defender_rect.centerx = cfg.WINDOW.centerx
16    defender_rect.bottom = cfg.WINDOW.bottom - 5
17    defender_speed = 2
18    defender_direction_v = -1
19
20    start_time = pygame.time.get_ticks()                # Movement starts
21    running = True
22    while running:
23        if pygame.time.get_ticks() > start_time + cfg.LIMIT:    # Ready?
24            defender_speed = 0
25        # Events
26        for event in pygame.event.get():
27            if event.type == pygame.QUIT:
28                running = False
29
30        # Update
31        defender_rect.top += defender_direction_v * defender_speed # New height
32        if defender_rect.bottom >= cfg.WINDOW.bottom:
33            defender_direction_v *= -1
34        elif defender_rect.top <= 0:
35            defender_direction_v *= -1
36
37        # Draw
38        screen.fill("white")
39        screen.blit(defender_image, defender_rect)
40        window.flip()
41        clock.tick(cfg.FPS)
42    print(f"top={defender_rect.top}")
43
44    pygame.quit()

```

In figure 2.29 on the next page, you can see screenshots of the distances the spaceship has traveled after half a second. In all experiments, the speed `defender_speed` remained the same – namely 2. Only the frame rate was increased.

How do these different heights come about? After all, only `defender_speed` is supposed to define the speed. The relationship should become clear in table 2.3 on the following page. The first column shows the speed of an object; in our example, this is the variable `defender_speed`. This value specifies how many pixels per frame the object is moved; this value does not change. The second column shows the frame rate, that is, the number of frames per second. In our example, this value is defined in `cfg.FPS`. The duration of the movement is shown in the third column. We have a duration of 500 ms, that is 0.5 s. In our example, this value is stored in `cfg.LIMIT` and is also the same for all experiments.

The last column shows the calculated distance in pixels that the moving object has traveled. Now the relationship becomes clear: because we repeat the main program loop a different number of times depending on the frame rate, different distances are covered within the same amount of time.

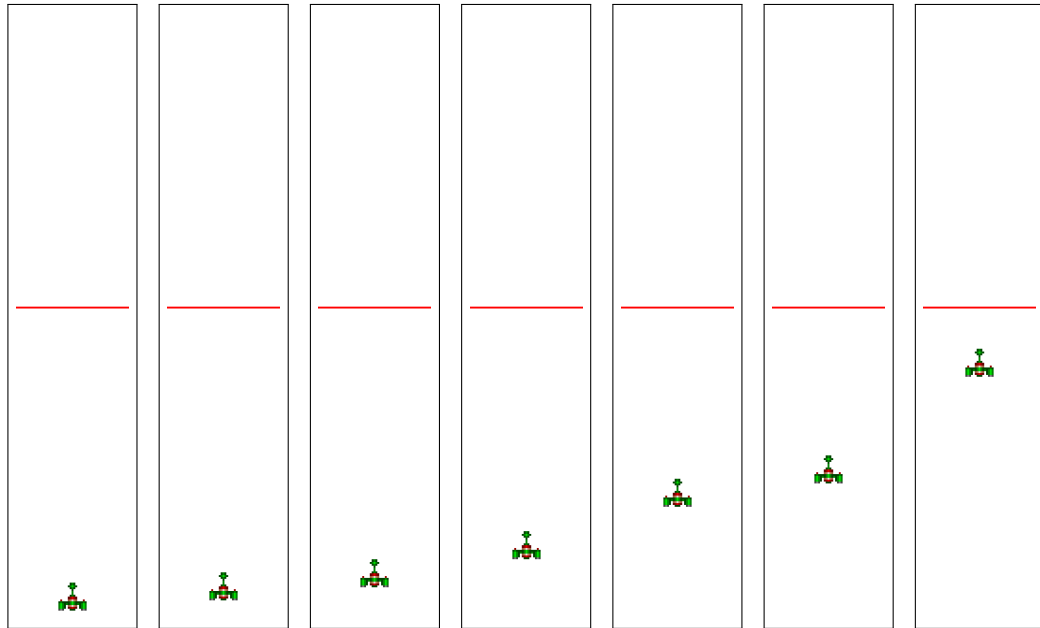


Figure 2.29: Non-normalized movement with identical speed but different frame rates (from left to right: 10, 30, 60, 120, 240, 300, 600)

Table 2.3: Distance without normalized movement

speed ($\frac{px}{f}$) * FPS ($\frac{f}{s}$) * time (s) = distance (px)			
2 *	10 *	0.5 =	10
2 *	30 *	0.5 =	30
2 *	60 *	0.5 =	60
2 *	120 *	0.5 =	120
2 *	240 *	0.5 =	240
2 *	300 *	0.5 =	300

What we therefore need is a mechanism that removes the influence of the frame rate again. This factor has to be constructed in such a way that, when multiplied by the frame rate, it always yields 1 as a result. In that case, the frame rate would effectively act like a multiplication by 1 in the overall product and would therefore no longer have any influence.

The obvious approach is to take the inverse of the frame rate, that is $\frac{1}{fps}$. This correction value is called *delta time* (dt). The calculation would then look, for example, as shown

deltatime

in table 2.4. The second and third columns cancel each other out, so that the distance remains the same – independent of the chosen frame rate.

That is exactly what we tried to achieve.

Table 2.4: Distance with normalized movement

speed ($\frac{px}{s}$) * FPS ($\frac{f}{s}$) * dt ($\frac{s}{f}$) * time (s) = distance (px)	
2 * 10 * $\frac{1}{10}$ * 0.5 =	1
2 * 30 * $\frac{1}{30}$ * 0.5 =	1
2 * 60 * $\frac{1}{60}$ * 0.5 =	1
2 * 120 * $\frac{1}{120}$ * 0.5 =	1
2 * 240 * $\frac{1}{240}$ * 0.5 =	1
2 * 300 * $\frac{1}{300}$ * 0.5 =	1

In this context, it becomes apparent that the distance is surprisingly short: only 1 px per second. Please note that the unit of the first column has changed as well. The speed no longer specifies the number of pixels per frame, but the number of pixels per second! We therefore need to choose a different speed value; based on our window size, I decided on 600 px/s. After one second, our spaceship will have reached the top.

In table 2.5, I calculated the expected final position (`.top`) after half a second. In the left half of the table, the traveled distance is calculated. Surprisingly, it always amounts to 300 px. From the window height (`WINDOW.height`), we have to subtract this distance. In addition, we subtract the height of our spaceship (30 px) and the small offset of 5 px, since we did not want to start the spaceship directly at the bottom edge. We therefore expect our spaceship to reach the final position calculated in table 2.5 after half a second.

Table 2.5: Pixel coordinates with normalized speed

speed * FPS * dt * time = distance → <code>WINDOW.height</code> - height - offset = <code>.top</code>	
600 * 10 * $\frac{1}{10}$ * 0.5 = 300 → 650-300 - 30 - 5 =	315
600 * 30 * $\frac{1}{30}$ * 0.5 = 300 → 650-300 - 30 - 5 =	315
600 * 60 * $\frac{1}{60}$ * 0.5 = 300 → 650-300 - 30 - 5 =	315
600 * 120 * $\frac{1}{120}$ * 0.5 = 300 → 650-300 - 30 - 5 =	315
600 * 240 * $\frac{1}{240}$ * 0.5 = 300 → 650-300 - 30 - 5 =	315
600 * 300 * $\frac{1}{300}$ * 0.5 = 300 → 650-300 - 30 - 5 =	315

Although table 2.5 may look complicated, the implementation is surprisingly simple. First, the adjustment in `config.py`. In line 6, the correction factor is defined – as discussed above – as the inverse of the frame rate.

Listing 2.47: Movement with normalization and $dt = 1/fps$: config.py

```

1 import pygame
2
3 WINDOW = pygame.Rect((0, 0), (120, 650))
4 FPS = 60 # 10 30 60 120 240 300 600
5 LIMIT = 500
6 DELTATIME = 1.0 / FPS #

```

The speed is adjusted from 2 to 600 in line 17, and in line 31 the correction factor DELTATIME is included as a factor in the calculation. That's it; in figure 2.33 on page 74 we can admire the *perfect* result on one of my slower computers.

Listing 2.48: Movement with normalization and $dt = 1/fps$

```

6 def main():
7     pygame.init()
8     window = pygame.Window(size=cfg.WINDOW.size, title="Movement", position=(10, 50))
9     screen = window.get_surface()
10    clock = pygame.time.Clock()
11
12    defender_image = pygame.image.load("images/defender01.png").convert_alpha()
13    defender_image = pygame.transform.scale(defender_image, (30, 30))
14    defender_rect = defender_image.get_rect()
15    defender_rect.centerx = cfg.WINDOW.centerx
16    defender_rect.bottom = cfg.WINDOW.bottom - 5
17    defender_speed = 600 # Not px/f but px/s
18    defender_direction_v = -1
19
20    start_time = pygame.time.get_ticks()
21    running = True
22    while running:
23        if pygame.time.get_ticks() > start_time + cfg.LIMIT:
24            defender_speed = 0
25        # Events
26        for event in pygame.event.get():
27            if event.type == pygame.QUIT:
28                running = False
29
30        # Update
31        defender_rect.top += defender_direction_v * defender_speed * cfg.DELTATIME #
32        if defender_rect.bottom >= cfg.WINDOW.bottom:
33            defender_direction_v *= -1
34        elif defender_rect.top <= 0:
35            defender_direction_v *= -1
36
37        # Draw
38        screen.fill("white")
39        pygame.draw.line(screen, "red", (0, 315), (cfg.WINDOW.width, 315), 2)
40        screen.blit(defender_image, defender_rect)
41        window.flip()
42        clock.tick(cfg.FPS)
43    print(f"top={defender_rect.top}")

```

2.4.3.2 Optimizing Normalized Speed

Two issues cause the error shown

in figure 2.33 on page 74:

- ⚠ **Rounding errors:** In theory, multiplying the frame rate by the delta time should always yield 1.0. Unfortunately, this is not the case. When computing delta time, a value close to the exact value is stored due to the way a [floating-point number](#) is represented; for example, instead of the exact value $0.0\bar{3}$ for $\frac{1.0}{30.0}$, the stored value is 0.03333333333333330. Over time, this rounding error accumulates to perceptible amounts.
- ⚠ **Incorrect understanding of *fps*:** The frame rate does not define that the main program loop is executed *exactly* 60 times per second, for example, but that it is executed *at most* 60 times per second. If the game logic or rendering takes more time than $\frac{1}{60}$ s, at least one frame will be skipped. This can also happen if the computer loses performance due to other operations (for example, cloud synchronization).

We cannot solve the first problem without a significant loss of performance, so we will not consider it any further. The second problem, however, can be addressed. Instead of a fixed delta time, we need a value that is based on the actual duration of a frame. The method `pygame.clock.tick()` in line 47 provides a good estimate of the frame time. Luckily, this feature is already built in and can therefore be used directly (see source code 2.49). The result in figure 2.34 on page 74 is better, but still not satisfying :-(. In figure 2.30 on the following page, you can see that the red line more or less dances around the green one, and no clear trend is visible.

`tick()`

Listing 2.49: Normalized Movement with `pygame.clock.tick()`

```

46     window.flip()
47     Settings.DELTATIME = clock.tick(Settings.FPS) / 1000.0  #
48     print(f"top={defender_rect.top}")

```

The cause is a problem that we should have fixed immediately. In the assignment in line 31 in source code 2.48 on the preceding page, the right-hand side is a floating-point value, while the left-hand side is an [integer](#). As a result, the decimal places are truncated in every loop iteration. For example, if the spaceship were supposed to move by 5.8 px in each frame, the following values would occur:

Float
for logic,
Int for
rendering

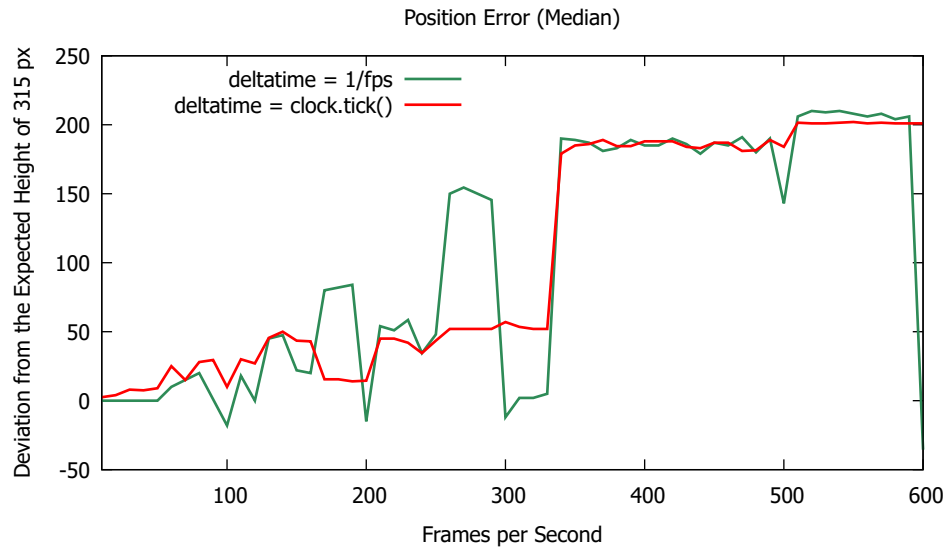
Figure 2.30: Position Error of $1/fps$ and `pygame.clock.tick()`

Table 2.6: Error Propagation

Frame	1	2	3	4	5	6	7	8	9
Actual Value	5.0	10.0	15.0	20.0	25.0	30.0	35.0	40.0	45.0
Correct Value	5.8	11.6	14.4	23.2	29.0	34.8	40.6	46.4	52.2
Error	0.8	1.3	2.4	3.2	4.0	4.8	5.6	6.4	7.2

FRect

Recently, Pygame introduced a variant of `Rect`, namely `FRect`. In this class, all values are stored as `floats`, so fractional parts are no longer truncated. Alternatively, we would have to store the position values independently of the `Rect` object in an additional float variable in order to preserve the fractional parts, for example in a `pygame.math.Vector2` object.

Listing 2.50: Normalized Movement with Positions in Float

```

11 def main():
12     pygame.init()
13     window = pygame.Window(size=Settings.WINDOW.size, title="Movement", position=(10, 50))
14     screen = window.get_surface()
15     clock = pygame.time.Clock()
16
17     defender_image = pygame.image.load("images/defender01.png").convert_alpha()
18     defender_image = pygame.transform.scale(defender_image, (30, 30))
19     defender_rect = pygame.Rect(defender_image.get_rect()) # float
20     defender_rect.centerx = Settings.WINDOW.centerx
21     defender_rect.bottom = Settings.WINDOW.bottom - 5
22     defender_speed = 600
23     defender_direction_v = -1
24
25     start_time = pygame.time.get_ticks()
26     running = True
27     while running:
28         if pygame.time.get_ticks() > start_time + Settings.LIMIT:
29             defender_speed = 0
30         # Events
31         for event in pygame.event.get():
32             if event.type == pygame.QUIT:
33                 running = False
34
35         # Update
36         defender_rect.top += defender_direction_v * defender_speed * Settings.DELTATIME
37         if defender_rect.bottom >= Settings.WINDOW.bottom:
38             defender_direction_v *= -1
39         elif defender_rect.top <= 0:
40             defender_direction_v *= -1
41
42         # Draw
43         screen.fill("white")
44         pygame.draw.line(screen, "red", (0, 315), (Settings.WINDOW.width, 315), 2)
45         screen.blit(defender_image, defender_rect)
46         window.flip()
47         Settings.DELTATIME = clock.tick(Settings.FPS) / 1000.0
48     print(f"top={defender_rect.top}")

```

In figure 2.35 on page 75, we can see that the result has already improved significantly. The deviation from the optimal value 315px has also been reduced dramatically. The difference is visualized in figure 2.31 on the next page.

However, there is yet another source of error: `pygame.clock.tick()` does not provide enough decimal precision. Over long runtimes, the missing fractional parts accumulate and again lead to noticeable errors. There are better Python functions for measuring elapsed time.

line 22 of source code 2.51 on page 73, `time.time()` is used to return the number of seconds since January 1, 1970 as a floating-point number. The fractional part represents fractions of a second. This measurement is more precise than the one provided by `pygame.clock.tick()` and, depending on the time-measurement capabilities of the computer architecture and the operating system, can provide more decimal places—up to the nanosecond range.

`time()`

In line 45, the current time is measured after one frame has elapsed, and in the following line the elapsed time is computed. This value represents the actual *delta time* of the

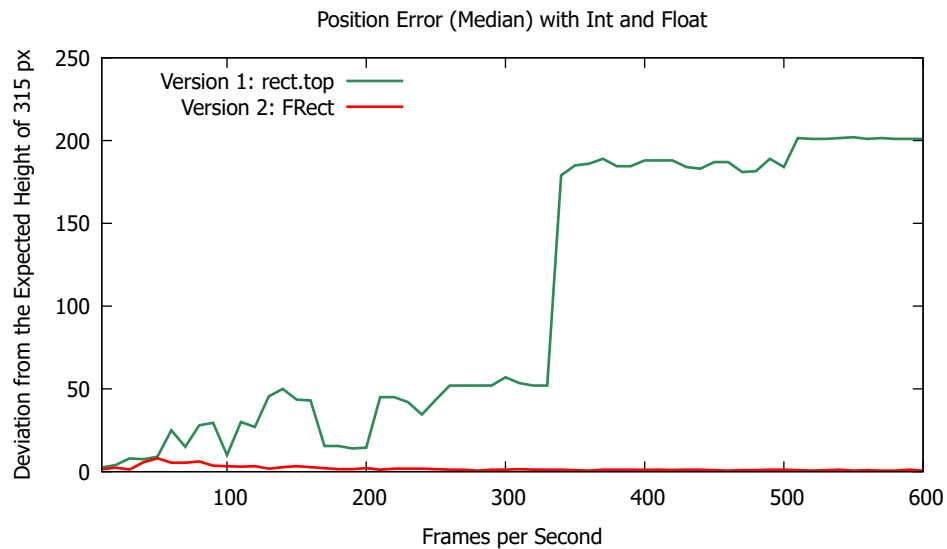


Figure 2.31: Position Error of Rect and FRect

frame, now with higher precision. Afterwards, in line 47, the new start time of the next frame is stored so that the elapsed time can be computed again after the next frame.

figure 2.36 on page 75 shows that the target positions are reached almost perfectly for all frame rates. However, comparing the position errors in figure 2.32 on the facing page does not allow for a clear evaluation. I suspect that experiments with significantly longer runtimes would make a difference visible. We must – and can – live with the remaining error.

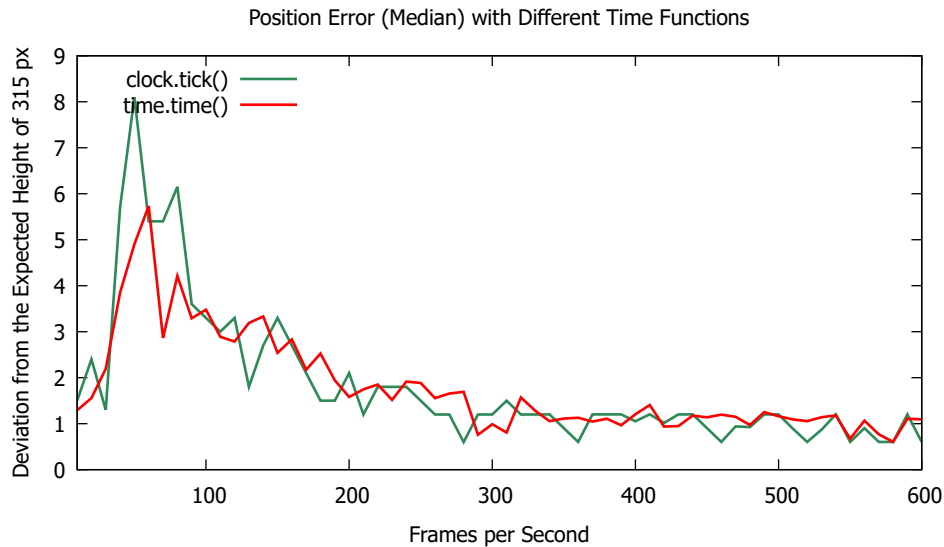


Figure 2.32: Position Error with Different Time Functions

Listing 2.51: Movement with normalization and `time.time()`

```

7 def main():
8     pygame.init()
9     window = pygame.Window(size=config.WINDOW.size, title="Movement", position=(10, 50))
10    screen = window.get_surface()
11    clock = pygame.time.Clock()
12
13    defender_image = pygame.image.load("images/defender01.png").convert_alpha()
14    defender_image = pygame.transform.scale(defender_image, (30, 30))
15    defender_rect = pygame.Rect(defender_image.get_rect())
16    defender_rect.centerx = config.WINDOW.centerx
17    defender_rect.bottom = config.WINDOW.height - 5
18    defender_speed = 600
19    defender_direction_v = -1
20
21    start_time = pygame.time.get_ticks()
22    time_previous = time() # remember start time
23    running = True
24    while running:
25        if pygame.time.get_ticks() > start_time + config.LIMIT:
26            defender_speed = 0
27        # Events
28        for event in pygame.event.get():
29            if event.type == pygame.QUIT:
30                running = False
31
32        # Update
33        defender_rect.top += defender_direction_v * defender_speed * config.DELTATIME
34        if defender_rect.bottom >= config.WINDOW.height:
35            defender_direction_v *= -1
36        elif defender_rect.top <= 0:
37            defender_direction_v *= -1
38
39        # Draw
40        screen.fill("white")
41        pygame.draw.line(screen, "red", (0, 315), (config.WINDOW.width, 315), 2)

```

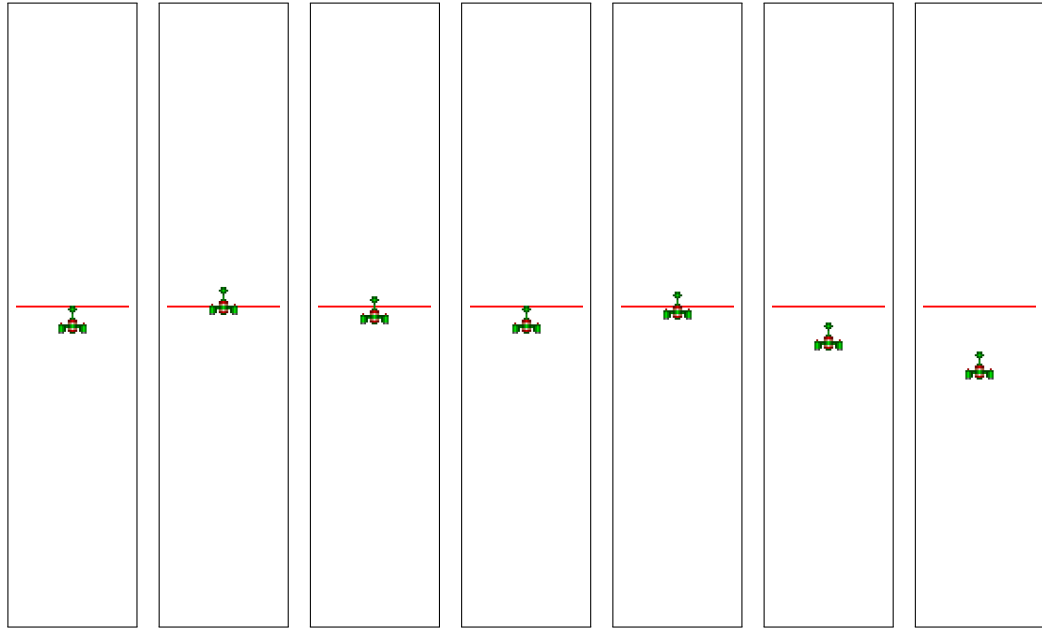


Figure 2.33: Movement with normalization and $dt = 1/fps$ using constant speed but different fps (from left to right: 10, 30, 60, 120, 240, 300, 600)

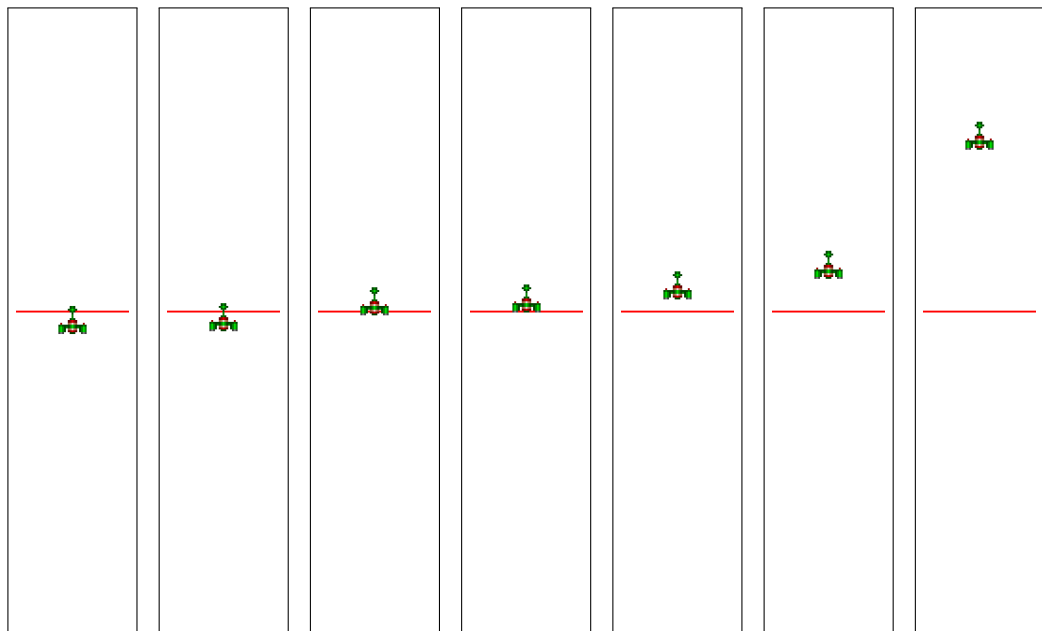


Figure 2.34: Movement with normalization and `pygame.clock.tick()` using constant speed but different fps (from left to right: 10, 30, 60, 120, 240, 300, 600)

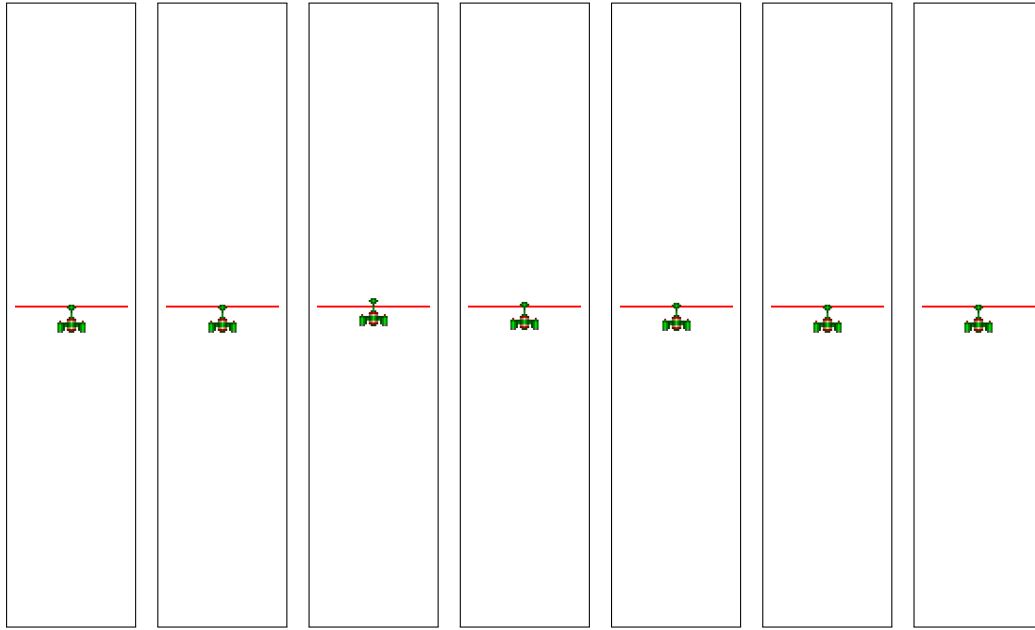


Figure 2.35: Movement with normalization and `pygame.clock.tick()` (float) using constant speed but different fps (from left to right: 10, 30, 60, 120, 240, 300, 600)

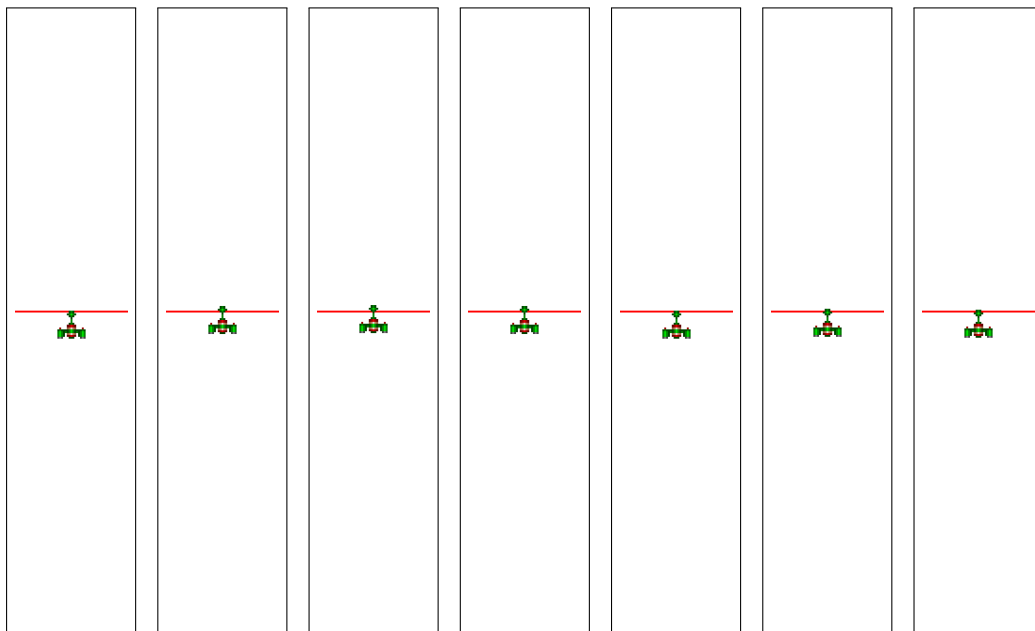


Figure 2.36: Movement with normalization and `time.time()` using constant speed but different fps (from left to right: 10, 30, 60, 120, 240, 300, 600), Version 3

```

42     screen.blit(defender_image, defender_rect)
43     window.flip()
44     clock.tick(cfg.FPS)
45     time_current = time() # remember stop time
46     cfg.DELTATIME = time_current - time_previous # Time consumption
47     time_previous = time_current # New start time
48     pygame.quit()

```

2.4.4 What was new?

- 👉 The position of an object is stored in a `Rect` or `FRect` object. In each frame, the position is checked and modified if necessary. When the screen is updated, this creates the impression of movement. The result of a movement is usually first stored temporarily in a variable and checked before it is applied as the new position.
- 👉 The direction of movement is encoded by the sign, and the speed by the value of the speed variable. Horizontal and vertical movement are handled separately.
- 👉 To become independent of the actual frame rate, a correction factor (delta time) must be used when calculating the new position. This value can either be computed manually or obtained from a call to `pygame.time.Clock.tick()`.



The following Pygame elements were introduced



- + 🎮 `pygame.rect.FRect`:
<https://pyga.me/docs/ref/rect.html>
- + 🎮 `pygame.rect.FRect.move()`:
<https://pyga.me/docs/ref/rect.html#pygame.Rect.move>
- + 🎮 `pygame.rect.Rect`:
<https://pyga.me/docs/ref/rect.html>
- + 🎮 `pygame.rect.Rect.move()`:
<https://pyga.me/docs/ref/rect.html#pygame.Rect.move>
- + 🎮 `pygame.Surface.get_rect()`:
https://pyga.me/docs/ref/surface.html#pygame.Surface.get_rect
- + 🎮 `pygame.time.get_ticks()`:
https://pyga.me/docs/ref/surface.html#pygame.time.get_ticks

 `pygame.math.Vector2:`
<https://pyga.me/docs/ref/math.html#pygame.math.Vector2>

 `pygame.math.Vector3:`
<https://pyga.me/docs/ref/math.html#pygame.math.Vector3>

2.4.5 Homework

1. Use `centerx`, `centery`, or `center` instead of `top` and `left`. Does it work? Do you notice any remarkable differences?
2. Create an application in which two identical objects travel the same horizontal distance of 800 px. One object uses position values stored as `int`, the other uses position values stored as `float`. Both should move at a speed of 50 px/s. What can you observe?
3. Create an application where the speed is not constant. Four objects should travel the same horizontal distance of 800 px in parallel, just like before. The behavior should be as follows:
 - a) Object 1: It continuously accelerates and reaches its maximum speed at the right end.
 - b) Object 2: It accelerates over time, reaches its maximum speed at the midpoint of the distance, and then slows down again. At the right end, its speed is 0 px/s. The increase and decrease in speed are linear.
 - c) Object 3: It accelerates over time, reaches its maximum speed at the midpoint of the distance, and then slows down again. At the right end, its speed is 0 px/s. The increase and decrease in speed follow a sine curve on $[0, \pi]$.
 - d) Object 4: Like object 3, but a variable controls how many intervals of $[0, \pi]$ are completed before reaching the right end. For example, the object speeds up and slows down again 5 times.
4. Something for the ambitious among you: All four objects reach the right edge at the same time.

2.5 Class Sprite

2.5.1 Introduction

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/05%20Sprite/invader

In section 2.4.3.1 on page 64, it became apparent that many variables start with `defender_`. In other words, they are attributes of a single entity and almost demand to be expressed as a class.

This class should contain all information related to updating and rendering the bitmap. Some elements, such as `defender_image` and `defender_rect`, seem to play a role in virtually every bitmap processing task. Furthermore, every bitmap will require some form of state update and screen output. In fact,

Pygame already provides a class that offers exactly such a **framework**

`pygame.sprite.Sprite`.

Let us therefore define the `Defender` class as a subclass of `Sprite` (line 7).

Listing 2.52: Sprites (1), Version 1.0

```

7  class Defender(pygame.sprite.Sprite):                                # Child class of Sprite
8
9      def __init__(self) -> None:                                       # Constructor
10         super().__init__()
11         self.image = pygame.image.load("images/defender01.png").convert_alpha()
12         self.image = pygame.transform.scale(self.image, (30, 30))
13         self.rect = pygame.Rect(self.image.get_rect())
14         self.rect.centerx = cfg.WINDOW.centerx
15         self.rect.bottom = cfg.WINDOW.bottom - 5
16         self.speed = 300
17
18     def update(self) -> None:                                           # State update
19         newpos = self.rect.move(self.speed * cfg.DELTATIME, 0)
20         if newpos.right >= cfg.WINDOW.right:
21             self.change_direction()
22             newpos.right = cfg.WINDOW.right
23         elif newpos.left <= cfg.WINDOW.left:
24             self.change_direction()
25             newpos.left = cfg.WINDOW.left
26         self.rect = newpos
27
28     def draw(self, screen: pygame.Surface) -> None: # Drawing
29         screen.blit(self.image, self.rect)
30
31     def change_direction(self) -> None:                               # 00 style
32         self.speed *= -1

```

The lines in the constructor (line 9ff.) correspond to those of the previous version. Only the prefix `defender_` is replaced by `self.`, which turns the variables into attributes of the class. You should have no difficulty understanding these changes.

Every subclass of `Sprite` must provide two attributes: `rect` and `image`. These two attributes are accessed by the already predefined solutions for collision detection, rendering, and related tasks. We will see their usefulness later on.

self.rect

self.image

In line 18ff., the boundary collisions and state changes are implemented. One detail that stands out is the computation of the new position using `move()`.

New is the call to the method `change_direction()`. This method (line 31) is more *object-oriented* than the previous version. In object-oriented programming, algorithms are not implemented directly; instead, messages are sent to objects, which then handle the details internally – in a way that is not visible from the outside. In this case, this means that instead of performing the direction change directly at the relevant point, I send a message to the object telling it that the direction needs to be changed.

With the method `draw()` in line 28, the screen output is encapsulated.

Listing 2.53: Sprites (2), Version 1.0

```

35 def main():
36     pygame.init()
37     window = pygame.Window(size=cfg.WINDOW.size, title="Sprite", position=(10, 50))
38     screen = window.get_surface()
39     clock = pygame.time.Clock()
40     defender = Defender()                                # Create object
41
42     time_previous = time()
43     running = True
44     while running:
45         # Events
46         for event in pygame.event.get():
47             if event.type == pygame.QUIT:
48                 running = False
49
50         # Update
51         defender.update()                                # Call
52
53         # Draw
54         screen.fill("white")
55         defender.draw(screen)                            # Call
56         window.flip()
57
58         clock.tick(cfg.FPS)
59         time_current = time()
60         cfg.DELTATIME = time_current - time_previous
61         time_previous = time_current
62     pygame.quit()

```

Using the `Defender` class has now become straightforward. In line 40, an object of the class is created. In line 51, `update()` is called, and in line 55, `draw()` is executed.

One advantage of the new architecture is the improved clarity and readability of the main program. By following naming conventions (descriptive class and method names), the overall control flow becomes clearer and is no longer obscured by implementation details.

I now want to make use of the capabilities of the `Sprite` class so that boundary collision checks no longer have to be implemented manually.

Let us get started: Since we want to organize collision detection differently, we first simplify `update()` again. We now only compute the new position. In doing so, the method `pygame.FRect.move_ip()` is introduced in line 19. It works like `move()`, but in this case the modification is applied directly to the rectangle; `ip` stands for *in place*. With `move()`, the original rectangle remains unchanged.

move_ip()

Listing 2.54: Sprites (1), Version 1.1

```

18     def update(self) -> None:
19         self.rect.move_ip(self.speed * cfg.DELTATIME, 0) # Move in-place

```

To make the boundaries visible and to better recognize collisions, the edges are now replaced by two stone walls on the left and right. These bitmaps are also implemented as subclasses of `pygame.sprite.Sprite`. Since the state of the two walls never changes, the implementation of `update()` can be omitted.

Listing 2.55: Sprites (2), Version 1.1

```

28 class Border(pygame.sprite.Sprite):
29
30     def __init__(self, leftright: str) -> None:
31         super().__init__()
32         self.image = pygame.image.load("images/brick01.png").convert_alpha()
33         self.image = pygame.transform.scale(self.image, (35, cfg.WINDOW.height))
34         self.rect = self.image.get_rect()
35         if leftright == "right":
36             self.rect.left = cfg.WINDOW.width - self.rect.width
37
38     def draw(self, screen: pygame.Surface) -> None:
39         screen.blit(self.image, self.rect)

```

I now create the two boundaries:

Listing 2.56: Sprites (3), Version 1.1

```

49     border_left = Border("left")
50     border_right = Border("right")

```

So far, everything has been easy.

Listing 2.57: Sprites (4), Version 1.1

```

60     # Update
61     if pygame.sprite.collide_rect(defender, border_left):
62         defender.change_direction()
63     elif pygame.sprite.collide_rect(defender, border_right):
64         defender.change_direction()
65     defender.update()

```

collide_rect()

What is happening here? The method `pygame.sprite.collide_rect()` checks whether

the rectangles of two `Sprite` objects collide. This means that I no longer have to manually check the left and right boundaries myself.

Here, the collision of a single object with both boundaries – or more generally, with many `Sprite` objects – is tested.

Spritegroups

In practice, sprites rarely exist on their own; they are usually organized into groups. This concept is also built into Pygame and leads to further simplifications.

Listing 2.58: Sprites (1), Version 1.2

```

36 def main():
37     pygame.init()
38     window = pygame.Window(size=cfg.WINDOW.size, title="Sprite", position=(10, 50))
39     screen = window.get_surface()
40     clock = pygame.time.Clock()
41
42     defender = pygame.sprite.GroupSingle(Defender())
43     all_border = pygame.sprite.Group()
44     all_border.add(Border("left"))
45     all_border.add(Border("right"))
46
47     time_previous = time()
48     running = True
49     while running:
50         # Events
51         for event in pygame.event.get():
52             if event.type == pygame.QUIT:
53                 running = False
54
55         # Update
56         if pygame.sprite.spritecollide(defender.sprite, all_border, False): # !
57             defender.sprite.change_direction() #
58         defender.update()
59
60         # Draw
61         screen.fill((255, 255, 255))
62         defender.draw(screen)
63         all_border.draw(screen) # In one go
64         window.flip()
65
66         clock.tick(cfg.FPS)
67         time_current = time()
68         cfg.DELTATIME = time_current - time_previous
69         time_previous = time_current
70     pygame.quit()

```

The defender is no longer addressed directly, but is instead packed into a luxury box. I will come back to this later. The two `Border` objects are no longer stored in two separate object variables either; instead, they are placed into a luxury box as well: a `pygame.sprite.Group`. Here, I could also store additional boundaries or walls. From the point of view of the game logic, they could then all be processed together in one go. This becomes clear in this mini example in two places.

Group

The first location is line 56, where a different variant of collision detection is used:

spritecollide()

`pygame.sprite.spritecollide()`. The first parameter is a *single* **Sprite** object – in our case, the defender. The second parameter is a sprite group containing all **Border** objects. Thus, the defender is checked for collisions with all members of the group. This only works if all sprites provide a **Rect** or **FRect** object named `rect` as an attribute. The third parameter – **False** in this example – controls whether the colliding sprite should be removed from the group. This feature is quite useful in games, for example when rocks that are shot by a spaceship should be deleted.

The second location is line 63. Here, `draw()` is no longer called for each object individually, but once for the entire group. When using this service, the `draw()` method can be removed from your own classes (here **Border** and **Defender**), which simplifies things even further.

It therefore seems like a good idea to pack sprites into such luxury boxes. But what about the defender? To take advantage of sprite groups, you can also create groups that contain only a single element. To allow these groups to work more efficiently – after all, it is known that they contain only one element – Pygame provides the special case `pygame.sprite.GroupSingle`. Since there is often a need to access the single **Sprite** object of the *group*, this group provides the additional attribute `sprite` (see line 56f.).

GroupSingle

2.5.2 More Input

2.5.2.1 OO Issues

 So ... where are the files?

- https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/05%20Sprite/invader/v04
- https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/05%20Sprite/invader/v05

I want to pursue my object-oriented approach even further and also turn the main program into a **Game** class. What is important to me is to establish a sense of structural discipline right from the beginning. The longer you stay in software development, the more you will grow fond of terms like *order* and *structure*. They help you avoid losing the thread, even in more complex games. A particularly helpful concept here is the [Single Responsibility Principle \(SRP\)](#).

Single Responsibility Principle (SRP)

Each class or function should have exactly one responsibility. It should focus on that single task and do it well. A solution to a specific problem should be encapsulated in one class or one method, and changes to that responsibility should affect only that part of the code.

Listing 2.59: Game-Klasse

```

37
38     def __init__(self) -> None:
39         pygame.init()
40         self.window = pygame.Window(size=config.WINDOW.size, title="Sprite", position=(10, 50))
41         self.screen = self.window.get_surface()
42         self.clock = pygame.time.Clock()
43
44         self.defender = pygame.sprite.GroupSingle(Defender())
45         self.all_border = pygame.sprite.Group()
46         self.all_border.add(Border("left"))
47         self.all_border.add(Border("right"))
48         self.running = False
49
50     def run(self) -> None:
51         time_previous = time()
52         self.running = True
53         while self.running:
54             self.watch_for_events()
55             self.update()
56             self.draw()
57             self.clock.tick(config.FPS)
58             time_current = time()
59             config.DELTATIME = time_current - time_previous
60             time_previous = time_current
61         pygame.quit()
62
63     def watch_for_events(self) -> None:
64         for event in pygame.event.get():
65             if event.type == pygame.QUIT:
66                 self.running = False
67
68     def update(self) -> None:
69         if pygame.sprite.spritecollide(self.defender.sprite, self.all_border, False):
70             self.defender.sprite.change_direction() # I don't like this!
71         self.defender.update()
72
73     def draw(self) -> None:
74         self.screen.fill((255, 255, 255))
75         self.defender.draw(self.screen)
76         self.all_border.draw(self.screen)
77         self.window.flip()

```

An example of this last point is the design of the `Game` class. Here, the source code is no longer simply placed in `__main__`, but is instead encapsulated, structured, and thus made flexibly reusable. A clear example of the SRP can be seen in the methods `watch_for_events()`, `update()`, and `draw()`. It is simply not the responsibility of `run()` to organize everything. From the perspective of the main loop, it is irrelevant which events are queried or how they are processed. All that matters is that events are

handled once per frame. Likewise, `run()` should not be concerned with the order in which sprites are drawn to the screen. That task belongs to the `draw()` method. The `run()` method merely ensures that sprites first update their state and that rendering happens afterwards.

One aspect remains that I would still like to address here: the call to `change_direction()` in line 70 does not appeal to me. It violates object-oriented design rules, specifically the [Liskov Substitution Principle \(LSP\)](#).

Liskov Substitution Principle (LSP)

A principle of object-oriented programming stating that objects of a derived class must be usable anywhere an object of the base class is expected, without altering the correct behavior of the program. Formulated by Barbara Liskov in 1987. The LSP ensures that inheritance does not introduce unexpected side effects and that class hierarchies remain consistent.

The sprite group is a collection of `Sprite` objects. However, the class `pygame.sprite.Sprite` does not define a method called `change_direction()`. Calling such a method here is therefore not entirely clean. Python does not have a problem with this, but that should not be the benchmark.

`update()`

A better approach is to adapt the `update()` method instead. If you take a closer look at the [signature](#) of `pygame.sprite.Sprite.update()`, you will see that it is designed to accept freely definable parameters. I have developed the habit of using a parameter named `action` to trigger specific behavior in subclasses. With this approach, `change_direction()` is called internally from `update()` (see line 23) rather than being invoked from the outside.

Listing 2.60: `Defender.update()`

```

20     if "action" in kwargs.keys():
21         if kwargs["action"] == "newpos":           # Compute new position
22             self.rect.move_ip(self.speed * cfg.DELTATIME, 0)
23         elif kwargs["action"] == "direction":      # Change direction
24             self.change_direction()

```

The call then takes place indirectly in source code 2.61 at line 75 by using the argument passed to the method.

Note: This also complies with the object-oriented design principle [Don't ask – tell](#).

Listing 2.61: `Game.update()`

```

74     if pygame.sprite.spritecollide(self.defender.sprite, self.all_border, False):
75         self.defender.update(action="direction") # Better (don't aks - tell!)
76         self.defender.update(action="newpos")

```


2.5.2.2 Add Sprite Objects to a Group Right Away



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/05%20Sprite/invadergroup/v01

It is often very convenient to assign a sprite object to a group already at the time it is created. To do this, the signature of `__init__()` only needs to be adjusted accordingly.

The parameter

`*groups`

together with the corresponding call to the constructor of the [superclass](#)

`super().__init__(*groups)`

ensures that the sprite object is immediately added to the given group or groups. Below is the complete source code. In line 61, the group is then simply passed to the constructor as the last argument.

Listing 2.62: Add Sprite Objects to a Group Right Away

```

1  from random import randint
2  from time import time
3  from typing import Any
4
5  import pygame
6
7
8  class Ship(pygame.sprite.Sprite):
9      def __init__(self, position:tuple[int,int], *groups: pygame.sprite.AbstractGroup[Any])
10         -> None:
11         super().__init__(*groups)
12         self.image = pygame.image.load("images/defender01.png").convert_alpha()
13         self.image = pygame.transform.scale(self.image, (30, 30))
14         self.rect = pygame.Rect(self.image.get_rect())
15         self.rect.left = position[0]
16         self.rect.bottom = position[1]
17         self.speed = -300
18
19     def update(self, *args: Any, **kwargs: Any) -> None:
20         self.rect.move_ip(0, self.speed * Game.DELTATIME)
21         self.rect.clamp_ip(Game.WINDOW)
22         return super().update(*args, **kwargs)
23
24
25  class Game():
26      FPS = 60
27      DELTATIME = 1.0 / FPS
28      SPWAN = 15
29      WINDOW = pygame.Rect(0,0,300,600)
30
31      def __init__(self) -> None:
32          pygame.init()
33          self.window = pygame.Window(size=Game.WINDOW.size, title="Spritegroup")

```

```

34     self.screen = self.window.get_surface()
35     self.clock = pygame.time.Clock()
36     self.ships = pygame.sprite.Group()
37     self.running = True
38     self.counter = 0
39
40
41     def run(self) -> None:
42         time_previous = time()
43         while self.running:
44             self.watch_for_events()
45             self.update()
46             self.draw()
47             self.clock.tick(Game.FPS)
48             time_current = time()
49             Game.DELTATIME = time_current - time_previous
50             time_previous = time_current
51
52     def watch_for_events(self) -> None:
53         for event in pygame.event.get():
54             if event.type == pygame.QUIT:
55                 self.running = False
56
57     def update(self) -> None:
58         self.counter += 1
59         if self.counter > Game.SPWAN:
60             self.counter = 0
61             Ship((randint(0, 300-30), 600), self.ships) #
62             self.ships.update()
63
64     def draw(self) -> None:
65         self.screen.fill((255, 255, 255))
66         self.ships.draw(self.screen)
67         self.window.flip()

```

2.5.2.3 Delete Sprites from Groups

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/05%20Sprite/invadergroup/v02

If you follow the program logic in source code 2.62 on the previous page, you will notice that a spaceship is created at the bottom edge four times per second and then flies upward. Once it reaches the top, it simply stops.

The latter behavior is usually rather pointless. A more natural approach is to remove all spaceships that have crossed an upper boundary. Think, for example, of projectiles that leave the playfield.

kill()

This can be achieved by calling `pygame.sprite.Sprite.kill()`, which instructs Pygame to remove the `Sprite` object from *all* sprite groups. If no further references to the `Sprite` object exist, it will then be deleted by Python's [garbage collector](#).

You can see an example of this in source code 2.63. Once the spaceship has completely passed the midpoint of the window, it is removed.

Listing 2.63: Kill a Sprite

```

18 def update(self, *args: Any, **kwargs: Any) -> None:
19     self.rect.move_ip(0, self.speed * Game.DELTATIME)
20     if self.rect.bottom < Game.WINDOW.centery:
21         self.kill()
22     return super().update(*args, **kwargs)

```

2.5.3 What was new?

From a behavioral point of view: *nothing at all*. The existing application has merely been embedded into a flexible framework.



The following Pygame elements were introduced



-  `pygame.Rect.move()`:
<https://pyga.me/docs/ref/rect.html#pygame.Rect.move>
-  `pygame.Rect.move_ip()`:
https://pyga.me/docs/ref/rect.html#pygame.Rect.move_ip
-  `pygame.sprite.Group`:
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.Group>
-  `pygame.sprite.GroupSingle`:
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.GroupSingle>
-  `pygame.sprite.GroupSingle.sprite`:
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.GroupSingle.sprite>
-  `pygame.sprite.Sprite`:
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.Sprite>
-  `pygame.sprite.Sprite.kill()`:
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.Sprite.kill>
-  `pygame.sprite.collide_rect()`:
https://pyga.me/docs/ref/sprite.html#pygame.sprite.collide_rect
-  `pygame.sprite.spritecollide()`:
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.spritecollide>

2.5.4 Homework

1. Modify the source code from section [2.3.2.3](#) on page [54](#) so that `Meadow`, `Sky`, `Tree`, `House`, and `Sun` are subclasses of `pygame.sprite.Sprite`.
2. Manage all `Sprite` classes that implement an `update()` method in one `pygame.sprite.Group` object, and store the others in a separate group.

2.6 Handling Keyboard Input

2.6.1 Introduction

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/06%20Keyboard/keyboard/v01

I do not intend to cover the keyboard exhaustively here, but merely to illustrate the basic principle. The movement of the spaceship is controlled by keyboard events. Every time I press a arrow key – left, right, up, down – the spaceship moves in the corresponding direction. If I release the arrow key, the spaceship stops. The game can now also be exited using the Escape key ([Boss key](#)).

As a first step, a dictionary of possible directions is created in `config.py`.

Vector2D

These directions are managed as `Vector2` objects, since they are easier to use for mathematical operations.

Listing 2.64: Control Direction by Keys (1), `config.py`

```
1 import pygame
2
3 WINDOW = pygame.Rect((0, 0), (300, 300))
4 FPS = 60
5 DELTATIME = 1.0 / FPS
6 DIRECTIONS = {
7     "stop": pygame.math.Vector2(0, 0),
8     "right": pygame.math.Vector2(1, 0),
9     "left": pygame.math.Vector2(-1, 0),
10    "up": pygame.math.Vector2(0, -1),
11    "down": pygame.math.Vector2(0, 1),
12 }
```

Next, we prepare the `Defender` class or modify it slightly (source code [2.65](#) on the following page). The sprite is no longer placed at the bottom but centered (line [15](#)), and the spaceship should now also be able to move vertically. For this, we either need two separate variables or a `Vector2` object. I choose a `Vector2` object, where the first component represents the horizontal direction vector and the second the vertical direction. Each direction vector is set according to the semantics introduced earlier.

I would like to draw special attention to line [23](#). The methods `clamp_ip()` as well as `clamp()` provide a very convenient shortcut in programming. Both methods check whether the inner rectangle has crossed the boundary of the outer rectangle on any side and, if necessary, move it back to the edge of the outer rectangle.

`clamp_ip()`
`clamp()`

Here is an equivalent check without using `clamp()`, shown only for the left edge of the outer rectangle:

```

1         if inner_rect.right < outer_rect.left:
2             inner_rect.right = outer_rect.left + 1

```

The method `clamp()` performs this kind of check for all sides of the inner rectangle. Using `clamp()` or `clamp_ip()`, you can therefore ensure that a sprite never leaves a defined play area.

The difference between the two methods is that `clamp()` does not modify the values of the inner rectangle but instead returns a new, adjusted rectangle, whereas `clamp_ip()` modifies the values of the inner rectangle directly.

Listing 2.65: Control Direction by Keys (2), Class Defender

```

8 class Defender(pygame.sprite.Sprite):
9
10     def __init__(self) -> None:
11         super().__init__()
12         self.image = pygame.image.load("images/defender01.png").convert_alpha()
13         self.image = pygame.transform.scale(self.image, (30, 30))
14         self.rect = pygame.Rect(self.image.get_rect())
15         self.rect.center = cfg.WINDOW.center #
16         self.speed = 100
17         self.direction = cfg.DIRECTIONS["stop"]
18
19     def update(self, *args: Any, **kwargs: Any) -> None:
20         if "action" in kwargs.keys():
21             if kwargs["action"] == "move":
22                 self.rect.move_ip(self.direction.elementwise() * self.speed *
23                                 cfg.DELTATIME)
24                 self.rect.clamp_ip(cfg.WINDOW) # Keep inside window
25             elif "direction" in kwargs.keys():
26                 self.direction = cfg.DIRECTIONS[kwargs["direction"]]

```

KEY-
DOWN
KEYUP

key

Let us now turn to the actual handling of keyboard input: Pressing a key can trigger the event types `pygame.KEYDOWN` or `pygame.KEYUP`. In our example (line 57), we are interested in which key is *pressed*, so we use `KEYDOWN`. After that, we can determine which key was pressed via `pygame.event.key`. For this purpose, Pygame provides a set of predefined constants in `pygame.key` (see table 2.7 on page 101 and table 2.8 on page 104).

Listing 2.66: Control Direction by Keys (3)), `Game.watch_for_events()`

```

54 for event in pygame.event.get():
55     if event.type == pygame.QUIT:
56         self.running = False
57     elif event.type == pygame.KEYDOWN: # Key pressed
58         if event.key == pygame.K_ESCAPE: #
59             self.running = False
60         elif event.key == pygame.K_RIGHT: # Arrows
61             self.defender.update(direction="right")
62         elif event.key == pygame.K_LEFT:
63             self.defender.update(direction="left")
64         elif event.key == pygame.K_UP:
65             self.defender.update(direction="up")

```

```

66         elif event.key == pygame.K_DOWN:
67             self.defender.update(direction="down")
68     elif event.type == pygame.KEYUP:           # Key released
69         if event.key in (pygame.K_RIGHT, pygame.K_LEFT,
70                         pygame.K_UP, pygame.K_DOWN):
71             self.defender.update(direction="stop")

```

Let us start with the boss key. In line 58, the constant `K_ESCAPE` is used to check whether the pressed key is the `[Esc]`. As with clicking the window's close button, the flag of the main program loop is then simply set to `False`. Try it out!

`K_ESCAPE`

After that, the four arrow keys are handled starting at line 60 ff. Using `K_LEFT`, `K_RIGHT`, `K_UP`, and `K_DOWN`, the corresponding arrow key is checked and the appropriate message is sent to the defender.

`K_LEFT`
`K_RIGHT`
`K_UP`
`K_DOWN`

If one of the arrow keys is released (`pygame.KEYUP` in line 68), the spaceship is stopped. This should be sufficient for now. The keyboard is only one possible way to control a game. Mouse input, game controllers, and joysticks are also supported in Pygame.

2.6.2 More Input

2.6.2.1 Example: Shift and Related Keys

So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/06%20Keyboard/keyboard/v02

With the help of `[↑]`, `[Ctrl]`, or other modifier keys, additional meanings can be assigned to regular keys. But how can we detect that such keys are being pressed? In the source code of the previous example, only a reaction to *one* key press per condition is implemented.

I will now extend the example so that pressing a movement key together with the left Shift key makes the spaceship move faster, while pressing it together with the right Shift key makes it move slower.

To do this, we first adapt the `update()` method of the `Defender`. As you can see, the signals to speed up or slow down can now be handled as well. It is also possible to reset the speed to its normal value. The concrete values are more or less arbitrary, but they are chosen so that the difference in speed is easy to perceive.

Listing 2.67: Control Direction by Keys (4)), Defender.update()

```

20     if "action" in kwargs.keys():
21         if kwargs["action"] == "move":
22             self.rect.move_ip(self.direction.elementwise() * self.speed *
23                               cfg.DELTATIME)
24             self.rect.clamp_ip(cfg.WINDOW)
25         elif "direction" in kwargs.keys():
26             self.direction = cfg.DIRECTIONS[kwargs["direction"]]
27         elif "speed" in kwargs.keys():
28             if kwargs["speed"] == "faster":
29                 self.speed = 300
30             elif kwargs["speed"] == "slower":
31                 self.speed = 10
32             elif kwargs["speed"] == "normal":
33                 self.speed = 100

```

Now we adapt `watch_for_events()`. To be able to detect simultaneous key presses, a slightly different mechanism must be used. Internally, a modifier bit is set. This bitmask must be checked using the [binary AND operation](#). In our example, this is done in line 68 using `KMOD_LSHIFT` and in line 70 using `KMOD_RSHIFT`.

event.mod

KMOD_LSHIFT
KMOD_RSHIFT

Listing 2.68: Control Direction by Keys (5)), Game.watch_for_events()

```

61     for event in pygame.event.get():
62         if event.type == pygame.QUIT:
63             self.running = False
64         elif event.type == pygame.KEYDOWN:
65             if event.mod == pygame.KMOD_NONE:           # No modifier keys
66                 self.defender.update(speed="normal")
67             else:
68                 if event.mod & pygame.KMOD_LSHIFT:      # Left shift pressed
69                     self.defender.update(speed="faster")
70                 if event.mod & pygame.KMOD_RSHIFT:      # Right shift pressed
71                     self.defender.update(speed="slower")
72             if event.key == pygame.K_ESCAPE:
73                 self.running = False
74             elif event.key == pygame.K_RIGHT:
75                 self.defender.update(direction="right")
76             elif event.key == pygame.K_LEFT:
77                 self.defender.update(direction="left")
78             elif event.key == pygame.K_UP:
79                 self.defender.update(direction="up")
80             elif event.key == pygame.K_DOWN:
81                 self.defender.update(direction="down")
82         elif event.type == pygame.KEYUP:
83             self.defender.update(speed="normal")
84             if event.key in (pygame.K_RIGHT, pygame.K_LEFT,
85                             pygame.K_UP, pygame.K_DOWN):
86                 self.defender.update(direction="stop")

```

Querying modifier keys

You cannot check the modifier keys directly. Instead, you have to use a binary AND operation to determine whether the corresponding bit is set in a mask or not.

In this way, the state of multiple modifier keys can be encoded and queried within a

single integer. For example, if you want to check whether the keys  and  are pressed at the same time, you can do so as follows:

```
69     if (event.mod & pygame.KMOD_SHIFT) and (event.mod & pygame.KMOD_ALT):
70         ...
```

Or

```
69     if (event.mod & (pygame.KMOD_SHIFT | pygame.KMOD_ALT)):
70         ...
```

As a complement, it should also be noted that `KMOD_NONE` can be used to check whether no modifier key is pressed at all (see line 65).

`KMOD-
_NONE`

2.6.2.2 In Which Window was the Key Pressed?

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/06%20Keyboard/keyboard/v03

When working with multiple windows, it is certainly important to determine in which window a keyboard input was made. There are many ways to achieve this, but the two approaches presented here are probably the most straightforward.

As a basis, we use the example from section 2.1.2.1 on page 21. Two windows are displayed next to each other.

The variables `window_first` and `window_second` are handle to the two windows. Internally, these handles are essentially memory addresses ([Random Access Memory \(RAM\)](#)) through which the windows can be accessed via their properties and methods. Naturally, this address is unique for each window.

When a keyboard event is triggered, the event also carries a handle to the window in which it occurred. This handle can be accessed via `event.window`, which is exactly what happens in line 26 and line 29. By comparing these handles, it is therefore possible to determine unambiguously in which window the event was triggered.

`event.win-
dow`

Alternatively, one can use the `id` attribute. This value is also unique for each window and reflects the order in which the windows were created.

`Window.id`

Listing 2.69: In which window was the key pressed?

```

4 def main():
5     pygame.init()
6     window_first = pygame.Window(size=(300, 50),
7         title="Main Window",
8         position=(500, 50))
9     window_second = pygame.Window(size=(300, 50),
10        title="Side Window",
11        position=(820, 50))
12     screen_first = window_first.get_surface()
13     screen_second = window_second.get_surface()
14     clock = pygame.time.Clock()
15
16
17     running = True
18     while running:
19         for event in pygame.event.get():
20             if event.type == pygame.QUIT:
21                 running = False
22             elif event.type == pygame.WINDOWCLOSE:
23                 running = False
24                 event.window.destroy()
25             elif event.type == pygame.KEYDOWN:
26                 if event.window == window_first:      # Check which window the event belongs to
27                     window_id = window_first.id
28                     event.window.title = "Main Window (Key Pressed: '" +
29                         pygame.key.name(event.key) + "')"
30                 elif event.window == window_second: #
31                     window_id = window_second.id
32                     event.window.title = "Side Window (Key Pressed: '" +
33                         pygame.key.name(event.key) + "')"
34                 else:
35                     window_id = None
36
37                 print(f"ID {window_id}: {event.window.title}")
38             if running:
39                 screen_first.fill((0, 255, 0))
40                 window_first.flip()
41                 screen_second.fill((255, 0, 0))
42                 window_second.flip()
43                 clock.tick(60)
44
45     pygame.quit()

```

Running the program produces the following console output when the corresponding keys are pressed:

```

1 ID 1: Main Window (Key Pressed: 'a')
2 ID 1: Main Window (Key Pressed: 's')
3 ID 1: Main Window (Key Pressed: '5')
4 ID 1: Main Window (Key Pressed: 'j')
5 ID 2: Side Window (Key Pressed: 'm')
6 ID 2: Side Window (Key Pressed: 'space')
7 ID 2: Side Window (Key Pressed: '3')
8 ID 2: Side Window (Key Pressed: '4')

```

2.6.2.3 Example: Visualizing the Keyboard

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/06%20Keyboard/typewriter

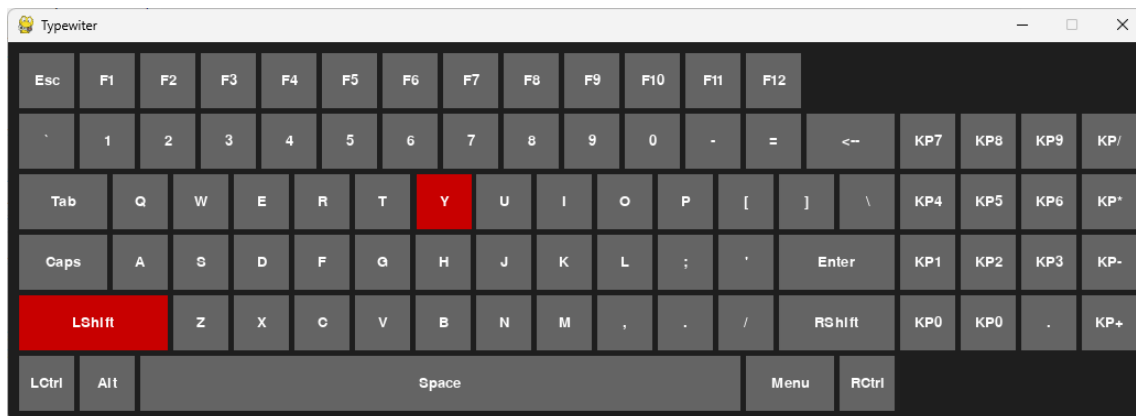


Figure 2.37: Typewriter

Just as a small exercise, let us write a simple program that visualizes a key press (see figure 2.37). For this purpose, I want to define the keyboard layout in `config.py` so that it can be easily adapted to different variants – here a simple U.S. keyboard layout.

Listing 2.70: Typewriter, `config.py`

```
0 import pygame
1
2 WINDOW = pygame.Rect((0, 0), (1035, 345))
3 FPS = 30
4
5 ROWS = [
6     ['Esc'] + [f'F{i}' for i in range(1, 13)],
7     ['`', '1', '2', '3', '4', '5', '6', '7', '8', '9', '0', '-', '=', '<--', 'KP7', 'KP8', 'KP9', 'KP/'],
8     ['Tab', 'Q', 'W', 'E', 'R', 'T', 'Y', 'U', 'I', 'O', 'P', '[', ']', '\\', 'KP4', 'KP5', 'KP6', 'KP*'],
9     ['Caps', 'A', 'S', 'D', 'F', 'G', 'H', 'J', 'K', 'L', ';', "'", 'Enter', 'KP1', 'KP2', 'KP3', 'KP-'],
10    ['LShift', 'Z', 'X', 'C', 'V', 'B', 'N', 'M', ',', '.', '/', 'RShift', 'KP0', 'KP0', '.', 'KP+'],
11    ['LCtrl', 'Win', 'Alt', 'Space', 'AltGr', 'Menu', 'RCtrl']
12 ]
13
14 KEY = {'width': 50, 'height': 50, 'spacing': 5}
```

The constructor of `KeySprite` receives its label (i.e., its meaning), its value according to table 2.7 on page 101, and its position. Based on this data, the sprite, the text label, its position, and its size are computed and processed.

A *normal* key has exactly the width and height defined in `config.py` by the dictionary

KEY. Therefore, in line 24 the width is multiplied by the factor 1, and no additional spacing is needed.

Other keys have different widths; for example, the `Space` key has a width of 10 keys. This is determined using a case distinction and stored in `factor_width`.

From this factor, we can also determine how many gaps between keys must be included in the width calculation. For a normal key, no additional gap is consumed. For two keys, there is exactly one gap between them; for three keys, there are two gaps, and so on. So the number of gaps is the number of normal key widths minus 1. However, since we also use factor values such as 1.5, we must always compute the next higher integer. The result is the calculation in line 26.

In the next line, the total width of the key can then be computed based on the two factors.

The attribute `pressed` is a flag that stores whether the key is currently pressed or not. After that, the rectangle is created using the given position, and the label is rendered. Text output using fonts will be explained in more detail in a later section.

In `update()`, the key is filled either gray or red. From a performance point of view, this is not ideal; strictly speaking, we would only need to redraw the key when the state of `pressed` changes, but let us keep it simple here for now.

Listing 2.71: Typewriter, Class `KeySprite`

```

8 class KeySprite(pygame.sprite.Sprite):
9     def __init__(self, label: str, key: int, left: int, top: int) -> None:
10         super().__init__()
11         self.font = pygame.font.SysFont(None, 20)
12         self.label = label
13         self.key = key
14         if label == 'Enter':
15             factor_width = 2.0
16         elif label == 'Tab' or label == 'Caps' or label == '<--' or label == 'Menu':
17             factor_width = 1.5
18         elif label == 'LShift':
19             factor_width = 2.5
20         elif label == 'RShift':
21             factor_width = 2.0
22         elif label == 'Space':
23             factor_width = 10.0
24         else:
25             # Normal key
26             factor_width = 1.0
27         factor_spacing = ceil(factor_width - 1) # Used space
28         width = int(factor_width * cfg.KEY['width'] + (factor_spacing * cfg.KEY['spacing']))
29         self.image = pygame.Surface((width, cfg.KEY['height']))
30         self.pressed = False
31         self.rect = self.image.get_rect(topleft=(left, top))
32         self.txt_surf = self.font.render(label, True, (255, 255, 255))
33         self.txt_rect = self.txt_surf.get_rect()
34         self.txt_rect.center = self.image.get_rect().center
35         self.update()
36
37     def update(self) -> None:
38         if self.pressed:
39             self.image.fill((200, 0, 0))
40             self.image.blit(self.txt_surf, self.txt_rect)
41         else:

```

```

41         self.image.fill((100, 100, 100))
42         self.image.blit(self.txt_surf, self.txt_rect)

```

The constructor and the `run()` method of `Game` should be self-explanatory. In the dictionary `keyboard`, all keys are stored using the identifiers defined in table 2.7 on page 101 as dictionary keys.

Listing 2.72: Typewriter, `Game.init()` and `Game.run()`

```

45 class Game(object):
46
47     def __init__(self) -> None:
48         pygame.init()
49         self.window = pygame.Window(size=cfg.WINDOW.size, title="Typewriter")
50         self.screen = self.window.get_surface()
51         self.clock = pygame.time.Clock()
52         self.running = False
53         self.all_sprites = pygame.sprite.Group()
54         self.keyboard = self.generate_sprites()
55
56     def run(self) -> None:
57         self.running = True
58         while self.running:
59             self.watch_for_events()
60             if self.running:
61                 self.update()
62                 self.draw()
63                 self.clock.tick(cfg.FPS)
64         pygame.quit()

```

The creation of this dictionary takes place in the method `generate_sprites()`. First, I define the starting position of the first key. It is placed in the upper-left corner and starts with twice the vertical key spacing – a purely arbitrary choice, but one that results in a visually pleasing layout.

Next, the keyboard layout defined in `config.py` is traversed using a nested loop. In line 125, the corresponding Pygame key value is determined from the key label. This value is used as the dictionary key in line 130, allowing it to be easily compared with and processed from the parameters of keyboard events later on.

For each key, a corresponding sprite is created. The new horizontal position `left` is calculated by adding the key spacing to the actual width of the key. Once all keys in a row have been processed, `left` is reset to its initial value and the vertical position `top` is shifted downward accordingly.

Finally, the generated dictionary is returned. Alternatively, it could have been stored directly as a class attribute; however, the approach chosen here keeps the method self-contained.

Listing 2.73: Typewriter, `Game.generate_sprites()`

```

120 def generate_sprites(self) -> dict[int, KeySprite]:
121     keyboard = {}
122     left = top = 2 * cfg.KEY['spacing']
123     for row in cfg.ROWS:

```

```

124         for label in row:
125             key = self.label2key(label)                # Get pygame key constant
126             if key is not None:
127                 keysprite = KeySprite(label, key, left, top)
128                 self.all_sprites.add(keysprite)
129                 left += keysprite.rect.width + cfg.KEY['spacing']
130                 keyboard[keysprite.key] = keysprite # Map key constant to sprite
131             left = 2 * cfg.KEY['spacing']
132             top += cfg.KEY['height'] + cfg.KEY['spacing']
133         return keyboard

```

The tricky part is the method `label2key()`. Its task is to determine the corresponding Pygame key code, as listed in table 2.7 on page 101, based on a given key label. For many keys, such as , we can directly use `pygame.key.key_code()` in line 116.

key_code()

For all other keys, we construct our own mapping tables. The underlying logic is more about Python than about Pygame itself and is therefore left to the interested reader to explore.

Listing 2.74: Typewriter, `Game.label2key()`

```

85     def label2key(self, label: str) -> int | None:
86         specials = {
87             'Space': pygame.K_SPACE,
88             'Enter': pygame.K_RETURN,
89             '<--': pygame.K_BACKSPACE,
90             'Tab': pygame.K_TAB,
91             'LShift': pygame.K_LSHIFT,
92             'LCtrl': pygame.K_LCTRL,
93             'RShift': pygame.K_RSHIFT,
94             'RCtrl': pygame.K_RCTRL,
95             'Alt': pygame.K_ALT,
96             'Caps': pygame.K_CAPSLOCK,
97             'Esc': pygame.K_ESCAPE,
98         }
99         for i in range(1, 13):
100             specials[f'F{i}'] = getattr(pygame, f'K_F{i}')
101
102         numpad = {
103             'KP+': pygame.K_KP_PLUS,
104             'KP-': pygame.K_KP_MINUS,
105             'KP*': pygame.K_KP_MULTIPLY,
106             'KP/': pygame.K_KP_DIVIDE,
107         }
108         for i in [0,1,2,3,4,5,6,7,8,9]:
109             numpad[f'KP{i}'] = getattr(pygame, f'K_KP{i}')
110
111         if label in specials:
112             return specials[label]
113         elif label in numpad:
114             return numpad[label]
115         try:
116             return pygame.key.key_code(label.lower()) # Alphanumeric keys
117         except Exception:
118             return None

```

The implementations of the methods `update()` and `draw()` are straightforward and require no further explanation.

Listing 2.75: Typewriter, `Game.update()` and `Game.draw()`

```

77 def update(self) -> None:
78     self.all_sprites.update()
79
80 def draw(self) -> None:
81     self.screen.fill((30, 30, 30))
82     self.all_sprites.draw(self.screen)
83     self.window.flip()

```

2.6.2.4 Not by Event, but by Function

Another way to find out which key was pressed or released

- 👉 `pygame.key.get_pressed()`
- 👉 `pygame.key.get_just_pressed()`
- 👉 `pygame.key.get_just_released()`

These functions each return a dictionary containing all available keys, where every entry is associated with a Boolean flag:

- the value is `True` if the key is currently pressed (or was released, depending on the function),
- otherwise it is `False`.

The *infix just* therefore refers to the time span within two frames.

So, if you want to check whether `K` is currently pressed, you can simply write something like:

```

1 if pygame.key.get_pressed()[pygame.K_k]:
2     ...

```

Often you will see constructions like this:

```

1 while pygame.key.get_pressed()[pygame.K_LEFT]:
2     ...

```

With this approach, actions inside the loop can react immediately to a key being held down – for example, *walking to the right*.

However, you then get *trapped* inside the loop. If the game is supposed to react to something else as well, or if other game objects need to be updated, this does not work.

In that case, it is better to query the keyboard once per frame:

```
1 keys = pygame.key.get_pressed()
2 if keys[pygame.K_LEFT]:
3     ...
```

This can be quite useful, but I have never really grown comfortable with this style of logic. Handling events as described above (see section 2.6.1 on page 89) does not trap me in loops and also feels “cleaner” to me, even though I cannot fully justify that feeling. One important note from the documentation: with this function, you cannot determine the order in which keys were used. This is only possible via event handling.

What was new?

The keyboard sends event messages that can be intercepted and evaluated. First, a distinction is made as to what kind of keyboard action occurred (`event.type`), and then which key was involved (`event.key`). Using `event.mod`, it is possible to query bitwise which modifier keys on the keyboard were used.



The following Pygame elements were introduced



-  `pygame.rect.FRect.clamp()`:
<https://pyga.me/docs/ref/rect.html#pygame.FRect.clamp>
-  `pygame.rect.FRect.clamp_ip()`:
https://pyga.me/docs/ref/rect.html#pygame.FRect.clamp_ip
-  `pygame.rect.Rect.clamp()`:
<https://pyga.me/docs/ref/rect.html#pygame.Rect.clamp>
-  `pygame.rect.Rect.clamp_ip()`:
https://pyga.me/docs/ref/rect.html#pygame.Rect.clamp_ip
-  `pygame.key`:
<https://pyga.me/docs/ref/key.html>
-  `pygame.key.get_pressed()`:
https://pyga.me/docs/ref/key.html#pygame.key.get_pressed
-  `pygame.key.get_just_pressed()`:
https://pyga.me/docs/ref/key.html#pygame.key.get_just_pressed
-  `pygame.key.get_just_released()`:
https://pyga.me/docs/ref/key.html#pygame.key.get_just_released

 `pygame.key.key_code()`:
https://pyga.me/docs/ref/key.html#pygame.key.key_code

 `pygame.KEYDOWN`, `pygame.KEYUP`:
<https://pyga.me/docs/ref/event.html>

Table 2.7: Predefined Keyboard Constants

Constant	Meaning	Description
K_BACKSPACE	\b	backspace
K_TAB	\t	tabulator
K_CLEAR		clear
K_RETURN	\r	return, enter
K_PAUSE		pause
K_ESCAPE	~[	escape
K_SPACE		space
K_EXCLAIM	!	exclaim
K_QUOTEDBL	"	double quote
K_HASH	#	hash
K_DOLLAR	\$	dollar
K_AMPERSAND	&	ampersand
K_QUOTE	'	quote
K_LEFTPAREN	(	left parenthesis
K_RIGHTPAREN	)	right parenthesis
K_ASTERISK	*	asterisk
K_PLUS	+	plus
K_COMMA	,	comma
K_MINUS	-	minus
K_PERIOD	.	period
K_SLASH	/	slash
K_0	0	0
K_1	1	1
K_2	2	2
K_3	3	3
K_4	4	4
K_5	5	5
K_6	6	6
K_7	7	7
K_8	8	8
K_9	9	9
K_COLON	:	colon

continued on next page

Table 2.7: Predefined Keyboard Constants (continued)

Constant	Meaning	Description
K_SEMICOLON	;	semicolon
K_LESS	<	less-than
K_EQUALS	=	equals
K_GREATER	>	greater-than
K_QUESTION	?	question mark
K_AT	@	at
K_LEFTBRACKET	[	left bracket
K_BACKSLASH	\	backslash
K_RIGHTBRACKET	]	right bracket
K_CARET	^	caret
K_UNDERSCORE	_	underscore
K_BACKQUOTE	`	grave
K_a	a	a
K_b	b	b
K_c	c	c
K_d	d	d
K_e	e	e
K_f	f	f
K_g	g	g
K_h	h	h
K_i	i	i
K_j	j	j
K_k	k	k
K_l	l	l
K_m	m	m
K_n	n	n
K_o	o	o
K_p	p	p
K_q	q	q
K_r	r	r
K_s	s	s
K_t	t	t
K_u	u	u
K_v	v	v
K_w	w	w
K_x	x	x
K_y	y	y
K_z	z	z
K_DELETE		delete

continued on next page

Table 2.7: Predefined Keyboard Constants (continued)

Constant	Meaning	Description
K_KP0		keypad 0
K_KP1		keypad 1
K_KP2		keypad 2
K_KP3		keypad 3
K_KP4		keypad 4
K_KP5		keypad 5
K_KP6		keypad 6
K_KP7		keypad 7
K_KP8		keypad 8
K_KP9		keypad 9
K_KP_PERIOD	.	keypad period
K_KP_DIVIDE	/	keypad divide
K_KP_MULTIPLY	*	Nummernfeld multiply
K_KP_MINUS	-	keypad minus
K_KP_PLUS	+	keypad plus
K_KP_ENTER	\r	keypad return, enter
K_KP_EQUALS	=	keypad equals
K_UP		up arrow
K_DOWN		down arrow
K_RIGHT		right arrow
K_LEFT		left arrow
K_INSERT		insert
K_HOME		home
K_END		end
K_PAGEUP		page up
K_PAGEDOWN		page down
K_F1		F1
K_F2		F2
K_F3		F3
K_F4		F4
K_F5		F5
K_F6		F6
K_F7		F7
K_F8		F8
K_F9		F9
K_F10		F10
K_F11		F11
K_F12		F12
K_F13		F13

continued on next page

Table 2.7: Predefined Keyboard Constants (continued)

Constant	Meaning	Description
K_F14		F14
K_F15		F15
K_NUMLOCK		numlock
K_CAPSLOCK		capslock
K_SCROLLLOCK		scrolllock
K_RSHIFT		right shift
K_LSHIFT		left shift
K_RCTRL		right control
K_LCTRL		left control
K_RALT		right alt
K_LALT		left alt
K_RMETA		right meta
K_LMETA		left meta
K_LSUPER		left Windows key
K_RSUPER		right windows key
K_MODE		mode shift/AltGr
K_HELP		help
K_PRINT		print screen
K_SYSREQ		sysreq
K_BREAK		break
K_MENU		menu
K_POWER		power
K_EURO	€	Euro
K_AC_BACK		Android back button

Table 2.8: Predefined Keyboard Modifier

Constant	Description
KMOD_NONE	no modifier keys pressed
KMOD_LSHIFT	left shift
KMOD_RSHIFT	right shift
KMOD_SHIFT	left or right shift or both
KMOD_LCTRL	left control
KMOD_RCTRL	right control
KMOD_CTRL	left or right control or both
KMOD_LALT	left alt
KMOD_RALT	right alt
KMOD_ALT	left or right alt or both

continued on next page

Table 2.8: Predefined Keyboard Modifier (continued)

Constant	Description
KMOD_LMETA	left meta
KMOD_RMETA	right meta
KMOD_META	left or right meta or both
KMOD_CAPS	caps lock
KMOD_NUM	num lock
KMOD_MODE	AltGr

2.6.3 Homework

1. Place a ball bitmap with a non-uniform shape in the center of the window. The ball starts with a radius of 100 px. Pressing  increases the size of the ball, and pressing  decreases it. If the ball touches the border of the window, its size must not be reduced any further. The minimum radius is 10 px.
2. Use the arrow keys to move the ball inside the window. The ball must not leave the window.
3. Using  +  rotates the ball by -90° , and  +  rotates it by 90° . The function you need is `pygame.transform.rotate()`.
4. Change the rotation from -90° to -10° and from 90° to 10° . Do you notice anything strange? Find out how to avoid this behavior. It is indeed a little tricky.

scale()

rotate()

2.7 Text output using fonts

2.7.1 Introduction

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/07%20Fonts



Figure 2.38: Simple text output using fonts

In many games, information is not shown only in a symbolic way on the playfield (for example, three little figures to represent three lives), but also as written text. One way to achieve this is by displaying text using installed fonts.

The basic idea is simple: first, a `Font` object is created (see line 12). Then this font is used to generate a `Surface` object that contains the text – the text is **rendered** onto a `Surface` object (see line 26). Once rendered, the text behaves like any other bitmap and can be blitted onto the screen.

render

Listing 2.76: Simple text output using fonts

```

5 def main():
6     pygame.init()
7     window = pygame.Window(size=config.WINDOW.size, title="Text with Fonts")
8     screen = window.get_surface()
9     clock = pygame.time.Clock()
10    all_sprites = pygame.sprite.Group()
11
12    font = pygame.font.SysFont(None, 24)                                # None -> default font
13    text = "This is an example of printing text using a font"
14
15    running = True
16    while running:
17        for event in pygame.event.get():
18            if event.type == pygame.QUIT:
19                running = False
20            elif event.type == pygame.KEYDOWN:
21                if event.key == pygame.K_ESCAPE:
22                    running = False
23        all_sprites.update()
24        screen.fill("white")
25        # Render and center the text

```

```

26     text_surface = font.render(text, True, (0, 0, 0))           # Render as usual
27     text_rect = text_surface.get_rect(center=(cfg.WINDOW.width // 2, cfg.WINDOW.height
28         // 2))
29     screen.blit(text_surface, text_rect)
30     all_sprites.draw(screen)
31     window.flip()
    clock.tick(cfg.FPS)

```

But how does the constructor of `Font` know, which font I want? In line 12 we use the predefined default font. `pygame.font.SysFont()` takes either the name of a font or `None` as the first argument. In the latter case, the default font is used. Since Pygame always ships with a default font, this call will always return a valid font object. However, there is no guarantee that it will always be the same font, as the default font itself could change.

SysFont()

The method `pygame.font.get_default_font()` returns a unique string – the internal name of the font. You can find in section 2.7.2.2 on page 110 a program, which shows you all installed fonts and there internal names.

get_de-
fault_font()

If you want to use a specific font, you can get all required information with `pygame.font.match_font()`:

match_font()

```

14     font = pygame.font.Font(pygame.font.match_font("arial"), 24)

```

or

```

14     font = pygame.font.SysFont("Arial", 24)

```

Don't waste computing time

It is a waste of computing time to render the text every frame. Try to cache the rendered `Surface-Object` and use some kind of dirty-flag to indicate, if the text was updated.

2.7.2 More Input

2.7.2.1 A More Sophisticated Approach



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/07%20Fonts

For a small example, I have wrapped this functionality into a class. You can easily extend it, modify it, or adapt it to your own needs.

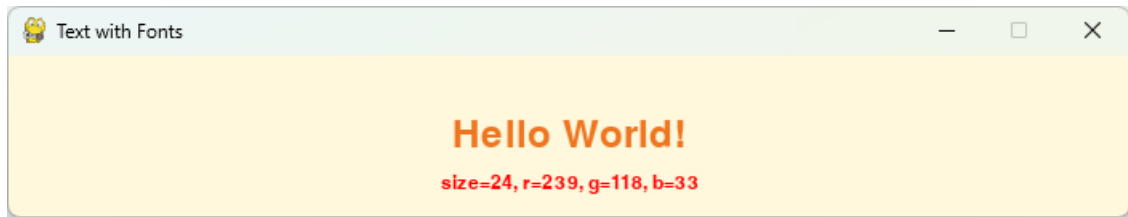


Figure 2.39: Text output using fonts

Listing 2.77: Text output using fonts (1), config.py

```

1 import pygame
2
3 WINDOW = pygame.Rect((0, 0), (700, 100))
4 FPS = 60

```

Font

And now the class `TextSprite`: do not let the OO approach confuse you. In fact, everything is quite simple. We need a `pygame.font.Font` object. This object, in turn, requires two pieces of information: which installed font it should use, and the font size in *pt*.

get_default_font()

One way to obtain an installed font is the method `pygame.font.get_default_font()`. Its call in line 34 returns the default font configured by the operating system. The font size (`fontsize`) can then be chosen freely according to our needs.

Listing 2.78: Text output using fonts (2), TextSprite

```

8 class TextSprite(pygame.sprite.Sprite):
9     def __init__(self, fontsize: int, fontcolor: list[int],
10                 center: Tuple[int, int], text: str = "Hello World!") -> None:
11         super().__init__()
12         self.image = None
13         self.rect = None
14         self.fontsize = fontsize
15         self.fontcolor = fontcolor
16         self.fontsize_update(0)      # 0!
17         self.text = text
18         self.center = center
19         self.dirty = True
20         self.render()                #
21
22     def render(self) -> None:
23         self.image = self.font.render(self.text, True, self.fontcolor) # Bitmap
24         self.rect = self.image.get_rect()
25         self.rect.center = self.center
26         self.dirty = False
27
28     def text_update(self, text: str) -> None:
29         self.text = text
30         self.dirty = True
31
32     def fontsize_update(self, step: int = 1) -> None:
33         self.fontsize += step
34         self.font = pygame.font.SysFont(None, self.fontsize) #
35         self.dirty = True
36

```

```

37     def fontcolor_update(self, delta: Tuple[int, int, int]) -> None:
38         for i in range(3):
39             self.fontcolor[i] = (self.fontcolor[i] + delta[i]) % 256
40             self.dirty = True
41
42     def update(self) -> None:
43         if self.dirty:
44             self.render()

```

Let us now take a closer look at the constructor. The attributes `image` and `rect` are initially created as dummy values; strictly speaking, this would not even be necessary. After storing the passed information about font size and font color in attributes, the `Font` object can be created. This is done by calling `fontsize_update()` in line 16. Passing the value 0 makes it clear that the size itself is not meant to be changed here, but that the object creation should take place.

Next, the actual text that is to be rendered as a label is stored, as well as the position where the center of the text should be placed. At this point, all required information is available, and by calling `render()` in line 20, the `Surface` object is created using `pygame.font.render()` (line 23). Afterwards, the rectangle of the bitmap is determined and its center is moved to the desired position.

`render()`

Finally, there are the two methods `fontsize_update()` and `fontcolor_update()`. Both allow the font size and font color to be changed at runtime. Their semantics should be self-explanatory.

How can such a class be used in practice? Here is a simple example. A greeting is displayed in the center using the object `hello` (line 53). Below it, the object `info` displays which font size and font color were used to render the greeting (line 53).

Listing 2.79: Text output using fonts (3), Main program

```

47 def main():
48     pygame.init()
49     window = pygame.Window(size=config.WINDOW.size, title="Text with Fonts", position=(10, 50))
50     screen = window.get_surface()
51     clock = pygame.time.Clock()
52
53     hello = TextSprite(24, [255, 255, 255], (config.WINDOW.center)) #
54     info = TextSprite(12, [255, 0, 0], (config.WINDOW.centerx, config.WINDOW.bottom - 20)) #
55         Fontinfo
56     all_sprites = pygame.sprite.Group()
57     all_sprites.add(hello, info)
58
59     running = True
60     while running:
61         for event in pygame.event.get():
62             if event.type == pygame.QUIT:
63                 running = False
64             elif event.type == pygame.KEYDOWN:
65                 if event.key == pygame.K_ESCAPE:
66                     running = False
67                 elif event.key == pygame.K_KP_PLUS or event.key == pygame.K_PLUS: # Bigger
68                     hello.fontsize_update(+1)
69                 elif event.key == pygame.K_KP_MINUS or event.key == pygame.K_MINUS: # Smaller
70                     hello.fontsize_update(-1)
71                 elif event.key == pygame.K_r:

```

```

71         if event.mod & pygame.KMOD_SHIFT:
72             hello.fontcolor_update((-1, 0, 0)) # Less red
73         else:
74             hello.fontcolor_update((+1, 0, 0)) # More red
75     elif event.key == pygame.K_g:
76         if event.mod & pygame.KMOD_SHIFT:
77             hello.fontcolor_update((0, -1, 0)) # Less green
78         else:
79             hello.fontcolor_update((0, +1, 0)) # More green
80     elif event.key == pygame.K_b:
81         if event.mod & pygame.KMOD_SHIFT:
82             hello.fontcolor_update((0, 0, -1)) # Less blue
83         else:
84             hello.fontcolor_update((0, 0, +1)) # More blue
85
86     info.text_update(f"size={hello.fontsize}, r={hello.fontcolor[0]},
87                      g={hello.fontcolor[1]}, b={hello.fontcolor[2]}")
88     all_sprites.update()
89     screen.fill("cornsilk1")
90     all_sprites.draw(screen)
91     window.flip()
92     clock.tick(cfg.FPS)

```

The greeting can be resized using  and  (line 66 ff.). The keys , , and  are used to manipulate the corresponding color channel. An uppercase letter increases the value (for example in line 72), while the lowercase letter decreases it (for example in line 74).

In figure 2.39 on page 108 you can see one possible visual result.

2.7.2.2 List of all Installed Fonts

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/07%20Fonts/fontslist

As another example, I would like to show you a small program that lists all installed fonts. This may be useful for getting ideas for visual design. The first part of the code should not cause any problems in terms of understanding.



Figure 2.40: List of all installed fonts

Listing 2.80: List of all installed fonts (1), config.py

```

1 import pygame
2
3 WINDOW = pygame.Rect((0, 0), (900, 300))
4 FPS = 15

```

The class `TextSprite` was customized a little bit, but has still the same logic.

Listing 2.81: List of all installed fonts (2), TextSprite

```

6 class TextSprite(pygame.sprite.Sprite):
7     def __init__(self, fontname: str, fontsize: int = 24, fontcolor: list[int] = [255, 255,
8         255], text: str = "") -> None:
9         super().__init__()
10        self.image = None
11        self.fontname = fontname
12        self.fontsize = fontsize
13        self.fontcolor = fontcolor
14        self.fontsize_update(0)
15        if text == "":
16            self.text = f"{self.fontname}: abcdefghijklmnopqrstuvwxyzßöäü0123456789"
17        else:
18            self.text = text
19            self.render()
20
21    def render(self) -> None:
22        self.image = self.font.render(self.text, True, self.fontcolor)
23        self.rect = self.image.get_rect()
24
25    def fontsize_update(self, step: int = 1) -> None:
26        self.fontsize += step
27        self.font = pygame.font.Font(pygame.font.match_font(self.fontname), self.fontsize) #
28
29    def fontcolor_update(self, delta: list[int]) -> None:
30        for i in range(3):
31            self.fontcolor[i] = (self.fontcolor[i] + delta[i]) % 256
32
33    def update(self) -> None:
34        self.render()

```

The class `BigImage` is responsible for managing all `FontSprite` images as one large bitmap. Later on, only a subsection of this bitmap is blitted onto the screen. This subsection depends on the current position within the list and is controlled by the attribute `offset`, which is updated in the method `update()` (line 46).

First, it is checked whether the upper or lower end of the bitmap has been reached. If this is the case, `top` or `bottom` is set accordingly, so that the entire screen is always filled. Otherwise, the `offset` rectangle is shifted up or down, and the corresponding subsection is determined using `pygame.Surface.subsurface()`.

subsurface()

Listing 2.82: List of all installed fonts (3), `BigImage`

```

36 class BigImage(pygame.sprite.Sprite):
37     def __init__(self):
38         super().__init__()
39         self.offset = pygame.Rect(cfg.WINDOW)
40
41     def create_image(self, width: int, height: int) -> None:
42         self.image_total = pygame.Surface((width, height))
43         self.image_total.fill("white")
44         self.update(0)
45
46     def update(self, delta: int) -> None: # Compute offset
47         if self.offset.top + delta >= 0:
48             if self.offset.bottom + delta <= self.image_total.get_rect().height:
49                 self.offset.move_ip(0, delta)
50             else:
51                 self.offset.bottom = self.image_total.get_rect().height
52         else:
53             self.offset.top = 0
54         self.image = self.image_total.subsurface(self.offset)
55         self.rect = self.image.get_rect()

```

And now the main program. In the first part, a list of all installed font names is obtained via `pygame.font.get_fonts()` (line 64). Each of these names is passed to the constructor of `TextSprite`.

get_fonts()

match_font()

Using the method `pygame.font.match_font()` (line 26), the actual font file is then searched for on the system. This method takes advantage of the fact that the font file name can usually be derived from the font name and the file extension `.ttf`.

Listing 2.83: List of all installed fonts (4), `main()`

```

58 def main():
59     pygame.init()
60     window = pygame.Window(size=cfg.WINDOW.size, title="List of all Installed Fonts",
61                             position=(10, 50))
62     screen = window.get_surface()
63     clock = pygame.time.Clock()
64
65     fonts = pygame.font.get_fonts() # All installed Fonts
66
67     list_of_fontsprites = pygame.sprite.Group()
68     height = 0
69     width = 0
70     for name in sorted(fonts):
71         try:
72             t = TextSprite(name, 24, "black")

```

```

72         t.rect.top = height
73         height += t.rect.height
74         width = t.rect.width if t.rect.width > width else width
75         list_of_fontsprites.add(t)
76     except OSError as err:
77         print(f"OS error {err}")
78     except pygame.error as perr:
79         print(f"Pygame error: {perr} with font {name}")
80
81 bigimage = pygame.sprite.GroupSingle(BigImage())
82 bigimage.sprite.create_image(width, height)
83 list_of_fontsprites.draw(bigimage.sprite.image_total) #
84
85 running = True
86 while running:
87     for event in pygame.event.get():
88         if event.type == pygame.QUIT:
89             running = False
90         elif event.type == pygame.KEYDOWN:
91             if event.key == pygame.K_ESCAPE:
92                 running = False
93             if event.key == pygame.K_UP:
94                 bigimage.update(-cfg.WINDOW.height // 2)
95             if event.key == pygame.K_DOWN:
96                 bigimage.update(cfg.WINDOW.height // 2)
97
98         bigimage.draw(screen)
99         window.flip()
100         clock.tick(cfg.FPS)
101
102 pygame.quit()

```

In the for-loop, TextSprite objects are now created for all fonts, and their height and width are determined. All of these individual bitmaps are then blitted onto the large bitmap (line 83).

The main loop is now only responsible for scrolling (each time by one third of the screen height) and for terminating the program.

2.7.2.3 Using Locally Installed Fonts



So ... where are the files?

- https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/07%20Fonts/localfont
- https://github.com/adamsralf/pygame_book/blob/main/src/00%20Introduction/07%20Fonts/localfont/fonts/Alex%20Winterbottom%20License.txt

In almost all aspects, using local fonts is identical to using system fonts. So what does this distinction actually mean? System fonts are fonts that have been registered and installed in the operating system via an installation process. Local fonts, on the other



Figure 2.41: Example of using a locally installed font

hand, are font files – such as `.ttf` files – that are stored in a subdirectory of the game itself. In our example, this is the file `rothenbg.ttf` located in the `fonts` subdirectory.

There are a few practical reasons why local fonts are often preferred in games. By shipping a font file together with the game, you ensure that the visual appearance of the text is identical on all systems, independent of which fonts are installed on the player's operating system.

In the example above (source code 2.78 on page 108, line 34), the constructor of `Font` was informed which system font to use via the function `get_default_font()`. In source code 2.81 on page 111, this was done by specifying a name known to the system – that is, a string under which the font is registered in the operating system.

In the example source code 2.84, however, a file name including a relative path is passed directly to the constructor of `Font`. In this case, `Font` looks for the corresponding file and uses it to build its `Font` object (see line 14).

From the programmer's point of view, the difference between system fonts and local fonts is very small. In practice, it only affects how the constructor of `Font` is called. Once the `Font` object has been created, rendering and using text works in exactly the same way for both system and local fonts.

When using local fonts, it is important to pay attention to licensing. Not every font may be freely redistributed. Before including a font file in your project, you should always check whether its license allows redistribution as part of a game or application. Our example was taken from <https://www.fontsquirrel.com/fonts/list/tag/historical> and was kindly made available as freeware by Alex Winterbottom.

Listing 2.84: Using locally installed fonts

```

7 def main():
8     pygame.init()
9     window = pygame.Window(size=cfg.WINDOW.size, title="Text with Local Fonts")
10    screen = window.get_surface()
11    clock = pygame.time.Clock()
12    all_sprites = pygame.sprite.Group()
13
14    font = pygame.font.Font("./fonts/rothenbg.ttf", 24)    # Adjust the path as necessary
15    text = "This is an example of printing text using a locally installed font"
16

```

```

17     running = True
18     while running:
19         for event in pygame.event.get():
20             if event.type == pygame.QUIT:
21                 running = False
22             elif event.type == pygame.KEYDOWN:
23                 if event.key == pygame.K_ESCAPE:
24                     running = False
25         all_sprites.update()
26         screen.fill("white")
27         # Render and center the text
28         text_surface = font.render(text, True, (0, 0, 0))      # Render as usual
29         text_rect = text_surface.get_rect(center=(cfg.WINDOW.width // 2, cfg.WINDOW.height
30             // 2))
31         screen.blit(text_surface, text_rect)
32         all_sprites.draw(screen)
33         window.flip()
34         clock.tick(cfg.FPS)
35     pygame.quit()

```

In figure 2.41 on the facing page you can admire the result ;-)

2.7.2.4 Text output



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/07%20Fonts/textbitmaps

Text output is often not done via fonts, but via a [SpriteLib](#). Such a library contains character sprites, symbols, or digits, usually in a special design that matches the style of the game. In figure 2.42 on the next page you can see a spritelib that provides sprites for a World War II dogfight game. Among other things, it contains the sprites for the digits 0–9 and the letters of the Latin alphabet.

One advantage of this approach is that you do not have to rely on a specific game font being available on the target system. If you render text using a font such as *Calibri*, that font must be installed on the player's computer.

A disadvantage is that bitmaps usually scale poorly, so you often do not have many different font sizes available.

The idea is to *punch out* the individual letters from the spritelib and store them in a suitable data structure. Whenever text needs to be displayed, the string is split into its characters, and the corresponding letter sprites are blitted from the data structure onto a target bitmap – for example onto the screen.

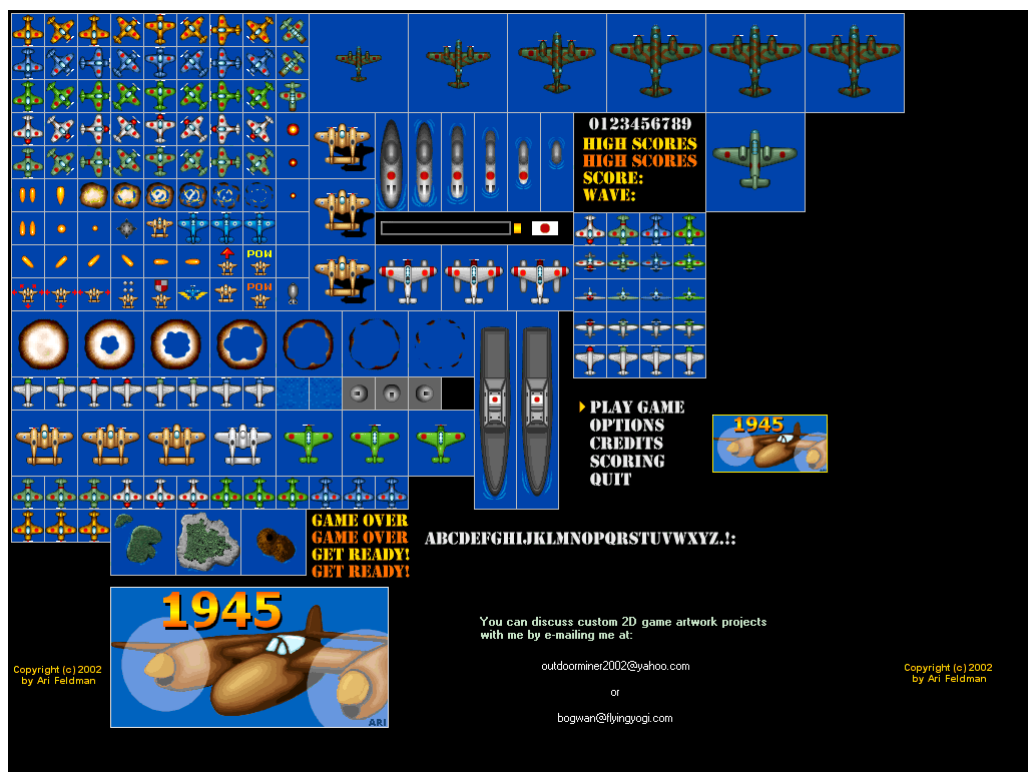


Figure 2.42: Example of a spritelib

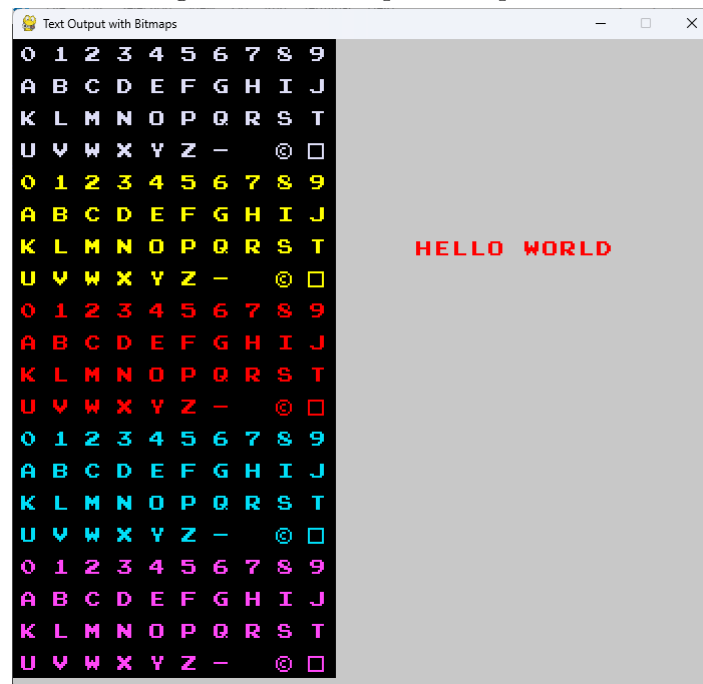


Figure 2.43: Text output using bitmaps

I will demonstrate this with a simple example. Our starting point is a spritelib that contains a character set in five different colors (see figure 2.43 on the facing page).

The first part of source code 2.85 should look familiar and is only extended by a few convenience features. The file paths are now determined via the functions `filepath()` and `imagepath()`.

Listing 2.85: Textbitmaps (1), `config.py`

```

1 import os
2
3 import pygame
4
5 WINDOW = pygame.Rect((0, 0), (700, 650))
6 PATH: dict[str, str] = {}
7 PATH["file"] = os.path.dirname(os.path.abspath(__file__))
8 PATH["image"] = os.path.join(PATH["file"], "images")
9 FPS = 60
10
11 def filepath(name: str) -> str:
12     return os.path.join(PATH["file"], name)
13
14 def imagepath(name: str) -> str:
15     return os.path.join(PATH["image"], name)

```

The class `Spritelib` is mainly used as a container. It loads the spritelib containing the letters and symbols and stores several parameters that are needed to extract individual letters or symbols precisely from the bitmap:

- **nof**: Stores the number of rows and columns. In our case, the symbol set is arranged in the bitmap in 4 rows and 10 columns. Since we are only interested in one color at a time, this information is sufficient.
- **letter**: Each sprite has a fixed width and height. In our case, this is particularly convenient because all sprites have the same dimensions. Take a look at the three squares around the letters N, W, and X in figure 2.44 on the following page. All sprites have a width and a height of 18 px.
- **offset**: The first sprite in the top-left corner has a distance from the left edge and from the top edge of the bitmap. This can be seen clearly for the sprite of the digit 0 in figure 2.44. There is a square around the bitmap and a gap between this square and the upper and left edges (marked by the green line). In our example, both offsets have a value of 6 px.
- **distance**: Each sprite has a fixed distance to the next sprite to the right and to the one below. Fortunately, the sprites in our spritelib are arranged equidistantly, which simplifies things a lot. Using the sprite for X in figure 2.44 as an example, you can see these distances. In our case, they are 14 px each.

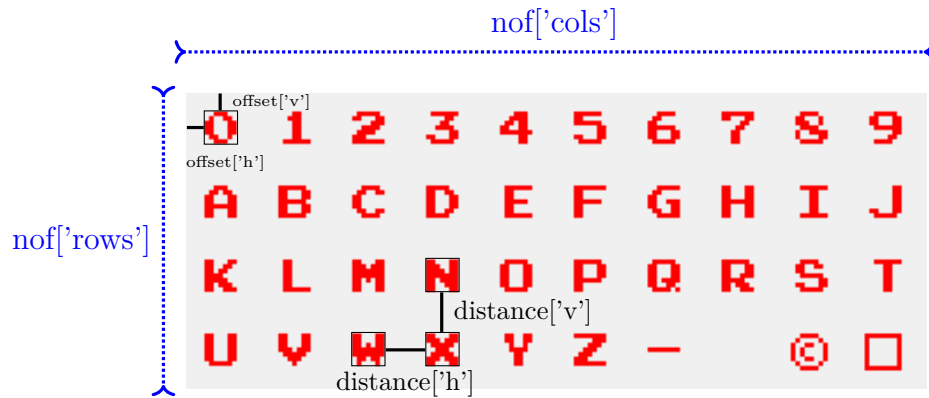


Figure 2.44: Meaning of the variables in Spritelib

Listing 2.86: Textbitmaps (2), Spritelib

```

6 class Spritelib(pygame.sprite.Sprite):
7
8     def __init__(self, filename: str) -> None:
9         super().__init__()
10        self.image = pygame.image.load(cfg.imagepath(filename)).convert()
11        self.rect = self.image.get_rect()
12        self.nof = {"rows": 4, "cols": 10}
13        self.letter = {"width": 18, "height": 18}
14        self.offset = {"h": 6, "v": 6}
15        self.distance = {"h": 14, "v": 14}
16
17    def draw(self, screen: pygame.Surface) -> None:
18        screen.blit(self.image, self.rect)

```

Let us now move on to the actually interesting class: **Letters**. This class cuts out all sprites of a single color from the **spritelib** and makes them available as **Surface** objects in a **dictionary**. This involves quite a bit of calculation, but do not let that intimidate you; in the end, it is nothing more than basic arithmetic.

Dictionary

Let us start with the constructor. The constructor has two parameters. The first parameter, **spritelib**, is a reference to the **Spritelib** object, which has loaded the original bitmap and provides several spacing and layout parameters. The second parameter, **colnumber**, allows us to extract the complete set of symbols for exactly one color later on: 0 stands for the white sprites, 1 for the yellow ones, and so on.

Listing 2.87: Textbitmaps (3), Constructor of Letters

```

21 class Letters(object):
22
23     def __init__(self, spritelib: Spritelib, colnumber: int) -> None:
24         super().__init__()
25         self.spritelib = spritelib
26         self.letters: dict[str, pygame.Surface] = {}
27         self.create_letter_bitmap(colnumber)

```

In the method `create_letter_bitmap()`, the individual sprites are now cut out and stored in a dictionary. The indices of this dictionary are defined in line 71. Here, the order must of course match the order in which the sprites are cut out. The variable `index` ensures exactly this: with each loop iteration, the next `lettername` is used as the key for the dictionary.

In line 76, the position – i.e. the pixel coordinates – of the first sprite is calculated. Try to follow the arithmetic yourself using the information given in figure 2.44 on the preceding page! Don't worry: it is not difficult, just a bit lengthy.

Starting at line 77, a nested `for`-loop begins. The outer loop iterates over all rows of the `spritelib`, and the inner loop over the columns. The goal of this construction is to create one `Rect` object for each sprite, in which the position and size of the sprite are stored. In line 78, the top coordinate is calculated, and in line 80 the left coordinate. If you have understood line 76, these two calculations should no longer pose any problems.

Height and width in line 80 are straightforward, since all sprites always have the same dimensions. After that, the `Rect` object is created and used to cut out the bitmap with the help of `subsurface()`. This extracted bitmap is then stored in the dictionary under its symbol name.

Listing 2.88: Textbitmaps (4): `create_letter_bitmap()` von `Letters`

```

29  def create_letter_bitmap(self, colornumber: int):
30      lettername = (
31          "0",
32          "1", # The rows between 34 and 65 are skipped!!
33          "2",
34          "z",
35          "-",
36          " ",
37          "copy",
38          "square",
39      ) #
40      index = 0
41      startpos = (
42          self.spritelib.offset["h"],
43          self.spritelib.offset["v"] + colornumber * self.spritelib.nof["rows"] *
44              (self.spritelib.letter["height"] + self.spritelib.distance["v"]),
45      ) #
46      for row in range(self.spritelib.nof["rows"]): # Rows
47          for col in range(self.spritelib.nof["cols"]): # Columns
48              left = startpos[0] + col * (self.spritelib.letter["width"] +
49                  self.spritelib.distance["h"]) #
50              top = startpos[1] + row * (self.spritelib.letter["height"] +
51                  self.spritelib.distance["v"]) #
52              width = self.spritelib.letter["width"] # Size
53              height = self.spritelib.letter["height"]
54              r = pygame.Rect(left, top, width, height)
55              self.letters[lettername[index]] = self.spritelib.image.subsurface(r) #
56              index += 1

```

The method `get_text()` finally returns the matching sequence of bitmap sprites for a given text. To do this, it uses the method `get_letter()`, which is necessary so that the program does not crash when an undefined letter or symbol is used. For example, if you type an ü, a square placeholder will be displayed.

Listing 2.89: Textbitmaps (5), get_letter() and get_text() von Letters

```

87 def get_letter(self, letter: str) -> pygame.Surface:
88     if letter in self.letters:
89         return self.letters[letter]
90     else:
91         return self.letters["square"]
92
93 def get_text(self, text: str) -> pygame.Surface:
94     l = len(text) * self.spritelib.letter["width"]
95     h = self.spritelib.letter["height"]
96     bitmap = pygame.Surface((l, h))
97     bitmap.set_colorkey((0, 0, 0))
98     for a in range(len(text)):
99         bitmap.blit(self.get_letter(text[a]), (a * self.spritelib.letter["width"], 0))
100    return bitmap

```

The actual main program is encapsulated in the class `TextBitmaps`. Since the source code does not introduce anything fundamentally new, it should be largely self-explanatory. However, I would like to take a closer look at two specific lines:

- line 123: Here, [slicing of arrays](#) is used. The value `-1` causes the end index of the slice to start at the last element and then move one step to the left. The result is a new string that is shortened by its last character.
- line 125: The attribute `unicode` provides, where applicable, the value of the pressed key in [unicode](#) format. This means that meaningful letters, digits, and similar characters are added directly to the string.

unicode

Listing 2.90: Textbitmaps (6), TextBitmaps

```

103 class TextBitmaps(object):
104
105     def __init__(self) -> None:
106         pygame.init()
107         self.window = pygame.Window(size=cfg.WINDOW.size, title="Text Output with Bitmaps",
108                                     position=(10, 50))
109         self.screen = self.window.get_surface()
110         self.clock = pygame.time.Clock()
111
112         self.filename = "chars.png"
113         self.running = False
114         self.input = ""
115
116     def watch_for_events(self) -> None:
117         for event in pygame.event.get():
118             if event.type == pygame.QUIT:
119                 self.running = False
120             elif event.type == pygame.KEYDOWN:
121                 if event.key == pygame.K_ESCAPE:
122                     self.running = False
123                 elif event.key == pygame.K_BACKSPACE:
124                     self.input = self.input[:-1] # Remove last character
125                 else:
126                     self.input += event.unicode # Keyboard input as Unicode character
127
128     def run(self) -> None:
129         spritelib = Spritelib(self.filename)
130         letters = Letters(spritelib, 2)
131         self.running = True

```

```

131     while self.running:
132         self.watch_for_events()
133         self.screen.fill((200, 200, 200))
134         self.screen.blit(letters.get_text(self.input), (400, 200))
135         spritelib.draw(self.screen)
136         self.window.flip()
137         self.clock.tick(cfg.FPS)
138
139     pygame.quit()

```

2.7.3 What was new?

- 👉 To produce text output, you can use either system-installed fonts or local font files. In the first step, a suitable font object is created. In the second step, this object is used to render a given text into a bitmap — a `Surface` object. This bitmap can then be blitted to the desired position just like any other `Surface` object.
- 👉 But Text output is not only created using fonts, but also by means of `spritelibs` that contain character bitmaps. These bitmaps are cut out and then assembled into new composite bitmaps.



The following Pygame elements were introduced



- 🎮 `pygame.event.Event.unicode`:
<https://pyga.me/docs/ref/event.html>
- 🎮 `pygame.font.Font`:
<https://pyga.me/docs/ref/font.html>
- 🎮 `pygame.font.Font.render()`:
<https://pyga.me/docs/ref/font.html#pygame.font.Font.render>
- 🎮 `pygame.font.get_default_font()`:
https://pyga.me/docs/ref/font.html#pygame.font.get_default_font
- 🎮 `pygame.font.get_fonts()`:
https://pyga.me/docs/ref/font.html#pygame.font.get_fonts
- 🎮 `pygame.font.match_font()`:
https://pyga.me/docs/ref/font.html#pygame.font.match_font
- 🎮 `pygame.font.SysFont()`:
<https://pyga.me/docs/ref/font.html#pygame.font.SysFont>

```
pygame.Surface.subsurface():  
https://pyga.me/docs/ref/surface.html#pygame.Surface.subsurface
```

```
+ pygame.Surface.subsurface():  
https://pyga.me/docs/ref/surface.html#pygame.Surface.subsurface
```

2.7.4 Homework

Create a program that simulates a analog clock. The program contains the following status variables: number of lives, score, high score, the number of seconds since the program started, and a game title. Please position the information inside the window as follows:

- Title line: on the left the current date, in the center the game title, and on the right the current time
- Status line at the very bottom: on the left the number of lives, in the center score/high score, and on the right the number of seconds since the game started

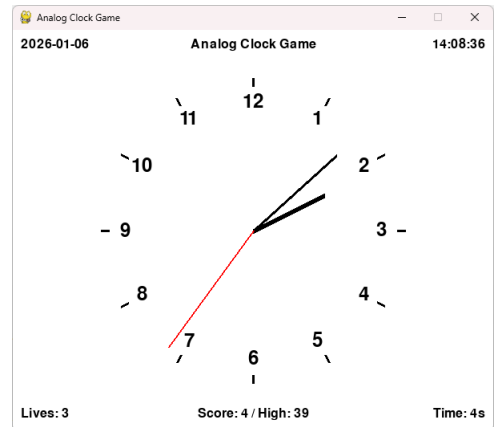


Fig. 2.45: Clock

The values should update dynamically. The score increases by 1 every second. This score is written to the file `highscore.txt` if it is greater than the current high score. Date, time, and elapsed time are obtained from suitable functions of the `time` module. Draw an analog clock in the center with second hand and dial like in figure 2.45.

2.8 Collision Detection

2.8.1 Introduction

Collision detection is used very often in game programming: characters must not walk through obstacles, projectiles hit targets, balls bounce off walls, and so on. For this reason, Pygame provides a whole variety of collision detection methods:

- **Rectangle overlap:** When we looked at the `Sprite` class, we already saw that the attribute `rect` is required. It contains the position and size of the surrounding rectangle. If two sprites meet, it is checked whether their rectangles overlap. This is a very *cheap* detection method, because only a few comparisons are needed to decide whether two rectangles touch or overlap. This method does not consider the actual shape of the sprite, only its bounding rectangle. Here is an example implementation:

```

1 def rectangle_collision(rect1, rect2):
2     return rect1.left < rect2.right and
3         rect2.left < rect1.right and
4         rect1.top < rect2.bottom and
5         rect2.top < rect1.bottom

```

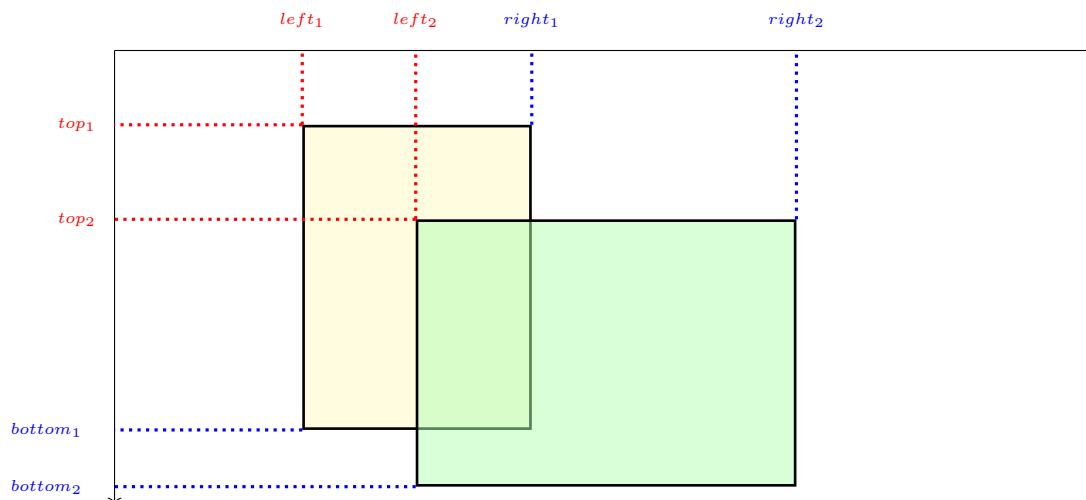


Figure 2.46: Collision detection with rectangles

- **Circle overlap:** For rather round sprites, it is recommended not to check rectangles, but to use an bounding circle for collision detection instead. This collision test is also quite fast, because only the distance between the centers has to be compared: $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} < r_1 + r_2$. For performance reason the check is usually computed as: $(x_2 - x_1)^2 + (y_2 - y_1)^2 < (r_1 + r_2)^2$

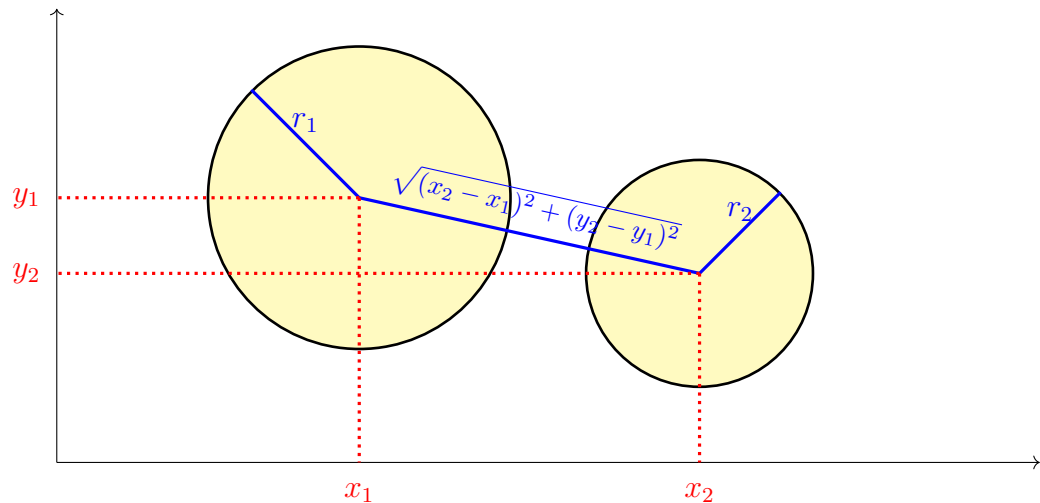


Figure 2.47: Collision detection with circles

- **Pixel overlap:** In pixel-perfect collision detection, every pixel of both sprites is checked to see whether they occupy the same position. If *yes*, the sprites overlap; if *no*, they do not. This is the most expensive collision test, but also the most accurate one.

To reduce the computational effort, the intersection rectangle of the two sprites is determined first. As with rectangle collision detection, it is first checked whether the two rectangles overlap at all. If they do not, the test can stop immediately. If they do, the intersection of the two rectangles is itself a rectangle (see figure 2.48).

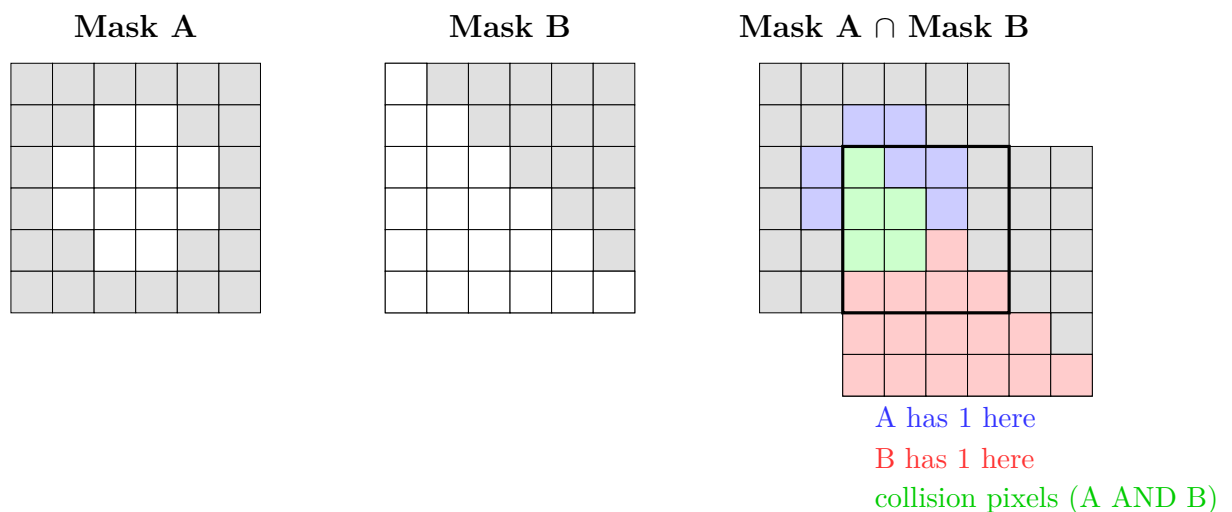


Figure 2.48: Collision detection using masks

If two pixels have the same position, they must lie inside this intersection rectangle.

Therefore, the pixel test can be limited to this usually much smaller area (see figure 2.49).

Another problem with pixel-perfect collision detection is distinguishing background from foreground. How should the collision test know whether a blue pixel belongs to the object or to the background? There are several approaches to this problem. The simplest one is to create a black-and-white image for each sprite (a [mask](#)); the white pixels are relevant, while the black pixels can be ignored. The pixel collision test is then performed only on these masks.

mask

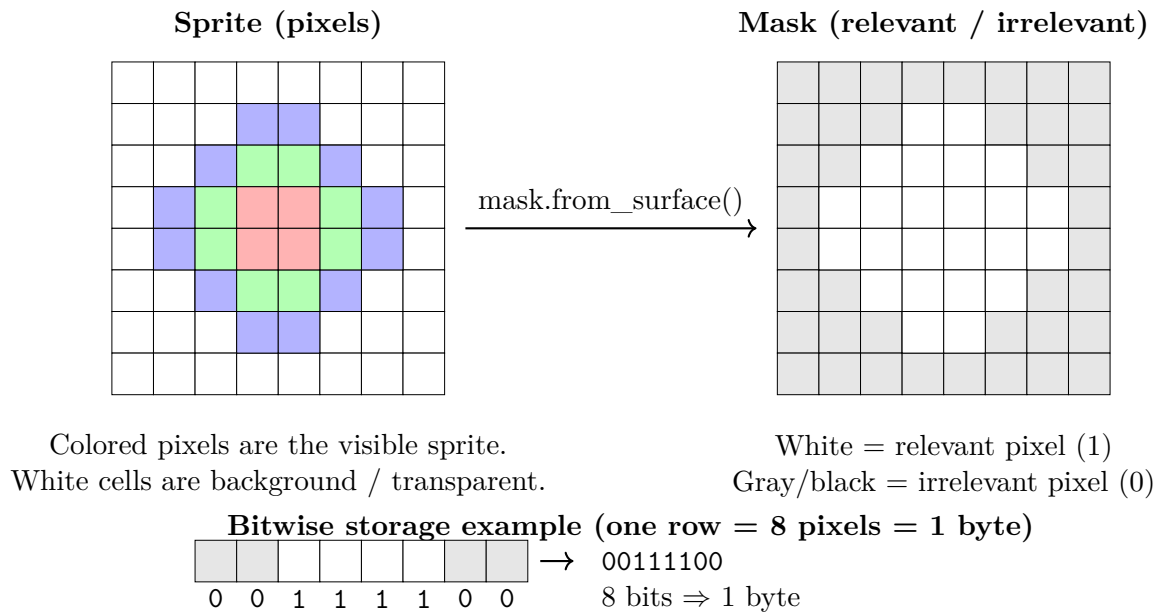


Figure 2.49: From sprite to mask

Let us look at the collision detection behaviour in more detail. In figure 2.50 on the following page we see four sprites: a wall, a spaceship, a monster, and a projectile. None of the sprites are touching each other.

In figure 2.51 on the next page you can clearly see the effect of collision detection using the bounding rectangles (bounding boxes). For the wall, everything is perfect: the projectile hits the wall, and the color indicates that the program has detected the collision.

However, we can also see the disadvantage when looking at the spaceship. A collision is detected even though the two sprites do not actually touch. The reason is that the spaceship's bounding rectangle also includes the empty corners, so the rectangles overlap and a collision is reported. The same effect can also be observed with the monster.

The situation is different when we use collision detection based on bounding circles (figure 2.52 on the following page). Now the collision with the wall is no longer detected

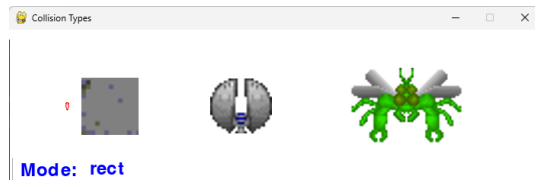


Figure 2.50: Four sprites

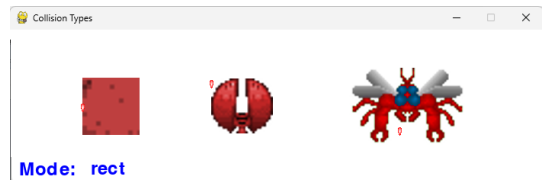


Figure 2.51: Collision detection using rectangles (montage)

correctly, because the corners of the wall do not belong to the inner circle. For the spaceship, however, this method produces exactly the desired result, since the empty corners are not part of the bounding circle. If we move a little further to the right, the spaceship would also turn red, because a collision would then be detected. The monster still produces an incorrect result.

Finally, there is pixel-perfect collision detection (figure 2.53). The collision with the wall is detected correctly. Even more interesting are the results for the spaceship and the monster. Both correctly report no collision, because the projectile is inside the rectangle and the inner circle, but only on transparent pixels. Feel free to try it yourself: move the projectile slightly to the left or right, and you will immediately see the pixel-perfect collision detection in action through the color change.

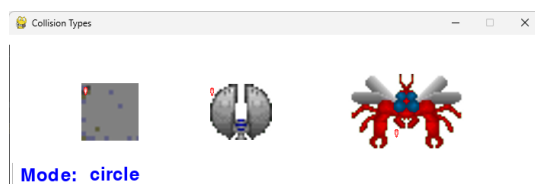


Figure 2.52: Collision detection using circles (montage)

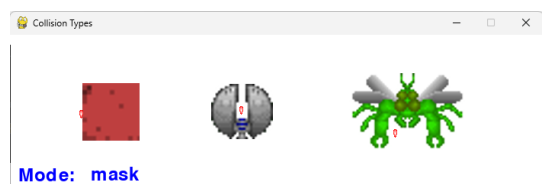


Figure 2.53: Collision detection using masks (montage)

2.8.2 More Input

2.8.2.1 Three Types of Collision Detection (of a Bullet)

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/08%20Collision/v01

Let us now take a closer look at the corresponding source code. However, I will skip another discussion of the `config.py`.

Listing 2.91: Collision types (1), config.py

```

1 from os import path
2
3 import pygame
4
5 WINDOW = pygame.Rect((0, 0), (700, 200))
6 FPS = 60
7 TITLE = "Collision Types"
8 PATH: dict[str, str] = {}
9 PATH["file"] = path.dirname(path.abspath(__file__))
10 PATH["image"] = path.join(PATH["file"], "images")
11 MODE = "rect"
12
13 @staticmethod
14 def filepath(name: str) -> str:
15     return path.join(PATH["file"], name)
16
17 @staticmethod
18 def imagepath(name: str) -> str:
19     return path.join(PATH["image"], name)

```

Things become more interesting with the `Obstacle` class. This class is used for the wall, the spaceship, and the monster. For rectangle-based collision detection, the surrounding rectangle is required. As usual, it is obtained in line 14 using `pygame.Surface.get_rect()` and stored in the attribute `rect`.

get_rect()

For sprites with implicit transparency or explicit transparency set via `set_colorkey()`, the mask can be created very easily using `pygame.mask.from_surface()` (line 15). In order for the predefined collision detection functions to work, this mask must be stored in the `Sprite` object using the attribute `mask`.

self.rect

from_surface()

In line 16, the bounding radius is calculated. This is implemented in a somewhat unclean way. Strictly speaking, one should determine the minimum of width and height and divide it by two.

self.mask

```

15 self.radius = min(self.rect.width, self.rect.height) // 2

```

As with the mask, the radius must also be stored in an attribute so that the predefined collision methods can work: `radius`.

self.radius

The flag `hit` is only used to ensure that the correct image is displayed depending on the detected collision. As you have probably already noticed, two images are loaded for these sprites: one for the *not hit* state and one for the *hit* state.

Listing 2.92: Collision types (2), Obstacle

```

7 class Obstacle(pygame.sprite.Sprite):
8
9     def __init__(self, filename1: str, filename2: str) -> None:
10         super().__init__()
11         self.image_normal = pygame.image.load(cfg.imagepath(filename1)).convert_alpha()
12         self.image_hit = pygame.image.load(cfg.imagepath(filename2)).convert_alpha()
13         self.image = self.image_normal
14         self.rect: pygame.Rect = self.image.get_rect()           # Bounding rectangle
15         self.mask = pygame.mask.from_surface(self.image)         # Pixel mask
16         self.radius = self.rect.width // 2                       # Bounding circle

```

```

17         self.rect.centery = cfg.WINDOW.centery
18         self.hit = False
19
20     def update(self, *args: Any, **kwargs: Any) -> None:
21         if "hit" in kwargs.keys():
22             self.hit = kwargs["hit"]
23         self.image = self.image_hit if (self.hit) else self.image_normal

```

The `Bullet` class is similar in many ways to the `Obstacle` class. Since we also want to use this class for all three types of collision detection, we need the same three attributes here as well: `rect`, `radius`, and `mask`.

In addition, the class contains a few lines of code to allow the bullet to move; this should be self-explanatory. Note: for the sake of simplicity, no boundary check has been implemented. There is no real need for it here.

Listing 2.93: Collision types (3), `Bullet`

```

26 class Bullet(pygame.sprite.Sprite):
27
28     def __init__(self, picturefile: str) -> None:
29         super().__init__()
30         self.image = pygame.image.load(cfg.imagepath(picturefile)).convert_alpha()
31         self.rect = self.image.get_rect()
32         self.radius = self.rect.centery
33         self.mask = pygame.mask.from_surface(self.image)
34         self.rect.center = (10, 10)
35         self.directions = {"stop": (0, 0), "down": (0, 1), "up": (0, -1),
36                           "left": (-1, 0), "right": (1, 0)}
37         self.set_direction("stop")
38
39     def update(self, *args: Any, **kwargs: Any) -> None:
40         if "action" in kwargs.keys():
41             if kwargs["action"] == "move":
42                 self.rect.move_ip(self.speed)
43         elif "direction" in kwargs.keys():
44             self.set_direction(kwargs["direction"])
45
46     def set_direction(self, direction: str) -> None:
47         self.speed = self.directions[direction]

```

And now the `Game` class. In the constructor, the usual things happen. There is nothing particularly noteworthy here.

Listing 2.94: Collision types (4), Constructor of `Game`

```

50 class Game(object):
51
52     def __init__(self) -> None:
53         pygame.init()
54         self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
55         self.screen = self.window.get_surface()
56         self.clock = pygame.time.Clock()
57
58         self.font = pygame.font.Font(pygame.font.get_default_font(), 24)
59         self.bullet = pygame.sprite.GroupSingle(Bullet("shoot.png"))
60         self.all_obstacles = pygame.sprite.Group()
61         self.all_obstacles.add(Obstacle("brick1.png", "brick2.png"))
62         self.all_obstacles.add(Obstacle("ship1.png", "ship2.png"))

```

```

63     self.all_obstacles.add(Obstacle("alienbig1.png", "alienbig2.png"))
64     self.running = False

```

The methods `run()` and `watch_for_events()` also follow well-established patterns.

Listing 2.95: Collision types (5), `run()` and `watch_for_events()` of Game

```

66     def run(self) -> None:
67         self.resize()
68         self.running = True
69         while self.running:
70             self.watch_for_events()
71             self.update()
72             self.draw()
73             self.clock.tick(cfg.FPS)
74         pygame.quit()
75
76     def watch_for_events(self) -> None:
77         for event in pygame.event.get():
78             if event.type == pygame.QUIT:
79                 self.running = False
80             elif event.type == pygame.KEYDOWN:
81                 if event.key == pygame.K_ESCAPE:
82                     self.running = False
83                 elif event.key == pygame.K_DOWN:
84                     self.bullet.sprite.update(direction="down")
85                 elif event.key == pygame.K_UP:
86                     self.bullet.sprite.update(direction="up")
87                 elif event.key == pygame.K_LEFT:
88                     self.bullet.sprite.update(direction="left")
89                 elif event.key == pygame.K_RIGHT:
90                     self.bullet.sprite.update(direction="right")
91                 elif event.key == pygame.K_r:
92                     cfg.MODE = "rect"
93                 elif event.key == pygame.K_c:
94                     cfg.MODE = "circle"
95                 elif event.key == pygame.K_m:
96                     cfg.MODE = "mask"
97             elif event.type == pygame.KEYUP:
98                 self.bullet.sprite.update(direction="stop")

```

The same applies to the methods `update()` and `draw()`.

Listing 2.96: Collision types (6), `update()` and `draw()` of Game

```

100    def update(self) -> None:
101        self.check_for_collision()
102        self.bullet.update(action="move")
103        self.all_obstacles.update()
104
105    def draw(self) -> None:
106        self.screen.fill("white")
107        self.all_obstacles.draw(self.screen)
108        self.bullet.draw(self.screen)
109        text_surface_modus = self.font.render(f"Mode: {cfg.MODE}", True, "blue")
110        self.screen.blit(text_surface_modus, dest=(10, cfg.WINDOW.bottom - 30))
111        self.window.flip()

```

The method `resize()` is not related to collision detection itself. Its only purpose is to ensure that the `Obstacle` objects are distributed evenly across the width of the window.

The first `for`-loop calculates the total width of all `Obstacle` objects. This information is needed to compute the spacing in line 117. To do this, the total obstacle width is subtracted from the window width. The remaining number of pixels can then be distributed across the gaps.

And how many gaps do we have? There are two gaps between the three `Obstacle` objects, one gap to the left border, and one to the right border – a total of four gaps. The resulting spacing is stored in `padding`.

In the second `for`-loop, the left position of each `Obstacle` object can then be calculated and set accordingly.

Listing 2.97: Collision types (7), `resize()` of `Game`

```

113     def resize(self) -> None:
114         total_width = 0
115         for s in self.all_obstacles:
116             total_width += s.rect.width
117         padding = (cfg.WINDOW.width - total_width) // 4 # Spacing between obstacles
118         for i in range(len(self.all_obstacles)):
119             if i == 0:
120                 self.all_obstacles.sprites()[i].rect.left = padding
121             else:
122                 self.all_obstacles.sprites()[i].rect.left = self.all_obstacles.sprites()[i -
1]}.rect.right + padding

```

Drum roll

And now the actual collision detection. Depending on which collision method we have selected, the corresponding collision function is called inside the `for`-loop: `pygame.sprite.collide_circle()`, `pygame.sprite.collide_mask()`, or `pygame.sprite.collide_rect()`.

The semantics are actually quite simple. Each of these methods is given two `Sprite` objects and returns `True` if a collision is detected; otherwise it returns `False`. As already mentioned above, it is important to ensure that the method being used can find the information it requires in the sprite:

Required information

- 👉 `pygame.sprite.collide_circle()` needs `self.radius`
- 👉 `pygame.sprite.collide_mask()` needs `self.mask`
- 👉 `pygame.sprite.collide_rect()` needs `self.rect`

Listing 2.98: Collision types (8), `check_for_collision()` of `Game`

```

124 def check_for_collision(self) -> None:
125     if cfg.MODE == "circle":
126         for s in self.all_obstacles:
127             s.update(hit=pygame.sprite.collide_circle(self.bullet.sprite, s))
128     elif cfg.MODE == "mask":
129         for s in self.all_obstacles:
130             s.update(hit=pygame.sprite.collide_mask(self.bullet.sprite, s))
131     else:
132         for s in self.all_obstacles:
133             s.update(hit=pygame.sprite.collide_rect(self.bullet.sprite, s))

```

2.8.2.2 Checking all Sprites in a List



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/08%20Collision/v03

Rectangle-based collision detection between a single sprite and a list of sprites – that is, checking whether one sprite collides with any sprite in a `SpriteGroup` – is used so often that a dedicated method exists for this purpose: `pygame.sprite.spritecollide()`.

`spritecollide()`

The first parameter is a single `Sprite` object – in this case, our fireball. The second parameter is the list of sprites in which a collision should be checked. The third parameter controls whether the colliding objects should be removed from the list. This is very useful, for example, when an obstacle should disappear upon contact.

Below a minimal example (printed only partially). In line 54 the key part happens: one sprite – `player` – is checked for collisions with many sprites – `blocks`. Every sprite that the player collides with is removed from all groups using `kill()`, and is therefore most likely deleted completely.

Listing 2.99: `spritecollide()`

```

31 def main():
32     pygame.init()
33     screen = pygame.display.set_mode((640, 480))
34     clock = pygame.time.Clock()
35
36     player = Player((320, 240))
37     blocks = pygame.sprite.Group()
38     for x in range(100, 541, 80):
39         for y in (120, 200, 280):
40             blocks.add(Block((x, y)))
41
42     all_sprites = pygame.sprite.Group(player, *blocks.sprites())
43
44     font = pygame.font.SysFont(None, 24)
45
46     while True:
47         for event in pygame.event.get():

```

```

48         if event.type == pygame.QUIT:
49             pygame.quit()
50
51     keys = pygame.key.get_pressed()
52     player.update(keys)
53
54     pygame.sprite.spritecollide(player, blocks, True) # spritecollide(sprite, group,
55                                                         dokill)
56
57     screen.fill((30, 30, 30))
58     all_sprites.draw(screen)
59     text = font.render(f"Blocks remaining: {len(blocks)}", True, (200, 200, 200))
60     screen.blit(text, (10, 10))
61
62     pygame.display.flip()
63     clock.tick(60)

```

2.8.2.3 Using Function Pointer/Collision Callback

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/08%20Collision/v02

function
pointer

collision
callback

The method `pygame.sprite.spritecollide()` has a fourth parameter as well. This parameter can be used to pass a [function pointer](#) or *collision callback* to a different collision detection method. This function must accept two `Sprite` objects as parameters. This means you can either use your own custom collision function or one of the three pre-defined methods: `collide_circle()`, `collide_mask()`, or `collide_rect()`. If nothing is specified here – as in our source code – `collide_rect()` is used automatically.

Listing 2.100: Dynamic collision callback

```

124     def check_for_collision(self) -> None:
125         match cfg.MODE:
126             case "circle":
127                 func = pygame.sprite.collide_circle
128             case "mask":
129                 func = pygame.sprite.collide_mask
130             case _:
131                 func = pygame.sprite.collide_rect
132         hits = pygame.sprite.spritecollide(self.bullet.sprite, self.all_obstacles, False,
133                                           func)
134         for s in self.all_obstacles:
135             s.update(hit=s in hits)

```


2.8.3 What was new?

There are three standard ways to test the collision of two sprites:

- 👉 checking whether their rectangles intersect,
- 👉 whether their bounding circles intersect, or
- 👉 whether the pixels of the objects overlap.

In order to perform these collision tests, a sprite must provide the required information: `rect`, `radius`, or `mask`.

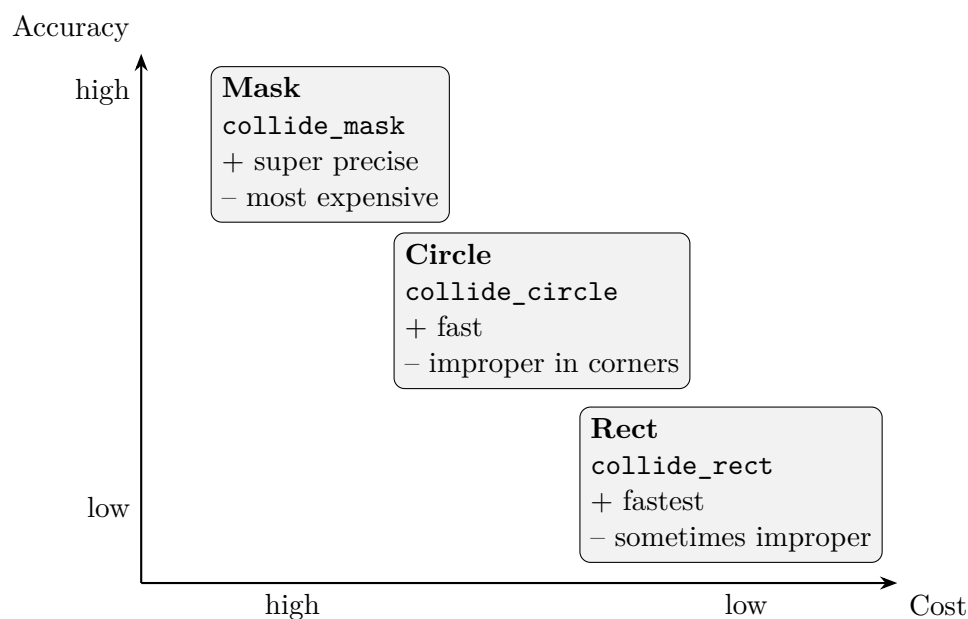


Figure 2.54: Trade-off Accuracy vs. Costs



The following Pygame elements were introduced



- 👉 `pygame.mask.from_surface()`:
https://pyga.me/docs/ref/mask.html#pygame.mask.from_surface
- 👉 `pygame.sprite.collide_circle()`:
https://pyga.me/docs/ref/sprite.html#pygame.sprite.collide_circle
- 👉 `pygame.sprite.collide_mask()`:

https://pyga.me/docs/ref/sprite.html#pygame.sprite.collide_mask

+ pygame.sprite.collide_rect():
https://pyga.me/docs/ref/sprite.html#pygame.sprite.collide_rect

+ pygame.sprite.spritecollide():
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.spritecollide>

2.8.4 Homework

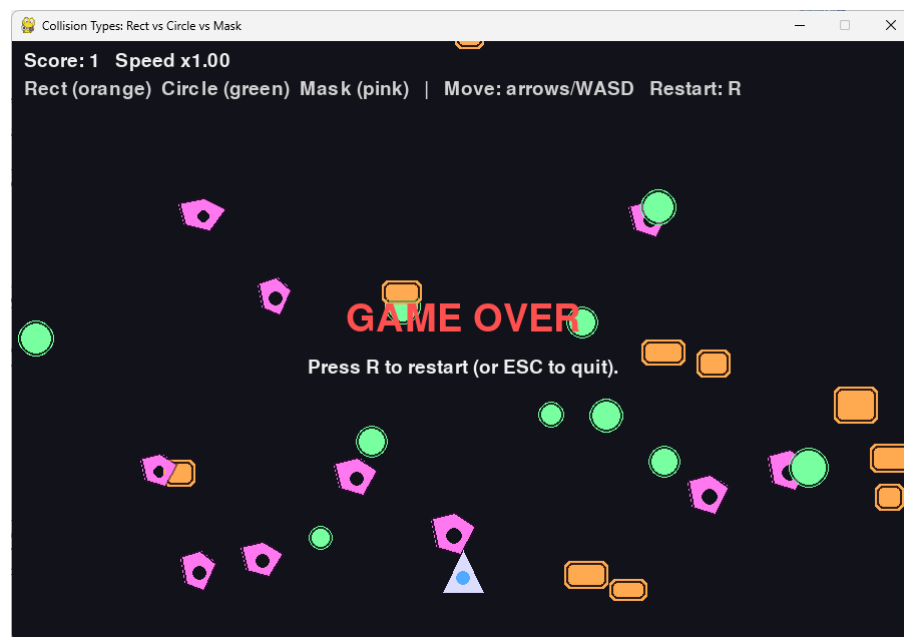


Figure 2.55: A simple collision game

Create a game with the following properties:

1. A player sprite is displayed at the bottom center of the playfield.
2. The player can move in all four directions using the keyboard, but cannot leave the playfield.
3. Thirty obstacles are placed on the playfield.
4. Ten obstacles each are detected using rectangle collision, circle collision, and mask-based collision detection.
5. The obstacles are placed in such a way that they do not overlap with each other or with the player.
6. The obstacles move downward at different speeds.

7. There is a score counter that starts at 0. It is to display on the top edge of the window.
8. If an obstacle leaves the playfield at the bottom, it reappears at the top and the score counter is increased by 1.
9. If an obstacle hits the player, the game is lost.
10. Bonus: Over time, the obstacles become faster and faster.
11. Bonus: A Game Over message and a restart option.

2.9 Time-based Actions

2.9.1 Introduction

 So ... where are the files?

- https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/09%20Time/v01
- https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/09%20Time/v02
- https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/09%20Time/v03

In games, time-based actions are needed in many situations: a bomb drops every half second, a shield is active for 10 seconds, after 3 jumps the *jump* ability is not available for 5 minutes, animation frames should be displayed every 1/30 second, and so on.

Let us first look at the screen output of source code 2.101 on the facing pageff. shown in figure 2.56. The fireballs are obviously released in very quick succession, so that they appear like a chain. Because the enemy is moving horizontally, this results in a slanted line – which is clearly not the intended behaviour.

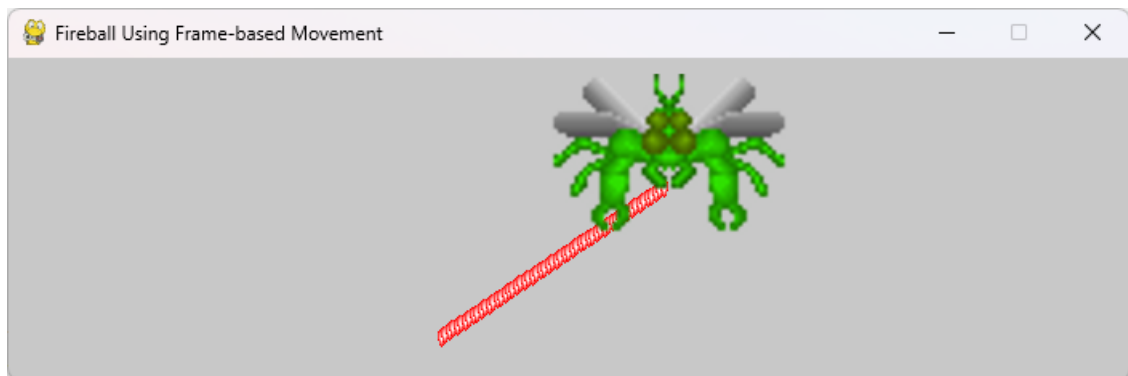


Figure 2.56: Fireball using frame-based movement

Before we take a closer look at time control itself, let us briefly look at the program. The `config.py` doesn't introduce anything new.

Listing 2.101: Time-based actions (1), config.py

```

1 from os import path
2
3 import pygame
4
5 WINDOW = pygame.Rect((0, 0), (700, 200))
6 FPS = 60
7 DELTATIME = 1.0 / FPS
8 TITLE = "Fireball Using Counter-based Movement"
9 PATH: dict[str, str] = {}
10 PATH["file"] = path.dirname(path.abspath(__file__))
11 PATH["image"] = path.join(PATH["file"], "images")
12
13 def filepath(name: str) -> str:
14     return path.join(PATH["file"], name)
15
16 def imagepath(name: str) -> str:
17     return path.join(PATH["image"], name)

```

The `Enemy` class does not introduce anything particularly exciting either. With a spacing of 10 px, the enemy continuously moves back and forth from left to right and vice versa.

Listing 2.102: Time-based actions (2), Version 1.0 – Enemy

```

8 class Enemy(pygame.sprite.Sprite):
9
10     def __init__(self, filename: str) -> None:
11         super().__init__()
12         self.image = pygame.image.load(cfg.imagepath(filename)).convert_alpha()
13         self.rect = pygame.FRect(self.image.get_rect())
14         self.rect.topleft = (10, 10)
15         self.direction = 1
16         self.speed = pygame.math.Vector2(150, 0)
17
18     def update(self, *args: Any, **kwargs: Any) -> None:
19         newpos = self.rect.move(self.speed * cfg.DELTATIME * self.direction)
20         if newpos.left < 10 or newpos.right >= cfg.WINDOW.right - 10:
21             self.direction *= -1
22         else:
23             self.rect = newpos

```

The `Bullet` class is also largely a repetition of what we have seen before. What may be more interesting is line 39. The method `pygame.sprite.Sprite.kill()` is not a true self-destruction mechanism. Instead, this method removes the `Sprite` object from all sprite groups.

kill()

If all references to the object are lost as a result, the object is of course destroyed. However, if a reference still exists somewhere, the object will remain alive. In practice, `Sprite` objects are usually managed in groups (that is, in `pygame.sprite.Group` objects) and are therefore effectively destroyed by calling `kill()`.

You can see this effect in figure 2.56 on the facing page: the fireball disappears about 30 px before reaching the bottom edge of the screen.

Listing 2.103: Time-based actions (3), Version 1.0 – Bullet

```

26 class Bullet(pygame.sprite.Sprite):
27
28     def __init__(self, picturefile: str, startpos: Tuple[int, int]) -> None:
29         super().__init__()
30         self.image = pygame.image.load(cfg.imagepath(picturefile)).convert_alpha()
31         self.rect = pygame.FRect(self.image.get_rect())
32         self.rect.center = startpos
33         self.direction = 1
34         self.speed = pygame.math.Vector2(0, 100)
35
36     def update(self, *args: Any, **kwargs: Any) -> None:
37         self.rect.move_ip(self.speed * cfg.DELTATIME * self.direction)
38         if self.rect.top > cfg.WINDOW.bottom - 30:
39             self.kill() # self-destruction (remove sprite)

```

In the constructor of the `Game` class, a sprite group for the fireballs is created, as well as a `GroupSingle` object for the enemy. In `run()`, the usual game loop tasks are carried out by calling the appropriate methods.

tick()

I would like to briefly draw attention to line 61ff. By calling `pygame.time.Clock.tick()`, the game loop is timed – in this case to 1/60 of a second – and the *delta time* is calculated afterwards.

delta time

Listing 2.104: Time-based actions (4), Version 1.0 – Constructor and `run()` of `Game`

```

42 class Game(object):
43
44     def __init__(self) -> None:
45         pygame.init()
46         self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
47         self.screen = self.window.get_surface()
48         self.clock = pygame.time.Clock()
49
50         self.enemy = pygame.sprite.GroupSingle(Enemy("alienbig1.png"))
51         self.all_bullets = pygame.sprite.Group()
52         self.running = False
53
54     def run(self) -> None:
55         time_previous = time()
56         self.running = True
57         while self.running:
58             self.watch_for_events()
59             self.update()
60             self.draw()
61             self.clock.tick(cfg.FPS) # frame rate limiting
62             time_current = time()
63             cfg.DELTATIME = time_current - time_previous
64             time_previous = time_current
65         pygame.quit()

```

The methods `watch_for_events()` and `draw()` also do not contain anything special.

Listing 2.105: Time-based actions (5), Version 1.0 – `watch_for_events()` and `draw()` of `Game`

```

67     def watch_for_events(self) -> None:
68         for event in pygame.event.get():
69             if event.type == pygame.QUIT:
70                 self.running = False

```

```

71         elif event.type == pygame.KEYDOWN:
72             if event.key == pygame.K_ESCAPE:
73                 self.running = False
74
75     def draw(self) -> None:
76         self.screen.fill((200, 200, 200))
77         self.all_bullets.draw(self.screen)
78         self.enemy.draw(self.screen)
79         self.window.flip()

```

The `update()` method is only worth mentioning with regard to line 82, because a new fireball is created (dropped) there by calling the `new_bullet()` method. The starting position is derived from the current position of the enemy. The horizontal center of the fireball and the enemy should be the same, while the vertical center is shifted slightly downward, which looks better visually.

Listing 2.106: Time-based actions (6), Version 1.0 – `update()` and `new_bullet()` of `Game`

```

81     def update(self) -> None:
82         self.new_bullet()                # spawn bullet
83         self.all_bullets.update()
84         self.enemy.update()
85
86     def new_bullet(self) -> None:
87         self.all_bullets.add(Bullet("shoot.png", self.enemy.sprite.rect.move(0, 20).center))

```

Back to the actual problem. As we saw above, the application is timed to 1/60 of a second by FPS and the call to `tick()` in line 61. In other words, up to 60 fireballs per second are currently created, which is nonsense.

A naive idea would be to reduce the frame rate. So, if I want to create a fireball only every half second, I would have to set the tick rate to 2. Try it! The result is disappointing: the entire game becomes slower. That is not the point.

Time-based counter

A next – and actually quite good – idea is to introduce a counter. The idea is: if the tick rate is 1/60, I count up to 30 and only then drop a fireball.

In the first step, two attributes are added to the `Game` class for this purpose (line 52 and line 53).

Listing 2.107: Time-based actions (7), Version 1.1 – Constructor of `Game`

```

51     self.all_bullets = pygame.sprite.Group()
52     self.time_counter = 0                # Counter
53     self.time_range = 30                # Threshold
54     self.running = False

```

In the `new_bullet()` method, these two values are now used to control the time interval between two drops. First, the counter is increased by 1 each time the method is called.

Since the method is called once per iteration of the main program loop and each iteration is timed, this effectively counts the number of ticks.

If the counter exceeds its upper limit (30 in our example), half a second has passed since the last drop, and a new fireball is released.

Finally, the counter must be reset to 0, because we now have to wait for the next 30 ticks again. The result can be seen in figure 2.57: only two fireballs are visible now.

Listing 2.108: Time-based actions (8), Version 1.1 – `new_bullet()` of `Game`

```

88 def new_bullet(self) -> None:
89     self.time_counter += 1                                # Increment per frame
90     if self.time_counter >= self.time_range:               # If threshold reached
91         self.all_bullets.add(Bullet("shoot.png", self.enemy.sprite.rect.move(0,
92                                     20).center))
93         self.time_counter = 0                               # reset counter

```

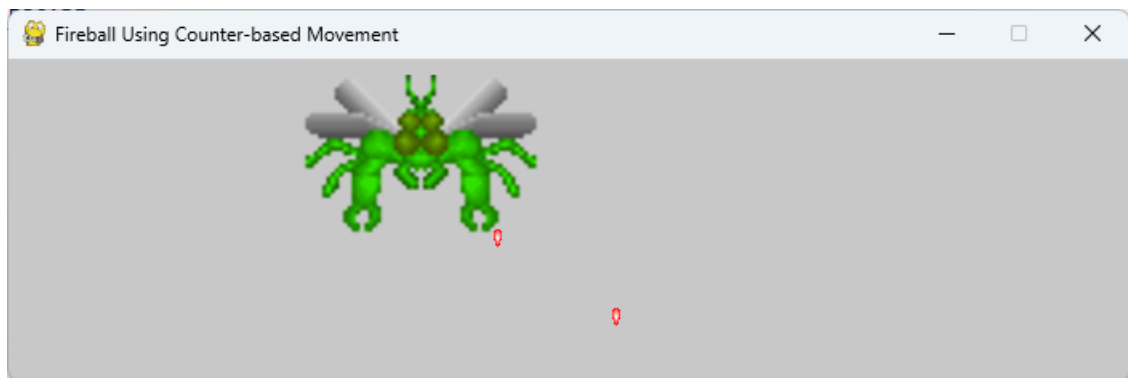


Figure 2.57: Fireball using counter-based movement

The advantages of this approach are clear: it is easy to implement, and the speed of the game itself is not affected.

However, there is a decisive disadvantage: this approach only works if the tick rate does not change and always behaves as expected. In reality, this is not guaranteed. As we remember, calling `tick()` ensures that the loop is executed *at most* 60 times per second. Under heavy load, it may run less often. In addition, many games determine the number of *frames per second* dynamically in order to adapt to different hardware performance. Therefore, coupling time control to the tick rate is not a truly stable solution.

`get_ticks()`

A better approach is to couple time control to a real time source. The method `pygame.time.get_ticks()` is very helpful here. This method returns the amount of time since the start of the game in [Milliseconds \(ms\)](#), and this value is independent of the performance of the hardware or the program.

Now the source code can be reworked. First, in line 52, the current number of *ms* since program start is measured, and in line 53 it is defined how many *ms* a time interval should last. We want to drop a fireball every half second, so this value is 500.

Listing 2.109: Time-based actions (9), Version 1.2 – Constructor of `Game`

```

50     self.enemy = pygame.sprite.GroupSingle(Enemy("alienbig1.png"))
51     self.all_bullets = pygame.sprite.Group()
52     self.time_stamp = pygame.time.get_ticks()      # Store timestamp
53     self.time_duration = 500                      # Interval duration (ms)
54     self.running = False

```

After that, `new_bullet()` checks whether the end of the interval has been reached. In line 89, the current time is measured again using `pygame.time.get_ticks()`. If this value is greater than the previous interval start plus the interval duration – which is the same as the interval end – then 500 ms have passed and a new fireball is dropped.

Now only the new interval start has to be determined, which is done in line 91.

Listing 2.110: Time-based actions (10), Version 1.2 – `new_bullet()` of `Game`

```

88     def new_bullet(self) -> None:
89         if pygame.time.get_ticks() >= self.time_stamp + self.time_duration: #
90             self.all_bullets.add(Bullet("shoot.png", self.enemy.sprite.rect.move(0,
91                                     20).center))
91             self.time_stamp = pygame.time.get_ticks() # Start of next interval

```

2.9.2 More Input

Hint

Time control using events is introduced in section 2.12.2.1 on page 178.

2.9.2.1 The Class `Timer`

So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/09%20Time/v04

Class
Timer

Since we need this logic multiple times, I encapsulated it in the `Timer` class. The core of this class again consists of two attributes, which store the interval duration (`duration`) and the end of the interval (`next`). Unlike before, the interval start is no longer stored; instead, the interval end is saved – which slightly reduces the required computation.

The optional parameter `with_start` is particularly interesting. It allows us to control whether the code should wait until the first interval ends, or whether the very first call to `is_next_stop_reached()` should already return `True`.

What does this mean for our example? If `with_start` is set to `True`, the first fireball is dropped immediately in the very first loop iteration. If the value is `False`, the first fireball is only dropped after 500 ms.

In `is_next_stop_reached()`, it is checked whether the end of the interval has been reached, and, if necessary, a new interval end is calculated. The method returns `True` if the interval end has been reached or exceeded; otherwise, it returns `False`.

Listing 2.111: Time-based actions (11), Version 1.3 – Class `Timer`

```

8  class Timer(object):
9
10     def __init__(self, duration: int, with_start: bool = True) -> None:
11         self.duration = duration
12         if with_start:
13             self.next = pygame.time.get_ticks()
14         else:
15             self.next = pygame.time.get_ticks() + self.duration
16
17     def is_next_stop_reached(self) -> bool:
18         if pygame.time.get_ticks() > self.next:
19             self.next = pygame.time.get_ticks() + self.duration
20             return True
21         return False

```

How is this timer used now? First, an appropriate object is created in the constructor (line 68); the two variables used previously are no longer needed.

Listing 2.112: Time-based actions (12), Version 1.3 – creating a `Timer` object

```

67     self.all_bullets = pygame.sprite.Group()
68     self.bullet_timer = Timer(500)           # Timer without initial delay
69     self.running = False

```

The `new_bullet()` method has now become simpler, since it no longer has to take care of the internal timer logic. It only checks in line 104 whether the interval end has been reached – and that's it!

Listing 2.113: Time-based actions (13), Version 1.3 – Using a `Timer` object verwenden

```

103     def new_bullet(self) -> None:
104         if self.bullet_timer.is_next_stop_reached(): # If interval boundary reached
105             self.all_bullets.add(Bullet("shoot.png", self.enemy.sprite.rect.move(0,
                20).center))

```

For the sake of comparability – which may have been a bit confusing just now – I have placed the four workflows side by side in figure 2.58 on the next page.

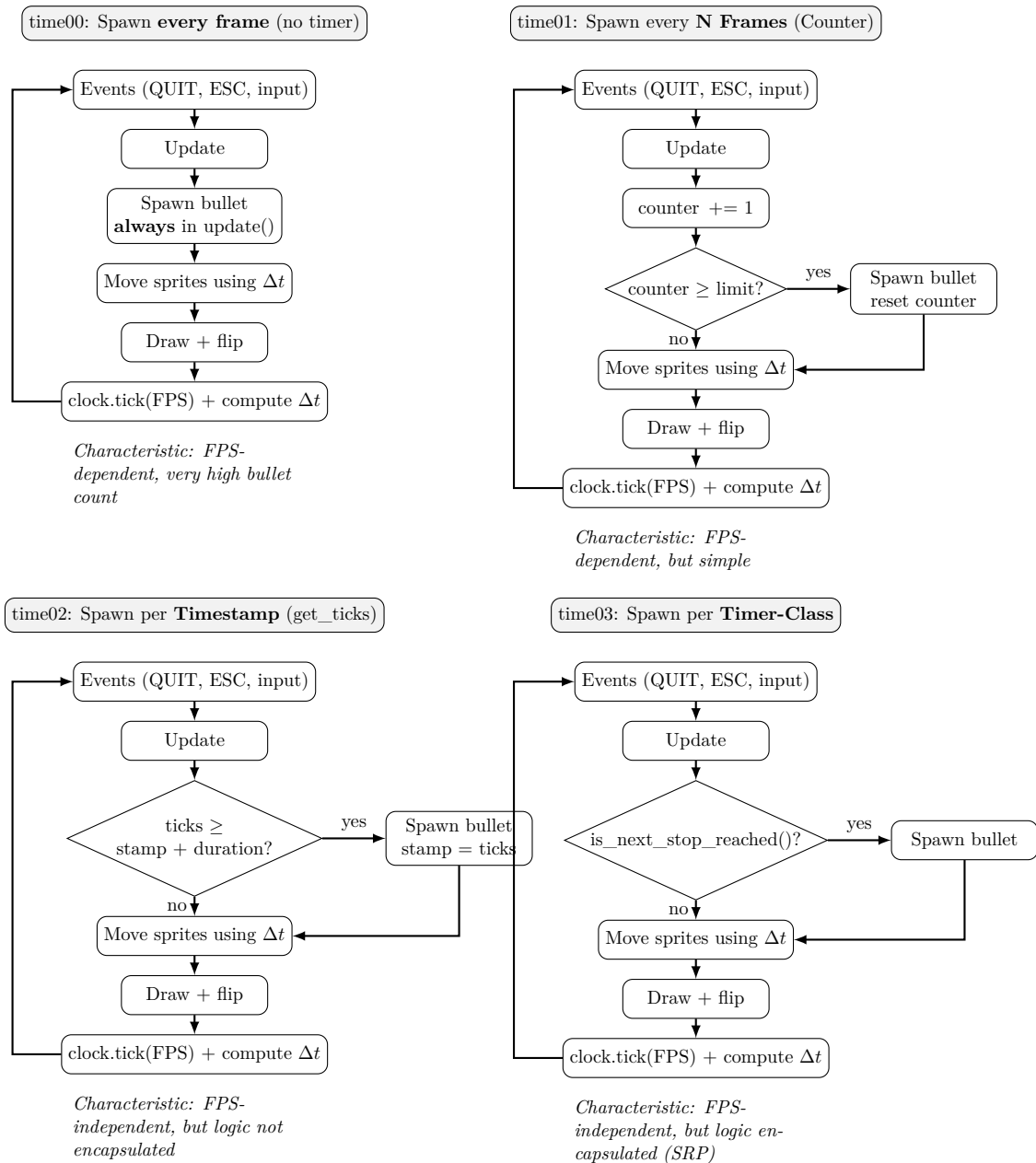


Figure 2.58: Comparison of the 4 algorithms

2.9.2.2 Accumulated Time

Sometimes it is sufficient to locally detect the progression of a time interval. For this purpose, one can use the `Timer` class, but it is also possible to accumulate the elapsed time and then compare the sum against a threshold.

Here is a small example:

```

1     elapsed += cfg.DELTATIME
2     if elapsed >= 0.5:
3         elapsed = 0.0
4         spawn()

```

By using `DELATIME`, a frame-rate-independent and fairly precise measurement of elapsed time is achieved. This logic is well suited for periodic actions or for animations.

2.9.2.3 Cool Down

If you want to let a certain amount of time pass – for example for shields or the time between two shots – the following small logic is commonly used:

```

1     if now() - last_shot >= cooldown:
2         shoot()
3         last_shot = now()

```

2.9.2.4 Start Delay

If you want to let a certain amount of time pass between two actions – for example because a start screen should be visible for a certain time – you can use the following approach:

```

1     start_time = pygame.time.get_ticks()
2     if pygame.time.get_ticks() - start_time > 3000:
3         do_something()

```

2.9.3 What was new?

Time-based events or time intervals should be made independent of the frame rate and should be based on the actual elapsed time. Since this is a frequently used logic, it is encapsulated in a separate class.



The following Pygame elements were introduced



-  `pygame.time.get_ticks()`:
https://pyga.me/docs/ref/time.html#pygame.time.get_ticks
-  `pygame.sprite.Sprite.kill()`:
<https://pyga.me/docs/ref/sprite.html#pygame.sprite.Sprite.kill>

2.10 Mouse

2.10.1 Introduction

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/10%20Mouse/example01

While many games are controlled using the keyboard or a controller, the mouse is also frequently used. In this script, basic mouse actions such as *clicking* and *position queries* are covered. Our example implements the following functionalities:

- A small transparent bubble appears in the center.
- When the mouse moves inside an inner rectangle, the bubble acts as the mouse cursor.
- When the mouse leaves the inner rectangle, the usual system mouse cursor appears.
- A left mouse click rotates the bubble 90° to the left.
- A right mouse click rotates the bubble 90° to the right.
- The mouse wheel is used to scale the size of the bubble.
- Clicking the mouse wheel terminates the application.

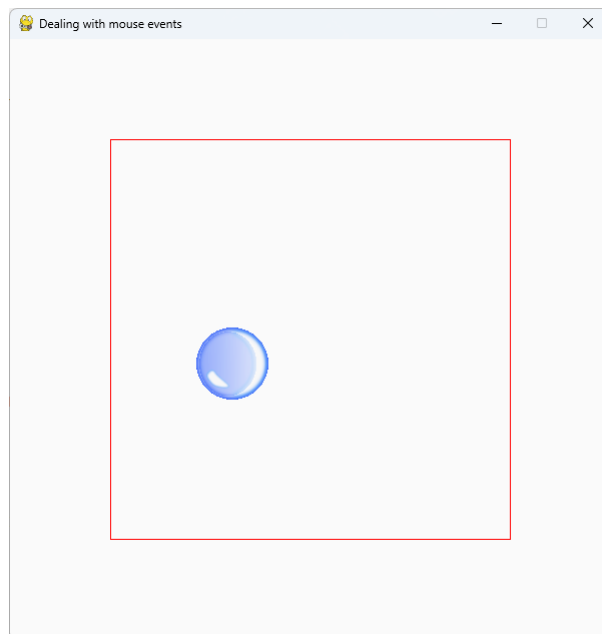


Figure 2.59: Example of actions with a mouse

The main action takes place in the `Game` class, since this is where the mouse actions are processed. Instead of using a separate `config.py` file, I implemented here static variables and methods in the `Game` class – that works as well. In the constructor, the usual suspects are initialized, and in line 67 the `Ball` object is created.

Listing 2.114: Mouse actions: statics and constructor of `Game`

```

55 class Game:
56     WINDOW = pygame.Rect((0, 0), (600, 600))
57     INNER_RECT = pygame.Rect(100, 100, WINDOW.width - 200, WINDOW.height - 200)
58     FPS = 60
59     DELTATIME = 1.0 / FPS
60
61     def __init__(self) -> None:
62         pygame.init()
63         self.window = pygame.Window(size=Game.WINDOW.size, title="Dealing with mouse events")
64         self.screen = self.window.get_surface()
65         self.clock = pygame.time.Clock()
66
67         self.ball = Ball()                # Create ball object
68         self.running = True

```

The `run()` method also holds no surprises.

Listing 2.115: Mouse actions – `Game.run()`

```

70 def run(self) -> None:
71     time_previous = time()
72     while self.running:
73         self.watch_for_events()
74         self.update()
75         self.draw()
76         self.clock.tick(Game.FPS)
77         time_current = time()
78         Game.DELTATIME = time_current - time_previous
79         time_previous = time_current
80     pygame.quit()

```

In `watch_for_events()`, we encounter the first interesting parts. Just as `KEYDOWN` and `KEYUP` mark the pressing and releasing of keys, there are corresponding events for the mouse as well: `MOUSEBUTTONDOWN` and `MOUSEBUTTONUP`.

MOUSE-
BUTTON-
DOWN

In line 89, the value of `event.type` is checked and then it is determined which mouse button was pressed.

MOUSE-
BUT-
TONUP

For this purpose, these two mouse events provide two attributes: `event.button` and `event.pos`. The numeric codes of `event.button` are shown in table 2.9 on page 154. Interestingly, there are no predefined constants here, unlike with keyboard input. After the check, the corresponding messages are sent to the `Ball` object.

event.but-
ton

If the left mouse button is pressed (line 90), a message is sent to the ball to rotate by 90° to the left, and if the right mouse button is pressed, to rotate by 90° to the right (hence -90°, see line 94).

The mouse wheel is also handled like a mouse button. Depending on the direction of rotation, a different numeric code is returned (see line 96 and line 98). If the mouse

wheel is pressed – that is, clicked – the game should terminate. This is checked and implemented in line 92.

Using `event.pos`, one could immediately query the mouse position of this very moment – which we do not do here.

`event.pos`

Listing 2.116: Mouse actions – `Game.watch_for_events()`

```

82     def watch_for_events(self) -> None:
83         for event in pygame.event.get():
84             if event.type == QUIT:
85                 self.running = False
86             elif event.type == KEYDOWN:
87                 if event.key == K_ESCAPE:
88                     self.running = False
89             elif event.type == MOUSEBUTTONDOWN:           # Mouse button pressed
90                 if event.button == 1:                     # Left
91                     self.ball.update(rotate=90)
92                 elif event.button == 2:                   # Middle
93                     self.running = False
94                 elif event.button == 3:                   # Right
95                     self.ball.update(rotate=-90)
96                 elif event.button == 4:                   # Scroll up
97                     self.ball.update(scale=2)
98                 elif event.button == 5:                   # Scroll down
99                     self.ball.update(scale=-2)

```

One requirement was that the system mouse cursor should only be visible outside the inner rectangle. Inside the rectangle, the ball is supposed to act as the mouse cursor. In line 104, this is achieved using the method `pygame.mouse.set_visible()`. This method controls whether the system mouse cursor – in whatever visual form – is shown or hidden.

`set_visible()`

The decision is based on whether the current mouse position lies inside the inner rectangle. The method `pygame.mouse.get_pos()` returns the current mouse position. This position is then simply passed into an already familiar collision test: `pygame.Rect.collidepoint()`. If the mouse position is inside the rectangle, this method returns `True`; otherwise, it returns `False`.

`get_pos()`

`collidepoint()`

Listing 2.117: Mouse actions – `Game.update()` and `Game.draw()`

```

101     def update(self):
102         newpos = pygame.mouse.get_pos()
103         self.ball.update(center=newpos)
104         if Game.INNER_RECT.collidepoint(pygame.mouse.get_pos()): # Hide cursor?
105             pygame.mouse.set_visible(False)
106         else:
107             pygame.mouse.set_visible(True)
108         self.ball.update(go=True)
109
110     def draw(self) -> None:
111         self.screen.fill((250, 250, 250))
112         pygame.draw.rect(self.screen, "red", Game.INNER_RECT, 1)
113         self.ball.draw(self.screen)
114         self.window.flip()

```

The only class left is `Ball`. Although it no longer contains any direct mouse actions, the `update()` method now looks quite different from what we have seen in the previous examples.

And again: LSP

In earlier examples, methods such as `rotate()` or `resize()` were called directly from `watch_for_events()` or comparable methods of the `Game` class. This is perfectly fine in principle. However, problems arise once such subclasses of `pygame.sprite.Sprite` are added to a `pygame.sprite.Group` or a `pygame.sprite.GroupSingle`. These classes expect only `Sprite` objects as elements. Therefore, in terms of object-oriented programming, one should only use methods and attributes that are known to the parent class `pygame.sprite.Sprite` – for example, `update()`. Methods such as `rotate()` would be unknown to the sprite group.

Consider, for example, line 70 in source code 2.59 on page 83. The method `change_direction()` is completely unknown to the `GroupSingle` object `defender`, because it expects a `Sprite` and not a `Defender` object. Syntax checkers such as `Pylance` will report errors here.

Dispatching hub

One way to work around this problem is to use `update()` as a dispatching hub. In the class `pygame.sprite.Sprite`, this method is defined with the following signature:

```
update(self, *args: Any, **kwargs: Any) -> None
```

In other words, any number of freely definable parameters can be passed to this method. This is exactly what happens in our `update()` method. For rotation, the parameter `rotate` is passed with the corresponding angle; for scaling, the parameter `scale`; and in `update()` of `Game`, the parameter `go` is passed with the value `True`.

Each caller can therefore define its own parameters spontaneously and assign values to them. The `update()` method in the subclass – here `Ball` – only needs to check for these parameters.

In the first step, it is checked whether a parameter was provided, as shown in line 21, line 29, line 32, and line 35. Afterwards, the parameter value is forwarded to the corresponding method of the subclass. This way, the sprite group does not need to access methods of the subclass directly, but can rely on the method provided by the parent class.

Listing 2.118: Mouse actions – Ball

```

9 class Ball(pygame.sprite.Sprite):
10     def __init__(self) -> None:
11         super().__init__()
12         path = os.path.dirname(os.path.abspath(__file__))
13         path = os.path.join(path, "images")
14         fullfilename = os.path.join(path, "blue2.png")
15         self.image_orig = pygame.image.load(fullfilename).convert_alpha()
16         self.scale = 10
17         self.image = pygame.transform.scale(self.image_orig, (self.scale, self.scale))
18         self.rect = self.image.get_rect()
19
20     def update(self, *args: Any, **kwargs: Any) -> None:
21         if "go" in kwargs.keys(): # Parameter present?
22             if kwargs["go"]:
23                 self.rect.clamp_ip(Game.INNER_RECT)
24                 c = self.rect.center # Store previous center
25                 self.image = pygame.transform.scale(self.image_orig, (self.scale,
26                                                         self.scale))
27                 self.rect = self.image.get_rect()
28                 self.rect.center = c # Reset center
29
30             if "rotate" in kwargs.keys(): #
31                 self.rotate(kwargs["rotate"])
32
33             if "scale" in kwargs.keys(): #
34                 self.resize(kwargs["scale"])
35
36             if "center" in kwargs.keys(): #
37                 self.set_center(kwargs["center"])
38
39     def draw(self, screen: pygame.Surface) -> None:
40         screen.blit(self.image, self.rect)
41
42     def rotate(self, angle: float) -> None:
43         self.image_orig = pygame.transform.rotate(self.image_orig, angle)
44
45     def resize(self, delta: int) -> None:
46         self.scale += delta
47         if self.scale > Game.INNER_RECT.width:
48             self.scale = Game.INNER_RECT.width
49         elif self.scale < 5:
50             self.scale = 5
51
52     def set_center(self, center: Tuple[int, int]) -> None:
53         self.rect.center = center

```

Two final notes on `pygame.transform.rotate()`

- ⚠ Unlike many other systems that work with angles, the angle here is measured in [degree](#) rather than in [radian](#).
- ⚠ The approach used here works only because the bitmap represents a ball. In general, you should not repeatedly rotate the already rotated image. Instead, keep the original image unchanged and update only an angle variable. If you rotate the same image, the contents may be scaled up or down depending on the bitmap's shape, resulting in visual artifacts and severe pixel degradation.

2.10.2 More Input

2.10.2.1 In Which Window Took the Mouse Action Place?

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/10%20Mouse/example02

Completely analogous to the example and explanation in section 2.6.2.2 on page 93, it is also possible to determine in which window a mouse action occurred.

Window.id

event.window

Listing 2.119: In which window took the mouse action place?

```

4 def main():
5     pygame.init()
6     window_first = pygame.Window(size=(300, 50),
7         title="Main Window",
8         position=(500, 50))
9     window_second = pygame.Window(size=(300, 50),
10        title="Side Window",
11        position=(820, 50))
12    screen_first = window_first.get_surface()
13    screen_second = window_second.get_surface()
14    clock = pygame.time.Clock()
15
16
17    running = True
18    while running:
19        for event in pygame.event.get():
20            if event.type == pygame.QUIT:
21                running = False
22            elif event.type == pygame.WINDOWCLOSE:
23                running = False
24                event.window.destroy()
25            elif event.type == pygame.MOUSEBUTTONDOWN:
26                if event.window == window_first: # Check which window the event belongs to
27                    window_id = window_first.id
28                    event.window.title = f"Main Window (Mouse Pressed: '{event.button}' at
29                                           '{event.pos}')"
30                elif event.window == window_second: #
31                    window_id = window_second.id
32                    event.window.title = f"Side Window (Mouse Pressed: '{event.button}' at
33                                           '{event.pos}')"
34                else:
35                    window_id = None
36                print(f"ID {window_id}: {event.window.title}")
37            if running:
38                screen_first.fill((0, 255, 0))
39                window_first.flip()
40                screen_second.fill((255, 0, 0))
41                window_second.flip()
42                clock.tick(60)
43
44    pygame.quit()

```

Running the program produces the following console output when the corresponding keys are pressed:

```

1 ID 1: Main Window (Mouse Pressed: '1' at (55, 25))
2 ID 1: Main Window (Mouse Pressed: '3' at (101, 27))
3 ID 1: Main Window (Mouse Pressed: '2' at (171, 23))
4 ID 2: Side Window (Mouse Pressed: '1' at (74, 12))
5 ID 2: Side Window (Mouse Pressed: '2' at (123, 25))
6 ID 2: Side Window (Mouse Pressed: '3' at (224, 25))

```

2.10.2.2 Surprise: No Double Click

In Pygame, there is no built-in event for a mouse double click. Unlike classic desktop GUI frameworks, pygame only reports individual mouse button presses.

This means you cannot ask directly whether a double click happened. Instead, you have to detect it yourself.

The usual approach is to measure the time between two mouse clicks. If the same mouse button is pressed twice within a short time window, it is treated as a double click. Optionally, the mouse position can also be checked to make sure the cursor has not moved too far between clicks.

This may sound a bit low-level at first, but it is actually very common in game programming. Many higher-level interactions are built by combining simple events with a bit of timing logic.

Once implemented, this technique works reliably and gives you full control over what *double click* really means in your game or tool.

```

1 import pygame
2 import time
3
4 DOUBLE_CLICK_TIME = 400 # ms
5
6 last_click_time = 0
7 last_button = None
8
9 for event in pygame.event.get():
10     if event.type == pygame.MOUSEBUTTONDOWN:
11         now = pygame.time.get_ticks()
12
13         if (event.button == last_button and now - last_click_time <
14             DOUBLE_CLICK_TIME):
15             print("Double click detected!")
16
17             last_click_time = now
18             last_button = event.button

```

Options

- 👉 A `DOUBLE_CLICK_TIME` between 250 and 300 ms feels very snappy.
- 👉 A `DOUBLE_CLICK_TIME` between 400 and 500 ms is easier for beginners.
- 👉 In addition, you can check the distance between the `event.pos` values of both clicks.

2.10.2.3 Not by Event, but by Function

In contrast to mouse button events, mouse button states can also be queried directly. Using the function `pygame.mouse.get_pressed()`, it is possible to check at any time which mouse buttons are currently being held down.

`get_pressed()`

The function returns a tuple of boolean values that indicate whether the left, middle, or right mouse button is pressed. This allows continuous mouse input to be processed without relying on discrete events such as `MOUSEBUTTONDOWN` or `MOUSEBUTTONUP`.

This approach is particularly useful when mouse buttons should influence behaviour as long as they are held down, for example for dragging objects or continuous actions.

A completely analogous discussion can be found in section 2.6.2.4 on page 99.

2.10.3 What was new?

Mouse actions are processed in a similar way to keyboard events. The mouse position can be queried easily. It is often simpler to hide the mouse cursor and let a bitmap follow the mouse position than to define a new mouse cursor.



The following Pygame elements were introduced



- 🎮 `pygame.constants:`
<https://pyga.me/docs/ref/locals.html>
- 🎮 `pygame.mouse.get_pressed()`
https://pyga.me/docs/ref/mouse.html#pygame.mouse.get_pressed
- 🎮 `pygame.MOUSEBUTTONDOWN`, `pygame.MOUSEBUTTONUP`:
<https://pyga.me/docs/ref/event.html>
- 🎮 List of mouse events: table 2.9 on the following page

 `pygame.mouse.get_pos():`
https://pyga.me/docs/ref/mouse.html#pygame.mouse.get_pos

 `pygame.mouse.set_visible():`
https://pyga.me/docs/ref/mouse.html#pygame.mouse.set_visible

 `pygame.Rect.collidepoint():`
<https://pyga.me/docs/ref/rect.html#pygame.Rect.collidepoint>

 `pygame.transform.rotate():`
<https://pyga.me/docs/ref/transform.html#pygame.transform.rotate>

Table 2.9: List of mouse events

Constant	Description
0	undefined
1	left mouse button
2	middle mouse button/mouse wheel
3	right mouse button
4	scroll the mouse wheel up
5	Scroll the mouse wheel down

2.10.4 Homework

Time for a small game:

1. Create a fairly large playfield with a space look.
2. Your spaceship appears in the center.
3. Control the spaceship using the mouse. If one of the  keys is pressed, the mouse movement is interpreted as a control command. Hint: you need to compare the mouse position coordinates between two frames.
4. Rocks (asteroids) of different sizes fly into the screen from all sides.
5. The goal is to avoid being hit by a rock for as long as possible.
6. Every 10 seconds, the score is increased.
7. If I collide with a rock, I lose one life.
8. At the bottom, the score and the remaining lives are displayed.
9. I start with three lives. Add new lives repeatedly when certain score values are reached.

10. When a certain score is reached, I can activate a shield for 15 seconds using the right mouse button. Whether the shield is available should be shown in the bottom line.
11. BONUS: The nose of the spaceship always points in the direction of movement.

2.11 Sound

Without background sounds and/or music, many games would simply be boring. Therefore, I would like to present three different topics here: background music or ambient sounds, sound events, and stereo effects.

2.11.1 Introduction

2.11.1.1 Sound: Music

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/11%20Sound/music

The first example covers the following features:

- Background music is loaded and played in an endless loop.
- The volume can be adjusted using the mouse wheel.
- Pressing `p` pauses the background music or resumes playback.
- Pressing `j` fades out the background music.

I will not explain the imports, `config.py`, and the other familiar building blocks in detail anymore, as they have already appeared many times before.

Listing 2.120: Sound – `config.py`

```

1  from os import path
2
3  import pygame
4
5  WINDOW: pygame.Rect = pygame.Rect(0, 0, 400, 200)    # Rect
6  FPS = 60
7  DELTATIME = 1.0 / FPS
8  PATH: dict[str, str] = {}
9  PATH["file"] = path.dirname(path.abspath(__file__))
10 PATH["image"] = path.join(PATH["file"], "images")
11 PATH["sound"] = path.join(PATH["file"], "sounds")
12 START_DISTANCE = 20
13 VOLUME_STEP = 0.05
14
15 def get_file(filename: str) -> str:
16     return path.join(PATH["file"], filename)
17
18 def get_image(filename: str) -> str:
19     return path.join(PATH["image"], filename)
20
21 def get_sound(filename: str) -> str:
22     return path.join(PATH["sound"], filename)

```

Before sound can be used, the corresponding subsystem must be initialized. This can be done explicitly using `pygame.mixer.init()`, or implicitly – as in the source code at line 10 – by calling `pygame.init()`.

init()

In the `sounds()` method, the preparatory steps for sound output are encapsulated. Background music is loaded into the mixer's internal memory using `pygame.mixer.music.load()`. However, loading the music does not start playback yet.

back-ground music

Playback begins after the volume has been set in line 21 using `pygame.mixer.music.set_volume()`, by calling the method in line 22. The method `pygame.mixer.music.play()` accepts three parameters:

set_volume()

play()

- The first parameter, `loops`, controls the number of repetitions; a value of `-1` means that the music is repeated indefinitely.
- The second parameter, `start`, specifies the position at which playback should begin; the default is `0.0`.
- If the music should start quietly and then become louder (*fade*), this can be achieved using the third parameter `fade`. Here, you can specify how many milliseconds are available for the fade-in; if nothing is specified, playback starts immediately at the target volume.

fade

Listing 2.121: Sound – Constructor and `sounds()` of Game

```

8 class Game:
9     def __init__(self) -> None:
10         pygame.init()
11         self.window = pygame.Window(size=config.WINDOW.size, title='Sound Background Music')
12         self.screen = self.window.get_surface()
13         self.clock = pygame.time.Clock()
14         self.font_bigsized = pygame.font.Font(pygame.font.get_default_font(), 40)
15         self.running = True
16         self.pause = False
17         self.sounds()
18
19     def sounds(self) -> None:
20         pygame.mixer.music.load(config.get_sound("Lucifer.mid"))
21         pygame.mixer.music.set_volume(0.1)
22         pygame.mixer.music.play(-1, 0.0)
```

The `watch_for_events()` method acts purely as a dispatcher. Depending on which key is pressed or which mouse element is used, the corresponding helper methods are called.

Listing 2.122: Sound – `Game.watch_for_events()`

```

24 def watch_for_events(self) -> None:
25     for event in pygame.event.get():
26         if event.type == QUIT:
27             self.running = False
28         elif event.type == KEYDOWN:
29             if event.key == K_ESCAPE:
30                 self.running = False
31         elif event.type == KEYUP:
32             if event.key == K_f:
33                 self.music_start_stop(fadeout=5000)
```

```

34         elif event.key == K_j:
35             self.music_start_stop(loop=-1)
36         elif event.key == K_p:
37             self.pause_alter()
38     elif event.type == pygame.MOUSEBUTTONDOWN:
39         if event.button == 4: # up
40             self.volume_alter(cfg.VOLUME_STEP)
41         elif event.button == 5: # down
42             self.volume_alter(-cfg.VOLUME_STEP)

```

I want to start the background music at some times and fade it out at others. This is handled by the helper method `music_start_stop()`. The background music is stopped using `pygame.mixer.music.fadeout()`. Here, you have to specify over how many milliseconds the music should gradually become quieter until it stops – in our example, this is 5000 ms. The method `pygame.mixer.music.play()` used to start the background music has already been explained above.

`fadeout()`

Listing 2.123: Sound – `Game.music_start_stop()`

```

45 def music_start_stop(self, **kwargs: Any) -> None:
46     if "fadeout" in kwargs.keys():
47         pygame.mixer.music.fadeout(kwargs["fadeout"]) #
48     if "loop" in kwargs.keys():
49         pygame.mixer.music.play(kwargs["loop"], 0.0) #

```

Pressing `p` pauses the background music or resumes playback. The current state is stored in the attribute `pause`. This attribute then determines which of the two music methods is called in the `pause_alter()` method – either `pygame.mixer.music.pause()` or `pygame.mixer.music.unpause()`. Finally, in line 49, the `pause` flag is toggled.

`pause()`

`unpause()`

Listing 2.124: Sound – `Game.pause_alter()`

```

51 def pause_alter(self) -> None:
52     if self.pause:
53         pygame.mixer.music.unpause() #
54     else:
55         pygame.mixer.music.pause()
56     self.pause = not self.pause #

```

As the final feature, volume control is introduced. It is encapsulated in the `volume_alter()` method. Instead of passing an absolute volume value to this method, a delta value is provided.

First, this value is added to the `volume` variable. Afterwards, the value is clamped to the interval `[0, 1]`, and finally the new volume is set using `pygame.mixer.music.set_volume()`.

`set_volume()`

Listing 2.125: Sound – `volume_alter()` von `Game`

```

58 def volume_alter(self, delta: float) -> None:
59     volume = pygame.mixer.music.get_volume()
60     volume += delta
61     volume = pygame.math.clamp(volume, 0.0, 1.0)
62     pygame.mixer.music.set_volume(volume) #

```

And finally, we deal with the remaining bits.

Listing 2.126: Sound – `draw()`, `update()`, `run()` of Game

```

64 def draw(self) -> None:
65     self.screen.fill("white")
66     volume = pygame.mixer.music.get_volume()
67     volume_surface = self.font_bigsized.render(f"Volume: {volume:3.2f}", True, "red")
68     volume_rect = volume_surface.get_rect()
69     volume_rect.center = cfg.WINDOW.center
70     self.screen.blit(volume_surface, volume_rect)
71
72     self.window.flip()
73
74 def update(self):
75     pass
76
77 def run(self):
78     self.running = True
79     while self.running:
80         self.watch_for_events()
81         self.update()
82         self.draw()
83         self.clock.tick(cfg.FPS)
84     pygame.quit()

```

2.11.1.2 Sound: Events



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/11%20Sound/effects

For sound effects, a separate `Sound` object is created in each case (line 18ff.). The constructor of `pygame.mixer.Sound` is given the file name including the path. If you already have an open file object, you can pass that instead; in this case, however, you should provide a second parameter specifying the sound encoding, for example `.OGG` or `.MP3`. As with background music, loading a sound is not the same as playing it.

sound
effects

Listing 2.127: Sound – `Game.sound()`

```

17 def sounds(self) -> None:
18     self.bubble = pygame.mixer.Sound(cfg.get_sound("plopp.mp3"))    #
19     self.clash = pygame.mixer.Sound(cfg.get_sound("glas.wav"))      #

```

In line 39, the current volume is first stored in a variable using `pygame.mixer.Sound.get_volume()`. To ensure that both sounds are played at the same volume, this value is modified and then applied to both sounds using `pygame.mixer.Sound.set_volume()`.

get_vol-
ume()

set_vol-
ume()

Listing 2.128: Sound – Game.volume_alter()

```

38     def volume_alter(self, delta: float) -> None:
39         volume = self.bubble.get_volume()                # For both
40         volume += delta
41         volume = pygame.math.clamp(volume, 0.0, 1.0)
42         self.bubble.set_volume(volume)
43         self.clash.set_volume(volume)

```

play()

For simplicity, the sounds are played directly in `watch_for_events()` (see line 30 and line 32). The actual playback is done using `pygame.mixer.Sound.play()`. You can see that the `play()` method is called on the corresponding `Sound` object.

The `play()` method provides three optional arguments:

- `loops`: number of repetitions (–1 means infinite playback, default)
- `maxtime`: maximum playback time in milliseconds (0 means unlimited, default)
- `fade_ms`: duration of the fade-in in milliseconds (default: 0)

If – as in this case – no arguments are provided, the sound starts playing immediately and stops automatically after it has finished playing. Any other sounds that are currently being played by other `Sound` objects are not interrupted. This means that multiple sounds can be played at the same time.

Listing 2.129: Sound – Game.watch_for_events()

```

21     def watch_for_events(self) -> None:
22         for event in pygame.event.get():
23             if event.type == QUIT:
24                 self.running = False
25             elif event.type == KEYDOWN:
26                 if event.key == K_ESCAPE:
27                     self.running = False
28             elif event.type == pygame.MOUSEBUTTONDOWN:
29                 if event.button == 1:                # left
30                     self.bubble.play()              #
31                 elif event.button == 3:              # right
32                     self.clash.play()                #
33                 elif event.button == 4:              # up
34                     self.volume_alter(0.05)
35                 elif event.button == 5:              # down
36                     self.volume_alter(-0.05)

```

2.11.2 More Input

2.11.2.1 Stereo

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/11%20Sound/stereo

A small example is intended to illustrate the use of channels and [stereo](#) effects. The topic is too extensive to be presented in full detail, but I hope that this chapter provides a helpful introduction.

In figure 2.60, you can see a tank driving from left to right or from right to left. While driving, it can fire up to 5 shots. It would be nice if the driving sound indicated acoustically where the tank is currently located. That is, if the tank is more to the right, the driving sound or the shot should be louder on the right speaker than on the left speaker ([stereo panning](#)). When driving from right to left, the driving sound would therefore move along with the tank.

stereo panning



Figure 2.60: Example Stereo Sound

First, the necessary boilerplate, which should not require any further explanation:

Listing 2.130: Sound-Stereo – config.py

```
1 from os import path
2
3 import pygame
4
5 WINDOW: pygame.Rect = pygame.Rect(0, 0, 800, 224)
6 FPS = 60
7 DELTATIME = 1.0 / FPS
8 PATH: dict[str, str] = {}
9 PATH["file"] = path.dirname(path.abspath(__file__))
10 PATH["image"] = path.join(PATH["file"], "images")
```

```

11 PATH["sound"] = path.join(PATH["file"], "sounds")
12 START_DISTANCE = 20
13 VOLUME_STEP = 0.05
14
15 def get_file(filename: str) -> str:
16     return path.join(PATH["file"], filename)
17
18 def get_image(filename: str) -> str:
19     return path.join(PATH["image"], filename)
20
21 def get_sound(filename: str) -> str:
22     return path.join(PATH["sound"], filename)

```

Listing 2.131: Sound-Stereo – Class Ground

```

11 class Ground(pygame.sprite.Sprite):
12
13     def __init__(self) -> None:
14         super().__init__()
15         fullfilename = cfg.get_image("tankbrigade_part64.png")
16         tile = pygame.image.load(fullfilename).convert()
17         rect = tile.get_rect()
18         self.image = pygame.Surface(cfg.WINDOW.size)
19         for row in range(cfg.WINDOW.width // rect.width):
20             for col in range(cfg.WINDOW.height // rect.height):
21                 self.image.blit(tile, (row * rect.width, col * rect.height))
22         self.rect = self.image.get_rect()

```

Sound ob-
ject

In line 18, a **Sound** object is created. This object is played to emphasize the movement of the tank with appropriate sounds. In the following line (line 44), the helper method **stereo()** is called (see below), and then the playback of the driving sound starts in an infinite loop (line 48).

It is noticeable that the output is not started using `pygame.mixer.Sound.play()`. Normally, this would be a good choice, since this command automatically selects one of the eight available Pygame sound channels.

channel

Selecting channels manually

However, it is also possible to address a Pygame channel directly and thus gain more control over the sound behavior. In line 45, a free `pygame.mixer.Channel` object is determined for this purpose. The method `pygame.mixer.find_channel()` returns the first pygame channel and stores it in the attribute `channel`.

play()

Playback in line 48 is then no longer started via a method of the **Sound** object, but by using `pygame.mixer.Channel.play()`.

This makes it possible to adjust volume and stereo panning dynamically while the sound is playing.

Listing 2.132: Sound-Stereo – Constructor of Tank

```

25 class Tank(pygame.sprite.Sprite):
26
27     def __init__(self) -> None:
28         super().__init__()
29         self.image_filename = (209, 190, 202, 214, 226, 238, 250, 262)
30         self.images: dict[str, list[pygame.Surface]] = {"up": [], "down": [], "left": [],
31             "right": []}
32         for number in self.image_filename:
33             fullfilename = cfg.get_image(f"tankbrigade_part{number}.png")
34             picture = pygame.image.load(fullfilename).convert()
35             picture.set_colorkey("black")
36             self.images["up"].append(picture)
37             self.images["down"].append(pygame.transform.rotate(picture, 180))
38             self.images["left"].append(pygame.transform.rotate(picture, +90))
39             self.images["right"].append(pygame.transform.rotate(picture, -90))
40         self.direction = "right"
41         self.imageindex = 0
42         self.image = self.images[self.direction][self.imageindex]
43         self.rect = pygame.FRect(self.image.get_rect())
44         self.rect.left, self.rect.top = 3 * self.rect.width, 2 * self.rect.height
45         self.sound_drive = pygame.mixer.Sound(cfg.get_sound("tank_drive1.wav")) #
46         self.channel = pygame.mixer.find_channel() # Find a free sound channel
47         if self.channel:
48             self.stereo() #
49             self.channel.play(self.sound_drive, -1) #
50         self.speed = 50

```

The `update()` method is shown here only for completeness. It does not contain any code related to sound playback.

Listing 2.133: Sound-Stereo – Tank.update()

```

51 def update(self, *args: Any, **kwargs: Any) -> None:
52     if "go" in kwargs.keys():
53         if kwargs["go"]:
54             self.update_imageindex()
55             self.image = self.images[self.direction][self.imageindex]
56             if self.direction == "up" or self.direction == "left":
57                 self.speed = -50
58             elif self.direction == "down" or self.direction == "right":
59                 self.speed = 50
60             if self.direction == "up" or self.direction == "down":
61                 self.rect.move_ip(0, self.speed * cfg.DELTATIME)
62                 if self.rect.top <= cfg.WINDOW.top:
63                     self.turn("down")
64                 if self.rect.bottom >= cfg.WINDOW.bottom:
65                     self.turn("up")
66             elif self.direction == "left" or self.direction == "right":
67                 self.rect.move_ip(self.speed * cfg.DELTATIME, 0)
68                 if self.rect.left <= cfg.WINDOW.left:
69                     self.turn("right")
70                 if self.rect.right >= cfg.WINDOW.right:
71                     self.turn("left")
72             self.stereo()
73         if "turn" in kwargs.keys():
74             self.turn(kwargs["turn"])

```

The `stereo()` method is surprisingly simple. The method `pygame.mixer.Channel.set_volume()` provides two parameters: *left* and *right*. Both parameters have a value range of [0, 1].

`set_volume()`

Stereo Panning

As discussed before, we want the right speaker to play the engine sound louder the further to the right the tank is positioned, and vice versa ([stereo panning](#)). To achieve this, I calculate the relative horizontal position of the tank's center with respect to the window width in line 48. This calculation also yields a value in the interval $[0, 1]$.

Once this value is known, the relative value for the left speaker can be determined in the following line by $left = 1 - right$. After that, both values are passed to the `set_volume()` method.

Hint

The method `pygame.mixer.Channel.set_volume()` allows different volume levels to be specified for the left and right Pygame channels, whereas the methods `pygame.mixer.Sound.set_volume()` and `pygame.mixer.music.set_volume()` do not.

Listing 2.134: Sound-Stereo – `Tank.stereo()`

```

76  def stereo(self) -> None:
77      volume_right = self.rect.centerx / cfg.WINDOW.width #
78      volume_left = 1 - volume_right
79      self.channel.set_volume(volume_left, volume_right)

```

What else could this effect be used for? For example, think of two people talking to each other, sound sources in a room, and so on. Whenever audio is meant to make localization easier, or when individual sounds should stand out or be easier to distinguish, different volume levels – i.e., stereo – are a good option.

Nothing related to sound output happens in `turn()` and `update_imageindex()`.

Listing 2.135: Sound-Stereo – `Tank.turn()` and `Tank.update_imageindex()`

```

81  def turn(self, direction: str) -> None:
82      self.direction = direction
83
84  def update_imageindex(self) -> None:
85      if self.speed == 0:
86          self.imageindex = 0
87      else:
88          self.imageindex = (self.imageindex + 1) % len(self.images[self.direction])

```

The sound output of the `Bullet` could also have been implemented in the `Tank` class. However, I find it more natural to place it in `Bullet`. After all, it might later be extended to include an impact sound or an explosion.

Before the constructor, the static variable `sound_fire` is defined in line 93. Although there are many bullets, they all use the same firing sound. Reading this sound file repeatedly and creating a new object each time would therefore waste memory and reduce performance. Instead, starting at line 113, a kind of [singleton](#) check is performed. This ensures that the sound file is read and the corresponding object is created exactly once.

After that, a free channel is searched for, just as with the tank, and the volume of the left and right speakers is determined based on the position. Finally, the sound is played based on the horizontal position of the bullet.

Listing 2.136: Sound-Stereo – Class Bullet

```

91 class Bullet(pygame.sprite.Sprite):
92
93     SOUND_FIRE = None                                # Only one shared sound is needed
94
95     def __init__(self, tank: Tank) -> None:
96         super().__init__()
97         bulletspeed = 300
98         number: dict[str, int] = {"left": 49, "right": 61, "up": 37, "down": 73}
99         directions = {
100             "left": pygame.Vector2(-bulletspeed, 0),
101             "right": pygame.Vector2(bulletspeed, 0),
102             "up": pygame.Vector2(0, -bulletspeed),
103             "down": pygame.Vector2(0, bulletspeed),
104         }
105         fullfilename = os.path.join(cfg.PATH["image"],
106                                     f"tankbrigade_part{number[tank.direction]}.png")
107         self.image = pygame.image.load(fullfilename).convert()
108         self.image.set_colorkey("black")
109         self.rect = self.image.get_rect()
110         self.direction = tank.direction
111         self.rect.center = tank.rect.center
112         self.speed = directions[tank.direction]
113
114         if Bullet.SOUND_FIRE == None:                  #
115             Bullet.SOUND_FIRE = pygame.mixer.Sound(cfg.get_sound("tank_fire1.wav"))
116             volume_right = self.rect.centerx / cfg.WINDOW.width
117             volume_left = 1 - volume_right
118             self.channel: pygame.mixer.Channel = pygame.mixer.find_channel()
119             if self.channel:
120                 self.channel.set_volume(volume_left, volume_right)
121                 self.channel.play(Bullet.SOUND_FIRE)
122
123     def update(self, *args: Any, **kwargs: Any) -> None:
124         self.rect.move_ip(self.speed * cfg.DELTATIME)
125         if not cfg.WINDOW.contains(self.rect):
126             self.kill()

```

The remaining source code is shown here only for the sake of completeness.

Listing 2.137: Sound-Stereo – Rest

```

128 class Game:
129
130     def __init__(self) -> None:
131         pygame.init()
132         self.window = pygame.Window(size=cfg.WINDOW.size, title="Steroe Panning Sound
133                               Example")
134         self.screen = self.window.get_surface()
135         self.clock = pygame.time.Clock()
136
137         self.ground = pygame.sprite.GroupSingle(Ground())
138         self.tankreference = Tank()
139         self.tank = pygame.sprite.GroupSingle(self.tankreference)
140         self.all_bullets = pygame.sprite.Group()
141         self.running = True

```

```

142     def watch_for_events(self) -> None:
143         for event in pygame.event.get():
144             if event.type == QUIT:
145                 self.running = False
146             elif event.type == KEYDOWN:
147                 if event.key == K_ESCAPE:
148                     self.running = False
149                 elif event.key == K_UP:
150                     self.tank.update(turn="up")
151                 elif event.key == K_DOWN:
152                     self.tank.update(turn="down")
153                 elif event.key == K_LEFT:
154                     self.tank.update(turn="left")
155                 elif event.key == K_RIGHT:
156                     self.tank.update(turn="right")
157                 elif event.key == K_SPACE:
158                     self.fire()
159
160     def fire(self) -> None:
161         if len(self.all_bullets) < 5:
162             self.all_bullets.add(Bullet(self.tankreference))
163
164     def draw(self) -> None:
165         self.ground.draw(self.screen)
166         self.tank.draw(self.screen)
167         self.all_bullets.draw(self.screen)
168         self.window.flip()
169
170     def update(self) -> None:
171         self.tank.update(go=True)
172         self.all_bullets.update()
173
174     def run(self) -> None:
175         time_previous = time()
176         self.running = True
177         while self.running:
178             self.watch_for_events()
179             self.update()
180             self.draw()
181             self.clock.tick(cfg.FPS)
182             time_current = time()
183             cfg.DELTATIME = time_current - time_previous
184             time_previous = time_current
185         pygame.quit()

```

2.11.2.2 Sound Formats and Technical Basics

Pygame does not support all audio formats equally well. The following formats have proven to be particularly reliable:

- .wav – uncompressed, fast to load, ideal for sound effects
- .ogg – compressed, well suited for music
- .mp3 – limited support, not recommended

Don'ts

- ⚠ Mono sounds are often used for sound effects because they require less memory and can be positioned spatially more effectively.
- ⚠ Large files and high sample rates can negatively affect loading times and performance.

2.11.2.3 Volume Hierarchies and Sound Mixing

Games often use multiple volume levels:

1. Master volume (everything)
2. Music volume
3. Effect volume

These can be combined:

```

1     master_volume = 0.8
2     effects_volume = 0.5
3
4     sound.set_volume(master_volume * effects_volume)

```

This makes it easy to implement audio settings for game menus later on.

2.11.2.4 Mono Sounds and Stereo Panning

Mono sounds are particularly suitable for position-dependent audio. Only mono sounds can be cleanly distributed between the left and right speakers.

```

1     channel.set_volume(left, right)

```

Don'ts

Stereo sounds already contain spatial information and may produce unexpected results when additional panning is applied.

2.11.2.5 Sound Lifetime and Resource Management

Sounds should not be reloaded for every event. Instead, they should be loaded once and reused.

```

1      class Bullet:
2          sound_fire = None
3
4          def __init__(self):
5              if sound_fire is None:
6                  sound_fire = pygame.mixer.Sound("fire.wav")

```

This saves memory and avoids unnecessary loading times.

2.11.2.6 Event-driven Sound Output

Sounds should be played in an event-driven manner, not frame-based.

It is incorrect to call `sound.play()` directly or indirectly inside the update loop. Instead, sounds should be triggered by events:

```

1      elif event.key == K_SPACE:
2          sound.play()

```

2.11.2.7 Looping and Transitions

Loops are used for continuous sounds (engines, wind, music):

```

1      channel.play(sound, loops=-1)

```

Smooth transitions can be achieved using fade-in and fade-out effects:

```

1      sound.fadeout(1000)    # 1 second

```

2.11.2.8 Muting and Pausing

Many games offer an option to mute or pause sound globally.

```

1      pygame.mixer.pause()
2      ...
3      pygame.mixer.unpause()

```

Alternatively, this can be done via volume control:

```
1 pygame.mixer.music.set_volume(0)
```

This is especially important for pause menus or when the game window loses focus.

2.11.2.9 Typical Errors and Debugging

Common problems with sound in Pygame include:

- ⚠ Mixer not initialized
- ⚠ Sound played too frequently
- ⚠ No free channels available
- ⚠ Distorted sound (incorrect format)
- ⚠ Meaningless increase of the number of sound channels, such as `pygame.mixer.set_num_channels(16)`

Checking the current state often helps:

```
1 print(pygame.mixer.music.get_busy())
```

2.11.3 What was new?

Two options are available for sound support. One option is background music, while the other uses individual sounds played on different channels and, if possible, distributed across the left and right speakers.



The following Pygame elements were introduced



- 🎮 `pygame.mixer.Channel` :
<https://pyga.me/docs/ref/music.html#pygame.mixer.Channel>
- 🎮 `pygame.mixer.Channel.play()`:
<https://pyga.me/docs/ref/mixer.html#pygame.mixer.Channel.play>
- 🎮 `pygame.mixer.Channel.set_volume()`:
https://pyga.me/docs/ref/mixer.html#pygame.mixer.Channel.set_volume

-  `pygame.mixer.find_channel()` :
https://pyga.me/docs/ref/music.html#pygame.mixer.find_channel
-  `pygame.mixer.init()`:
<https://pyga.me/docs/ref/mixer.html#pygame.mixer.init>
-  `pygame.mixer.set_num_channels()`:
https://pyga.me/docs/ref/mixer.html#pygame.mixer.set_num_channels
-  `pygame.mixer.music.fadeout()`:
<https://pyga.me/docs/ref/music.html#pygame.mixer.music.fadeout>
-  `pygame.mixer.music.get_busy()` :
https://pyga.me/docs/ref/music.html#pygame.mixer.music.get_busy
-  `pygame.mixer.music.get_volume()`:
https://pyga.me/docs/ref/music.html#pygame.mixer.music.get_volume
-  `pygame.mixer.music.load()`:
<https://pyga.me/docs/ref/music.html#pygame.mixer.music.load>
-  `pygame.mixer.music.pause()`:
<https://pyga.me/docs/ref/music.html#pygame.mixer.music.pause>
-  `pygame.mixer.music.play()`:
<https://pyga.me/docs/ref/music.html#pygame.mixer.music.play>
-  `pygame.mixer.music.set_volume()`:
https://pyga.me/docs/ref/music.html#pygame.mixer.music.set_volume
-  `pygame.mixer.music.unpause()`:
<https://pyga.me/docs/ref/music.html#pygame.mixer.music.unpause>
-  `pygame.mixer.Sound`:
<https://pyga.me/docs/ref/mixer.html#pygame.mixer.Sound>
-  `pygame.mixer.Sound.get_volume()`:
https://pyga.me/docs/ref/mixer.html#pygame.mixer.Sound.get_volume
-  `pygame.mixer.Sound.play()`:
<https://pyga.me/docs/ref/mixer.html#pygame.mixer.Sound.play>
-  `pygame.mixer.Sound.set_volume()`:
https://pyga.me/docs/ref/mixer.html#pygame.mixer.Sound.set_volume

2.12 Events

2.12.1 Introduction

We have already used events ([event](#)) in two places without examining them in more detail. On the one hand, this happened when we talked about the keyboard in chapter 2.6 on page 89, and on the other hand when we discussed the mouse in chapter 2.10 on page 146.

Here, we will take a closer look at three aspects

- 👉 What information is actually contained in an event?
- 👉 How can I create an event myself?
- 👉 How can events be generated periodically?

2.12.1.1 What Information is Contained in an Event?

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/12%20Events/event00

The program shown in source code 2.138 simply creates a gray window and prints the event to the console using `print()` in line 31.

Listing 2.138: Events – outputting information

```

29  def watch_for_events(self) -> None:
30      for event in pygame.event.get():
31          print(event)                                # Print event information
32          if event.type == QUIT:
33              self.running = False
34          elif event.type == KEYDOWN:
35              if event.key == K_ESCAPE:
36                  self.running = False

```

If you now move the mouse back and forth, press a few keys, or close the application, something like the following will appear in the console. Many redundant lines have been removed here:

Listing 2.139: Events – console output

```

1 <Event(769-KeyUp {'unicode': 'd', 'key': 100, 'mod': 4096, 'scancode': 7, 'window': None})>
2 <Event(768-KeyDown {'unicode': 'a', 'key': 97, 'mod': 4096, 'scancode': 4, 'window': None})>
3 <Event(771-TextInput {'text': 'a', 'window': None})>
4 <Event(768-KeyDown {'unicode': ' ', 'key': 32, 'mod': 4096, 'scancode': 44, 'window': None})>
5 <Event(771-TextInput {'text': ' ', 'window': None})>
6 <Event(1024-MouseMotion {'pos': (299, 143), 'rel': (-1, 0), 'buttons': (0, 0, 0), 'touch':
  False, 'window': None})>
7 <Event(1024-MouseMotion {'pos': (297, 143), 'rel': (-2, 0), 'buttons': (0, 0, 0), 'touch':
  False, 'window': None})>
8 <Event(1025-MouseButtonDown {'pos': (230, 118), 'button': 1, 'touch': False, 'window':
  None})>
9 <Event(1026-MouseButtonUp {'pos': (230, 118), 'button': 1, 'touch': False, 'window': None})>
10 <Event(1027-MouseWheel {'flipped': False, 'x': 0, 'y': 1, 'precise_x': 0.0, 'precise_y':
  1.0, 'touch': False, 'window': None})>
11 <Event(1025-MouseButtonDown {'pos': (230, 118), 'button': 5, 'touch': False, 'window':
  None})>
12 <Event(1026-MouseButtonUp {'pos': (230, 118), 'button': 5, 'touch': False, 'window': None})>
13 <Event(1027-MouseWheel {'flipped': False, 'x': 0, 'y': -1, 'precise_x': 0.0, 'precise_y':
  -1.0, 'touch': False, 'window': None})>
14 <Event(1024-MouseMotion {'pos': (572, 0), 'rel': (3, -1), 'buttons': (0, 0, 0), 'touch':
  False, 'window': None})>
15 <Event(32768-ActiveEvent {'gain': 0, 'state': 1})>
16 <Event(32784-WindowLeave {'window': None})>
17 <Event(32787-WindowClose {'window': None})>
18 <Event(256-Quit {})>

```

At first, it becomes apparent that the event information is provided in the form of a dictionary. The first entry (the number with a hyphen followed by a name) can be accessed via `event.type`. So that you do not have to memorize these numbers, Pygame provides corresponding constants; an overview for the keyboard can be found in table 2.7 on page 101, and for the mouse in table 2.9 on page 154.

The key/value pairs inside the curly braces contain the information associated with the event. For keyboard events, this includes, for example, the representation as a Unicode character or its Unicode number. Mouse events are sensibly provided with the position and the button number. Clicking the *window close* button in the upper right corner triggers several events; the last four of the list are shown here.

We will soon see that, for user-defined events, this information can be defined according to our own requirements.

2.12.1.2 How can I Create and Use User-defined Events?

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/12%20Events/event01

As an example, I will use two simple buttons here, each of which should generate an event when the left mouse button is clicked. Inside the screen, `NOFPARTICLES`

many particles move around. Using the **Stop** and **Start** buttons, the particles can be stopped and started again.

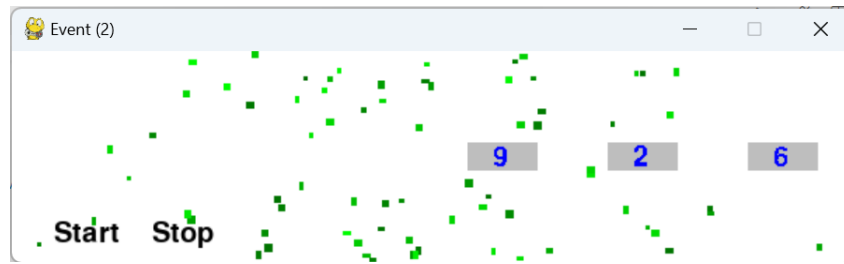


Figure 2.61: User-defined events

As an additional feature, a kind of counter is implemented. The boxes in the center absorb the particles and count them. The logic works as follows: Each time a particle hits a box, a counting event is triggered. In this process, a 1 is always added to the box on the far right.

When the rightmost box reaches the value 10, it generates an overflow to the next digit to its left and resets itself to 0. This process continues from right to left. In this way, the boxes display the total number of particles that have already been absorbed.

Now let us look at the whole setup in detail. In the console output above (see page 172), a unique number can be seen for each event, which can be used to identify the event.

cus-
tom_type()

Pygame reserves a range of numbers for user-defined events between the constants `pygame.USEREVENT` and `pygame.USEREVENTS - 1`.

For each user-defined event, such a unique number must be assigned. The simplest approach is to call `pygame.event.custom_type()`. You can find corresponding examples in line 11 and line 12.

I encapsulate these definitions in a static class for no other reason than that it allows me to make good use of the editor's auto-completion (line 10).

Listing 2.140: Events (2) – config.py

```

1 import pygame
2
3 WINDOW = pygame.Rect((0, 0), (600, 150))
4 FPS = 60
5 DELTATIME = 1.0 / FPS
6 STARTNOFPARTICLES = 999
7 NOFBBOXES = 3
8 BOXWIDTH = 50
9
10 class MyEvents:                                # Only for autocompletion (convenience)
11     BUTTONPRESSED = pygame.event.custom_type() # Event id for button presses
12     OVERFLOW = pygame.event.custom_type()      # Event id for overflow

```

Event

The `Button` class should also be understandable for the most part. The first interesting section can be found in line 25. Here, a new `pygame.event.Event` object is created. As the first parameter, the previously mentioned ID must be specified. After that, any number of additional pieces of information can be passed as event data. In our example, the button text is included so that it can later be determined which button was pressed.

post()

Afterwards, in line 26, the event is dispatched using `pygame.event.post()`.

Listing 2.141: Events (2) – Class Button

```

12 class Button(pygame.sprite.Sprite):
13
14     def __init__(self, text: str, position: Tuple[int], *groups: Tuple[pygame.sprite.Group])
15         -> None:
16         super().__init__(*groups)
17         self.font = pygame.font.SysFont(None, 30)
18         self.centerxy = (cfg.WINDOW.centerx, self.font.get_height() // 2)
19         self.text = text
20         self.image = self.font.render(self.text, True, "black")
21         self.rect = self.image.get_rect(topleft=(position))
22
23     def update(self, *args: Any, **kwargs: Any) -> None:
24         if "action" in kwargs.keys():
25             if kwargs["action"] == "pressed":
26                 evt = pygame.event.Event(MyEvents.BUTTONPRESSED, text=self.text) #
27                 pygame.event.post(evt) #
28                 return super().update(*args, **kwargs)

```

The `Particle` class consists of a lot of source code with little that is new. Particles of random size, color, direction, and speed move across the screen and may bounce off the edges. They do not contain any event-specific functionality. The attribute `halted` is used to stop the particle or let it move again after the buttons have been pressed.

Listing 2.142: Events (2) – Class Particle

```

30 class Particle(pygame.sprite.Sprite):
31
32     def __init__(self, *groups: Tuple[pygame.sprite.Group]) -> None:
33         super().__init__(*groups)
34         self.image = pygame.Surface((randint(3, 6), randint(3, 6)))
35         self.image.fill((0, randint(100, 255), 0))
36         self.rect = pygame.FRect(self.image.get_rect())
37         self.rect.topleft = (
38             randint(30, cfg.WINDOW.right - 30),
39             randint(30, cfg.WINDOW.bottom - 30),
40         )
41         self.speed = randint(100, 400)
42         self.direction = pygame.Vector2(choice((-1, 1)), choice((-1, 1)))
43         self.halted = False
44
45     def update(self, *args: Any, **kwargs: Any) -> None:
46         if "action" in kwargs.keys():
47             if kwargs["action"] == "move":
48                 if not self.halted:
49                     self._move()
50             elif kwargs["action"] == "Start":
51                 self.halted = False
52             elif kwargs["action"] == "Stop":
53                 self.halted = True

```

```

54
55     def _move(self) -> None:
56         self.rect.move_ip(self.speed * self.direction * cfg.DELTATIME)
57         if self.rect.left < cfg.WINDOW.left or self.rect.right > cfg.WINDOW.right:
58             self.direction[0] *= -1
59         if self.rect.top < cfg.WINDOW.top or self.rect.bottom > cfg.WINDOW.bottom:
60             self.direction[1] *= -1
61         self.rect.clamp_ip(cfg.WINDOW)

```

With Box, a kind of digit box is implemented. The constructor receives a position and an index as parameters. The meaning of the parameter **position** should be clear. Using **index**, it can later be determined which box caused an overflow to the next higher power of ten.

In **update()**, the internal counter **count** is increased by 1 each time. If the value 10 is reached (line 79), an event is generated and the index is passed as event data. This allows the main program to determine which box now needs to receive an **update()** call.

Listing 2.143: Events (2) – Klasse Box

```

64 class Box(pygame.sprite.Sprite):
65
66     def __init__(self, index: int, position: Tuple[int], *groups:
67         Tuple[pygame.sprite.Group]) -> None:
68         super().__init__(*groups)
69         self.image = pygame.Surface((cfg.BOXWIDTH, 20))
70         self.rect = self.image.get_rect(center=position)
71         self.font = pygame.font.SysFont(None, 30)
72         self.counter = 0
73         self.index = index
74         self.fill()
75
76     def update(self, *args: Any, **kwargs: Any) -> None:
77         if "counter" in kwargs.keys():
78             if kwargs["counter"] == "inc":
79                 self.counter += 1
80                 if self.counter == 10: # Overflow
81                     evt = pygame.event.Event(MyEvents.OVERFLOW, index=self.index)
82                     pygame.event.post(evt)
83                     self.counter = 0
84                     self.fill()
85                 return super().update(*args, **kwargs)
86
87     def fill(self) -> None:
88         self.image.fill("gray")
89         number = self.font.render(f"{self.counter}", False, "Blue")
90         self.image.blit(number, (18, 1))

```

And now the main program: In the constructor, the buttons, boxes, and particles are created and assigned to sprite groups.

Listing 2.144: Events (2) – Constructor of Game

```

92 class Game:
93
94     def __init__(self) -> None:
95         pygame.init()
96         self.window = pygame.Window(size=cfg.WINDOW.size, title="Event (1)",
97             position=WINDOWPOS_CENTERED)

```

```

97         self.screen = self.window.get_surface()
98         self.clock = pygame.time.Clock()
99         self.running = True
100         self.all_sprites = pygame.sprite.Group()
101         self.all_particles = pygame.sprite.Group()
102         self.generate_particles(cfg.STARTNOFPARTICLES)
103         self.all_buttons = pygame.sprite.Group()
104         self.all_buttons.add(Button("Start", (30, cfg.WINDOW.bottom - 30), self.all_sprites))
105         self.all_buttons.add(Button("Stop", (100, cfg.WINDOW.bottom - 30), self.all_sprites))
106         self.all_boxes = pygame.sprite.Group()
107         self.generate_boxes(cfg.NOFBBOXES)
108         self.running = True

```

The `run()` method is almost boring.

Listing 2.145: Events (2) – `Game.run()`

```

110     def run(self) -> None:
111         time_previous = time()
112         while self.running:
113             self.watch_for_events()
114             self.update()
115             self.draw()
116             self.clock.tick(cfg.FPS)
117             time_current = time()
118             cfg.DELTATIME = time_current - time_previous
119             time_previous = time_current
120         pygame.quit()

```

User-defined events are handled in exactly the same way as predefined ones.

First, you check the `type`, and then you process the event data. In line 132, it is checked whether one of the two buttons was pressed. Afterwards, the message is forwarded to the particles via the event field `text`, telling them whether they should stop or keep moving. The same idea is used starting at line 134. First, it is checked whether a box has overflowed, and then the next box is informed – using the event field `index` – that it has to increase by 1.

Listing 2.146: Events (2) – `Game.watch_for_events()`

```

122     def watch_for_events(self) -> None:
123         for event in pygame.event.get():
124             if event.type == QUIT:
125                 self.running = False
126             elif event.type == KEYDOWN:
127                 if event.key == K_ESCAPE:
128                     self.running = False
129             elif event.type == MOUSEBUTTONDOWN:
130                 if event.button == 1:
131                     self.check_button_pressed(event.pos)
132             elif event.type == MyEvents.BUTTONPRESSED: #
133                 self.all_particles.update(action=event.text)
134             elif event.type == MyEvents.OVERFLOW: #
135                 if event.index < cfg.NOFBBOXES - 1:
136                     self.all_boxes.sprites()[event.index + 1].update(counter="inc")

```

The rest is shown here for completeness.

Listing 2.147: Events (2) – The Rest of Game

```

138 def update(self):
139     self.all_buttons.update()
140     self.all_particles.update(action="move")
141     self.check_boxcollision()
142
143 def draw(self) -> None:
144     self.screen.fill("white")
145     self.all_sprites.draw(self.screen)
146     self.window.flip()
147
148 def generate_boxes(self, number: int) -> None:
149     for i in range(number):
150         self.all_boxes.add(Box(i, (cfg.WINDOW.right - 50 - i * 100, cfg.WINDOW.centery),
151             self.all_sprites))
152
153 def generate_particles(self, number: int) -> None:
154     for i in range(number):
155         self.all_particles.add(Particle(self.all_sprites))
156
157 def check_button_pressed(self, position: Tuple[int]) -> None:
158     for b in self.all_buttons.sprites():
159         if b.rect.collidepoint(position):
160             b.update(action="pressed")
161
162 def check_boxcollision(self) -> None:
163     c = pygame.sprite.groupcollide(self.all_particles, self.all_boxes, True, False)
164     for _ in c:
165         self.all_boxes.sprites()[0].update(counter="inc")

```

Finally, I would like to briefly explain the whole mechanism of the counter again using figure 2.62 on the next page, in order to make it clearer how the event `MyEvent.OVERFLOW` works in this context.

- t_0 : First, a standard collision check determines that a particle has hit the rectangle of the rightmost field (see figure 2.61 on page 173, the box with the 6). As a result, the method `update(counter="inc")` is called.
- t_1 : This causes the event `MyEvents.OVERFLOW` to be triggered in the `Box` class with the index value 0. This event is caught in `watch_for_event()` and forwarded to the appropriate box – that is, the one with index `index+1` – together with the instruction to also execute `update(counter="inc")` there.
- t_2 : Since this box currently contains the value 9, `MyEvents.OVERFLOW` is triggered again in this box, but now with the next index value, namely 1. This event is again caught in `watch_for_event()` and forwarded to box 2 with `update(counter="inc")`.
- t_3 : The value of the leftmost box is currently 0 and is increased by 1 by the call to `update(counter="inc")`. Since no overflow is generated in this case, the chain of events stops here.

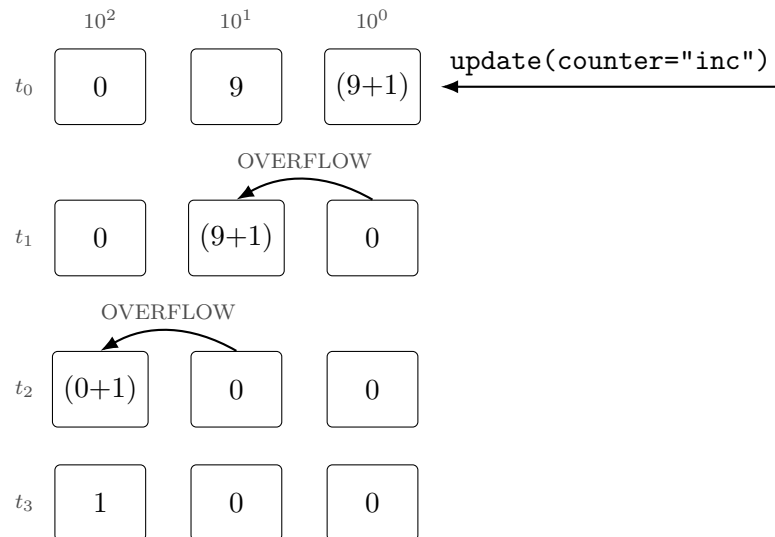


Figure 2.62: How the counter works

2.12.2 More Input

2.12.2.1 How can periodic events be generated?

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/00%20Introduction/12%20Events/event02

This is actually quite simple. The previous example is extended so that new particles are created at intervals of 500 ms. To achieve this, a new ID `NEWPARTICLES` is first defined for the user event.

Listing 2.148: Events (3) – `config.py`

```

4 class MyEvents:
5     BUTTONPRESSED = pygame.event.custom_type()
6     OVERFLOW = pygame.event.custom_type()
7     NEWPARTICLES = pygame.event.custom_type()
8
9
10 WINDOW = pygame.Rect((0, 0), (600, 150))
11 FPS = 60
12 DELTATIME = 1.0 / FPS
13 STARTNOPARTICLES = 500
14 NEWNOPARTICLES = 10
15 NOFBBOXES = 5
16 BOXWIDTH = 50

```

In the constructor of `Game`, a periodic timer is set in line 123 using `pygame.time.set_timer()`. This timer fires the corresponding event ID every 500 ms.

`set_timer()`

Listing 2.149: Events (3) – Timer in the constructor of `Game`

```
122     self.generate_boxes(cfg.NOBOXES)
123     pygame.time.set_timer(MyEvents.NEWPARTICLES, 500) # Periodic event
124     self.running = True
```

Like the other events, this one is caught in `watch_for_event()` (line 153) and processed. In this case, this is done by calling the method `generate_particles()`.

Listing 2.150: Events (3) – Catching a periodical event

```
153         elif event.type == MyEvents.NEWPARTICLES: # Periodic Event
154             self.generate_particles(cfg.NEWNOPARTICLES)
```

2.12.2.2 Structuring the Event Loop Correctly

Best practice

In each frame, the event loop should follow a clear and consistent structure:

1. Retrieve all events from the event queue
2. Process the events
3. Update the game state
4. Render the scene

```
1 ...
2     for event in pygame.event.get():
3         if event.type == pygame.QUIT:
4             running = False
5         handle_event(event)
6 ...
7     update()
8     draw()
```

Common mistakes

- ⚠ Failing to regularly process events can cause the application window to become unresponsive.
- ⚠ Forgetting to call `pygame.event.get()` or processing events only sporadically.

 Using `event.wait()` in the main loop, which can block rendering and updates.

2.12.2.3 Choosing the Right Event Retrieval Method

Pygame provides different ways to access events:

-  `pygame.event.get()` retrieves all pending events (standard choice for games)
-  `pygame.event.poll()` retrieves a single event
-  `pygame.event.wait()` blocks until an event occurs

2.12.2.4 Avoid Generating Events Every Frame

Events should represent state changes, not continuous states

Common mistake: Posting custom events or playing sounds inside the `update()` method every frame, which can flood the event queue and cause performance issues. Generate events only when a condition changes (edge-triggered behavior).

```

1     if not was_pressed and is_pressed:
2         pygame.event.post(pygame.event.Event(MyEvents.FIRE))
3         was_pressed = is_pressed

```

2.12.2.5 Defining Event Data Clearly and Consistently

When creating custom events, meaningful and consistent event data should be attached.

```

1     pygame.event.post(pygame.event.Event(MyEvents.BUTTON, {"action": "start"}))

```

Use clear, descriptive keys (e.g. `action`, `index`, `pos`) and stick to a consistent naming scheme (snake_case). Using inconsistent field names across different events, making event handling error-prone.

2.12.2.6 Managing User-Defined Event IDs

Best practice

Define all custom event IDs centrally, for example in a configuration file.

```
1 BUTTON_EVENT    = pygame.event.custom_type()
2 OVERFLOW_EVENT  = pygame.event.custom_type()
3 NEWPARTICLES    = pygame.event.custom_type()
```

2.12.2.7 Avoid Manually Set Event IDs

Bad practice

If you use the method shown below, event IDs may be assigned multiple times when they are defined across different source files. It is therefore preferable to use `pygame.event.custom_type()` instead.

```
1 BUTTON_EVENT    = pygame.USEREVENT + 1
2 OVERFLOW_EVENT  = pygame.USEREVENT + 2
3 NEWPARTICLES    = pygame.USEREVENT + 3
```

2.12.2.8 Use `set_timer()` Correctly!

Best practice

Set timers only once (e.g. in the constructor) and disable timers when they are no longer needed.

```
1 pygame.time.set_timer(NEWPARTICLES, 0)
```

Setting the same timer multiple times and forgetting to disable timers when restarting a game or switching scenes.

2.12.2.9 Filtering Events for Performance

In event-heavy applications, it may be useful to restrict which events are allowed.

```

1  pygame.event.set_allowed([pygame.QUIT,
2                             pygame.KEYDOWN,
3                             pygame.MOUSEBUTTONDOWN,
4                             NEWPARTICLES])

```

Use event filtering sparingly and only when necessary. Blocking too many events can accidentally preventing essential input from being processed.

2.12.2.10 Event-Based Input vs. State-Based Input

There are two complementary approaches to input handling:

1. Event-based input: Reacts to discrete events (KEYDOWN, KEYUP) and is ideal for actions like shooting, opening menus, or triggering sounds.
2. State-based input: Uses continuous state queries (key.get_pressed()) and can be used for movement and continuous control.

Combine both approaches appropriately.

2.12.2.11 Window Focus and Application State

Best practice

Games should respond appropriately when the window loses focus or is minimized. Pause the game or mute sound when focus is lost and always handle the QUIT event reliably.

2.12.2.12 Debugging Events Effectively

Printing events to the console is useful during development but should be done selectively.

```

1  for event in pygame.event.get():
2      if event.type != pygame.MOUSEMOTION:
3          print(event)

```

Common mistake

Logging every mouse movement event, which can overwhelm the console and reduce performance.

2.12.2.13 Structuring Event Handling Code

Best practice

As projects grow, large if-elif blocks become hard to maintain. Use handler functions or dispatch tables.

```

1     handlers = {
2         pygame.KEYDOWN: handle_keydown,
3         pygame.MOUSEBUTTONDOWN: handle_mouse,
4         NEWPARTICLES: handle_newparticles,
5     }
6
7     for event in pygame.event.get():
8         handlers.get(event.type, handle_default)(event)

```

This approach improves readability and scalability.

2.12.3 What was new?

The advantage of user-defined events becomes very clear here. If this were implemented in a different way, the objects would have to know about each other. For example, all boxes would have to know their predecessor or successor via references in order to report an overflow. While this can also be a valid approach, using events decouples the classes, and the main program can control and organize the forwarding of information via the event data.

In particular, clicking on the buttons can be implemented very easily using events.



The following Pygame elements were introduced



 **USEREVENT :**

<https://pyga.me/docs/ref/event.html#pygame.event>

 **NUMEVENTS :**

<https://pyga.me/docs/ref/event.html#pygame.event>

 **pygame.event.custom_type():**

https://pyga.me/docs/ref/event.html#pygame.event.custom_type

 **pygame.event.Event:**

<https://pyga.me/docs/ref/event.html#pygame.event.Event>

-  `pygame.event.get()` :
<https://pyga.me/docs/ref/event.html#pygame.event.get>
-  `pygame.event.poll()` :
<https://pyga.me/docs/ref/event.html#pygame.event.poll>
-  `pygame.event.post()`:
<https://pyga.me/docs/ref/event.html#pygame.event.post>
-  `pygame.event.wait()`:
<https://pyga.me/docs/ref/event.html#pygame.event.wait>
-  `pygame.time.set_allowed()`:
https://pyga.me/docs/ref/time.html#pygame.time.set_allowed
-  `pygame.time.set_timer()`:
https://pyga.me/docs/ref/time.html#pygame.time.set_timer
-  `pygame.WINDOWPOS_CENTERED`:
<https://pyga.me/docs/ref/window.html#pygame.Window.position>

3 Techniques

3.1 Animation

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/01%20Animation

An animation is essentially a small *movie* inside a game. Examples of useful animations include movements, explosions, pulsing effects, and changes in appearance. Here, I would like to present two examples: a small movement and an explosion.

3.1.1 The Running Cat



Figure 3.1: Animation of a cat: frame sprites

You can see the individual frames of the movement example in figure 3.1. If these individual sprites are displayed one after another at a certain speed, they appear as a smooth movement. The following rule applies: the more individual frames are used, the smoother the animation appears.

At first the `config.py`:

Listing 3.1: The running cat, `config.py`

```
1 from os import path
2
3 import pygame
4
5 WINDOW = pygame.Rect((0, 0), (300, 200))
```

```

6 FPS = 60
7 DELTATIME = 1.0 / FPS
8 TITLE = "Animation"
9 PATH: dict[str, str] = {}
10 PATH["file"] = path.dirname(path.abspath(__file__))
11 PATH["image"] = path.join(PATH["file"], "images")
12
13 @staticmethod
14 def filepath(name: str) -> str:
15     return path.join(PATH["file"], name)
16
17 @staticmethod
18 def imagepath(name: str) -> str:
19     return path.join(PATH["image"], name)

```

The source code in source code 3.2 differs from the chapter above (see section 2.9.2.1 on page 142 by only one feature. The `Timer` class has been extended by the method `change_duration()`. This method makes it possible to change the duration of the time interval at runtime, with a lower limit of 0ms. We will use this feature shortly to manually adjust the animation speed.

Listing 3.2: The running cat (1), Version 1.0 – Timer

```

8 class Timer:
9
10     def __init__(self, duration: int, with_start: bool = True):
11         self.duration = duration
12         if with_start:
13             self.next = pygame.time.get_ticks()
14         else:
15             self.next = pygame.time.get_ticks() + self.duration
16
17     def is_next_stop_reached(self) -> bool:
18         if pygame.time.get_ticks() > self.next:
19             self.next = pygame.time.get_ticks() + self.duration
20             return True
21         return False
22
23     def change_duration(self, delta: int = 10):
24         self.duration += delta
25         if self.duration < 0:
26             self.duration = 0

```

If we want to animate something, this animation does not require just a single sprite for display, but several. For this reason, in addition to the `image` attribute, I introduced another one: the list `images`. Using a for-loop starting at line 34, I now load all bitmaps of the animation into this list.

We now need an attribute that keeps track of which of the 6 sprites should currently be displayed: `imageindex`. If the images are stored in the `images` array in the same order in which they are supposed to be displayed, `imageindex` only needs to be incremented. We also need a `Timer` object so that the animation does not run absurdly fast – we start here with 100ms.

In the `update()` method, the `imageindex` attribute is incremented by 1 depending on the `Timer` object, and the corresponding bitmap is then assigned to the `image` attribute so that the familiar `Sprite` features can be used. The method `change_animation_time()`

simply forwards its parameter to the `Timer` object. With this, all preparatory steps are essentially complete.

Listing 3.3: The running cat (2), Version 1.0 – Cat

```

29 class Cat(pygame.sprite.Sprite):
30
31     def __init__(self) -> None:
32         super().__init__()
33         self.images: list[pygame.Surface] = []
34         for i in range(6):
35             # Load animation sprites
36             bitmap = pygame.image.load(cfg.imagepath(f"cat{i}.bmp")).convert()
37             bitmap.set_colorkey("black")
38             self.images.append(bitmap)
39         self.imageindex = 0
40         self.image: pygame.Surface = self.images[self.imageindex]
41         self.rect: pygame.Rect = self.image.get_rect()
42         self.rect.center = cfg.WINDOW.center
43         self.animation_time = Timer(100)
44
45     def update(self, *args: Any, **kwargs: Any) -> None:
46         if "animation_delta" in kwargs.keys():
47             self.change_animation_time(kwargs["animation_delta"])
48         if self.animation_time.is_next_stop_reached():
49             self.imageindex += 1
50             if self.imageindex >= len(self.images):
51                 self.imageindex = 0
52             self.image = self.images[self.imageindex]
53             # implement game logic here
54
55     def change_animation_time(self, delta: int) -> None:
56         self.animation_time.change_duration(delta)

```

The `CatAnimation` class is merely the usual encapsulation of the main program. In line 67, the `Cat` object is created and placed into a `GroupSingle`.

Listing 3.4: The running cat (3), Version 1.0 – Constructor and `run()`

```

58 class CatAnimation:
59
60     def __init__(self) -> None:
61         pygame.init()
62         self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
63         self.screen = self.window.get_surface()
64         self.clock = pygame.time.Clock()
65
66         self.font = pygame.font.Font(pygame.font.get_default_font(), 12)
67         self.cat = pygame.sprite.GroupSingle(Cat()) # My cat sprite
68         self.running = False
69
70     def run(self) -> None:
71         time_previous = time()
72         self.running = True
73         while self.running:
74             self.watch_for_events()
75             self.update()
76             self.draw()
77             self.clock.tick(cfg.FPS)
78             time_current = time()
79             cfg.DELTATIME = time_current - time_previous
80             time_previous = time_current
81         pygame.quit()

```

In `watch_for_events()`, the only noteworthy aspect is that `+` and the `-` key are used to manipulate the animation speed. To increase the animation speed, the time interval of the `Timer` object has to be reduced, hence `-10`. To slow down the animation, the time interval of the `Timer` object has to be increased, hence `+10`.

Listing 3.5: The running cat (4), Version 1.0 – `watch_for_events()`

```

83     def watch_for_events(self) -> None:
84         for event in pygame.event.get():
85             if event.type == pygame.QUIT:
86                 self.running = False
87             elif event.type == pygame.KEYDOWN:
88                 if event.key == pygame.K_ESCAPE:
89                     self.running = False
90                 elif event.key == pygame.K_PLUS:
91                     self.cat.sprite.update(animation_delta=-10)
92                 elif event.key == pygame.K_MINUS:
93                     self.cat.sprite.update(animation_delta=10)

```

The remaining source code (source code 3.6) should be self-explanatory. When you start the program, an animated cat movement will be displayed. Feel free to try changing the animation speed.

Listing 3.6: The running cat (5), Version 1.0 – `update()` and `draw()`

```

97
98     def draw(self) -> None:
99         self.screen.fill("gray")
100         self.cat.draw(self.screen)
101         text_image = self.font.render(f"animation time:
102                                     {self.cat.sprite.animation_time.duration}", True, "white")
103         text_rect = text_image.get_rect()
104         text_rect.centerx = cfg.WINDOW.centerx
105         text_rect.bottom = cfg.WINDOW.bottom - 50
106         self.screen.blit(text_image, text_rect)
107         self.window.flip()

```

3.1.2 The Class Animation

As with time control, I am bothered by the fact that the animation logic is spread across the `Cat` class, which in my opinion violates the SRP. So let us simply build a dedicated animation class (see source code 3.7 on the next page).

Let us take a look at the constructor parameters:

- **namelist:** A list of file names without path information. These are resolved automatically using the entries in `config.py`. The order of the file names must correspond to the animation order.
- **endless:** This flag controls whether the animation repeats indefinitely. `True` means that after the last sprite, the animation starts again with the first one. `False` means that the last sprite remains displayed.

- **animationtime**: The delay between individual sprites in ms.
- **colorkey**: This parameter handles the case where sprites may not have transparency and therefore require an explicit transparency color (see page 47). If no value is provided, the transparency of the loaded sprite is kept as is. If a color value is provided, it is applied using `set_colorkey()` in line 20.

In the `next()` method, the next `imageindex` is calculated and the corresponding sprite is returned. For this purpose, the internal `Timer` object is used so that the sprites appear with a defined time interval. The `imageindex` attribute is increased by 1 and then checked to see whether the end of the sprite list has been reached. If the animation is set to *endless*, the `imageindex` is reset to 0; otherwise, the last image of the list is displayed permanently.

Question to the audience

Why was `imageindex` initialized to `-1` in the constructor?

A feature that is often needed has been implemented in the `is_ended()` method. Frequently, the code that triggered the animation needs to know whether the animation has finished. We will make use of this later on.

Listing 3.7: The running cat (6), Version 1.1 – Animation

```

9 class Animation:
10
11     def __init__(self, namelist: list[str], endless: bool, animationtime: int, colorkey:
12         tuple[int, int, int] | None = None) -> None:
13         self.images: list[pygame.Surface] = []
14         self.endless = endless
15         self.timer = Timer(animationtime)
16         for filename in namelist:
17             if colorkey == None:
18                 bitmap = pygame.image.load(cfg.imagepath(filename)).convert_alpha()
19             else:
20                 bitmap = pygame.image.load(cfg.imagepath(filename)).convert()
21                 bitmap.set_colorkey(colorkey)          # Enable transparency
22             self.images.append(bitmap)
23         self.imageindex = -1
24
25     def next(self) -> pygame.Surface:
26         if self.timer.is_next_stop_reached():
27             self.imageindex += 1
28             if self.imageindex >= len(self.images):
29                 if self.endless:
30                     self.imageindex = 0
31                 else:
32                     self.imageindex = len(self.images) - 1
33             return self.images[self.imageindex]
34
35     def is_ended(self) -> bool:
36         if self.endless:
37             return False
38         return self.imageindex >= len(self.images) - 1

```

This simplifies the `Cat` class, allowing it to focus again on its – admittedly still non-

existent – game logic. The `Animation` object is created here in line 65. The file names can be generated very easily, since they are numbered consecutively. The cat is supposed to run endlessly, with a time interval of 100 ms between the sprites. In `update()`, the `next()` method is then simply called.

Listing 3.8: The running cat (7), Version 1.1 – Cat

```

61 class Cat(pygame.sprite.Sprite):
62
63     def __init__(self) -> None:
64         super().__init__()
65         self.animation = Animation([f"cat{i}.bmp" for i in range(6)], True, 100, (0, 0, 0))
66         #
67         self.image: pygame.Surface = self.animation.next()
68         self.rect: pygame.Rect = self.image.get_rect()
69         self.rect.center = cfg.WINDOW.center
70
71     def update(self, *args: Any, **kwargs: Any) -> None:
72         if "animation_delta" in kwargs.keys():
73             self.change_animation_time(kwargs["animation_delta"])
74             self.image = self.animation.next()
75             # implement game logic here
76
77     def change_animation_time(self, delta: int) -> None:
78         self.animation.timer.change_duration(delta)

```

3.1.3 The Exploding Rock

My second example spawns rocks (meteors) at random positions and at random time intervals. Each rock is also given a certain lifetime – again chosen randomly. After that, it explodes. This explosion is animated.

Let us first take a look at the `Rock` class. In line 66, a random number is generated, which is then used in the following line to load one of four possible rock bitmaps. After that, the coordinates of the rock's center are determined using a random number generator, while keeping a certain distance from the screen borders. In line 71, the `Animation` object is created. Here again, the file names of the animation bitmaps are loaded in the order of the animation. You can see these bitmaps in figure 3.2 on the facing page.

Since the animation should not repeat, the corresponding parameter is set to `False` here. After the explosion, the rock is supposed to disappear. The delay between the individual frames is set to 50 ms. In line 72, the lifetime of the rock is again determined randomly and a corresponding `Timer` object is created – as you can see, these are quite useful and can be reused often. The flag `bumm` is a marker that indicates whether the rock is currently exploding.

The `update()` method has now become quite interesting. First, the `Timer` object is used to check whether the end of the lifetime has been reached. If not, nothing happens here, although one could implement movement or some other meaningful behaviour in the

`else` branch. If the lifetime has been reached, the corresponding flag is set. Depending on this, the animation is then started.

What is the purpose of the three lines starting at line 80? They serve purely visual purposes. The dimensions of the explosion sprites are not always the same, and the `rect` object always aligns them to the upper-left corner, which would result in a visible jitter. To avoid this, the old center position is stored, the new rectangle of the next animation sprite is calculated, and its center is set to the previous position. This keeps the animation nicely aligned to the original center of the rock.

Finally, it is checked whether the animation has finished. If so, the sprite is no longer needed and can be removed from the sprite group using `kill()`.

`kill()`

Listing 3.9: The exploding rock (1) – Rock

```

62 class Rock(pygame.sprite.Sprite):
63
64     def __init__(self):
65         super().__init__()
66         rocknb = random.randint(6, 9)                # Rock image number
67         self.image = pygame.image.load(cfg.imagepath(f"felsen{rocknb}.png")).convert_alpha()
68         self.rect = self.image.get_rect()
69         self.rect.centerx = random.randint(self.rect.width, cfg.WINDOW.width -
            self.rect.width)
70         self.rect.centery = random.randint(self.rect.height, cfg.WINDOW.height -
            self.rect.height)
71         self.anim = Animation([f"explosion0{i}.png" for i in range(1, 5)], False, 50) #
72         self.timer_lifetime = Timer(random.randint(100, 2000), False) # Lifetime timer
73         self.bumm = False
74
75     def update(self, *args: Any, **kwargs: Any) -> None:
76         if self.timer_lifetime.is_next_stop_reached():
77             self.bumm = True
78         if self.bumm:
79             self.image = self.anim.next()
80             c = self.rect.center                      # Center position
81             self.rect = self.image.get_rect()
82             self.rect.center = c
83         if self.anim.is_ended():
84             self.kill()

```



Figure 3.2: The exploding rock: frame sprites

The `ExplosionAnimation` class should no longer pose any difficulty for you. There are only a few places that I would like to briefly address. In line 96, a `Timer` object is created that is supposed to spawn two rocks per second, and in line 121 this timer is checked.

Listing 3.10: The exploding rock (2) – ExplosionAnimation

```

87 class ExplosionAnimation(object):
88
89     def __init__(self) -> None:
90         pygame.init()
91         self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
92         self.screen = self.window.get_surface()
93         self.clock = pygame.time.Clock()
94
95         self.all_rocks = pygame.sprite.Group()
96         self.timer_newrock = Timer(500)           # Timer
97         self.running = False
98
99     def run(self) -> None:
100         time_previous = time()
101         self.running = True
102         while self.running:
103             self.watch_for_events()
104             self.update()
105             self.draw()
106             self.clock.tick(cfg.FPS)
107             time_current = time()
108             cfg.DELTATIME = time_current - time_previous
109             time_previous = time_current
110         pygame.quit()
111
112     def watch_for_events(self) -> None:
113         for event in pygame.event.get():
114             if event.type == QUIT:
115                 self.running = False
116             elif event.type == KEYDOWN:
117                 if event.key == K_ESCAPE:
118                     self.running = False
119
120     def update(self) -> None:
121         if self.timer_newrock.is_next_stop_reached(): # 500ms?
122             self.all_rocks.add(Rock())
123         self.all_rocks.update()
124
125     def draw(self) -> None:
126         self.screen.fill("black")
127         self.all_rocks.draw(self.screen)
128         self.window.flip()

```

Hint

There is also the source file `animation03.py`. In this variant, the rocks move and explode when they collide with each other. Take a look!

3.2 Tiles Are Beautiful

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/02%20Tilemap

Very often, the visual appearance of games consists of many small and large tiles that are assembled in an appropriate way. These tiles are usually combined into larger bitmaps ([SpriteLib](#)) and then have to be cut out correctly by the game developer. In figure 3.3, you can find such a simple sprite library.

SpriteLib

I will now show you how tiles can be cut out of a sprite library and used to assemble your own worlds. The required information is stored in a [CSV files](#). I will also use several example levels (not to be confused with difficulty levels or floors within the game world) in order to address different types of sprites in different ways.

Hint

There is an excellent tool that helps you create such levels and integrate them into a game. It is called [Tiled](#) and can be downloaded from [MapEditor](#). Since there are already sufficiently detailed and high-quality introductions available for this software, I will omit a description here.

3.2.1 Our Example



Figure 3.3: Sprite library (original)

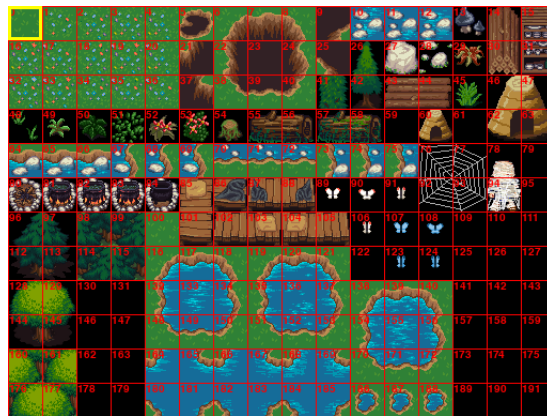


Figure 3.4: Sprite library (prepared)

In figure 3.3, we can see such a sprite library. It contains tiles for building a landscape consisting of lakes, meadows, and forests, along with some [gadget](#). In figure 3.4, I have

Gadget

made the individual tiles visible by adding a grid and numbering them. The numbers will become important again later. Our tiles have a width and a height of 32 px each; they are arranged in 16 columns and 12 rows.

The goal is a game surface as shown in figure 3.5. The playing field – coincidentally – also has 16 columns and 12 rows.



Figure 3.5: Example of tile based playground

3.2.2 A Green Meadow

In the first step, I will show how a single tile can be used to fill the entire game area. So let us get started. In source code 3.11, we first find the usual suspects. The parameters `TILESIZE`, `TILEMAP_NOF_COLS`, `TILEMAP_NOF_ROWS`, and `TILEMAP_WINDOW` should also be self-explanatory.

Listing 3.11: Forest – `config.py`

```

1 import pygame
2
3 FPS = 60
4 DELTATIME = 1.0 / FPS
5 TILESIZE = pygame.math.Vector2(32, 32)
6 TILEMAP_NOF_COLS = 16
7 TILEMAP_NOF_ROWS = 12
8 TILEMAP_WINDOW = pygame.Rect(0, 0,
9                               TILEMAP_NOF_COLS * TILESIZE.x,
10                              TILEMAP_NOF_ROWS * TILESIZE.y)

```

Also `Game` is very easy to understand.

Listing 3.12: Forest – Game

So far, everything happens inside the `WindowGame` class, which will of course change later on. In the constructor, the window is created with all its parameters and the sprite library is loaded. In `draw()`, a single tile is now cut out of the sprite library and stored in the variable `image`. The crucial part here is the parameters passed to `subsurface()`. Starting at position `(0,0)` in the sprite library – i.e. the top-left corner – a rectangle of size `32 px × 32 px` is cut out. This corresponds to the yellow-bordered rectangle 0 in figure 3.4 on page 193.

`subsur-
face()`

Using the two `for` loops, this image is then distributed across the entire surface. In each loop iteration, the variable `position` is calculated from the row number `row` (`[0 – 12]`) and the column number `col` (`[0 – 16]`). Multiplying the column number by the tile width yields the x-position, and multiplying the row number by the tile height yields the y-position at which the tile should be drawn.

The result then looks as shown in figure 3.6 on page 198.

Listing 3.13: Forest – WindowGame

```

7 class WindowGame:
8
9     def __init__(self) -> None:
10         self.window = pygame.Window(size=cfg.TILEMAP_WINDOW.size)
11         self.screen : pygame.Surface = self.window.get_surface()
12         self.spritelib = pygame.image.load("images/forest_tiles.png").convert_alpha()
13         self.rect = self.screen.get_frect()
14         self.window.title = "Tilemap Example"
15         self.clock = pygame.time.Clock()
16
17     def draw(self) -> None:
18         self.screen.fill("black")
19         image = self.spritelib.subsurface(pygame.Rect((0, 0), cfg.TILESIZE))
20         for row in range(cfg.TILEMAP_NOF_ROWS):
21             for col in range(cfg.TILEMAP_NOF_COLS):
22                 position = col * cfg.TILESIZE.x, row * cfg.TILESIZE.y
23                 self.screen.blit(image, position)
24         self.window.flip()

```

However, I would like to be able to select and display any arbitrary tile. For example, instead of a monotonous meadow, we might want it to be composed of the tiles with the numbers 0, 1, 2, 3, 4, 16, 17, 18, 19, 20, 32, 33, 34, 35, 36. This practically calls for a separate class.

In the constructor (see source code 3.14 on the next page), only the bitmap of the sprite library is loaded. The method `subsurface()` expects the tile number as a parameter. The tiles are assumed to be numbered from left to right and from top to bottom. This makes it fairly easy to compute the column number and the row number from the tile number.

The column number is the value that remains after all complete rows have been removed. Example: The tile number is 34. I want to determine the column number. All complete rows above tile 34 are irrelevant. Therefore, with 16 columns, from

$$34 \rightarrow 18 \rightarrow 2.$$

Mathematically, this is the remainder ([modulo](#)) of an integer division. Do you remember? Elementary school? *15 divided by 6 is 2 remainder 3*. Or, in our case,

$$34 \bmod 16 = 2.$$

This is exactly what happens in line 13; the only difference is that the result is then multiplied by the number of pixels per column in order to obtain the left position of the tile.

The row number – that is, the row in which tile 34 is located – is determined in a similar, but slightly different way. In other words: how many complete rows – i. e. 16 tiles – are contained in the tile number? For this, we use integer division:

$$34 \div 16 = 2.$$

And since in computer science we always start counting at 0, the tile is indeed located in row 2. Take a look at figure 3.4 on page 193! In line 13, the row number is then multiplied by the row height to obtain the top position of the tile. The rest should be self-explanatory.

Listing 3.14: Forest – Spritelib

```

8 class Spritelib:
9     def __init__(self, filename: str) -> None:
10         self.image = pygame.image.load(filename).convert_alpha()
11
12     def subsurface(self, tilenumber: int) -> pygame.Surface:
13         left = (tilenumber % cfg.TILEMAP_NOF_COLS) * cfg.TILESIZE.x #
14         top = (tilenumber // cfg.TILEMAP_NOF_COLS) * cfg.TILESIZE.y #
15         tile_rect = pygame.Rect((left, top), cfg.TILESIZE)
16         return self.image.subsurface(tile_rect)

```

Now we integrate the new class into `WindowGame`. The constructor is extended by determining random tile numbers:

Listing 3.15: Forest – Extension of the Constructor of `WindowGame`

```

21 def __init__(self) -> None:
22     self.window = pygame.Window(size=cfg.TILEMAP_WINDOW.size)
23     self.screen : pygame.Surface = self.window.get_surface()
24     self.rect = self.screen.get_frect()
25     self.window.title = "Tilemap Example"
26     self.clock = pygame.time.Clock()
27     self.spritelib = Spritelib("images/forest_tiles.png")
28     self.tiles = []
29     for row in range(cfg.TILEMAP_NOF_ROWS):
30         for col in range(cfg.TILEMAP_NOF_COLS):
31             self.tiles.append(choice((0,1,2,3,4,16,17,18,19,20,32,33,34,35,36)))

```

In `WindowGame.draw()`, we now only need to implement access to the tile number. The result then looks as shown in figure 3.7 on the next page.

Listing 3.16: Forest – Selecting specific tile numbers in `Game.draw()`

```

33 def draw(self) -> None:
34     self.screen.fill("black")
35     for row in range(cfg.TILEMAP_NOF_ROWS):
36         for col in range(cfg.TILEMAP_NOF_COLS):
37             index = self.tiles[row * cfg.TILEMAP_NOF_COLS + col]
38             image = self.spritelib.subsurface(index)
39             position = col * cfg.TILESIZE.x, row * cfg.TILESIZE.y
40             self.screen.blit(image, position)
41     self.window.flip()

```

So, what can we do already?

1. We can provide a sprite library to all components of the game via a separate class.
2. We can determine the row – and thus the vertical pixel position in the sprite library – from the tile number.

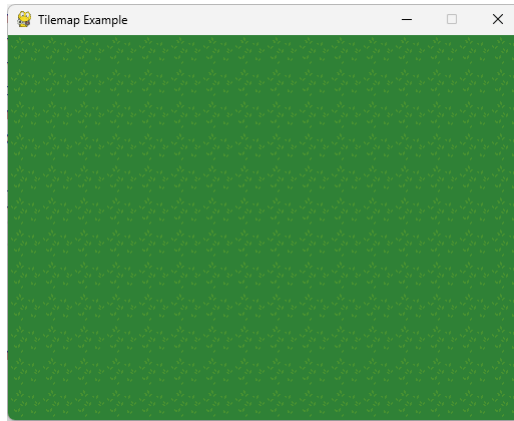


Figure 3.6: Forest playground (1)

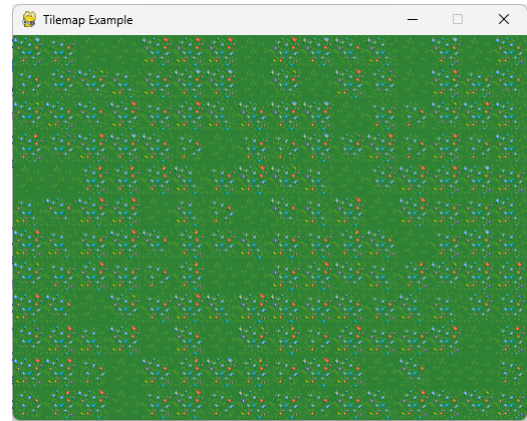


Figure 3.7: Forest playground (2)

3. We can determine the column – and thus the horizontal pixel position in the sprite library – from the tile number.
4. We can compute the upper-left pixel position in the target window – that is, the game playground – from a row and column number.

That is already quite a lot.

3.2.3 Tile Numbers and Two-Dimensional Arrays

In most cases, tiles should not be determined by a random generator. Instead, the tile numbers should be specified explicitly in some way. Two-dimensional arrays are well suited for this purpose, since they allow convenient access using row and column indices.

Nothing could be simpler, you might think. And the reader would be right ;-)

First, let us replace the random selection with a meaningful assignment using a two-dimensional array. The tile numbers are no longer random, but already correspond to those that we want to see later.

Listing 3.17: Forest – Tile numbers in a 2D array

```

18 class WindowGame:
19
20     def __init__(self) -> None:
21         self.window = pygame.Window(size=config.TILEMAP_WINDOW.size)
22         self.screen : pygame.Surface = self.window.get_surface()
23         self.rect = self.screen.get_frect()
24         self.window.title = "Tilemap Example"
25         self.clock = pygame.time.Clock()
26         self.spritelib = Spritelib("images/forest_tiles.png")
27         self.tiles = []
28         self.tiles.append([99,98,113,17,18,17,18,18,17,18,51,17,17,13,34,34])
29         self.tiles.append([115,114,97,60,18,20,34,29,34,34,34,13,34,36,116,117])
30         self.tiles.append([99,115,113,17,18,20,116,117,117,117,64,117,120,117,164,133])
31         self.tiles.append([115,113,18,18,18,20,132,133,133,136,10,136,133,136,155,155])

```

```

32     self.tiles.append([99,97,18,17,104,105,132,136,185,152,71,152,152,149,180,133])
33     self.tiles.append([115,113,18,13,17,20,132,136,137,55,56,0,0,53,132,155])
34     self.tiles.append([99,97,19,18,18,20,132,136,137,0,0,0,0,132,133])
35     self.tiles.append([115,113,18,51,18,20,75,10,69,0,50,0,53,0,135,133])
36     self.tiles.append([99,97,18,17,17,20,132,136,137,0,0,82,0,0,135,133])
37     self.tiles.append([115,113,128,129,18,20,132,136,134,0,0,0,0,135,155])
38     self.tiles.append([99,97,144,145,13,20,132,136,137,87,103,0,13,0,73,11])
39     self.tiles.append([115,113,19,18,19,36,132,133,169,139,139,139,139,139,164,136])

```

The output is now simplified, as can be seen starting at line 45.

Listing 3.18: Forest – `WindowGame.draw()` using a 2D array

```

42     self.screen.fill("black")
43     for row in range(cfg.TILEMAP_NOF_ROWS):
44         for col in range(cfg.TILEMAP_NOF_COLS):
45             index = self.tiles[row][col] #
46             image = self.spritelib.subsurface(index)
47             position = col * cfg.TILESIZE.x, row * cfg.TILESIZE.y
48             self.screen.blit(image, position)
49     self.window.flip()

```

The result is a bit unsatisfying.

- ⚠ Since there is only space for a single tile number at each tile position, background tiles are missing. As a result, an ugly black border becomes visible around these tiles.
- ⚠ The semantic meaning of the tiles is completely lost. This may be acceptable for pure background tiles, but what if some tiles represent ground, others obstacles, and yet others gadgets with a special meaning?

I usually get by with these three semantic layers and encapsulate the whole concept in a `Map` class.

In the constructor, tile numbers are specified only for these three layers. For collision detection, the main program—or wherever it makes sense—can then always check whether a tile number is contained in `layer_data[0]`, `layer_data[1]`, or `layer_data[2]`. The result with regard to the black backgrounds has also improved, as can be seen in figure 3.9 on page 201; since I can now first draw a green meadow as a background in level 0 and later place a tent or a tree in level 1 or 2.

Listing 3.19: Forest – `Map` with a 2D array

```

19     def __init__(self) -> None:
20         self.layer_data = []
21         self.layer_data.append([])
22         self.layer_data[0].append([18,17,18,17,18,17,18,18,17,18,18,17,17,19,34,34])
23         self.layer_data[0].append([17,18,17,17,18,20,34,34,34,34,34,34,34,36,0,0])
24         self.layer_data[0].append([18,1,18,17,18,20,0,0,0,0,64,0,0,0,0,0])
25         self.layer_data[0].append([17,17,18,18,18,20,0,0,0,0,10,0,0,0,0,0])
26         self.layer_data[0].append([17,18,18,17,18,20,0,0,0,0,71,0,0,0,0,0])
27         self.layer_data[0].append([18,17,18,17,17,20,0,0,0,0,0,0,0,0,0,0])

```

```

28     self.layer_data[0].append([18,1,19,18,18,20,0,0,0,0,0,0,0,0,0])
29     self.layer_data[0].append([18,18,18,18,18,20,75,10,69,0,0,0,0,0,0])
30     self.layer_data[0].append([18,18,18,17,17,20,0,0,0,0,0,0,0,0,0])
31     self.layer_data[0].append([18,18,17,1,18,20,0,0,0,0,0,0,0,0,0])
32     self.layer_data[0].append([18,18,18,18,17,20,0,0,0,0,0,0,0,73,11])
33     self.layer_data[0].append([18,18,19,18,19,36,0,0,0,0,0,0,0,0,0])
34     self.layer_data.append([])
35     self.layer_data[1].append([99,98,113,-1,-1,-1,-1,-1,-1,-1,51,-1,-1,-1,-1,-1])
36     self.layer_data[1].append([115,114,97,-1,-1,-1,-1,29,-1,-1,-1,-1,-1,-1,116,117])
37     self.layer_data[1].append([99,115,113,-1,-1,-1,116,117,117,117,-1,117,120,117,164,133])
38     self.layer_data[1].append([115,113,-1,-1,-1,-1,132,133,133,136,-1,136,133,136,155,155])
39     self.layer_data[1].append([99,97,-1,-1,104,105,132,136,185,152,-1,152,152,149,180,133])
40     self.layer_data[1].append([115,113,-1,-1,-1,-1,132,136,137,55,56,-1,-1,53,132,155])
41     self.layer_data[1].append([99,97,-1,-1,-1,-1,132,136,137,-1,-1,-1,-1,132,133])
42     self.layer_data[1].append([115,113,-1,51,-1,-1,-1,-1,-1,-1,50,-1,53,-1,135,133])
43     self.layer_data[1].append([99,97,-1,-1,-1,-1,132,136,137,-1,-1,82,-1,-1,135,133])
44     self.layer_data[1].append([115,113,128,129,-1,-1,132,136,134,-1,-1,-1,-1,-1,135,155])
45     self.layer_data[1].append([99,97,144,145,-1,-1,132,136,137,87,103,-1,-1,-1,-1,-1])
46     self.layer_data[1].append([115,113,-1,-1,-1,-1,132,133,169,139,139,139,139,139,164,136])
47     self.layer_data.append([])
48     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,13,-1,-1])
49     self.layer_data[2].append([-1,-1,-1,60,-1,-1,-1,-1,-1,-1,-1,-1,-1,13,-1,-1])
50     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
51     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
52     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
53     self.layer_data[2].append([-1,-1,-1,13,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
54     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
55     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
56     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
57     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
58     self.layer_data[2].append([-1,-1,-1,-1,13,-1,-1,-1,-1,-1,-1,-1,13,-1,-1,-1])
59     self.layer_data[2].append([-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1])
60
61     def get_layer_data(self, layer_index: int) -> list[list[int]]:
62         return self.layer_data[layer_index]

```

In `WindowGame`, the object of the class is now created:

Listing 3.20: Forest – Map object in `WindowGame`

```

73     self.map = Map()

```

In `draw()`, only the `Map` object is accessed from now on. Unassigned tiles (see line 82) are skipped in the process.

Listing 3.21: Forest – `WindowGame.draw()` using a `Map` object

```

76     self.screen.fill("black")
77     for layer_index in range(3):
78         tiles = self.map.get_layer_data(layer_index)
79         for row in range(cfg.TILEMAP_NOF_ROWS):
80             for col in range(cfg.TILEMAP_NOF_COLS):
81                 index = tiles[row][col]
82                 if index > -1:
83                     image = self.spritelib.subsurface(index)
84                     position = col * cfg.TILESIZE.x, row * cfg.TILESIZE.y
85                     self.screen.blit(image, position)
86     self.window.flip()

```

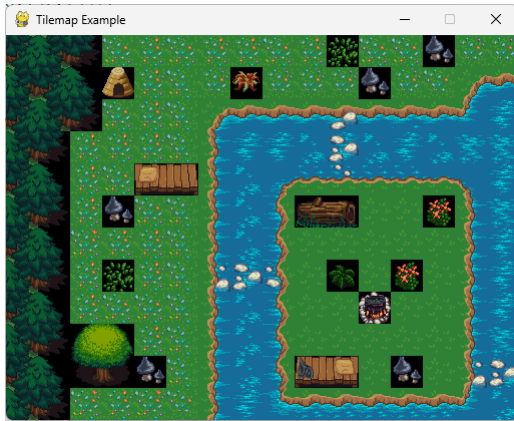


Figure 3.8: Forest playground (3)

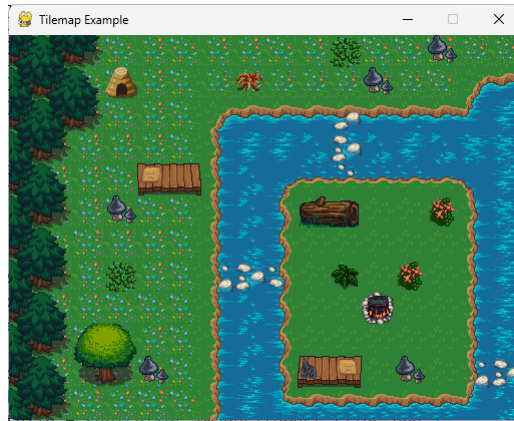


Figure 3.9: Forest playground (4)

Hard-coding the tile numbers directly in the source code is, of course, a nightmare.

It is much better to store them in separate CSV files. The appeal of this approach is that it allows the use of external programs such as *Tiled* for level design. Even self-made tools like `forest00.py` (which can be found in the GitHub repository of this script) are often sufficient and much more efficient than specifying the data directly in the source code.

So let us quickly switch to using CSV files. First, the `csv` module needs to be imported.

Listing 3.22: Forest – `csv` import

```
1 import csv
```

In the constructor, the CSV files located in the `level` subdirectory are now loaded. Please note that creating the `Map` object now requires specifying how many levels are available as CSV files.

Listing 3.23: Forest – Constructor of `Map` using `csv`

```
20 def __init__(self, levels:int) -> None:
21     self.level_data = []
22     for level in range(levels):
23         self.level_data.append([])
24         with open(f'levels/level_{level}.csv', 'r') as datei:
25             csvReader = csv.reader(datei, delimiter=',')
26             for row in csvReader:
27                 self.level_data[level].append([int(tile) for tile in row])
```

That should be sufficient at this point :-)

3.3 Very Large Worlds

In many games, the playable universe is too large to be displayed entirely within a single game window or across the whole screen. Therefore, solutions are required to determine how a section of the game world should be displayed relative to the player's position.

3.3.1 A Large Example World

 So ... where are the files?

- https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V01
- https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V02

Let us take a look at a simple example without any actual gameplay mechanics, so that it does not become too distracting. Our world consists of a large number of square tiles that differ only in their color. The closer a tile is to the center, the whiter its yellow color becomes.

The complete source code is split across several files to maintain clarity. Let us start with the file `globals.py`. By now, `FPS` and `DELTATIME` should be self-explanatory. The constant `TILESIZE_WORLD` defines the width and height of a tile in the large world; in our case 24 px. `NOF_COLS` and `NOF_ROWS` specify the number of columns and rows—that is, how many tiles per row and per column exist in the large world. As a result, `WORLD` becomes a rectangle with a width of 2160 px and a height of 1680 px in line 9; this is larger than what can be displayed on most monitors.

The setting `TILE_WITH_BORDER` controls whether the tiles should have an inner border. A value of 0 means *no*, while a value > 0 specifies the width of the inner border. This makes the individual tiles visible again; otherwise, the world would appear as a large color gradient.

Finally, there is `NOF_MOBS`. I let a few blue rectangles move aimlessly around the world so that something is happening visually and the scene appears a bit more representative of a real game; after all, real games usually contain more than just static elements and the player.

Listing 3.24: Big World – `config.py`

```
1 from pygame import FRect, Vector2
2
3 FPS = 60
4 DELTATIME = 1.0 / FPS
```

```

5 TILESIZE_WORLD = Vector2(24, 24)
6 TILESIZE_WINDOW = Vector2(6, 6)
7 NOF_COLS = 90
8 NOF_ROWS = 70
9 WORLD: FRect = FRect(0, 0, NOF_COLS * TILESIZE_WORLD.y, NOF_ROWS * TILESIZE_WORLD.y) #
10 WINDOW: FRect = FRect(0, 0, NOF_COLS * TILESIZE_WINDOW.x, NOF_ROWS * TILESIZE_WINDOW.y) #
11 TILE_WITH_BORDER = 0
12 NOF_MOBS = 50

```

Next, we take a look at the three classes `Tile`, `Player`, and `Mob` in `objects.py`. The `Tile` class is representative of any type of sprite placed in the world. These can be static background elements, walls in the foreground, or other movable objects. For our considerations here, this distinction does not matter.

In the constructor of `Tile`, an image with the size `TILESIZE_WORLD` is created. Starting at line 12, the relative distance of the tile to the center is calculated. This is possible because the position is passed as a parameter. The value range of `rel_dist_center` lies within the interval $[0, 1]$. The blue component for the yellow color is then computed in line 18 and used to color the tile in the following line.

The `Player` class represents a simple wanderer through the world. It is drawn as a simple red circle centered within a tile-sized image. The radius is chosen such that the circle fills the tile as much as possible. In `update()`, either the position of the tile in the world is set directly or a new position is calculated based on the chosen direction. Note that the position refers not to the top-left corner, but to the center of the circle.

Finally, there is the `Mob` class. A blue rectangle is placed at a random position – at a reasonable distance from the borders. The direction vectors are chosen from a uniform distribution, and the speed is also randomly selected between 100 px and 500 px. The size of the rectangle is likewise determined by a random padding. In `update()`, the mob is moved. If it has completely wandered out of the world, it reappears at the opposite edge.

Listing 3.25: Big World – `Tile`, `Player`, and `Mob`

```

1 from random import choice, randint
2 from typing import Any
3
4 import config as cfg
5 import pygame
6
7
8 class Tile(pygame.sprite.Sprite):
9     def __init__(self, position: tuple[float, float]) -> None:
10         super().__init__()
11         self.image = pygame.Surface(cfg.TILESIZE_WORLD)
12         # Yellow -> White according to the distance to the center
13         v1 = pygame.Vector2(position)
14         v2 = pygame.Vector2(cfg.WORLD.center)
15         distance = v2.distance_to(v1)
16         max_distance = v2.length()
17         rel_dist_center = min(1.0, distance / max_distance)
18         blue_value = int(255 * (1 - rel_dist_center)) #
19         color = (255, 255, blue_value)
20         self.image.fill(color)
21         if cfg.TILE_WITH_BORDER > 0:

```



```

22         pygame.draw.rect(self.image, "Black",
23                           ((0,0), cfg.TILESIZE_WORLD),
24                           cfg.TILE_WITH_BORDER)
25         self.rect = self.image.get_rect(topleft=position)
26
27
28 class Player(pygame.sprite.Sprite):
29
30     def __init__(self, position: tuple[float, float]) -> None:
31         super().__init__()
32         self.image : pygame.Surface = pygame.Surface(cfg.TILESIZE_WORLD)
33         self.image.set_colorkey((0, 0, 0))
34
35         self.radius = int(cfg.TILESIZE_WORLD.x // 2)
36         pygame.draw.circle(self.image, "red", (self.radius, self.radius), self.radius)
37         self.rect : pygame.FRect = self.image.get_frect(center=position)
38
39         self.speed = 400.0 # pixels per second
40         self.directions = {
41             "left": pygame.Vector2(-1, 0),
42             "right": pygame.Vector2(1, 0),
43             "up": pygame.Vector2(0, -1),
44             "down": pygame.Vector2(0, 1),
45             "stop": pygame.Vector2(0, 0),
46         }
47         self.direction = self.directions["stop"]
48
49     def update(self, *args: Any, **kwargs: Any) -> None:
50         if "position" in kwargs:
51             self.rect.center = kwargs["position"]
52         if "move" in kwargs:
53             self.direction = self.directions[kwargs["move"]]
54         new_position = pygame.Vector2(self.rect.center) + self.direction * (
55             self.speed * cfg.DELTATIME
56         )
57         self.rect.center = new_position
58         self.rect.clamp_ip(cfg.WORLD)
59         return super().update(*args, **kwargs)
60
61
62 class Mob(pygame.sprite.Sprite):
63
64     def __init__(self) -> None:
65         super().__init__()
66         x1 = int(cfg.TILESIZE_WORLD.x + 10)
67         x2 = int(cfg.WORLD.width - (cfg.TILESIZE_WORLD.x + 10))
68         y1 = int(cfg.TILESIZE_WORLD.y + 10)
69         y2 = int(cfg.WORLD.height - (cfg.TILESIZE_WORLD.y + 10))
70         position = (randint(x1, x2), randint(y1, y2))
71         self.image = pygame.Surface(cfg.TILESIZE_WORLD, pygame.SRCALPHA)
72         color = (0, 0, randint(10, 255))
73         pad = randint(0, int(cfg.TILESIZE_WORLD.x//2 - 1))
74         pygame.draw.rect(self.image,
75                           color,
76                           ((pad,pad), (cfg.TILESIZE_WORLD.x - 2*pad,
77                                       cfg.TILESIZE_WORLD.y - 2*pad)))
78         self.rect = self.image.get_frect()
79         self.rect.topleft = position
80         self.direction = pygame.Vector2(choice((-1, 1)), choice((-1,1)))
81         self.speed = randint(100, 500) # px/sec
82
83     def update(self, *args: Any, **kwargs: Any) -> None:
84         self.rect.move_ip(self.direction * self.speed * cfg.DELTATIME)
85         if self.rect.right < cfg.WORLD.left:
86             self.rect.left = cfg.WORLD.right
87         elif self.rect.left > cfg.WORLD.right:

```



```

88         self.rect.right = cfg.WORLD.left
89     if self.rect.bottom < cfg.WORLD.top:
90         self.rect.top = cfg.WORLD.bottom
91     elif self.rect.top > cfg.WORLD.bottom:
92         self.rect.bottom = cfg.WORLD.top
93     return super().update(*args, **kwargs)

```

Our output window – i.e. the first rather unworthy attempt – is defined in the file `windows.py`. Not much happens here. A window with the size defined in `config.py` is created. Finally, the window title is adjusted so that it conveys a bit of information about its properties. In `draw()`, the window is filled with black, and then all world objects – background tiles as well as moving objects, including the player – are rendered. I added the method `save()` only for this script, so that I can capture images of the current states, for example to show them here.

Listing 3.26: Big World – WindowPlain

```

1  import config as cfg
2  import pygame
3
4
5  class WindowPlain:
6
7      def __init__(self, tiles:pygame.sprite.Group, mobs:pygame.sprite.Group) -> None:
8          self.tiles = tiles
9          self.mobs = mobs
10         self.window = pygame.Window(size=cfg.WINDOW.size)
11         self.window.position = (0 * (cfg.WINDOW.width + 60),
12                                0 * (cfg.WINDOW.height) + 30)
13         self.screen : pygame.Surface = self.window.get_surface()
14         self.rect = self.screen.get_frect()
15         self.window.title = f"Plain Window (size={self.rect.size})"
16         self.clock = pygame.time.Clock()
17
18     def draw(self):
19         self.tiles.draw(self.screen)
20         self.mobs.draw(self.screen)
21         self.window.flip()
22
23     def save(self):
24         pygame.image.save(self.screen, "plain_image.png")

```

In `camera_demo.py`, the class `Game` is now defined and the call to `main()` is performed. In the constructor of `Game`, the functions `create_tiles()` and `create_mobs()` are called. These functions create the tiles and the moving objects and assign them the correct positions within the world; more on this later. In addition, the `WindowPlain` object is created as the output window, as well as the player – i.e. a `Player` object.

In `run()`, the basic structure of the main program loop that I typically use in my games is defined. There are plenty of explanations for this structure earlier and later in the script.

Listing 3.27: Big World – Constructor and run() of Game

```

9  class Game:
10
11     def __init__(self) -> None:
12         pygame.init()
13         self.clock = pygame.time.Clock()
14         self.create_tiles()
15         self.create_mobs()
16         self.window_plain = WindowPlain(self.tiles, self.mobs)
17         self.world_image = pygame.Surface(cfg.WORLD.size)
18         self.player = Player(cfg.WORLD.center)
19         self.mobs.add(self.player)
20         self.running = True
21
22     def run(self) -> None:
23         time_previous = time()
24         while self.running:
25             self.watch_for_events()
26             if self.running:
27                 self.update()
28                 self.draw()
29                 self.clock.tick(cfg.FPS)
30                 time_current = time()
31                 cfg.DELTATIME = time_current - time_previous
32                 time_previous = time_current
33         pygame.quit()

```

The methods shown in source code 3.28 also require no further explanation.

Listing 3.28: Big World – watch_for_events(), update(), and draw() of Game

```

35  def watch_for_events(self) -> None:
36      for event in pygame.event.get():
37          if event.type == pygame.QUIT:
38              self.running = False
39          elif event.type == pygame.WINDOWCLOSE:
40              self.running = False
41              event.window.destroy()
42          elif event.type == pygame.KEYDOWN:
43              if event.key == pygame.K_ESCAPE:
44                  self.running = False
45              elif event.key == pygame.K_UP:
46                  self.player.update(move="up")
47              elif event.key == pygame.K_DOWN:
48                  self.player.update(move="down")
49              elif event.key == pygame.K_LEFT:
50                  self.player.update(move="left")
51              elif event.key == pygame.K_RIGHT:
52                  self.player.update(move="right")
53              elif event.key == pygame.K_s:
54                  self.save()
55
56          elif event.type == pygame.KEYUP:
57              if event.key in (pygame.K_UP, pygame.K_DOWN, pygame.K_LEFT, pygame.K_RIGHT):
58                  self.player.update(move="stop")
59
60  def update(self) -> None:
61      self.player.update()
62      self.mobs.update()
63
64  def draw(self) -> None:
65      self.window_plain.draw()
66      self.tiles.draw(self.world_image)

```

```
67 self.mobs.draw(self.world_image)
```

The methods `create_tiles()` and `create_mobs()` create the game objects – that is, in our case, the static tiles and the moving game elements. The method `save()` triggers saving both the entire world and the game window as PNG files. As mentioned above, this functionality exists only to allow images to be included in this script (for example, figure 3.12 on the next page).

Listing 3.29: Big World – `create_tiles()`, `create_mobs()`, and `save()` of Game

```
69 def create_tiles(self) -> None:
70     self.tiles = pygame.sprite.Group()
71     for row in range(cfg.NOI_ROWS):
72         for col in range(cfg.NOI_COLS):
73             x = col * cfg.TILESIZE_WORLD.x
74             y = row * cfg.TILESIZE_WORLD.y
75             self.tiles.add(Tile((x, y)))
76
77 def create_mobs(self) -> None:
78     self.mobs = pygame.sprite.Group()
79     for _ in range(cfg.NOI_MOBS):
80         self.mobs.add(Mob())
81
82 def save(self):
83     pygame.image.save(self.world_image, "world_image.png")
84     self.window_plain.save()
```

If we take a look at the screen outputs of the current source code in figure 3.10 and figure 3.11, the fundamental problem becomes immediately apparent. The window `Plain-Window` is far too small to display the entire world (see figure 3.12 on the following page). We can only see the top-left corner of the large world – once without borders and once with borders. The visible section is so small that even the color gradient is hardly recognizable. Here, the borders help to make the many tiles visible. Later on, we will no longer need them.

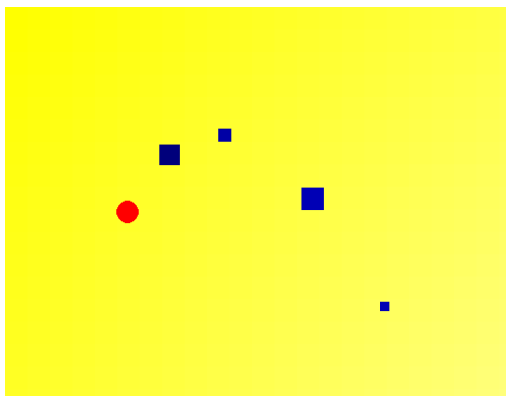


Figure 3.10: Tiles without borders

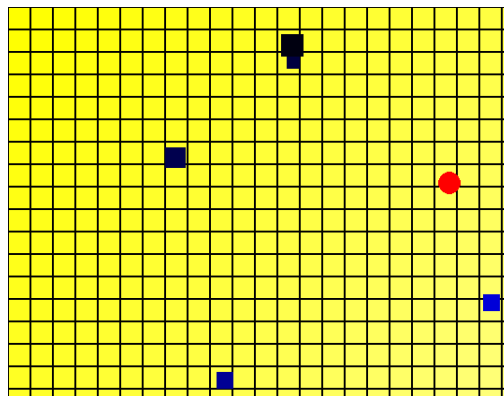


Figure 3.11: Tiles with borders

Before continuing with the different top-down views, I would like to address a performance issue. To do so, I extend the window title to display the actual FPS and increase

Visibility
culling

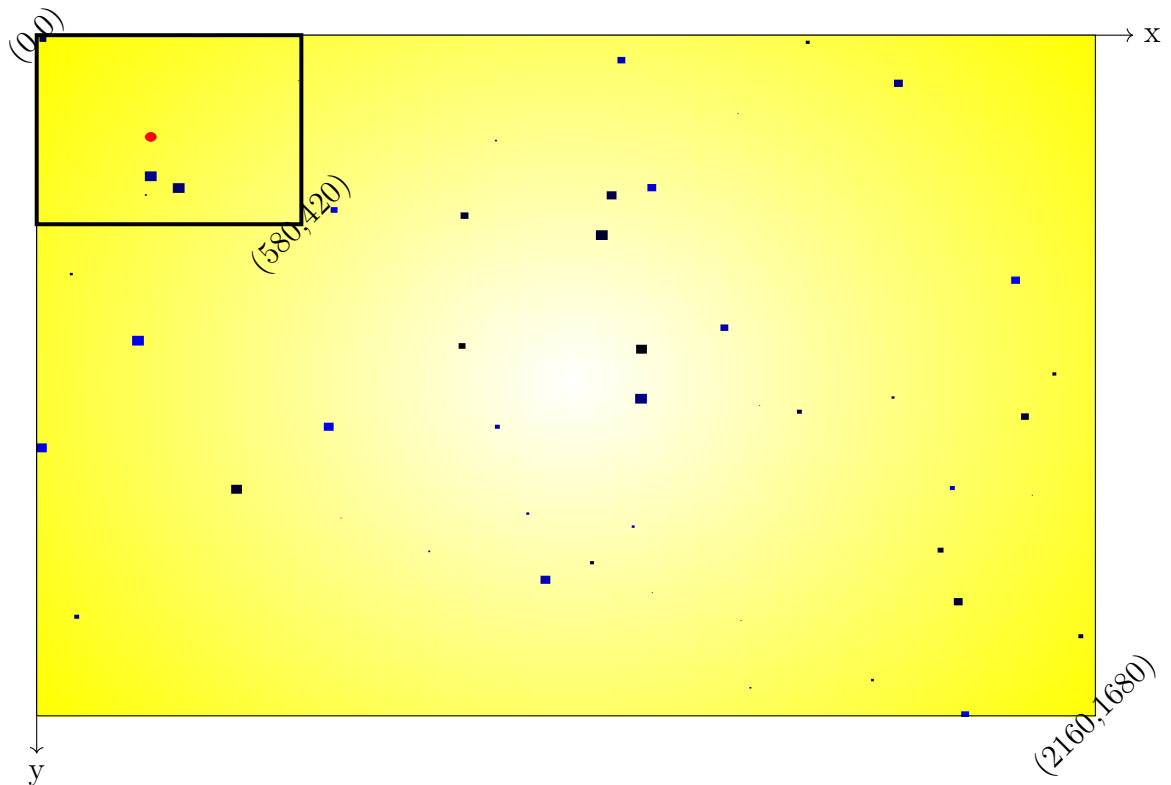


Figure 3.12: PlainWindow as a Viewport of the World

FPS in `config.py` to 600 fps – not because this is a realistic value for a game, but because I want to determine how many frames are actually achieved. I then store this actually achieved number of frames in a text file.

If we take a look at the method `draw()` in source code 3.26 on page 205, we see that all tiles and all moving objects are rendered into the target window – that is, drawn – even though the vast majority of these objects are not visible within the window at all.

One possible approach is to first check all sprites, or rather their rectangles, to see whether they are actually located inside the output window. The list `a` is created by iterating over all sprites in the group and, for each one, using `colliderect()` to check whether it lies within or at least touches the rectangle of the window.

Listing 3.30: Big World – `WindowPlain.draw()` with Visibility culling

```

18 def draw(self):
19     self.screen.fill("Black")
20     a = [r for r in self.tiles.sprites() if cfg.WINDOW.colliderect(r.rect)]
21     for sprite in a:
22         self.screen.blit(sprite.image, sprite.rect)
23     #self.tiles.draw(self.screen_plain)
24     a = [r for r in self.mobs.sprites() if cfg.WINDOW.colliderect(r.rect)]
25     for sprite in a:
26         self.screen.blit(sprite.image, sprite.rect)
27     #self.mobs.draw(self.screen_plain)

```

Afterwards, I performed the same performance measurement as before, and the result can be seen in figure 3.13. When visibility checking is enabled, significantly more frames per second are achieved than without it. Conclusion: it is worth using.

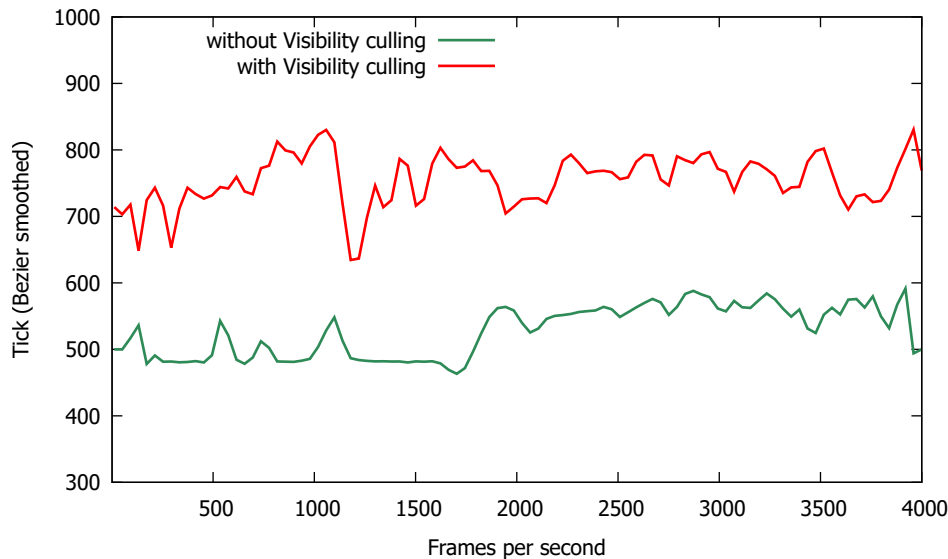


Figure 3.13: Performance without and with visibility culling

3.3.2 Top-Down View / Bird's-Eye View

 So ... where are the files?

- https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V03
- https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V04

First of all, one might want to have a complete overview of the entire world. This is not strictly necessary. It is not uncommon for a game to never allow the player to see the whole world at once. However, many games offer a top-down view (Bird's-Eye View).

A first naive approach would be to scale down the world using `transform.scale_by()` (see line 50).

This approach has the following advantages and disadvantages

- 👉 Advantage: Very easy to implement.
- 👉 Disadvantage: In every frame, the entire oversized bitmap has to be created.
- 👉 Disadvantage: Scaling can produce undesirable artifacts, since we have no control over which pixels are lost during scaling.
- 👉 Disadvantage: Scaling can create objects that are only one pixel in size or even smaller, and therefore hardly visible or not visible at all.

Listing 3.31: Big World – WindowBirdEyeView

```

35 class WindowBirdEyeView:
36
37     def __init__(self, world_screen:pygame.Surface) -> None:
38         self.world_screen = world_screen
39         zx = cfg.WINDOW.width / cfg.WORLD.width
40         zy = cfg.WINDOW.height / cfg.WORLD.height
41         self.zoom = pygame.Vector2(zx, zy)
42         self.window = pygame.Window(size=cfg.WINDOW.size)
43         self.window.position = (1 * (cfg.WINDOW.width + 60),
44                                0 * (cfg.WINDOW.height) + 30)
45         self.screen : pygame.Surface = self.window.get_surface()
46         self.rect = self.screen.get_frect()
47         self.window.title = f"Birdeye (zoom={self.zoom.x:0.2f}, {self.zoom.y:0.2f})"
48
49     def draw(self):
50         image = pygame.transform.scale_by(self.world_screen, self.zoom) #
51         self.screen.blit(image)
52         self.window.flip()
53
54     def save(self):
55         pygame.image.save(self.screen, "birdeye_image.png")

```

I therefore recommend that each sprite provides two or possibly even several variants of its image. This makes it easier to identify objects quickly in the Bird's-Eye View as well.

Keep in mind that the purpose of the top-down view is not to allow the game to be played in a full overview after all, but rather to provide orientation, to locate important points of interest, and possibly to identify friends and enemies.

1. Each stationary tile is simply displayed in a scaled-down version.
2. The player is represented by a smaller red circle.
3. The other moving objects are displayed as equally sized blue squares.

One more thing I would like to have is the ability to recognize the visible section of the PlainWindow as a rectangle within the Bird's-Eye View.

Let us implement this: First, the Bird's-Eye View window is integrated into the main program (see source code 3.32 and source code 3.33). Please note that at no point in the source code is a surface for the entire world created at its original size anymore.

Listing 3.32: Big World – BirdEyeView in the constructor of Game

```

16     self.window_plain = WindowPlain(self.tiles, self.mobs)
17     self.window_birdeye = WindowBirdEyeView(self.tiles, self.mobs)
18     self.player = Player(cfg.WORLD.center)

```

In `draw()`, an additional option is prepared, namely the visualization of which part of the entire world is currently covered by `PlainWindow`. To do this, I pass the rectangle of `PlainWindow` and the border color to the `draw()` method of `BirdEyeView` in line 68 (see figure 3.15 on the next page).

Listing 3.33: Big World – BirdEyeView in `Game.draw()`

```

64     def draw(self) -> None:
65         self.window_plain.draw()
66         self.window_plain.window.title = f"Plain Window (size={self.window_plain.rect.size},
        fps={self.clock.get_fps():.0f})"
67
68         self.window_birdeye.draw([{"rect":self.window_plain.rect, "color":"blue"}] )#

```

This data is then used in `draw()` of `BirdEyeView`, and the rectangle(s) are drawn starting at line 54.

Listing 3.34: Big World – `BirdEyeView.draw()`

```

49     def draw(self, rects:list):
50         for sprite in self.tiles:
51             self.screen.blit(sprite.image_small, self.zoom_rect(sprite.rect))
52         for sprite in self.mobs:
53             self.screen.blit(sprite.image_small, self.zoom_rect(sprite.rect))
54         if rects:                                     # Are rects to draw?
55             for item in rects:
56                 pygame.draw.rect(self.screen, item["color"], self.zoom_rect(item["rect"]), 2)
57         self.window.flip()

```

One more note on performance: Due to the preparations carried out – namely the one-time creation of a smaller, symbolic representation of the game elements – a significant performance improvement was achieved as well. Analogous to the measurements above, I performed a corresponding benchmark over 660 frames. The result can be read from figure 3.16 on the following page.

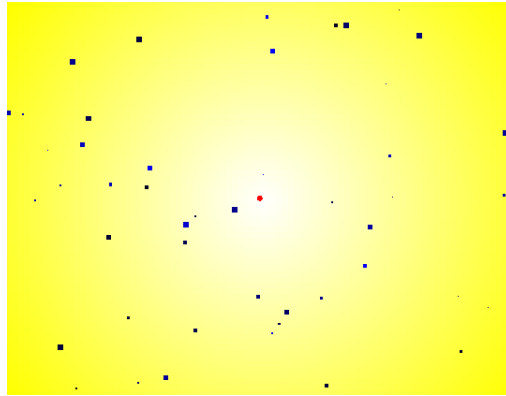


Figure 3.14: BirdEye (scaled)

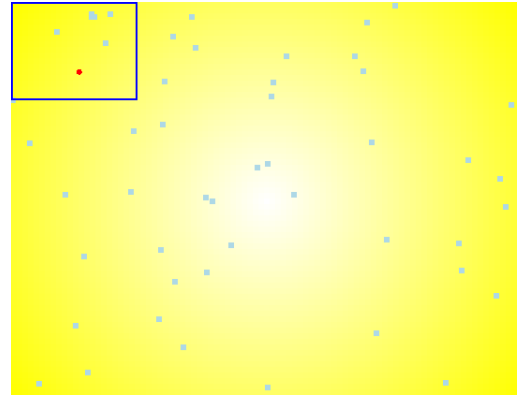


Figure 3.15: Bird's-Eye View (simplified and with visibility indicator)

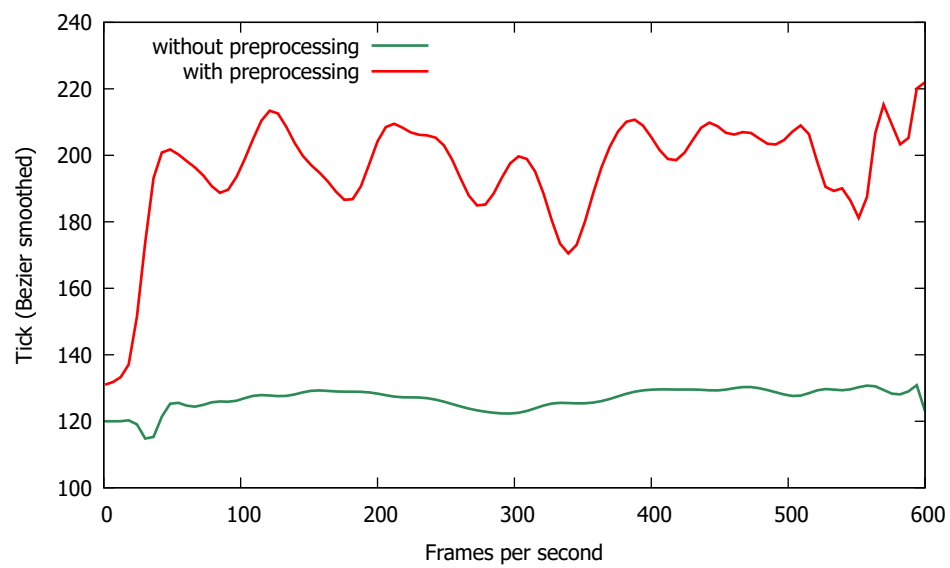


Figure 3.16: Performance without and with preprocessing

3.3.3 Player Centered Camera

 So ... where are the files?

- https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V05
- https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V06
- https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V07

As the next type of view, I would like to introduce the *Player-Centered Camera*. In this view, the player appears to be fixed at the center, while the elements of the game move according to the player's subjective direction of movement.

The basic idea behind a solution is actually quite simple – so do not be intimidated by the mathematics! Each player has a position in the world measured from the top-left corner, given by the vector $\vec{P}_W = (P_{Wx}, P_{Wy})$. The point P_W has a distance to the center of the world—that is, the large game world. Why is this important? Because the player is later supposed to appear at the center of the window.

Therefore, we need to find a correction value (*offset*) that transforms the player's world coordinates into the center coordinates of the camera view window. This offset must be subtracted from the world coordinates, since the coordinates of the view window are much smaller than those of the game world.

Offset

Let $\vec{P}_V$ be the player position in the camera view window and $\vec{O}_{ff}$ the correction value. In other words: the position in the world minus the correction value yields the position in the window. Let us now transform this relationship so that we can compute the correction value.

$$\vec{P}_W - \vec{O}_{ff} = \vec{P}_V \quad \parallel + \vec{O}_{ff} \quad (3.1)$$

$$\vec{P}_W = \vec{P}_V + \vec{O}_{ff} \quad \parallel - \vec{P}_V$$

$$\vec{P}_W - \vec{P}_V = \vec{O}_{ff} \quad \parallel reverseorder$$

$$\vec{O}_{ff} = \vec{P}_W - \vec{P}_V \quad \parallel coordinatenotation$$

$$\begin{pmatrix} O_{ffx} \\ O_{ffy} \end{pmatrix} = \begin{pmatrix} P_{Wx} \\ P_{Wy} \end{pmatrix} - \begin{pmatrix} P_{Vx} \\ P_{Vy} \end{pmatrix} \quad \parallel vectorsubtraction$$

$$\begin{pmatrix} O_{ffx} \\ O_{ffy} \end{pmatrix} = \begin{pmatrix} P_{Wx} - P_{Vx} \\ P_{Wy} - P_{Vy} \end{pmatrix} \quad (3.2)$$

Equation 3.3 reflects the fact that the new position of our player is supposed to be exactly the center of the window, that is, half the width and half the height:

$$\begin{pmatrix} P_{Vx} \\ P_{Vy} \end{pmatrix} = \begin{pmatrix} S_{Vx}/2 \\ S_{Vy}/2 \end{pmatrix} \quad (3.3)$$

Now we substitute equation 3.3 into equation 3.2:

$$\begin{pmatrix} O_{ffx} \\ O_{ffy} \end{pmatrix} = \begin{pmatrix} P_{Wx} - S_{Vx}/2 \\ P_{Wy} - S_{Vy}/2 \end{pmatrix} \quad (3.4)$$

With this, we have indeed derived – using nothing more than straightforward mathematics – the formula for computing the translation, that is, the offset.

It is time for a bit of source code. Let us first prepare everything in the main program. Even though we have not implemented the class yet, we can simply treat the new window like the other two and work with copy/paste. In line 20, the window is created. Please note that this has to happen after creating the **Player**, because we need its position.

Listing 3.35: Big World – Centered Camera in Game

```

11  def __init__(self) -> None:
12      pygame.init()
13      self.clock = pygame.time.Clock()
14      self.create_tiles()
15      self.create_mobs()
16      self.window_plain = WindowPlain(self.tiles, self.mobs)
17      self.window_birdeye = WindowBirdEyeView(self.tiles, self.mobs)
18      self.player = Player(cfg.WORLD.center)
19      self.mobs.add(self.player)
20      self.window_centered = WindowCenteredCamera(self.player, self.tiles, self.mobs) #
21      self.running = True

```

In the `draw()` method, three additions are necessary. In line 74, the rectangle for the Bird's-Eye View is added, showing the visible section of the new window. After that, `draw()` is called as with the other windows and the title line is updated.

Listing 3.36: Big World – Centered Camera in Game.draw()

```

66  def draw(self) -> None:
67      self.window_plain.draw()
68      self.window_plain.window.title = f"Plain Window (size={self.window_plain.rect.size},
69                                     fps={self.clock.get_fps():.0f})"
70
71      self.window_centered.draw() #
72      self.window_centered.window.title = f"Centered Window
73                                     (offset={self.window_centered.offset})"
74
75      self.window_birdeye.draw([{"rect":self.window_plain.rect, "color":"blue"},
76                                {"rect":self.window_centered.rect, "color":"green"}]) #

```

And `save()` also needs to be extended by line 92:

Listing 3.37: Big World – Centered Camera in `Game.save()`

```

89     def save(self):
90         self.window_plain.save()
91         self.window_birdeye.save()
92         self.window_centered.save() #

```

Now let us move on to the fun part: the `WindowCenteredCamera` class. First, the self-explanatory `__init__()`. In addition, two attributes are defined here: `self.offset` and `self.player`. Using the offset, I will later compute the new coordinates, and the player is used to compute the offset.

Listing 3.38: Big World – Constructor of `WindowCenteredCamera`

```

72     def __init__(self, player:pygame.sprite.Sprite, tiles:pygame.sprite.Group,
73                 mobs:pygame.sprite.Group) -> None:
74         self.tiles = tiles
75         self.mobs = mobs
76         self.player = player
77         self.offset = pygame.Vector2(0, 0) #
78         self.window = pygame.Window(size=cfg.WINDOW.size)
79         self.window.position = (2 * (cfg.WINDOW.width + 60),
80                                0 * (cfg.WINDOW.height) + 30)
81         self.screen : pygame.Surface = self.window.get_surface()
82         self.rect = self.screen.get_frect()
83         self.clock = pygame.time.Clock()

```

The offset is computed in the method `scroll()`. Keep equation 3.4 on the preceding page next to the source code. The implementation should be self-explanatory, as it serves as an example of how easily mathematical expressions can be translated into source code.

Why is the last line actually needed? It is not required for scrolling itself. However, by providing this value, I can inspect the rectangle of the visible world section in the Bird's-Eye View.

Listing 3.39: Big World – `WindowCenteredCamera.scroll()`

```

97     def scroll(self) -> None:
98         self.offset.x = self.player.rect.x - self.rect.width / 2
99         self.offset.y = self.player.rect.y - self.rect.height / 2
100        self.rect.topleft = self.offset

```

Now we have everything in place to implement the remaining parts. Let us start with two helper methods so that the coordinate transformations do not have to be implemented multiple times. In `world2camera()`, the coordinates of the game objects in the large world are transformed into the coordinates of the centered view. This calculation corresponds exactly to the initial idea shown in equation 3.1 on page 213.

Listing 3.40: Big World – `WindowCenteredCamera.world2camera()`

```

102    def world2camera(self, rect: pygame.FRect) -> pygame.FRect:
103        return pygame.FRect(rect.topleft - self.offset, rect.size)

```

What remains is the method `draw()`. This method looks almost the same as the `draw()` method of the other class. However, here the coordinates of the game objects are transformed using `camera2world()` before the visibility check. Take a moment to think about why this is necessary!

Listing 3.41: Big World – `WindowCenteredCamera.draw()`

```

84 def draw(self) -> None:
85     self.screen.fill("lightgrey")
86     a = [r for r in self.tiles.sprites() if
87          cfg.WINDOW.collidect(self.world2camera(r.rect))]
88     for sprite in a:
89         self.screen.blit(sprite.image, self.world2camera(sprite.rect))
90     a = [r for r in self.mobs.sprites() if
91          cfg.WINDOW.collidect(self.world2camera(r.rect))]
92     for sprite in a:
93         self.screen.blit(sprite.image, self.world2camera(sprite.rect))
94     self.window.flip()

```

If we now run the source code and move the player to the top-left corner of the game world, we obtain the views shown in figure 3.17 and figure 3.18. In the left image, the screen section visible in the right image can be identified by the green rectangle. The green rectangle only appears to be smaller than the blue one; in fact, three quarters of the view lie outside the visible area of the Bird's-Eye View.

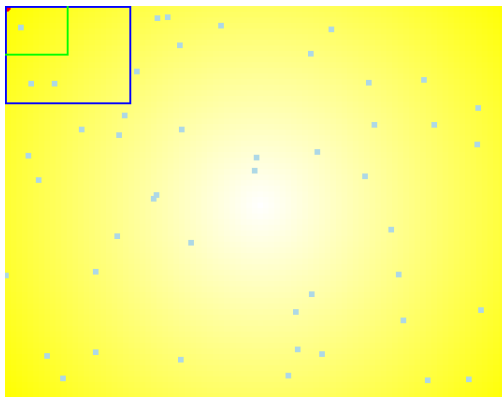


Figure 3.17: Bird's-Eye View: Green = Centered

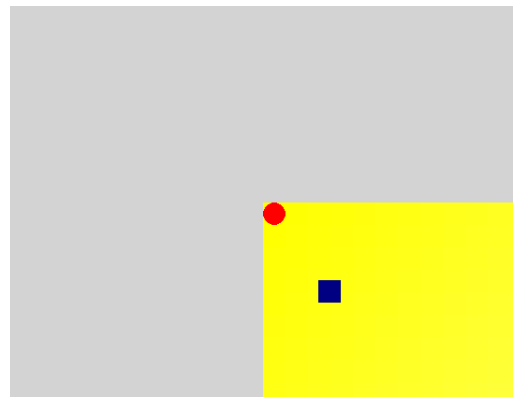


Figure 3.18: Centered Camera – with border error

We therefore need to adjust the method `scroll()` so that the borders are not exceeded. We now see that the offset is limited to 0 at the top and on the left, meaning it cannot become negative. This would otherwise indicate that we are extending beyond the world to the top or to the left. The same logic applies to the right and bottom edges. Here, it is checked whether the right edge of the object exceeds the right edge of the world, and analogously whether this also happens at the bottom. This procedure is known as [clamp](#).

Clamp

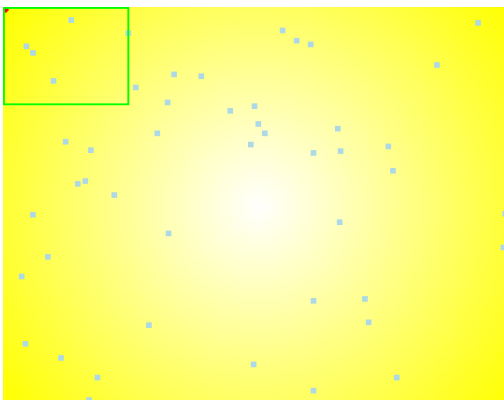
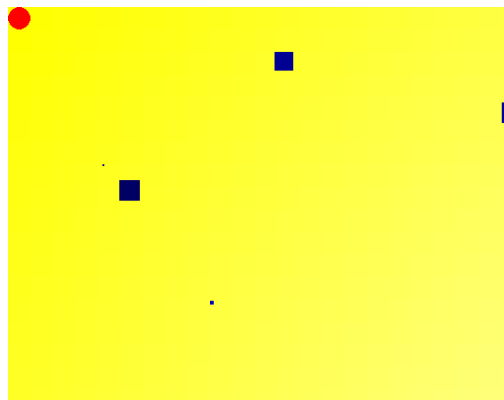
Listing 3.42: Big World – WindowCenteredCamera.scroll() with clamping

```

97 def scroll(self) -> None:
98     # Clamp left/top so we do not scroll past the world edges
99     self.offset.x = max(0, self.player.rect.x - self.rect.width / 2)
100     self.offset.y = max(0, self.player.rect.y - self.rect.height / 2)
101
102     # Clamp right/bottom so we do not scroll past the world edges
103     self.offset.x = min(cfg.WORLD.right - self.rect.width, self.offset.x)
104     self.offset.y = min(cfg.WORLD.bottom - self.rect.height, self.offset.y)
105
106     self.rect.topleft = self.offset

```

In figure 3.20, we no longer see any border artifacts. Instead, the player's position has shifted from the center toward the edge—exactly as intended. Note: In figure 3.19, the blue border of PlainWindow can no longer be seen, since both views now cover the same section.

Figure 3.19: BirdEyeView:
Grün=CenteredFigure 3.20: Centered Camera – with-
out border error

There is one more thing I would like to address

In the method `draw()` (see source code 3.41 on the preceding page), the method `world2camera()` is called for every single game object; that is, thousands of coordinates are transformed. Would it not be more efficient to transform the world coordinates just once and then compare them with the coordinates of the game objects?

To this end, we introduce a new method, appropriately called `camera2world()`. It is, so to speak, the inverse of `world2camera()`.

Listing 3.43: Big World – WindowCenteredCamera.camera2world() with clamping

```

111 def world2camera(self, rect: pygame.FRect) -> pygame.FRect:
112     return pygame.FRect(rect.topleft - self.offset, rect.size)

```

I now integrate this into `draw()`. I have left the old computation commented out above, so that the difference can be seen more clearly. Once again, simple reasoning has led to a performance gain (see figure 3.21).

Listing 3.44: Big World – `WindowCenteredCamera.draw()` with clamping

```

84 def draw(self):
85     self.screen.fill("Black")
86     #a = [r for r in self.tiles.sprites() if
87         Settings.WINDOW.collidect(self.world2camera(r.rect))]
88     w = self.camera2world(cfg.WINDOW)
89     a = [r for r in self.tiles.sprites() if w.collidect(r.rect)]
90     for sprite in a:
91         self.screen.blit(sprite.image, self.world2camera(sprite.rect))
92     #a = [r for r in self.mobs.sprites() if
93         Settings.WINDOW.collidect(self.world2camera(r.rect))]
94     a = [r for r in self.mobs.sprites() if w.collidect(r.rect)]
95     for sprite in a:
96         self.screen.blit(sprite.image, self.world2camera(sprite.rect))
97     self.window.flip()

```

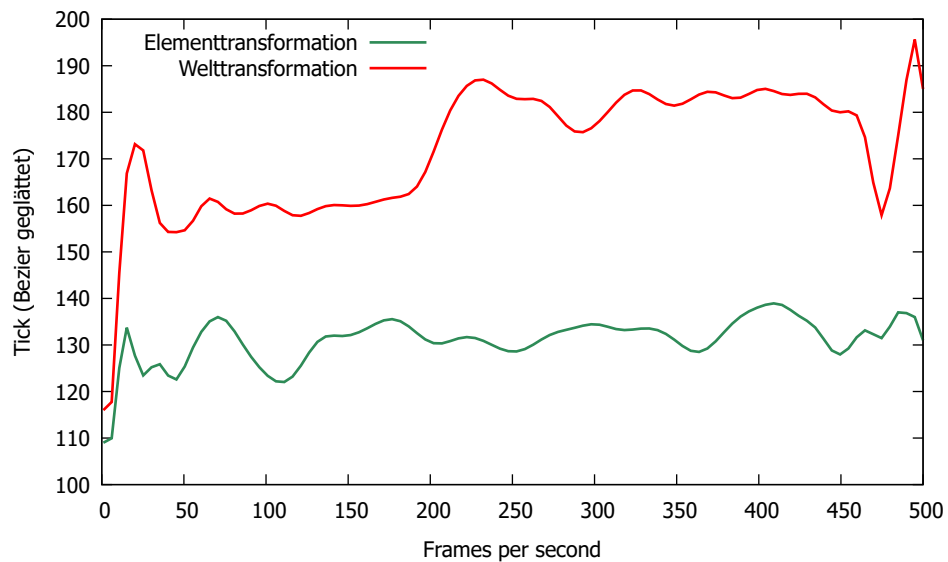


Figure 3.21: Performance with element-based and world-based transformation

3.3.4 Page Scrolling/Edge Scrolling

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V08

Keeping the player permanently centered can lead to a restless and confusing visual effect, depending on the game's visual style. It is also associated with many transformations that are often unnecessary for the gameplay itself. Page-wise scrolling (edge scrolling or page scrolling) represents a good compromise. The player initially moves normally within the visible game area. Only when a minimum distance (**padding**) to one of the borders is undershot does the background shift in the corresponding direction – that is, the view scrolls to the next page.

Edge

Page

Let us take a look at the constructor of `WindowPagewise`. Most of the elements have already been explained above. What is new is the attribute `inner_rect` and the parameter `padding`. This parameter controls the distance between the inner rectangle and the boundaries of the view—that is, the space between the inner brown rectangle and the outer green rectangle in figure 3.22 on the next page. The parameter is – purely arbitrarily – an integer here and serves as a factor for calculating the distance. As a second factor, I chose the width and height of the player. Semantically, this means that a value of 2 results in a padding of two player widths or heights.

Listing 3.45: Big World – Constructor of `WindowPagewise`

```

112 class WindowPagewise:
113
114     def __init__(self, player:pygame.sprite.Sprite, tiles:pygame.sprite.Group,
115                 mobs:pygame.sprite.Group, padding:int = 1) -> None:
116         self.tiles = tiles
117         self.mobs = mobs
118         self.player = player
119         self.offset = pygame.Vector2(0, 0)
120         self.window = pygame.Window(size=config.WINDOW.size)
121         self.window.position = (0 * (config.WINDOW.width + 60),
122                                1 * (config.WINDOW.height) + 30)
123         self.screen : pygame.Surface = self.window.get_surface()
124         self.rect = self.screen.get_frect()
125         self.clock = pygame.time.Clock()
126         self.inner_rect = pygame.FRect(
127             config.WINDOW.left + padding * player.rect.width,
128             config.WINDOW.top + padding * player.rect.height,
129             config.WINDOW.width - padding * 2 * player.rect.width,
130             config.WINDOW.height - padding * 2 * player.rect.height,
131         )

```

The actual work happens in the method `scroll()`. Here as well, the basic logic is fairly simple. If the player's rectangle lies within the inner rectangle, no scrolling needs to take place at all; the player simply moves normally. Once the player leaves the inner area, scrolling has to be performed. How do we test whether the player is still inside

the inner rectangle? By checking whether the player no longer collides with the inner rectangle (line 148).

Listing 3.46: Big World – WindowPagewise.scroll()

```

146 def scroll(self) -> None:
147     player_in_view = self.world2camera(self.player.rect)
148     if not player_in_view.colliderect(self.inner_rect):      # nicht mehr innerhalb?
149         self.offset.x = max(0, self.player.rect.x - cfg.WINDOW.centerx)
150         self.offset.y = max(0, self.player.rect.y - cfg.WINDOW.centery)
151         self.offset.x = min(cfg.WORLD.right - cfg.WINDOW.width, self.offset.x)
152         self.offset.y = min(cfg.WORLD.bottom - cfg.WINDOW.height, self.offset.y)
153     self.rect.topleft = self.offset

```

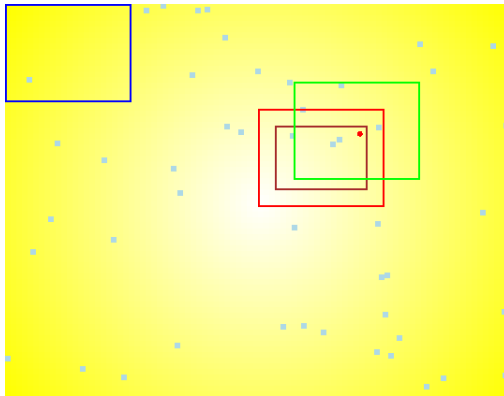
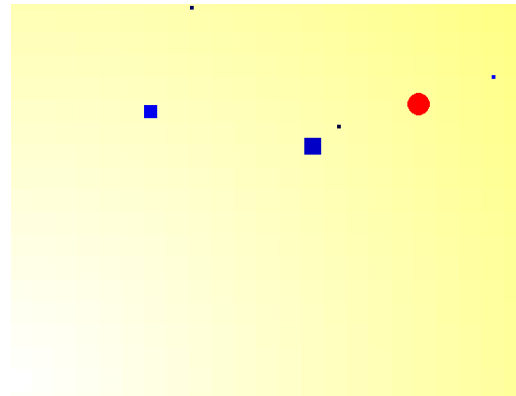
Figure 3.22: Bird's-Eye: Centered,
Pagewise, InnerRect

Figure 3.23: Page / Edge Scrolling

Integrating the new class is done in exactly the same way as integrating `WindowCenteredCamera`. Only the visualization of the rectangles in the Bird's-Eye View requires some explanation here: In line 82, the rectangle of the view is rendered in red. The second rectangle, shown in brown, is the inner rectangle whose boundary triggers scrolling when it is crossed. The coordinates for these rectangles are computed in advance in line 80.

Listing 3.47: Big World – Game.draw()

```

68
69 def draw(self) -> None:
70     self.window_plain.draw()
71     self.window_plain.window.title = f"Plain Window (size={self.window_plain.rect.size},
72                                     fps={self.clock.get_fps():.0f})"
73
74     self.window_pagewise.draw()
75     xy = int(self.window_pagewise.offset.x), int(self.window_pagewise.offset.y)
76     self.window_pagewise.window.title = f"Pagewise Window (offset={xy})"
77
78     self.window_centered.draw()
79     self.window_centered.window.title = f"Centered Window
80                                         (offset={self.window_centered.offset})"
81
82     inner = self.window_pagewise.camera2world(self.window_pagewise.inner_rect)
83     self.window_birdeye.draw([{"rect": self.window_plain.rect, "color": "blue"},

```



```

82         {"rect":self.window_pagewise.rect, "color":"red"},      #
83         {"rect":inner, "color":"brown"},                        #
84         {"rect":self.window_centered.rect, "color":"green"}] )

```

3.3.5 Auto Scrolling/Endless Scrolling



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/V09

Another variant is auto scrolling. In this approach, the background moves automatically and continuously in a fixed direction, while the player usually remains centered and can only evade vertically—for example by jumping. It is also quite common for the background to move downward, requiring the player to jump to higher platforms.

Let us therefore take a look at the new class `WindowAuto`. What is new or special here is that the constructor receives a direction parameter. This consists of two numbers: one for horizontal movement and one for vertical movement. The values represent the speed in px/s, while the sign determines the direction: positive values indicate movement to the right and downward, negative values movement to the left and upward.

Listing 3.48: Big World – Constructor of `WindowAuto`

```

163
164 class WindowAuto:
165
166     def __init__(self, player:pygame.sprite.Sprite, tiles:pygame.sprite.Group,
167                 mobs:pygame.sprite.Group, direction:Vec2Like) -> None:
168         self.tiles = tiles
169         self.mobs = mobs
170         self.player = player
171         self.offset = pygame.Vector2(0, 0) #
172         self.window = pygame.Window(size=cfg.WINDOW.size)
173         self.window.position = (1 * (cfg.WINDOW.width + 60),
174                                1 * (cfg.WINDOW.height) + 30)
175         self.screen : pygame.Surface = self.window.get_surface()
176         self.rect = self.screen.get_frect()
177         self.direction = pygame.math.Vector2(direction)

```

Here as well, essentially only the method `scroll()` needs to be adapted. Only the first line is of interest. According to the task, the offset is adjusted using the specified `direction`. Multiplying by `DELTATIME` allows the unit px/s to be used instead of px/frame.

Listing 3.49: Big World – `WindowAuto.scroll()`

```

193 def scroll(self) -> None:
194     self.offset += self.direction * cfg.DELTATIME

```

```

195     self.offset.x = max(0, self.offset.x)
196     self.offset.y = max(0, self.offset.y)
197     self.offset.x = min(cfg.WORLD.right - cfg.WINDOW.width, self.offset.x)
198     self.offset.y = min(cfg.WORLD.bottom - cfg.WINDOW.height, self.offset.y)

```

For the sake of completeness: do not forget to adjust the name in `save()` ;-)

3.3.6 As a Strategy Pattern

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/03%20WorldScrolling/Pattern

Although I hope that all the techniques presented so far have been explained clearly, it is worthwhile to further decouple the algorithms from the concrete game scenario of my *large world*. After all, these techniques occur repeatedly, making it worthwhile to formulate a reusable solution.

If we compare the three scenarios closely, we find that they differ only in the method `scroll()`, with one or two additional attributes being required in each case. In other words, we can encapsulate the algorithms in separate classes and then formulate a solution using the [strategy pattern](#). The resulting architecture is shown in figure 3.24.

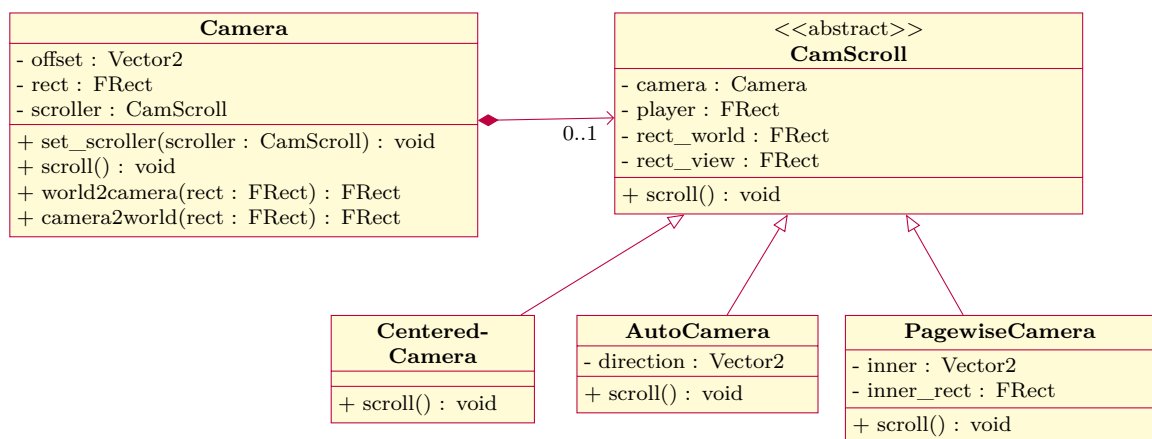


Figure 3.24: Strategy pattern applied on **Camera** and scroll strategies in `cameraview.py`

The class **Camera** is the class that we will later use in our game. It contains all attributes and methods required to render an oversized world. It also contains a *placeholder* for the actual scrolling behavior: the attribute `scroller`. In the method `scroll()`, the scrolling behavior of the behavior class is then invoked.

CamScroll(ABC) is the abstract interface class for the behavior classes and essentially consists only of the abstract method `scroll()`. The four attributes are required for computing the scrolling behavior.

The three concrete behavior classes are **CenteredCamera**, **AutoCamera**, and **Pagewise-Camera**. Here, the method `scroll()` is implemented. The logic of these implementations corresponds exactly to the approaches shown above and should therefore be easy to understand.

A video demonstration can be found here: <https://youtu.be/A2uXPimynnc>.

Further resources include:

- <https://www.youtube.com/watch?v=XmSv2V69Y7A>
- <https://www.youtube.com/watch?v=ARt6DLP38-Y>
- <https://www.youtube.com/watch?v=FDJU8IIObVE>

3.4 State Machine and Scenes

3.4.1 A Simple Example: Tic-Tac-Toe

3.4.1.1 Start Screen

Before we dive into the abstract concept of a state machine, let's look at a simple example that illustrates the problem it is designed to solve. In this case, using a state machine is a bit like using a sledgehammer to crack a nut – but our goal here is simply to understand the underlying idea.

Our example will be [Tic-Tac-Toe](#) – everyone knows it. Since both the game logic and the [User Interface \(UI\)](#) are very simple, we can focus on the architectural challenges rather than the details of the game itself.

So, let's start building the game. The first thing we need is a start screen.

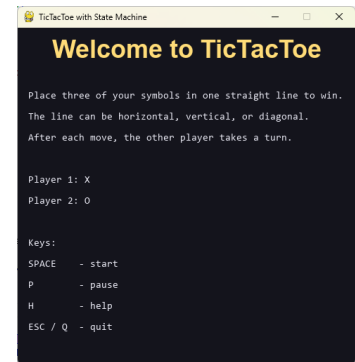


Fig. 3.25: Tic-Tac-Toe – Start

So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/05%20TicTacToe/v01

Requirement 1 Standard functionality

1. The window has a size of 640×480 px.
 2. A welcome message and a help screen are displayed.
 3. The game can be exited using `Esc` or by clicking the red X.
-

Let's start by defining some basic settings in `config.py`, following the requirements in requirement 1.

Listing 3.50: TicTacToe (requirement 1) – `config.py`

```

1 from pygame import Rect
2
3 TITLE = "TicTacToe with State Machine"
4 WINDOW = Rect(0, 0, 480, 480)
5 COLORS = {
6     'BG': (20, 20, 28),
7     'TEXT': (235, 235, 245),
8     'TITLE': (255, 220, 120),
9 }
```

```

10
11 FPS = 60

```

The start screen should display the game title and a short help text. This can be implemented quickly using the techniques we already know. In the `Game` class, the methods `run()`, `watch_for_events()`, `update()`, and `draw()` are implemented in a straightforward way without any surprises.

One small detail is worth mentioning: the fonts are stored in a static variable, namely the `FONTs` dictionary. Of course, there are other ways to organize this, but for a small project like this, the approach works well and keeps the code simple. The result is shown in figure 3.25 on the facing page.

Listing 3.51: TicTacToe (requirement 1) – `tictactoe.py`

```

1 import pygame
2
3 import config as cfg
4
5
6 class Game:
7     FONTs = {}
8
9     def __init__(self):
10         pygame.init()
11         self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
12         self.screen = self.window.get_surface()
13         self.clock = pygame.time.Clock()
14
15         Game.FONTs["title"] = pygame.font.SysFont("arial", 36, bold=True)
16         Game.FONTs["text"] = pygame.font.SysFont("consolas", 14)
17
18         self.running = True
19
20     def run(self) -> None:
21         while self.running:
22             self.watch_for_events()
23             self.update()
24             self.draw()
25             self.clock.tick(cfg.FPS)
26         pygame.quit()
27
28     def watch_for_events(self) -> None:
29         for event in pygame.event.get():
30             if event.type == pygame.QUIT:
31                 self.running = False
32             elif event.type == pygame.KEYDOWN:
33                 if event.key == pygame.K_ESCAPE:
34                     self.running = False
35
36     def update(self) -> None:
37         pass
38
39     def draw(self) -> None:
40         self.screen.fill(cfg.COLORS["BG"])
41         welcome = "Welcome to TicTacToe"
42         text = [
43             "Place three of your symbols in one straight line to win.",
44             "The line can be horizontal, vertical, or diagonal.",
45             "After each move, the other player takes a turn.",
46             ""

```

```

47         "Player 1: X",
48         "Player 2: O",
49         "",
50         "Keys:",
51         "SPACE    - start",
52         "P         - pause",
53         "H         - help",
54         "ESC / Q   - quit",
55     ]
56
57     headline = Game.FONTS["title"].render(welcome, True, cfg.COLORS["TITLE"])
58     rect = headline.get_rect()
59     rect.centerx = cfg.WINDOW.centerx
60     rect.top = 10
61     self.screen.blit(headline, rect)
62     left, top = 15, 90
63     for line in text:
64         textline = Game.FONTS["text"].render(line, True, cfg.COLORS["TEXT"])
65         self.screen.blit(textline, (left, top))
66         top += 30
67     self.window.flip()
68
69
70 def main():
71     Game().run()
72
73 if __name__ == "__main__":
74     main()

```

3.4.1.2 The Playground

Next, we want to create the playground.



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/05%20TicTacToe/v02

Requirement 2 Playground

1. After pressing `SPACE`, the playground is displayed.
 2. It has a size of 420×420 px.
 3. The playground is divided into nine equally sized squares by two vertical and two horizontal lines.
 4. The active player is shown below the playground.
 5. When the player clicks on an empty square with the left mouse button, player 1 places an *X* and player 2 places an *O*. A click on an occupied square is ignored.
 6. For now: If no square is left empty, the game ends.
-

To begin, we add a few basic definitions to `config.py`. The playground is defined as a rectangle whose center is positioned at the center of the window. Furthermore, the `COLORS` dictionary is extended with colors for the new game elements.

Listing 3.52: TicTacToe (requirement 2) – `config.py`

```

1 from pygame import Rect
2
3 TITLE = "TicTacToe with State Machine"
4 WINDOW = Rect(0, 0, 480, 480)
5 PLAYGROUND = Rect(0, 0, 400, 400)
6 PLAYGROUND.center = WINDOW.center
7 CELL_SIZE = PLAYGROUND.width // 3
8 COLORS = {
9     'BG': (20, 20, 28),
10    'TEXT': (235, 235, 245),
11    'TITLE': (255, 220, 120),
12    'GRID': (190, 190, 210),
13    'PLAYGROUND': (32, 32, 46),
14    'O': (238, 214, 130),
15    'X': (238, 132, 146),
16 }
17
18 FPS = 60

```

Similar to `FONTS`, several additional static variables are defined in `Game`. The variable `STATUS` stores the current state of the game. Depending on this status, event handling in `watch_for_events()`, game updates in `update()`, and screen rendering in `draw()` can be performed differently.

The variable `PLAYER` keeps track of the currently active player. The variable `BOARD` is a two-dimensional array that stores the state of each cell: either empty (`None`), occupied by player 1 (`X`), or occupied by player 2 (`O`).

Finally, `SYMBOLS` stores the two `Surface` objects that are used to display an `X` or an `O` on the playing field.

Listing 3.53: TicTacToe (requirement 2) – static data container in `Game`

```

6 class Game:
7     FONTS = {}
8     STATUS = "start"
9     PLAYER = "X"
10    BOARD = [[None, None, None], [None, None, None], [None, None, None]]
11    SYMBOLS = {}

```

Starting at line 22, the constructor of `Game` is extended to initialize the `SYMBOLS` dictionary. The variable `padding` defines the distance between a player symbol and the border of its cell, while `linewidth` specifies the thickness of the lines and the circle outline in pixels. The size of a cell is determined by dividing the playing field into three rows and three columns.

In line 26, a `Surface` object with a transparent background is created. Afterwards, two lines are drawn: first from the upper-left corner to the lower-right corner, and then from the upper-right corner to the lower-left corner, forming the symbol for player 1.

Similarly, in line 33, the `Surface` object for player 2 is created. A circle is then drawn in its center using the appropriate radius.

Listing 3.54: TicTacToe (requirement 2) – `Game.init()`

```

13  def __init__(self):
14      pygame.init()
15      self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
16      self.screen = self.window.get_surface()
17      self.clock = pygame.time.Clock()
18
19      Game.FONTS["title"] = pygame.font.SysFont("arial", 36, bold=True)
20      Game.FONTS["text"] = pygame.font.SysFont("consolas", 14)
21
22      # Creating the player bitmaps
23      padding = 10
24      linewidth = 7
25      cs = cfg.CELL_SIZE
26      Game.SYMBOLS["X"] = pygame.Surface((cs, cs), pygame.SRCALPHA) #
27      pygame.draw.line(Game.SYMBOLS["X"], cfg.COLORS["X"],
28                      (padding, padding), (cs - padding, cs - padding),
29                      linewidth)
30      pygame.draw.line(Game.SYMBOLS["X"], cfg.COLORS["X"],
31                      (cs - padding, padding), (padding, cs - padding),
32                      linewidth)
33      Game.SYMBOLS["O"] = pygame.Surface((cs, cs), pygame.SRCALPHA) #
34      pygame.draw.circle(Game.SYMBOLS["O"], cfg.COLORS["O"],
35                        (cs // 2, cs // 2),
36                        cs // 2 - padding, linewidth)
37
38      self.running = True

```

The variable `STATUS` is now used in `watch_for_events()`. Starting at line 55, keyboard and mouse events are handled differently depending on the current state of the game.

Only when the game is in the *start* state does pressing `[SPACE]` switch the game to play mode. When the game is already in the *play* state, the key press is ignored.

The left mouse button is handled in a similar way. Mouse clicks are processed only when the game is in the *play* state. In this case, `update()` is called with the information that a player symbol should be placed. The target cell can then be determined from the mouse position stored in `event.pos`.

Listing 3.55: TicTacToe (requirement 2) – `Game.watch_for_events()`

```

48  def watch_for_events(self) -> None:
49      for event in pygame.event.get():
50          if event.type == pygame.QUIT:
51              self.running = False
52          elif event.type == pygame.KEYDOWN:
53              if event.key == pygame.K_ESCAPE:
54                  self.running = False
55              if Game.STATUS == "start": #
56                  if event.type == pygame.KEYDOWN:
57                      if event.key == pygame.K_SPACE:
58                          Game.STATUS = "play"
59          elif Game.STATUS == "play":
60              if event.type == pygame.MOUSEBUTTONDOWN:
61                  if event.button == 1:
62                      self.update(action="place", pos=event.pos)

```

In `update()`, while the game is in the *play* state, we first check whether the mouse click was inside the playground. If this is the case, the row and column numbers are calculated.

Let us think this through together using a few concrete numerical examples. Starting from the mouse position, why do these two lines (line 69 and line 70) return values between 0 and 2?

To see this more clearly, please take a closer look at table 3.1 and table 3.2.

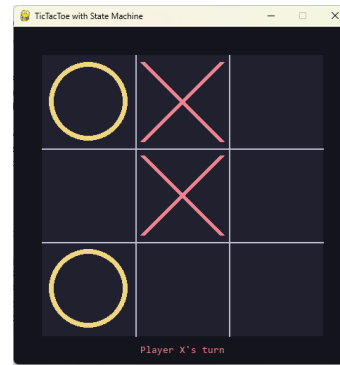


Fig. 3.26: Tic-Tac-Toe – Playground

Table 3.1: Computation of the row number (line 69 in listing 3.56)

pos[1]	top	height	computation			row
120	40	400	→	$(120 - 40) // (400 // 3)$	=	$80 // 133 = 0$
250	40	400	→	$(250 - 40) // (400 // 3)$	=	$210 // 133 = 1$
420	40	400	→	$(420 - 40) // (400 // 3)$	=	$380 // 133 = 2$
318	40	400	→	$(318 - 40) // (400 // 3)$	=	$278 // 133 = 2$
241	40	400	→	$(241 - 40) // (400 // 3)$	=	$201 // 133 = 1$
74	40	400	→	$(74 - 40) // (400 // 3)$	=	$34 // 133 = 0$
71	40	400	→	$(71 - 40) // (400 // 3)$	=	$31 // 133 = 0$
227	40	400	→	$(227 - 40) // (400 // 3)$	=	$187 // 133 = 1$
423	40	400	→	$(423 - 40) // (400 // 3)$	=	$383 // 133 = 2$

Table 3.2: Computation of the col number (line 70 in listing 3.56)

pos[0]	left	width	computation			col
101	40	400	→	$(101 - 40) // (400 // 3)$	=	$61 // 133 = 0$
50	40	400	→	$(50 - 40) // (400 // 3)$	=	$10 // 133 = 0$
163	40	400	→	$(163 - 40) // (400 // 3)$	=	$123 // 133 = 0$
189	40	400	→	$(189 - 40) // (400 // 3)$	=	$149 // 133 = 1$
247	40	400	→	$(247 - 40) // (400 // 3)$	=	$207 // 133 = 1$
281	40	400	→	$(281 - 40) // (400 // 3)$	=	$241 // 133 = 1$
338	40	400	→	$(338 - 40) // (400 // 3)$	=	$298 // 133 = 2$
357	40	400	→	$(357 - 40) // (400 // 3)$	=	$317 // 133 = 2$
418	40	400	→	$(418 - 40) // (400 // 3)$	=	$378 // 133 = 2$

Once the row and column numbers have been calculated, the current player's symbol can be stored in the corresponding cell of `BOARD`. Afterwards, `check_ending()` is called to determine whether there are any empty cells left. If the board is full, the game is terminated.

Listing 3.56: TicTacToe (requirement 2) – `Game.update()`

```

64 def update(self, *args, **kwargs) -> None:
65     if Game.STATUS == "play":
66         if kwargs.get("action") == "place":
67             pos = kwargs.get("pos")
68             if cfg.PLAYGROUND.collidepoint(pos):
69                 row = (pos[1] - cfg.PLAYGROUND.top) // cfg.CELL_SIZE #
70                 col = (pos[0] - cfg.PLAYGROUND.left) // cfg.CELL_SIZE #
71                 if Game.BOARD[row][col] is None:
72                     Game.BOARD[row][col] = Game.PLAYER
73                     Game.PLAYER = "O" if Game.PLAYER == "X" else "X"
74             if self.check_ending():
75                 print("Game Over!")
76                 Game.STATUS = "over"

```

In `check_ending()`, we simply check whether there is still a cell whose value is `None`. If such a cell exists, the method returns `False`. Otherwise, it returns `True`.

Listing 3.57: TicTacToe (requirement 2) – `Game.check_ending()`

```

78 def check_ending(self) -> bool:
79     for row in Game.BOARD:
80         for cell in row:
81             if cell is None:
82                 return False
83     return True

```

3.4.1.3 Do we have a winner?



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/05%20TicTacToe/v03

Requirement 3 Win or Draw?

1. *After each move, it is checked whether the active player has won the game.*
 2. *If nobody has won and the board is full, the game ends in a draw.*
-

To support these new requirements, `update()` has to be modified. Previously, the method only checked whether the board was full. Now the logic is slightly different.

First, in line 71, the game checks whether the active player has won. If so, the status is changed to `win`. This allows the rest of the program to determine whether the game is still running or whether a winner is currently being celebrated.

If the player has not won, the active player is switched.

Only afterwards does the game check whether the board is full and therefore a draw has been reached. In that case, the status is set to draw.

Listing 3.58: TicTacToe (requirement 3) – `Game.update()`

```

62 def update(self, *args, **kwargs) -> None:
63     if Game.STATUS == "play":
64         if kwargs.get("action") == "place":
65             pos = kwargs.get("pos")
66             if cfg.PLAYGROUND.collidepoint(pos):
67                 row = (pos[1] - cfg.PLAYGROUND.top) // (cfg.PLAYGROUND.height // 3)
68                 col = (pos[0] - cfg.PLAYGROUND.left) // (cfg.PLAYGROUND.width // 3)
69                 if Game.BOARD[row][col] is None:
70                     Game.BOARD[row][col] = Game.PLAYER
71                     if self.check_winning(): #
72                         Game.STATUS = "win"
73                         return
74                     else:
75                         self.change_player()
76             if self.check_ending():
77                 Game.STATUS = "draw"

```

The method `change_player()` simply contains the logic for switching the active player. It has been moved into a separate method to keep the code in `update()` shorter and easier to read.

Listing 3.59: TicTacToe (requirement 3) – `Game.change_player()`

```

102 def change_player(self) -> None:
103     Game.PLAYER = "O" if Game.PLAYER == "X" else "X"

```

Checking for a winning position in Tic-Tac-Toe is extremely simple—although it can certainly be implemented in much more complicated ways. The easiest approach is to test all possible winning combinations one by one. This is exactly what the method `check_winning()` does.

Listing 3.60: TicTacToe (requirement 3) – `Game.check_winning()`

```

86 def check_winning(self) -> bool:
87     for i in range(3):
88         # rows
89         if Game.BOARD[i][0] == Game.BOARD[i][1] == Game.BOARD[i][2] is not None:
90             return True
91         # cols
92         if Game.BOARD[0][i] == Game.BOARD[1][i] == Game.BOARD[2][i] is not None:
93             return True
94         # diagonal
95         if Game.BOARD[0][0] == Game.BOARD[1][1] == Game.BOARD[2][2] is not None:
96             return True
97         # anti-diagonal
98         if Game.BOARD[0][2] == Game.BOARD[1][1] == Game.BOARD[2][0] is not None:
99             return True
100     return False

```

Up to this point, everything has been fairly straightforward. But let's take a closer look at `draw()`. You will probably notice that the current implementation works perfectly fine – it displays the correct screen depending on the current game state.

However, is this approach really a good long-term strategy when additional game states or scenes are added? At some point, the growing chains of `if` decisions become difficult to read, maintain, and extend.

In fact, we have already seen the first signs of this problem in `update()` and `watch_for_events()`. There, too, the program had to decide whether certain actions should be performed based on the current status of the game. As more states are introduced, this logic quickly becomes more and more complex.

Listing 3.61: TicTacToe (requirement 3) – `Game.draw()`

```

105 def draw(self) -> None:
106     self.screen.fill(cfg.COLORS["BG"])
107     if Game.STATUS == "start":
108         self.draw_start()
109     elif Game.STATUS == "play":
110         self.draw_play()
111     elif Game.STATUS == "win":
112         self.draw_win()
113     elif Game.STATUS == "draw":
114         self.draw_draw()
115     self.window.flip()

```

We will come back to this issue later.

Avoid building program logic with `if` mountains

Representing complex program logic through large chains or deeply nested structures of `if` decisions leads to code that is difficult to maintain and prone to errors.

For the sake of completeness, we also add the methods `draw_win()` and `draw_draw()`.

Listing 3.62: TicTacToe (requirement 3) – `Game.draw_win()` und `Game.draw_draw()`

```

164 def draw_win(self) -> None:
165     print(f"Player {Game.PLAYER} wins!")
166
167 def draw_draw(self) -> None:
168     print("It's a draw!")

```

3.4.1.4 Restart

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/05%20TicTacToe/v04

Requirement 4 Win or Draw?

1. *After the game has ended, the player is asked whether to restart the game or quit.*
 2. *The game displays which player has won.*
 3. *The game displays that the match ended in a draw.*
-

Before we solve the problem from the previous section, we first add a small extension and a few visual improvements.

Because of requirement 2 and 3, we need a method that displays messages on the screen. Each message should also be able to have a headline. We solve this task in `draw_message()`. In the first three lines, a semi-transparent `Surface` is created, which is later placed on top of the game screen. Because of the transparency, the last board state remains visible in the background. After that, the headline and the individual text lines are blitted onto this `Surface` object. Finally, the whole `Surface` object is drawn onto the screen.

Listing 3.63: TicTacToe (requirement 4) – `Game.draw_message()`

```

178 def draw_message(self, headline: str, message: list[str]) -> None:
179     surface = pygame.Surface(cfg.WINDOW.size, pygame.SRCALPHA)
180     surface.fill((0, 0, 0, 180))
181     headline = Game.FONTS["title"].render(headline, True, cfg.COLORS["TITLE"])
182     rect = headline.get_rect()
183     rect.centerx = cfg.WINDOW.centerx
184     rect.top = 10
185     surface.blit(headline, rect)
186     left, top = 15, 90
187     for line in message:
188         textline = Game.FONTS["text"].render(line, True, cfg.COLORS["TEXT"])
189         surface.blit(textline, (left, top))
190         top += 30
191     self.screen.blit(surface, (0, 0))

```

Performance issues caused by unnecessary graphics creation

The method `draw_message()` creates the `Surface` object from scratch every time it is called. In addition, the unchanged text is rendered again and again for every frame, even though it does not change. Since the game runs at around 60 frames per second, this work is repeated approximately 60 times each second. A possible solution is left as an exercise for the interested reader :-)

All that remains is to provide the appropriate parameter values and call the method.

Listing 3.64: TicTacToe (requirement 4) – `draw_start()`, `draw_win()` und `draw_draw()`

```

144 def draw_start(self) -> None:
145     welcome = "Welcome to TicTacToe"
146     text = [

```

```

147         "Place three of your symbols in one straight line to win.",
148         "The line can be horizontal, vertical, or diagonal.",
149         "After each move, the other player takes a turn.",
150         "",
151         "Player 1: X",
152         "Player 2: O",
153         "",
154         "Keys:",
155         "SPACE - start",
156         "P      - pause",
157         "H      - help",
158         "ESC / Q - quit",
159     ]
160     self.draw_message(welcome, text)
161
162     def draw_win(self) -> None:
163         welcome = f"Player {Game.PLAYER} wins!"
164         text = [
165             "Press Y to play again.",
166             "Press N/ Q /ESC to quit.",
167         ]
168         self.draw_message(welcome, text)
169
170     def draw_draw(self) -> None:
171         welcome = "It's a draw!"
172         text = [
173             "Press Y to play again.",
174             "Press N/ Q /ESC to quit.",
175         ]
176         self.draw_message(welcome, text)

```

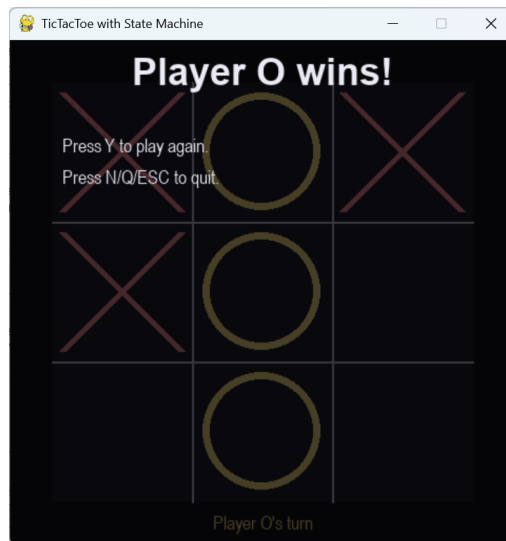


Figure 3.27: You've won

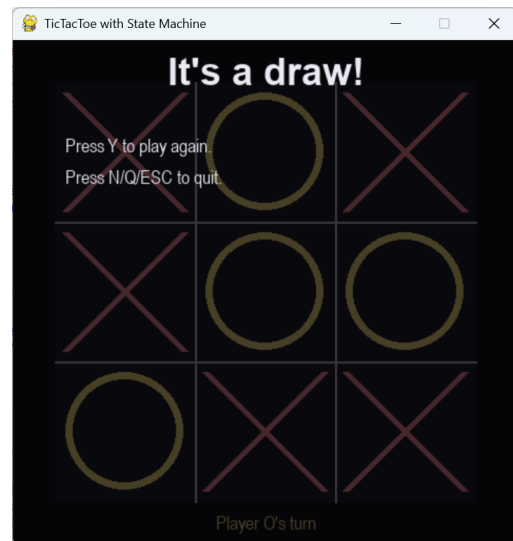


Figure 3.28: It's a draw

But how do we react to the user's answer? First, `watch_for_events()` has to be extended. Starting at line 61, the input is handled depending on the current status of the game.

Listing 3.65: TicTacToe (requirement 4) – `watch_for_events()`

```

46 def watch_for_events(self) -> None:
47     for event in pygame.event.get():
48         if event.type == pygame.QUIT:
49             self.running = False
50         elif event.type == pygame.KEYDOWN:
51             if event.key in (pygame.K_ESCAPE, pygame.K_q):
52                 self.running = False
53         if Game.STATUS == "start":
54             if event.type == pygame.KEYDOWN:
55                 if event.key == pygame.K_SPACE:
56                     Game.STATUS = "play"
57         elif Game.STATUS == "play":
58             if event.type == pygame.MOUSEBUTTONDOWN:
59                 if event.button == 1:
60                     self.update(action="place", pos=event.pos)
61         elif Game.STATUS in ["win", "draw"]:
62             if event.type == pygame.KEYDOWN:
63                 if event.key == pygame.K_y:
64                     self.reset()
65                 elif event.key == pygame.K_n:
66                     self.running = False

```

Finally, `reset()` restores all variables to their initial values.

Listing 3.66: TicTacToe (requirement 4) – `reset()`

```

194 def reset(self) -> None:
195     Game.STATUS = "start"
196     Game.PLAYER = "X"
197     Game.BOARD = [[None] * 3 for _ in range(3)]

```

3.4.1.5 Separating Strings from the Code



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/05%20TicTacToe/v05

For the sake of readability and to simplify the adaptation to other languages, string constants should be stored in separate files. In our example, moving them to `config.py` is sufficient.

Requirement 5 Externalize String Constants

String constants are stored in a separate file.

In our implementation, we store the messages in the dictionary `MSGSTRINGS`.

Listing 3.67: TicTacToe (requirement 5) – MSGSTRING in config.py

```

20 MSGSTRINGS = {}
21 MSGSTRINGS["start"] = {
22     "headline": "Welcome to Tic Tac Toe!",
23     "message": [
24         "Place three of your symbols in one straight line to win.",
25         "The line can be horizontal, vertical, or diagonal.",
26         "After each move, the other player takes a turn.",
27         "",
28         "Player 1: X",
29         "Player 2: O",
30         "",
31         "Keys:",
32         "SPACE    - start",
33         "P        - pause",
34         "H        - help",
35         "ESC / Q   - quit",
36     ]
37 }
38 MSGSTRINGS["win"] = {
39     "headline": "Player Wins!",
40     "message": [
41         "Congratulations!",
42         "You have won the game.",
43         "",
44         "Press Y to play again.",
45         "Press N / Q /ESC to quit.",
46     ]
47 }
48 MSGSTRINGS["draw"] = {
49     "headline": "It's a Draw!",
50     "message": [
51         "The game ended in a draw.",
52         "",
53         "Press Y to play again.",
54         "Press N / Q /ESC to quit.",
55     ]
56 }

```

As a consequence, the code in `tictactoe.py` becomes much cleaner and easier to understand.

Listing 3.68: TicTacToe (requirement 5) – Verwendung von MSGSTRING in Game

```

144 def draw_start(self) -> None:
145     self.draw_message(cfg.MSGSTRINGS["start"])
146
147 def draw_win(self) -> None:
148     self.draw_message(cfg.MSGSTRINGS["win"])
149
150 def draw_draw(self) -> None:
151     self.draw_message(cfg.MSGSTRINGS["draw"])
152
153 def draw_message(self, message: dict[str, str|list[str]]) -> None:
154     surface = pygame.Surface(cfg.WINDOW.size, pygame.SRCALPHA)
155     surface.fill((0, 0, 0, 180))
156     headline = Game.FONTS["title"].render(message["headline"], True, cfg.COLORS["TITLE"])
157     rect = headline.get_rect()
158     rect.centerx = cfg.WINDOW.centerx
159     rect.top = 10
160     surface.blit(headline, rect)
161     left, top = 15, 90
162     for line in message["message"]:

```

```

163         textline = Game.FONTS["text"].render(line, True, cfg.COLORS["TEXT"])
164         surface.blit(textline, (left, top))
165         top += 30
166     self.screen.blit(surface, (0, 0))

```

3.4.2 Introduction of State Machines

3.4.2.1 Refactoring Scenes



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/05%20TicTacToe/v06

Let us take look at the advantages and disadvantages of the current architecture in table 3.3.

Table 3.3: Evaluation of the STATUS approach

+ organic growth	The source code reflects the developer's thought process during development quite well.
+ quick and dirty	For smaller projects, this approach can be both effective and appropriate.
- many <code>if self.STATUS ==</code> chains	Poor readability.
- complex decision structures	Difficult to extend and highly error-prone.
- everything in a single source file	Collaborative development often leads to numerous merge conflicts.

Up to this point, we have moved around quite a lot of source code without actually talking much about state machines. In computer science, a state machine is a way of modeling software, just like class diagrams or sequence diagrams. A state machine describes how the state of a program changes in response to events.

State Machine

A state machine consists of a finite set of states and well-defined transitions between them. At any point in time, the program behavior depends on its current state.

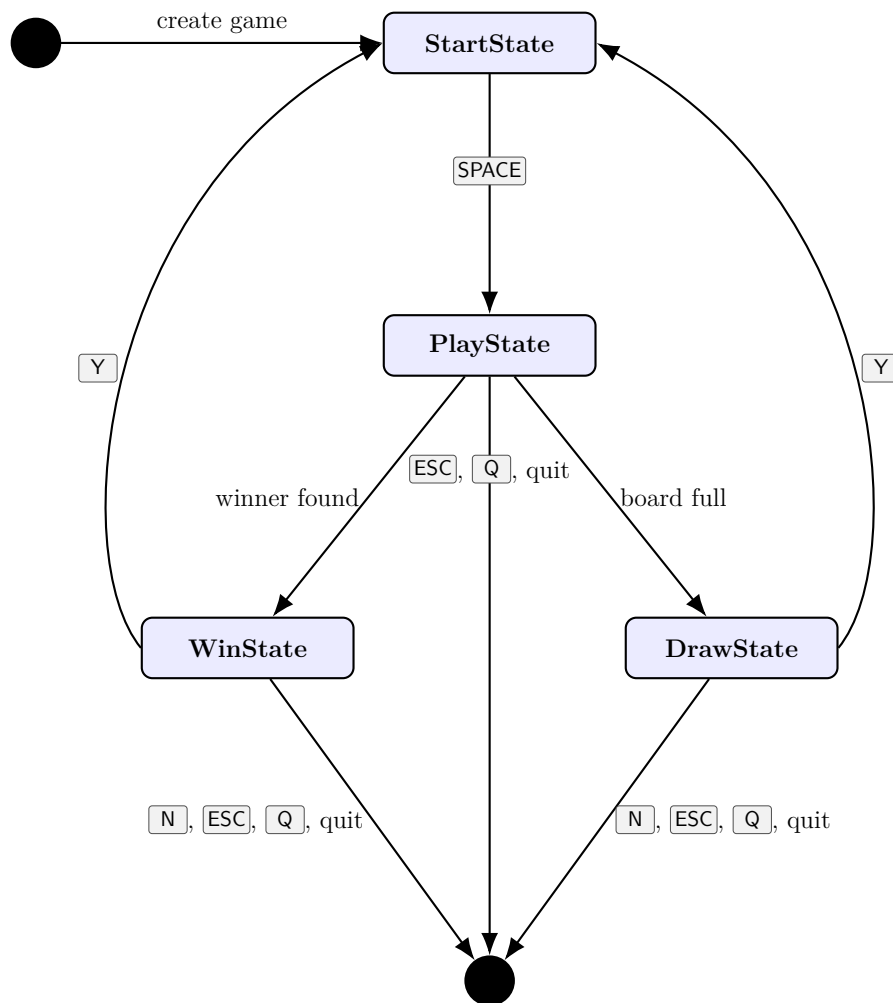


Figure 3.29: Tic-Tac-Toe – UML state machine diagram

For our example, the corresponding UML state machine diagram is shown in figure 3.29.

In our example, the initial *start* state is left when the player presses `SPACE`, causing the application to enter the *play* state. Throughout the program, chains of `if` statements are then used to determine the current state and decide how the program should react.

The idea behind using a state machine in game development is to treat each state as if it were a separate program within the program. For every state (or status), a dedicated variant of **Game** is implemented in such a way that it is responsible for handling only that single state. This dramatically reduces the complexity of the overall program and, following the principle of *divide et impera*, distributes the complexity among the individual state implementations.

If we take a look at the latest version of our game, we can identify the following states:

- *start*: The start screen is displayed.
- *play*: Xs and Os are placed on the playing field.

- *win*: The active player has won the game.
- *draw*: Neither player has won.

We can also identify the methods in which these game states are currently checked:

- `watch_for_events()`
- `update()`
- `draw()`

As a consequence, we need to provide a separate implementation of these three methods for each game state. A class hierarchy is a convenient way to achieve this.

Let us therefore define a base class called **State**. To keep the project structure organized, this class is placed in the `scenes` subdirectory (section 2.0.2 on page 15).

Listing 3.69: TicTacToe – **State**

```

1  from abc import ABC, abstractmethod
2
3
4  class State(ABC):
5      def __init__(self, game):
6          self.game = game
7
8      @abstractmethod
9      def watch_for_events(self):
10         pass
11
12     @abstractmethod
13     def update(self, **kwargs):
14         pass
15
16     @abstractmethod
17     def draw(self):
18         pass

```

Now the relevant parts of the methods `watch_for_events()`, `update()`, and `draw()` are distributed among the classes **StartState**, **PlayState**, **WinState**, and **DrawState**. As a result, each class contains only the source code that is actually needed for its particular *scene*. You can use the class diagram in figure 3.30 on the following page as a guide.

As a first example, let us look at the class **StartState**. Since the game starts in this scene, an object of this class is created inside **Game** (see line 37).

Listing 3.70: TicTacToe – **Game.__init__()**

```

12  def __init__(self):
13      pygame.init()
14      self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
15      self.screen = self.window.get_surface()
16      self.clock = pygame.time.Clock()
17
18      Game.FONTS["title"] = pygame.font.SysFont("arial", 36, bold=True)
19      Game.FONTS["text"] = pygame.font.SysFont("consolas", 14)
20

```

```

21     # Creating the player bitmaps
22     padding = 10
23     linewidth = 7
24     Game.SYMBOLS["X"] = pygame.Surface((cfg.CELL_SIZE, cfg.CELL_SIZE), pygame.SRCALPHA)
25     pygame.draw.line(Game.SYMBOLS["X"], cfg.COLORS["X"],
26                     (padding, padding), (cfg.CELL_SIZE - padding, cfg.CELL_SIZE -
27                                         padding),
28                                         linewidth)
29     pygame.draw.line(Game.SYMBOLS["X"], cfg.COLORS["X"],
30                     (cfg.CELL_SIZE - padding, padding), (padding, cfg.CELL_SIZE -
31                                         padding),
32                                         linewidth)
33     Game.SYMBOLS["O"] = pygame.Surface((cfg.CELL_SIZE, cfg.CELL_SIZE), pygame.SRCALPHA)
34     pygame.draw.circle(Game.SYMBOLS["O"], cfg.COLORS["O"],
35                       (cfg.CELL_SIZE // 2, cfg.CELL_SIZE // 2),
36                       cfg.CELL_SIZE // 2 - padding, linewidth)
37
38     self.running = True
39     self.state = StartState(self) #

```

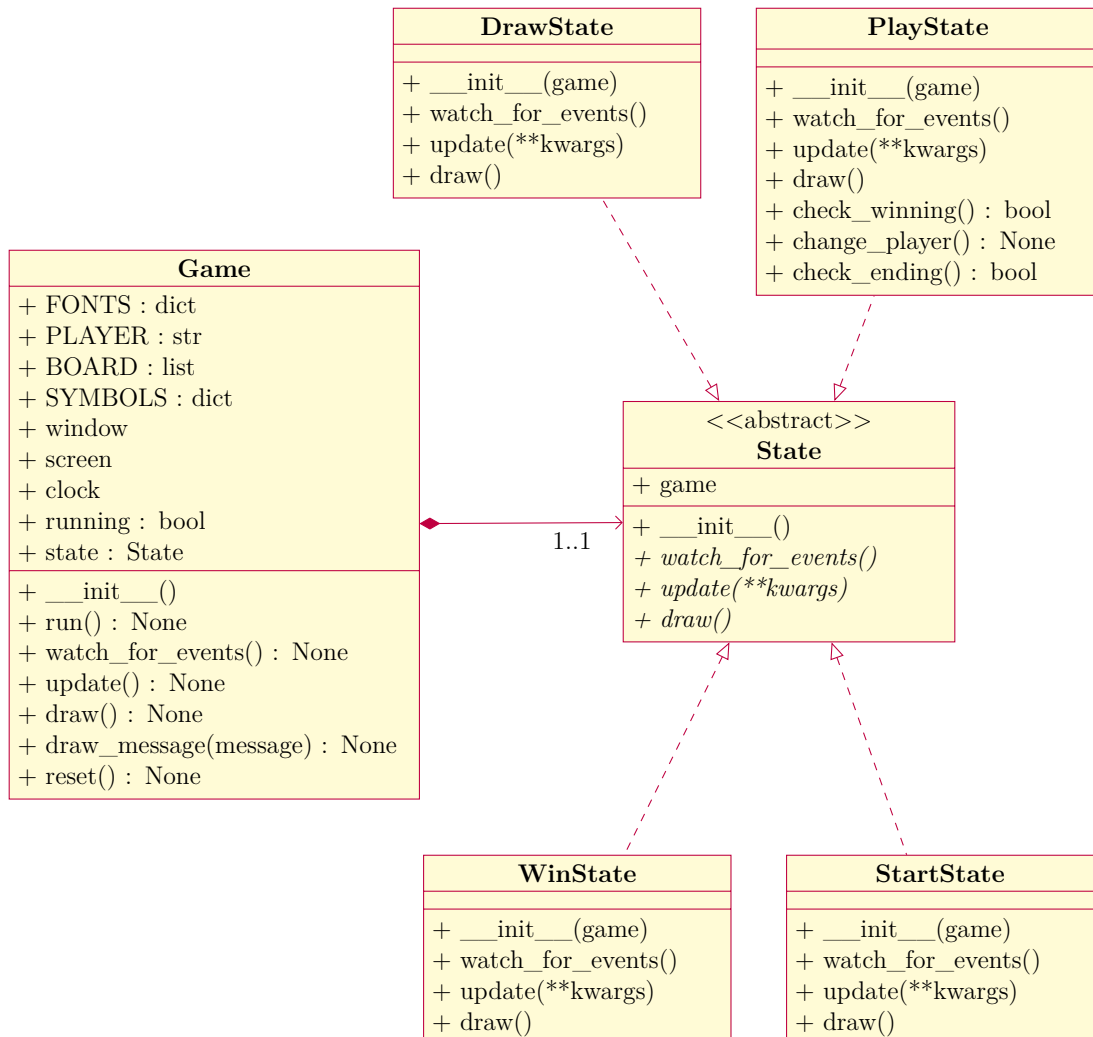


Figure 3.30: Example of a simple State Machine

The methods `watch_for_events()`, `update()`, and `draw()` are now greatly simplified. They no longer contain any state-specific logic themselves. Instead, they simply delegate their work to the corresponding methods of the currently active state object.

Listing 3.71: TicTacToe – `watch_for_events()`, `update()`, and `draw()`

```

47     def watch_for_events(self) -> None:
48         self.state.watch_for_events()
49
50     def update(self) -> None:
51         self.state.update()
52
53     def draw(self) -> None:
54         self.screen.fill(cfg.COLORS["BG"])
55         self.state.draw()
56         self.window.flip()

```

Within `StartState` itself, we only need to check for the keys used to quit the game and whether `SPACE` has been pressed. If `SPACE` is pressed, the next scene is created in line 21 and assigned to the attribute `Game.state`. Consequently, in the next frame, `Game` delegates the calls to `watch_for_events()`, `update()`, and `draw()` to `PlayState`.

Listing 3.72: TicTacToe – `StartState`

```

7  class StartState(State):
8      def __init__(self, game):
9          super().__init__(game)
10         self.game.reset()
11
12     def watch_for_events(self):
13         for event in pygame.event.get():
14             if event.type == pygame.QUIT:
15                 self.game.running = False
16             elif event.type == pygame.KEYDOWN:
17                 if event.key in (pygame.K_ESCAPE, pygame.K_q):
18                     self.game.running = False
19                 elif event.key == pygame.K_SPACE:
20                     from scenes.playstate import PlayState # to avoid circular imports
21                     self.game.state = PlayState(self.game) #
22
23     def update(self, **kwargs):
24         pass
25
26     def draw(self):
27         self.game.draw_message(cfg.MSGSTRINGS["start"])

```

Things work almost exactly the same way in `WinState` and `DrawState`. The only additional task is to ask the player whether another round should be played. Pressing `No` immediately quits the game, whereas pressing `Yes` calls `Game.reset()` and assigns `StartState` to `Game.state` again in order to restart the game.

Listing 3.73: TicTacToe – `WinState`

```

7  class WinState(State):
8      def __init__(self, game):
9          super().__init__(game)

```

```

10
11     def watch_for_events(self):
12         for event in pygame.event.get():
13             if event.type == pygame.QUIT:
14                 self.game.running = False
15             elif event.type == pygame.KEYDOWN:
16                 if event.key in (pygame.K_ESCAPE, pygame.K_q):
17                     self.game.running = False
18                 elif event.key == pygame.K_y:
19                     from scenes.startstate import StartState
20                     self.game.reset()
21                     self.game.state = StartState(self.game)
22                 elif event.key == pygame.K_n:
23                     self.game.running = False
24
25     def update(self, **kwargs):
26         pass
27
28     def draw(self):
29         cfg.MSGSTRINGS["win"]["headline"] = f"Player {self.game.PLAYER} Wins!"
30         self.game.draw_message(cfg.MSGSTRINGS["win"])

```

Listing 3.74: TicTacToe – DrawState

```

7 class DrawState(State):
8     def __init__(self, game):
9         super().__init__(game)
10
11     def watch_for_events(self):
12         for event in pygame.event.get():
13             if event.type == pygame.QUIT:
14                 self.game.running = False
15             elif event.type == pygame.KEYDOWN:
16                 if event.key in (pygame.K_ESCAPE, pygame.K_q):
17                     self.game.running = False
18                 elif event.key == pygame.K_y:
19                     from scenes.startstate import StartState
20                     self.game.reset()
21                     self.game.state = StartState(self.game)
22                 elif event.key == pygame.K_n:
23                     self.game.running = False
24
25     def update(self, **kwargs):
26         pass
27
28     def draw(self):
29         self.game.draw_message(cfg.MSGSTRINGS["draw"])

```

`PlayState` is slightly more complex because it contains more game logic. However, its overall structure is exactly the same as that of the other states – so don't let that confuse you ;-)

What happens inside the individual methods has already been explained in detail when we introduced the Tic-Tac-Toe example earlier in this chapter. The important point here is that the game remains in the `PlayState` scene until either a player wins the game (line 32) or the game ends in a draw (line 38).

Listing 3.75: TicTacToe – PlayState

```

7 class PlayState(State):
8     def __init__(self, game):
9         super().__init__(game)
10
11     def watch_for_events(self):
12         for event in pygame.event.get():
13             if event.type == pygame.QUIT:
14                 self.game.running = False
15             elif event.type == pygame.KEYDOWN:
16                 if event.key in (pygame.K_ESCAPE, pygame.K_q):
17                     self.game.running = False
18             elif event.type == pygame.MOUSEBUTTONDOWN:
19                 if event.button == 1:
20                     self.update(action="place", pos=event.pos)
21
22     def update(self, **kwargs):
23         if kwargs.get("action") == "place":
24             pos = kwargs.get("pos")
25             if cfg.PLAYGROUND.collidepoint(pos):
26                 row = (pos[1] - cfg.PLAYGROUND.top) // cfg.CELL_SIZE
27                 col = (pos[0] - cfg.PLAYGROUND.left) // cfg.CELL_SIZE
28                 if self.game.BOARD[row][col] is None:
29                     self.game.BOARD[row][col] = self.game.PLAYER
30                     if self.check_winning():
31                         from scenes.winstate import WinState
32                         self.game.state = WinState(self.game) #
33                         return
34                     else:
35                         self.change_player()
36                 if self.check_ending():
37                     from scenes.drawstate import DrawState
38                     self.game.state = DrawState(self.game) #
39
40     def draw(self):
41         self.game.screen.fill(cfg.COLORS["PLAYGROUND"], cfg.PLAYGROUND)
42         for i in range(1, 3):
43             pygame.draw.line(self.game.screen, cfg.COLORS["GRID"], (cfg.PLAYGROUND.left +
44                 i*cfg.CELL_SIZE, cfg.PLAYGROUND.top), (cfg.PLAYGROUND.left +
45                 i*cfg.CELL_SIZE, cfg.PLAYGROUND.bottom), 2)
46             pygame.draw.line(self.game.screen, cfg.COLORS["GRID"], (cfg.PLAYGROUND.left,
47                 cfg.PLAYGROUND.top + i*cfg.CELL_SIZE), (cfg.PLAYGROUND.right,
48                 cfg.PLAYGROUND.top + i*cfg.CELL_SIZE), 2)
49         player = self.game.FONTS["text"].render(f"Player {self.game.PLAYER}'s turn", True,
50             cfg.COLORS[self.game.PLAYER])
51         rect = player.get_rect()
52         rect.centerx = cfg.PLAYGROUND.centerx
53         rect.bottom = cfg.WINDOW.bottom - 10
54         for y in range(3):
55             for x in range(3):
56                 cell = self.game.BOARD[y][x]
57                 if cell is not None:
58                     symbol = self.game.SYMBOLS[cell]
59                     symbol_rect = symbol.get_rect()
60                     symbol_rect.center = (cfg.PLAYGROUND.left + (x + 0.5) * cfg.CELL_SIZE,
61                         cfg.PLAYGROUND.top + (y + 0.5) * cfg.CELL_SIZE)
62                     self.game.screen.blit(symbol, symbol_rect)
63         self.game.screen.blit(player, rect)
64
65     def check_winning(self) -> bool:
66         for i in range(3):
67             # rows
68             if self.game.BOARD[i][0] == self.game.BOARD[i][1] == self.game.BOARD[i][2] is
69                 not None:
70                 return True

```

```

64         # cols
65         if self.game.BOARD[0][i] == self.game.BOARD[1][i] == self.game.BOARD[2][i] is
           not None:
66             return True
67     # diagonal
68     if self.game.BOARD[0][0] == self.game.BOARD[1][1] == self.game.BOARD[2][2] is not
       None:
69         return True
70     # anti-diagonal
71     if self.game.BOARD[0][2] == self.game.BOARD[1][1] == self.game.BOARD[2][0] is not
       None:
72         return True
73     return False
74
75     def change_player(self) -> None:
76         self.game.PLAYER = "O" if self.game.PLAYER == "X" else "X"
77
78     def check_ending(self) -> bool:
79         for row in self.game.BOARD:
80             for cell in row:
81                 if cell is None:
82                     return False
83         return True

```

The elegance of this approach becomes apparent rather quickly. The individual scenes, or game states, can be implemented largely independently of one another. As a result, debugging, maintenance, and collaborative development become much easier.

3.4.2.2 Using Events

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/01%20Techniques/05%20TicTacToe/v07

What I really do not like is that objects of the state classes are created inside other state classes. What do I mean by that? In `StartState`, after pressing `SPACE`, an object of `PlayState` is created and assigned to `Game.state`. *Brr*. I even had to use local `import` statements to avoid circular dependencies.

A much cleaner solution would be for the state objects to merely notify `Game` that the state, or scene, should change. Then `watch_for_events()` would be responsible for performing the actual transition. `StartState` and the other state classes would no longer need to know about each other. In other words, we decouple these source files from one another.

The first implementation demonstrates the basic idea of encapsulating scenes into separate classes. The event-based version refines this design by reducing coupling between these classes.

To do this, we first define the appropriate events in `Game`.

Listing 3.76: TicTacToe – User defined events

```

1 import config as cfg
2 import pygame
3 from scenes import DrawState, PlayState, StartState, WinState
4
5
6 class Game:
7     FONTS = {}
8     PLAYER = "X"
9     BOARD = [[None, None, None], [None, None, None], [None, None, None]]
10    SYMBOLS = {}
11    EVENTS = {
12        "START": pygame.event.custom_type(),
13        "PLAY": pygame.event.custom_type(),
14        "WIN": pygame.event.custom_type(),
15        "DRAW": pygame.event.custom_type(),
16    }

```

And in `__init__()`, see line 43, we no longer assign a `StartState` object directly. Instead, the corresponding event is triggered.

Listing 3.77: TicTacToe – Using START event

```

18 def __init__(self):
19     pygame.init()
20     self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.TITLE)
21     self.screen = self.window.get_surface()
22     self.clock = pygame.time.Clock()
23
24     Game.FONTS["title"] = pygame.font.SysFont("arial", 36, bold=True)
25     Game.FONTS["text"] = pygame.font.SysFont("consolas", 14)
26
27     # Creating the player bitmaps
28     padding = 10
29     linewidth = 7
30     Game.SYMBOLS["X"] = pygame.Surface((cfg.CELL_SIZE, cfg.CELL_SIZE), pygame.SRCALPHA)
31     pygame.draw.line(Game.SYMBOLS["X"], cfg.COLORS["X"],
32                     (padding, padding), (cfg.CELL_SIZE - padding, cfg.CELL_SIZE -
33                                         padding),
34                     linewidth)
35     pygame.draw.line(Game.SYMBOLS["X"], cfg.COLORS["X"],
36                     (cfg.CELL_SIZE - padding, padding), (padding, cfg.CELL_SIZE -
37                                         padding),
38                     linewidth)
39     Game.SYMBOLS["O"] = pygame.Surface((cfg.CELL_SIZE, cfg.CELL_SIZE), pygame.SRCALPHA)
40     pygame.draw.circle(Game.SYMBOLS["O"], cfg.COLORS["O"],
41                       (cfg.CELL_SIZE // 2, cfg.CELL_SIZE // 2),
42                       cfg.CELL_SIZE // 2 - padding, linewidth)
43
44     self.running = True
45     pygame.event.post(pygame.event.Event(Game.EVENTS["START"])) #

```

This event is now processed in `watch_for_events()`, just like all other events. The first step is to handle immediate termination of the game triggered by keyboard input. As a consequence, this logic can now be removed from the individual `State` classes. After that

After that, the user defined events are processed, and the corresponding `State` object is created if necessary. When handling the events for restarting the game, `reset()` is now called at this point, which appears to be a more natural place from a design perspective.

Listing 3.78: TicTacToe – watch_for_events()

```

53 def watch_for_events(self) -> None:
54     events = pygame.event.get()
55     for event in events:
56         if event.type == pygame.QUIT:
57             self.running = False
58         elif event.type == pygame.KEYDOWN:
59             if event.key in (pygame.K_ESCAPE, pygame.K_q):
60                 self.running = False
61         elif event.type == Game.EVENTS["START"]:
62             self.reset()
63             self.state = StartState(self)
64         elif event.type == Game.EVENTS["PLAY"]:
65             self.state = PlayState(self)
66         elif event.type == Game.EVENTS["WIN"]:
67             self.reset()
68             self.state = WinState(self)
69         elif event.type == Game.EVENTS["DRAW"]:
70             self.reset()
71             self.state = DrawState(self)
72     self.state.watch_for_events(events)

```

The responsibilities of the `State` classes are now much more focused. Each class processes only the logic that is relevant to its particular scene. For instance, `StartState` merely checks whether `SPACE` has been pressed and, if so, triggers the appropriate event.

Listing 3.79: TicTacToe – StartState

```

9 class StartState(State):
10     def __init__(self, game):
11         super().__init__(game)
12
13     def watch_for_events(self, events:List[pygame.event.Event]) -> None:
14         for event in events:
15             if event.type == pygame.KEYDOWN:
16                 if event.key == pygame.K_SPACE:
17                     pygame.event.post(pygame.event.Event(self.game.EVENTS["PLAY"]))
18
19     def update(self, **kwargs):
20         pass
21
22     def draw(self):
23         self.game.draw_message(cfg.MSGSTRINGS["start"])

```

Things work in much the same way in `WinState` and `DrawState`. Each of these classes focuses solely on the behavior required for its own scene.

Listing 3.80: TicTacToe – WinState

```

13 def watch_for_events(self, events:List[pygame.event.Event]) -> None:
14     for event in events:
15         if event.type == pygame.KEYDOWN:
16             if event.key == pygame.K_y:
17                 pygame.event.post(pygame.event.Event(self.game.EVENTS["START"])) #
18             elif event.key == pygame.K_n:
19                 self.game.running = False

```

Listing 3.81: TicTacToe – DrawState

```

13 def watch_for_events(self, events:List[pygame.event.Event]) -> None:
14     for event in events:
15         if event.type == pygame.KEYDOWN:
16             if event.key == pygame.K_y:
17                 pygame.event.post(pygame.event.Event(self.game.EVENTS["START"])) #
18             elif event.key == pygame.K_n:
19                 self.game.running = False

```

Game ending scene is missing

It is worth noting that the game is still terminated directly by setting `running = False` within the `State` classes. This approach is now outdated and represents a break in the overall design.

Extend the game by introducing a new class called `StateEnd` and modify the program accordingly at all relevant points.

This leaves only the necessary adjustments to `PlayState`.

Listing 3.82: TicTacToe – PlayState

```

13 def watch_for_events(self, events:List[pygame.event.Event]) -> None:
14     for event in events:
15         if event.type == pygame.MOUSEBUTTONDOWN:
16             if event.button == 1:
17                 self.update(action="place", pos=event.pos)
18
19 def update(self, **kwargs):
20     if kwargs.get("action") == "place":
21         pos = kwargs.get("pos")
22         if cfg.PLAYGROUND.collidepoint(pos):
23             row = (pos[1] - cfg.PLAYGROUND.top) // cfg.CELL_SIZE
24             col = (pos[0] - cfg.PLAYGROUND.left) // cfg.CELL_SIZE
25             if self.game.BOARD[row][col] is None:
26                 self.game.BOARD[row][col] = self.game.PLAYER
27                 if self.check_winning():
28                     pygame.event.post(pygame.event.Event(self.game.EVENTS["WIN"])) #
29                     return
30                 else:
31                     self.change_player()
32             if self.check_ending():
33                 pygame.event.post(pygame.event.Event(self.game.EVENTS["DRAW"])) #
34                 return

```

Summary

In this chapter, we used the simple example of Tic-Tac-Toe to motivate the use of state machines in game development. Initially, the game logic was implemented using a status variable and increasingly large chains of `if` chains. While this approach works for small examples, it quickly becomes difficult to read, maintain, and extend as additional scenes and game states are introduced.

To address this problem, we introduced the concept of a state machine. Each

scene of the game was represented by its own class derived from the abstract base class `State`. Responsibilities such as event handling, updating the game logic, and rendering the screen were distributed among these state classes. As a consequence, each class focused only on the behavior relevant to its particular scene.

We further improved the design by decoupling the state classes from one another. Instead of directly creating other state objects, the states now communicate desired transitions through user-defined events, while `Game` remains responsible for performing the actual state changes.

The resulting architecture follows the principle of *divide et impera*: a complex problem is decomposed into smaller, more manageable parts. This leads to source code that is easier to understand, debug, maintain, and extend. Although the Tic-Tac-Toe example is intentionally simple, the same architectural ideas can be applied successfully to much larger game projects.

Using a sledgehammer to crack a nut

For small projects, a simple status variable may be entirely sufficient. State machines become particularly valuable when the number of scenes and transitions increases.

3.4.3 Stacked States / Pushdown Automaton

There are situations in which one state should not replace another but merely overlay it. Consider, for example, the still missing scenes `PauseState` and `HelpState`. Both of these scenes should display the current game state in the background, visible through a translucent overlay.

Up to this point, our implementation assumed that exactly one state is active at a time. However, some scenes such as pause menus, help screens, or inventory windows do not replace the underlying game. Instead, they temporarily overlay it while preserving the previous state.

One possible approach is to organize the states in a [stack](#). However, this also means that additional methods are required to support this extended functionality, since sometimes a state needs to be replaced, while at other times a new state should simply be pushed onto the stack.

Before we turn our attention to this problem, a few preparations are in order. Let us begin by defining two new events: one for entering the help screen and another for leaving it.

Listing 3.83: TicTacToe – help events

```

12  EVENTS = {
13      "START": pygame.event.custom_type(),
14      "PLAY": pygame.event.custom_type(),
15      "WIN": pygame.event.custom_type(),
16      "DRAW": pygame.event.custom_type(),
17      "HELP": pygame.event.custom_type(),
18      "UNHELP": pygame.event.custom_type(),
19  }

```

We also need to add an appropriate help message to `config.py`.

Listing 3.84: TicTacToe – help text

```

54  MSGSTRINGS["help"] = {"headline": "How to play Tic Tac Toe!",
55                          "message": [
56                              "Place three of your symbols in one straight line to
57                                  win.",
58                              "The line can be horizontal, vertical, or diagonal.",
59                              "After each move, the other player takes a turn.",
60                              "",
61                              "Player 1:  X",
62                              "Player 2:  O",
63                              "",
64                              "Keys:",
65                              "  SPACE    - start",
66                              "  P       - pause/unpause",
67                              "  H       - help/unhelp",
68                              "  ESC / Q - quit",
69                          ]

```

We also need to implement the `HelpState` class. Keep in mind that pressing `H` in this state should close the help screen again. Therefore, the event `UNHELP` must be triggered.

Listing 3.85: TicTacToe – HelpState

```

9  class HelpState(State):
10      def __init__(self, game):
11          super().__init__(game)
12
13      def watch_for_events(self, events:List[pygame.event.Event]) -> None:
14          for event in events:
15              if event.type == pygame.KEYDOWN:
16                  if event.key == pygame.K_h:
17                      pygame.event.post(pygame.event.Event(self.game.EVENTS["UNHELP"]))
18
19      def update(self, **kwargs):
20          pass
21
22      def draw(self):
23          self.game.draw_message(cfg.MSGSTRINGS["help"])

```

`PlayState` now needs to be extended to process the `H` key, since this is used to open the help screen. The method `watch_for_events()` is adapted accordingly.

Listing 3.86: TicTacToe – PlayState

```

13     def watch_for_events(self, events:List[pygame.event.Event]) -> None:
14         for event in events:
15             if event.type == pygame.KEYDOWN:
16                 if event.key == pygame.K_h:
17                     pygame.event.post(pygame.event.Event(self.game.EVENTS["HELP"]))    #
18             elif event.type == pygame.MOUSEBUTTONDOWN:
19                 if event.button == 1:
20                     self.update(action="place", pos=event.pos)

```

And now, finally, we arrive at the functional extensions to `Game`. The first step is to replace the single `state` attribute with `stack_state`.

Listing 3.87: TicTacToe – `stack_state`

```

47     pygame.event.post(pygame.event.Event(Game.EVENTS["START"]))

```

To simplify working with the stack, we introduce a few helper methods:

- `push_state()` appends a new state to the top of the stack (at the end of the Python list).
- `pop_state()` removes the state currently on top of the stack. Before doing so, it checks whether the stack is already empty. If no states remain, this can reasonably be interpreted as the end of the game, since there is no longer a playable state available.
- `replace_state()` discards the entire existing stack and initializes a new one containing only the state passed as an argument.
- `current_state` is simply a convenience attribute that returns the state currently on top of the stack.

Listing 3.88: TicTacToe – `push_state()`, `pop_state()`, `replace_state()`, `current_state`

```

111     def push_state(self, state: State) -> None:
112         self.stack_state.append(state)
113
114     def pop_state(self) -> None:
115         if self.stack_state:
116             self.stack_state.pop()
117         if not self.stack_state:
118             self.running = False
119
120     def replace_state(self, state: State) -> None:
121         self.stack_state = [state]
122
123     @property
124     def current_state(self) -> State | None:
125         return self.stack_state[-1] if self.stack_state else None

```

With these helper methods in place, `watch_for_events()` can now be rewritten. Whenever a new state is intended to completely replace the current one, `replace_state()` is used. This applies to the transitions triggered by `START` and `PLAY`.

In contrast, the states associated with `WIN`, `DRAW`, and `HELP` are added to the stack using `push_state()`. This makes it possible to access both `PlayState` and, for example, `HelpState`, rather than being limited to a single active state.

When the event `UNHELP` occurs, only the help state should disappear again. Therefore, `pop_state()` is used.

Finally, the current events are forwarded only to the most recently added `State` object – that is, the state currently on top of the stack – and not to all states. This is the main difference to a [broadcast system](#).

Listing 3.89: TicTacToe – `watch_for_events()`

```

57 def watch_for_events(self) -> None:
58     events = pygame.event.get()
59     for event in events:
60         if event.type == pygame.QUIT:
61             self.running = False
62         elif event.type == pygame.KEYDOWN:
63             if event.key in (pygame.K_ESCAPE, pygame.K_q):
64                 self.running = False
65         elif event.type == Game.EVENTS["START"]:
66             self.reset()
67             self.replace_state(StartState(self))
68         elif event.type == Game.EVENTS["PLAY"]:
69             self.replace_state(PlayState(self))
70         elif event.type == Game.EVENTS["WIN"]:
71             self.push_state(WinState(self))
72         elif event.type == Game.EVENTS["DRAW"]:
73             self.push_state(DrawState(self))
74         elif event.type == Game.EVENTS["HELP"]:
75             self.push_state(HelpState(self))
76         elif event.type == Game.EVENTS["UNHELP"]:
77             self.pop_state()
78     if self.current_state:
79         self.current_state.watch_for_events(events)

```

For `update()`, a design decision has to be made: should `update()` be executed for every state on the stack, or only for the state currently on top of it?

In the first case, moving game elements would continue to update even while an overlay state is active. In the second case, they would effectively freeze. Since we want the background to remain static, we forward the `update()` call only to the current state on top of the stack.

In contrast, `draw()` behaves quite differently. Here, all states stored in the stack – in our example, there are at most two – should render their graphics. This allows the game state in the background to remain visible through the overlay. As a result, the player can still inspect, for example, the winning position that led to the end of the game.

Rendering policies vary depending on the desired behavior. In our example, the underlying `PlayState` is drawn first, followed by the overlay state. Therefore, `draw()` traverses the stack from bottom to top, while event processing only targets the state on top.

Listing 3.90: TicTacToe – draw()

```

85     def draw(self) -> None:
86         self.screen.fill(cfg.COLORS["BG"])
87         for state in self.stack_state:
88             state.draw()
89         self.window.flip()

```

3.4.4 I'm confused!

Over the course of this chapter, we explored several ways of organizing game states, ranging from a simple status variable to stacked states. Each approach has its strengths and weaknesses, and the *right* choice depends on the requirements of the project. The following table provides a small rule of thumb for deciding which technique may be most appropriate in a given situation.

Table 3.4: Decision table for designing scene handling

If ...	Use ...
there are only a few scenes	a status variable
many scenes replace each other	a state machine
states should not know each other directly	events
scenes temporarily overlay other scenes	a state stack

4 Examples

4.1 Pong

The ultimate beginner classic. This game has been played in countless variations since 1972. Because the rules are simple and easy to understand, it is perfectly suited as a first programming project.

We will develop this game step by step in a systematic way, assuming that the techniques from chapter 2 are already familiar. I will deliberately omit docstring comments in the source code, since everything is explained in the text and including them here would only make the listings unnecessarily long. They are, of course, included in the final version.

4.1.1 Requirement 1: Standards

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v01

Requirement 1 Standard functionality

1. *The window has an appropriate size.*
 2. *The background is a dark red playing field with a dashed center line.*
 3. *The game can be exited using the `Esc` key or by clicking the red “X”.*
 4. *The game runs at a speed independent of the FPS.*
-

And off we go. Here, the `config.py`. I assume that you have sufficient Python knowledge to extend it as needed.

Listing 4.1: Pong (Requirement 1) – `config.py`

```

1 from pygame import Rect
2
3 WINDOW = Rect(0, 0, 1000, 600)
4 FPS = 60
5 DELTATIME = 1.0 / FPS

```

The background is not loaded from a bitmap this time, but created on the fly. There is no deep reason for this – apart from showing that bitmaps

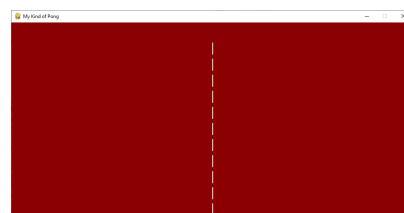


Fig. 4.1: Pong: the background

do not always have to come from image files (see section 2.3.2.3 on page 54). Instead, they can be generated dynamically as well.

To do this, a **Surface** object with the size of the screen is created first. It is then filled with a dark red color, meant to resemble a clay court.

In `paint_net()`, starting at line 21, the net is drawn as a sequence of small white rectangles.

Listing 4.2: Pong (Requirement 1) – the class **Background**

```

8 class Background(pygame.sprite.Sprite):
9     def __init__(self, *groups: Tuple[pygame.sprite.Group]) -> None:
10         super().__init__(*groups)
11         self.image = pygame.Surface(cfg.WINDOW.size).convert()
12         self.rect = self.image.get_rect()
13         self.image.fill("darkred")
14         self.paint_net()
15
16     def paint_net(self) -> None:
17         net_rect = pygame.Rect(0, 0, 0, 0)
18         net_rect.centerx = cfg.WINDOW.centerx
19         net_rect.top = 50
20         net_rect.size = (3, 30)
21         while net_rect.bottom < cfg.WINDOW.bottom: # Net as a seq of rectangles
22             pygame.draw.rect(self.image, "grey", net_rect, 0)
23             net_rect.move_ip(0, 40)

```

The class **Game** consists of the basic building blocks that we have already seen in chapter 2. In `__init__()`, Pygame is initialized, the window and the clock are created, and the control flag for the main game loop is set up. The background is stored in a **GroupSingle** object. The remaining methods should be fairly self-explanatory.

Listing 4.3: Pong (Requirement 1) – the class **Game**

```

26 class Game:
27     def __init__(self):
28         pygame.init()
29         self.window = pygame.Window(size=cfg.WINDOW.size, title="My Kind of Pong",
30                                     position=pygame.WINDOWPOS_CENTERED)
31         self.screen = self.window.get_surface()
32         self.clock = pygame.time.Clock()
33         self.background = pygame.sprite.GroupSingle(Background())
34         self.running = True
35
36     def run(self):
37         time_previous = time()
38         while self.running:
39             self.watch_for_events()
40             self.update()
41             self.draw()
42             self.clock.tick(cfg.FPS)
43             time_current = time()
44             cfg.DELTATIME = time_current - time_previous
45             time_previous = time_current
46         pygame.quit()

```

```

47     def update(self):
48         pass
49
50     def draw(self):
51         self.background.draw(self.screen)
52         self.window.flip()
53
54     def watch_for_events(self):
55         for event in pygame.event.get():
56             if event.type == pygame.QUIT:
57                 self.running = False
58             elif event.type == pygame.KEYDOWN:
59                 if event.key == pygame.K_ESCAPE:
60                     self.running = False

```

At this point, the application is not functional yet, but it already displays the background as you can see in figure 4.1 on page 253.

4.1.2 Requirement 2: The Paddles



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v02

Requirement 2 Paddles

1. *There is one rectangular paddle on the left side and one on the right side.*
 2. *The paddles have a width of 15 px and a height of one tenth of the screen height.*
 3. *The paddles have a speed of $\frac{\text{screen height}}{2}$ px/s.*
 4. *Each paddle is positioned at a distance of 50 px from the left or right edge, measured from its center.*
 5. *The left paddle is moved upward using and downward using .*
 6. *The right paddle is moved upward using and downward using .*
 7. *The paddles cannot leave the playing field.*
-

In line 32, the size of the paddle is calculated (requirements 2.1 and 2.2). Starting at line 33, the paddles are positioned. Vertically, they always start in the center of the screen. Horizontally, their start position depends on whether we are dealing with the left or the right paddle. In both cases, they are placed slightly away from the edge, exactly as specified in requirement 2.4.

The paddle speed is set in line 39 according to requirement 2.3. Just like the background, this bitmap is not loaded from a file but created directly in the code (line 41) and filled with a bright yellow color.

Listing 4.4: Pong (Requirement 2) – The constructor of Paddle

```

26 class Paddle(pygame.sprite.Sprite):
27     BORDERDISTANCE = {"horizontal": 50, "vertical": 10}
28     DIRECTION = {"up": -1, "down": 1, "halt": 0}
29
30     def __init__(self, player: str, *groups: Tuple[pygame.sprite.Group]) -> None:
31         super().__init__(*groups)
32         self.rect = pygame.Rect(0, 0, 15, cfg.WINDOW.height // 10) # Size
33         self.rect.centery = cfg.WINDOW.centery # Position
34         self.player = player
35         if self.player == "left":
36             self.rect.left = Paddle.BORDERDISTANCE["horizontal"]
37         else:
38             self.rect.right = cfg.WINDOW.right - Paddle.BORDERDISTANCE["horizontal"]
39         self.speed = cfg.WINDOW.height // 2 # Speed
40         self.direction = Paddle.DIRECTION["halt"] #
41         self.image = pygame.Surface(self.rect.size) # Surface
42         self.image.fill("yellow")

```

The method `update()` is responsible for distributing the tasks. With regard to movement, the attribute `self.direction` is adjusted accordingly (starting at line 48). If the paddle is supposed to change its position, the method `move()` is called in line 46.

Listing 4.5: Pong (Requirement 2) – `Paddle.update()`

```

44 def update(self, *args: Any, **kwargs: Any) -> None:
45     if "action" in kwargs.keys():
46         if kwargs["action"] == "move": # Change Position
47             self.move()
48         elif kwargs["action"] in Paddle.DIRECTION.keys(): # Direction
49             self.direction = Paddle.DIRECTION[kwargs["action"]]
50     return super().update(*args, **kwargs)

```

All that remains is the method `move()`. It looks more complicated than it actually is. After checking whether there is anything to do at all, the new vertical position is calculated in line 54 (the horizontal position remains unchanged). After that, it is checked whether the paddle has left the playing field. If so, the paddle is moved back to the top or bottom edge accordingly.

Listing 4.6: Pong (Requirement 2) – `Paddle.move()`

```

52 def move(self) -> None:
53     if self.direction != Paddle.DIRECTION["halt"]:
54         self.rect.move_ip(0, self.speed * self.direction * cfg.DELTATIME) #
55         if self.direction == Paddle.DIRECTION["up"]:
56             self.rect.top = max(self.rect.top, Paddle.BORDERDISTANCE["vertical"])
57         elif self.direction == Paddle.DIRECTION["down"]:
58             self.rect.bottom = min(self.rect.bottom, cfg.WINDOW.height -
                                   Paddle.BORDERDISTANCE["vertical"])

```

Now the paddles need to be integrated into the `Game` class. In line 69, a sprite group is created first, which will hold all sprites except the background. After that, the two paddles are created and immediately added to the sprite group via constructor arguments.

Listing 4.7: Pong (Requirement 2) – Constructor of `Game`

```

61 class Game:
62     def __init__(self):
63         pygame.init()
64         self.window = pygame.Window(size=cfg.WINDOW.size, title="My Kind of Pong",
65                                     position=pygame.WINDOWPOS_CENTERED)
66         self.screen = self.window.get_surface()
67         self.clock = pygame.time.Clock()
68         self.background = pygame.sprite.GroupSingle(Background())
69         self.all_sprites = pygame.sprite.Group()
70         self.paddle = {} # Schläger
71         self.paddle["left"] = Paddle("left", self.all_sprites)
72         self.paddle["right"] = Paddle("right", self.all_sprites)
73         self.running = True

```

In `update()` and `draw()`, the only thing that happens is the corresponding method call on the sprite group and now the paddles finally show up on screen.

Listing 4.8: Pong (Requirement 2) – `Game.update()` and `Game.draw()`

```

86 def update(self):
87     self.all_sprites.update(action="move") # Move
88
89 def draw(self):
90     self.background.draw(self.screen)
91     self.all_sprites.draw(self.screen) #
92     self.window.flip()

```

And now the keyboard events are handled. Pressing a key triggers a movement (starting at line 101), while releasing the key causes the corresponding paddle to stop (starting at line 109).

In each case, the method `Paddle.update()` is called with an appropriate parameter: for movement with `action="up"` or `action="down"`, and for stopping with `action="halt"`.

Listing 4.9: Pong (Requirement 2) – `Game.watch_for_events()`

```

94 def watch_for_events(self):
95     for event in pygame.event.get():
96         if event.type == pygame.QUIT:
97             self.running = False
98         elif event.type == pygame.KEYDOWN:
99             if event.key == pygame.K_ESCAPE:
100                 self.running = False
101             elif event.key == pygame.K_UP: # Paddle moves
102                 self.paddle["right"].update(action="up")
103             elif event.key == pygame.K_DOWN:
104                 self.paddle["right"].update(action="down")
105             elif event.key == pygame.K_w:
106                 self.paddle["left"].update(action="up")
107             elif event.key == pygame.K_s:
108                 self.paddle["left"].update(action="down")

```

```

109         elif event.type == pygame.KEYUP:             # Paddle stops
110             if event.key == pygame.K_UP or event.key == pygame.K_DOWN:
111                 self.paddle["right"].update(action="halt")
112             elif event.key == pygame.K_w or event.key == pygame.K_s:
113                 self.paddle["left"].update(action="halt")

```

4.1.3 Requirement 3: The Ball

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v03

Requirement 3 Ball

1. The ball is a circle with a radius of 10 px.
 2. Its speed is $\frac{\text{screen width}}{3}$ px/s.
 3. It starts in the center of the screen with a random horizontal and vertical direction.
 4. It bounces off the top and bottom edges of the screen.
 5. When it touches the left edge, it is reset to the center. The same happens when it touches the right edge.
 6. If the right edge is hit, player 1 scores a point; if the left edge is hit, player 2 scores a point.
-

Since we need to keep track of the players' scores according to requirement 3.6, a corresponding array is added to `config.py` (line 6).

Listing 4.10: Pong (Requirement 3) – `config.py`

```

1  from pygame import Rect
2
3  WINDOW = Rect(0, 0, 1000, 600)
4  FPS = 60
5  DELTATIME = 1.0 / FPS
6  POINTS = [0, 0]                                     # Score

```

In accordance with requirements 3.1 and 3.2, the size and speed of the ball are defined in line 65 and line 69. Since the ball needs to be restarted frequently, the initialization of its starting position and direction is moved into the separate method `service()` (line 71).

Listing 4.11: Pong (Requirement 3) – Constructor of Ball

```

62 class Ball(pygame.sprite.Sprite):
63     def __init__(self, *groups: Tuple[pygame.sprite.Group]) -> None:
64         super().__init__(*groups)
65         self.rect = pygame.Rect(0, 0, 20, 20)           # Size
66         self.image = pygame.Surface(self.rect.size).convert()
67         self.image.set_colorkey("black")
68         pygame.draw.circle(self.image, "green", self.rect.center, self.rect.width // 2)
69         self.speed = cfg.WINDOW.width // 3             # Speed
70         self.speedxy = pygame.Vector2()
71         self.service()                                 #

```

In `update()`, the responsibilities are distributed.

Listing 4.12: Pong (Requirement 3) – `Ball.update()`

```

73 def update(self, *args: Any, **kwargs: Any) -> None:
74     if "action" in kwargs.keys():
75         if kwargs["action"] == "move":
76             self.move()
77         elif kwargs["action"] == "service":
78             self.service()
79     return super().update(*args, **kwargs)

```

Let us now take a closer look at the helper methods, one by one. We start with `move()`. As expected, the position is updated using the velocity values. After that, starting at line 83, it is checked whether the ball has reached any of the four edges of the screen.

If the top or bottom edge is hit (requirement 3.4), the sign of the vertical velocity is inverted by calling `vertical_flip()` (source code 4.15 on the next page). After the flip, the ball is clamped to the top or bottom edge, since it may already have crossed the boundary.

Things are different when the ball reaches the left or right edge. In that case, the ball is served again according to requirement 3.5 (see source code 4.14 on the following page), and – as specified in requirement 3.6 – the appropriate player’s score is increased.

Listing 4.13: Pong (Requirement 3) – `Ball.move()`

```

81 def move(self) -> None:
82     self.rect.move_ip(self.speedxy * cfg.DELTATIME)
83     if self.rect.top <= 0:                                     #
84         self.vertical_flip()
85         self.rect.top = 0
86     elif self.rect.bottom >= cfg.WINDOW.bottom:
87         self.vertical_flip()
88         self.rect.bottom = cfg.WINDOW.bottom
89     elif self.rect.right < 0:
90         cfg.POINTS[1] += 1
91         self.service()
92     elif self.rect.left > cfg.WINDOW.right:
93         cfg.POINTS[0] += 1
94         self.service()

```

When serving, the center of the ball is set to the center of the screen (requirement 3.3). After that, the signs of the two velocity components are chosen randomly, which deter-

mines the direction of movement (left or right, and up or down). Since we do not have a score display yet, a temporary console output is implemented in line 99.

Listing 4.14: Pong (Requirement 3) – `Ball.service()`

```

96  def service(self) -> None:
97      self.rect.center = cfg.WINDOW.center
98      self.speedxy = pygame.Vector2(choice([-1, 1]), choice([-1, 1])) * self.speed
99      print(cfg.POINTS)                                # Ugly

```

The direction change is simply a sign flip. The method `flip_horizontal()` is not used yet, but we will need it later when we want the ball to bounce off the paddle.

Listing 4.15: Pong (Requirement 3) – The flip methods of `Ball`

```

101 def horizontal_flip(self) -> None:
102     self.speedxy.x *= -1
103
104 def vertical_flip(self) -> None:
105     self.speedxy.y *= -1

```

Typical reflection pitfalls when handling the ball

⚠ Sticky edge / multiple flips per frame

If the ball is still inside the wall after a flip (because it has already crossed the boundary), the velocity is inverted again in the next frame. The result is a ball that appears to jitter or stick to the edge.

Fix: After flipping the velocity, clamp the position explicitly (e.g. `rect.top = 1` or `rect.bottom = WINDOW.HEIGHT - 1`).

⚠ Checking the wrong reference (center vs. rect)

A common mistake is to compute movement using the ball center or a position vector, but perform collision checks against `rect.left/right/top/bottom` (or vice versa). This usually leads to off-by-radius errors.

Fix: Be consistent: either check collisions exclusively using `rect.*`, or use `center` together with $\pm$ radius – but do not mix both approaches.

⚠ Flipping the wrong axis

A classic error: when hitting the top or bottom wall, `speed.x` is inverted instead of `speed.y` (or the other way around).

Fix:

- Top / bottom collision: `speed.y *= -1`
- Left / right collision: `x *= -1` (or trigger a service/reset)

⚠ Tunneling at high speed

With large `DELTATIME` values or high velocities, the ball may jump over a wall between two frames and never register a collision.

Fix (simple): After moving the ball, check whether it has crossed a boundary and clamp it back.

Fix (robust): Split the movement into smaller steps (sub-stepping) or use swept collision detection.

⚠ Incorrect handling of multiple collisions in one frame

If the ball hits, for example, a corner (top wall and paddle edge at the same time), naively flipping both axes can cancel out the reflection entirely.

Fix: Prioritize collisions (e.g. wall before paddle), or decide based on the smaller penetration depth.

⚠ Not pushing the ball out of the paddle after a hit

If the ball remains inside the paddle rectangle after a bounce, it will flip direction again in the next frame and appear to vibrate.

Fix: After a paddle collision, move the ball explicitly in front of the paddle edge (clamp), and only then invert `vx`.

4.1.4 Requirement 4: Scoring**📁 So ... where are the files?**

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v04

Requirement 4 Scoring

1. *The score is displayed centered at the top of the screen.*

For displaying the score, the class `Score` is used. In the end, it is just another sprite—but one that needs to be recreated from time to time, namely whenever the score changes. Since the current score is now stored in line 118, it can be removed from `config.py`.

Listing 4.16: Pong (Requirement 4) – Constructor of Score

```

114
115     def __init__(self, *groups: Tuple[pygame.sprite.Group]):
116         super().__init__(*groups)
117         self.font = pygame.font.SysFont(None, 30)
118         self.score = {1: 0, 2: 0} # Not in Settings anymore!
119         self.image: pygame.Surface = None
120         self.rect: pygame.Rect = None
121         self.render()

```

In this method, the current score is rendered using a font object and then positioned accordingly.

Listing 4.17: Pong (Requirement 4) – Score.render()

```

130         self.image = self.font.render(f"{self.score[1]} : {self.score[2]}", True, "white")
131         self.rect = self.image.get_rect(centerx=cfg.WINDOW.centerx, top=15)

```

In `update()`, the appropriate score value is updated and `render()` is called.

Listing 4.18: Pong (Requirement 4) – Score.update()

```

124         if "player" in kwargs.keys():
125             self.score[kwargs["player"]] += 1
126             self.render()
127         return super().update(*args, **kwargs)

```

What is still missing is a trigger for updating the score display. This is a perfect opportunity to introduce a user-defined event. Starting at line 7, everything required for such a user event is implemented. First, an event ID is defined, followed by the corresponding `pygame.event.Event` object.

Listing 4.19: Pong (Requirement 4) – MyEvent

```

1  import pygame
2
3  WINDOW = pygame.Rect(0, 0, 1000, 600)
4  FPS = 60
5  DELTATIME = 1.0 / FPS
6
7  class MyEvents: # User events
8      POINT_FOR = pygame.event.custom_type()
9      MYEVENT = pygame.event.Event(POINT_FOR, player=0)

```

Now the `Ball` class only has to trigger the appropriate event, and `Game` needs to handle it. Here are the required changes in `Ball`. Inside the method `move()`, the relevant code sections are replaced. For example, in line 94 the number of the player who scores the point is packed into the event, and in line 95 the event is dispatched.

Listing 4.20: Pong (Requirement 4) – `Ball.move()`

```

86     self.rect.move_ip(self.speedxy * cfg.DELTATIME)
87     if self.rect.top <= 0:
88         self.vertical_flip()
89         self.rect.top = 0
90     elif self.rect.bottom >= cfg.WINDOW.bottom:
91         self.vertical_flip()
92         self.rect.bottom = cfg.WINDOW.bottom
93     elif self.rect.right < 0:
94         MyEvents.MYEVENT.player = 2           # Player
95         pygame.event.post(MyEvents.MYEVENT)   # Shoot event
96         self.service()
97     elif self.rect.left > cfg.WINDOW.right:
98         MyEvents.MYEVENT.player = 1
99         pygame.event.post(MyEvents.MYEVENT)
100        self.service()

```

Now all that remains is to catch the user-defined event inside `watch_for_events()` (starting at line 189).

Listing 4.21: Pong (Requirement 4) – `Ball.watch_for_events()`

```

189     elif event.type == MyEvents.POINT_FOR: # User event
190         self.score.update(player=event.player)

```

Why is a user-defined event more elegant than direct access?

- 👉 **No direct access from the ball to the score:** If the `Ball` were to call `Score.render()` or `Score.update()` directly, it would need to know about the `Score` object – or even the `Game` class. This creates unnecessary dependencies and tightly couples classes that should remain independent.
- 👉 **Clear separation of responsibilities:** The `Ball` only knows one thing: *A point was scored by player X*. The `Game`, on the other hand, decides what that means in practice. Update the score data, re-render the score display, maybe play a sound, reset the ball, or start the next serve. Each class focuses on its own responsibility (SRP).
- 👉 **A clean extension point:** Later on, additional reactions can easily be attached to the same event – such as sound effects, particle effects, a short pause, a change in serve direction, or logging – without touching the `Ball` code again.
- 👉 **Better testability and maintainability:** All scoring-related behavior is handled centrally in `Game.watch_for_events()`, instead of being scattered across multiple classes. This makes the code easier to understand, test, and maintain.

The ball reports what happened and the game decides what to do about it.

4.1.5 Requirement 5: Paddle hit

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v05

At first glance, the game already looks finished – but it is still not really playable, because the paddles are not doing anything yet.

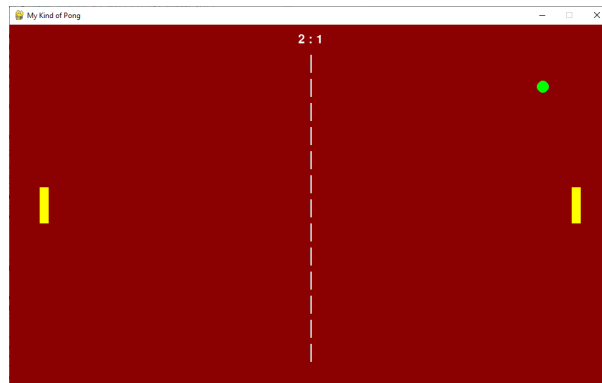


Figure 4.2: Pong: paddles, ball, and score

Requirement 5 Paddle hit

1. *When the ball touches a paddle, it bounces off and is returned to the opponent's side of the field.*
 2. *Each time the ball hits a paddle, its directional velocities are increased by a small random amount.*
-

To achieve this, we add the method `check_collision()` to the `Game` class. This method checks whether the ball has hit one of the paddles. A good choice here is the method `pygame.sprite.collide_rect()`.

If a collision is detected, the previously unused method `horizontal_flip()` (see source code 4.15 on page 260) is triggered via `update()`. Afterwards, the positions are adjusted so that the ball and the paddle no longer overlap. In addition, the method `respeed()` is called via `update()` to fulfill requirement 5.2.

Listing 4.22: Pong (Requirement 5) – `Game.check_collision()`

```

199 def check_collision(self):
200     if pygame.sprite.collide_rect(self.ball, self.paddle["left"]):
201         self.ball.update(action="hflip")

```

```

202         self.ball.rect.left = self.paddle["left"].rect.right + 1
203     elif pygame.sprite.collide_rect(self.ball, self.paddle["right"]):
204         self.ball.update(action="hflip")
205         self.ball.rect.right = self.paddle["right"].rect.left - 1

```

In `respeed()`, small random values are added to the velocity components. Via the attribute `speed`, this variation is indirectly tied to the screen size.

Listing 4.23: Pong (Requirement 5) – `Ball.respeed()`

```

114 def respeed(self) -> None:
115     self.speedx.x += randrange(0, self.speed // 4)
116     self.speedy.y += randrange(0, self.speed // 4)

```

Now the game finally becomes playable.

4.1.6 Requirement 6: Computer-controlled player



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v06

Strictly speaking, we would be finished at this point – but I would like to add a computer-controlled player. This allows the game to be played against the computer, or simply to let the computer play against itself for hours.

Requirement 6 Computer player

1. Pressing ☐ 1 toggles control of the left paddle between human and computer.
2. Pressing ☐ 2 toggles control of the right paddle between human and computer.
3. When control is switched back to manual, the paddle should initially remain stationary.

In `config.py`, a dictionary of flags is defined in line 6. These flags control, for each player, whether the paddle is controlled manually or by the computer.

Listing 4.24: Pong (Requirement 6) – `config.py`

```

1 import pygame
2
3 WINDOW = pygame.Rect(0, 0, 1000, 600)
4 FPS = 60

```

```

5 DELTATIME = 1.0 / FPS
6 KI = {"left": False, "right": False}           # Flag computer player
7
8 class MyEvents:
9     POINT_FOR = pygame.event.custom_type()
10    MYEVENT = pygame.event.Event(POINT_FOR, player=0)

```

In the `update()` method, starting at line 187, the flags are checked to determine whether a paddle is controlled by the computer. If so, a corresponding controller method is called.

Listing 4.25: Pong (Requirement 6) – `Game.update()`

```

185    def update(self):
186        self.check_collision()
187        for i in cfg.KI.keys():                 # Computer commands
188            if cfg.KI[i]:
189                self.paddlecontroller(self.paddle[i])
190        self.all_sprites.update(action="move")

```

Let us now take a look at the controller method. The basic idea is simple: the paddle moves upward as long as the center of the ball is above the center of the paddle, and it moves downward as long as the ball's center is below the paddle's center.

There is no need to move all the way to the very top or bottom. The last few pixels can be ignored, since a collision will usually be triggered before that anyway.

Why does this simple computer player work so well?

At first glance, this controller logic looks almost trivial: the paddle simply follows the vertical position of the ball. Surprisingly, this already produces a reasonably strong computer opponent.

The reason is that Pong is a very simple game in terms of physics and decision-making. The ball moves along a straight line between collisions, and its vertical position is the most important piece of information needed to intercept it. By continuously aligning the paddle's center with the ball's center, the computer ensures that the paddle is usually in the right place at the right time.

Another advantage of this approach is that it is stable and predictable. The paddle does not overreact, oscillate wildly, or make unnecessary movements. Since the paddle speed is limited, it also cannot instantly teleport to the ball's position, which keeps the game fair.

Finally, stopping the paddle slightly before reaching the exact ball position is intentional. This avoids jitter and unnecessary micro-movements, and in practice a collision will occur anyway once the ball reaches the paddle.

follow
the
ball

In short: For simple games like Pong, a straightforward *follow the ball* strategy is often more than sufficient – and a great example of how simple rules can lead to convincing behavior.

Listing 4.26: Pong (Requirement 6) – Game.paddlecontroller()

```

242 def paddlecontroller(self, paddle: pygame.sprite.Sprite) -> None:
243     if paddle.rect.centery > self.ball.rect.centery and paddle.rect.top > 10:
244         paddle.update(action="up")
245     elif paddle.rect.centery < self.ball.rect.centery and paddle.rect.bottom <
246         cfg.WINDOW.bottom - 10:
247         paddle.update(action="down")
248     else:
249         paddle.update(action="halt")

```

In `watch_for_events()`, more extensive changes are required. First, manual control for a paddle must be disabled whenever that paddle is set to computer control. So, before calling the corresponding `update()` method, we first check whether the computer player currently has control. An example can be found in line 205.

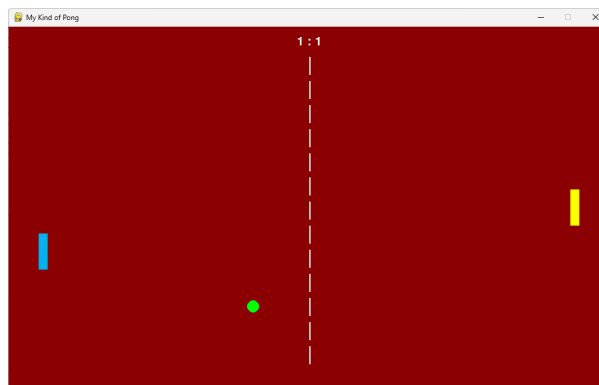


Figure 4.3: Pong: paddle color indicates AI mode (left AI, right manual)

One remaining detail is requirement 6.3. For this, the corresponding flag is checked as shown in line 218, and the paddle is sent a halt signal.

Listing 4.27: Pong (Requirement 6) – Game.watch_for_events()

```

197 def watch_for_events(self):
198     for event in pygame.event.get():
199         if event.type == pygame.QUIT:
200             self.running = False
201         elif event.type == pygame.KEYDOWN:
202             if event.key == pygame.K_ESCAPE:
203                 self.running = False
204             elif event.key == pygame.K_UP:
205                 if not cfg.KI["right"]: #
206                     self.paddle["right"].update(action="up")
207             elif event.key == pygame.K_DOWN:
208                 if not cfg.KI["right"]:
209                     self.paddle["right"].update(action="down")
210             elif event.key == pygame.K_w:
211                 if not cfg.KI["left"]:
212                     self.paddle["left"].update(action="up")
213             elif event.key == pygame.K_s:
214                 if not cfg.KI["left"]:
215                     self.paddle["left"].update(action="down")
216             elif event.key == pygame.K_1:

```

```

217         cfg.KI["left"] = not cfg.KI["left"]
218         if not cfg.KI["left"]: # Stop!
219             self.paddle["left"].update(action="halt")
220     elif event.key == pygame.K_2:
221         cfg.KI["right"] = not cfg.KI["right"]
222         if not cfg.KI["right"]:
223             self.paddle["right"].update(action="halt")
224     elif event.type == pygame.KEYUP:
225         if event.key == pygame.K_UP or event.key == pygame.K_DOWN:
226             if not cfg.KI["right"]:
227                 self.paddle["right"].update(action="halt")
228         elif event.key == pygame.K_w or event.key == pygame.K_s:
229             if not cfg.KI["left"]:
230                 self.paddle["left"].update(action="halt")
231     elif event.type == MyEvents.POINT_FOR:
232         self.score.update(player=event.player)

```

4.1.7 Requirement 7: Sound



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v07

A bit of sound would make the game feel much more lively.

Requirement 7 Sound

1. *Hitting the ball with a paddle should be accompanied by an appropriate sound effect.*
2. *Bouncing off the top and bottom edges should also be accompanied by a suitable sound effect.*
3. *Sound should be toggleable on and off using the **F2**.*

As a first step, we extend **Settings** by adding the flag **SOUNDFLAG** in line 9. This flag controls whether sound should be played or not and provides access to the sound files.

Listing 4.28: Pong (Requirement 7) – config.py

```

1  import os
2
3  import pygame
4
5  WINDOW = pygame.Rect(0, 0, 1000, 600)
6  FPS = 60
7  DELTATIME = 1.0 / FPS
8  KI = {"left": False, "right": False}
9  SOUNDFLAG = True # Sound flag
10 PATH = {}
11 PATH["file"] = os.path.dirname(os.path.abspath(__file__))

```



```

12 PATH["sound"] = os.path.join(PATH["file"], "sounds")
13
14 def get_sound(filename: str) -> str:
15     return os.path.join(PATH["sound"], filename)
16
17 class MyEvents:
18     POINT_FOR = pygame.event.custom_type()
19     MYEVENT = pygame.event.Event(POINT_FOR, player=0)

```

The actual sound playback is implemented in the `Ball` class. In the constructor, starting at line 84, the sound effects are loaded and a channel is selected through which the sounds will be played.

Listing 4.29: Pong (Requirement 7) – Constructor of `Ball`

```

81 class Ball(pygame.sprite.Sprite):
82     def __init__(self, *groups: Tuple[pygame.sprite.Group]) -> None:
83         super().__init__(*groups)
84         self.sounds: dict[str, pygame.mixer.Sound] = {} # Sound container
85         self.sounds["left"] = pygame.mixer.Sound(cfg.get_sound("playerl.mp3"))
86         self.sounds["right"] = pygame.mixer.Sound(cfg.get_sound("playerr.mp3"))
87         self.sounds["bounce"] = pygame.mixer.Sound(cfg.get_sound("bounce.mp3"))
88         self.channel = pygame.mixer.find_channel()
89         self.rect = pygame.FRect(0, 0, 20, 20)
90         self.image = pygame.Surface(self.rect.size).convert()
91         self.image.set_colorkey("black")
92         pygame.draw.circle(self.image, "green", self.rect.center, self.rect.width // 2)
93         self.speed = cfg.WINDOW.width // 3
94         self.speedxy = pygame.Vector2()
95         self.service()

```

The first sound effect is implemented for paddle collisions in `horizontal_flip()`. After checking whether sound output is enabled at all, it is determined whether the ball is bouncing off the left or the right paddle. This is done indirectly by checking the current horizontal direction of the ball (line 132). Based on this information (see section 2.11.2.1 on page 161, the volume of the sound is adjusted so that it creates the impression that the bounce happens to the left or right of the listener.

stereo panning

Listing 4.30: Pong (Requirement 7) – `Ball.horizontal_flip()`

```

130 def horizontal_flip(self) -> None:
131     if cfg.SOUNDFLAG:
132         if self.speedxy.x < 0: # Ball to left?
133             self.channel.set_volume(0.9, 0.1)
134             self.channel.play(self.sounds["left"])
135         else:
136             self.channel.set_volume(0.1, 0.9)
137             self.channel.play(self.sounds["right"])
138     self.speedxy.x *= -1
139     self.resped()

```

This sound effect becomes a bit more dynamic in `vertical_flip()`. In line 143, the relative horizontal position of the ball is calculated. If the center of the ball is on the left side, `rel_pos` will be close to 0; if the ball is far to the right, the value will be close to 1.

These values can then be used directly as the left and right volume levels when calling `set_volume()`, creating a simple but effective stereo panning effect.

Listing 4.31: Pong (Requirement 7) – `Ball.vertical_flip()`

```

141 def vertical_flip(self) -> None:
142     if cfg.SOUNDFLAG:
143         rel_pos = self.rect.centerx / cfg.WINDOW.width # Where am I?
144         self.channel.set_volume(1.0 - rel_pos, rel_pos)
145         self.channel.play(self.sounds["bounce"])
146     self.speedxy.y *= -1

```

All that remains is toggling sound output on and off inside `watch_for_events()` in line 227 using the function key `F2`.

Listing 4.32: Pong (Requirement 7) – `Ball.watch_for_events()`

```

213 def watch_for_events(self):
214     for event in pygame.event.get():
215         if event.type == pygame.QUIT:
216             self.running = False
217         elif event.type == pygame.KEYDOWN:
218             if event.key == pygame.K_ESCAPE:
219                 self.running = False
220             elif event.key == pygame.K_UP:
221                 if not cfg.KI["right"]:
222                     self.paddle["right"].update(action="up")
223             elif event.key == pygame.K_DOWN:
224                 if not cfg.KI["right"]:
225                     self.paddle["right"].update(action="down")
226             elif event.key == pygame.K_F2:
227                 cfg.SOUNDFLAG = not cfg.SOUNDFLAG # Toggle Soundflag

```

4.1.8 Requirement 8: Pause and Help Screen

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/01%20Pong/v08

Requirement 8 Pause and help

1. Pressing `p` pauses all activity and stops the game. Pressing `p` again resumes the game.
2. Pressing `h` pauses the game and displays a help text. Pressing `h` again resumes the game.

For the pause functionality, we create a separate class—perhaps a bit overengineered, but nicely self-contained. The essential part can be found in line 33. There, a semi-transparent gray overlay is created using a **Surface** object with the same size as the screen. The surface is filled with a gray color whose alpha channel is set to 200, allowing the background to shine through.

Listing 4.33: Pong (Requirement 8) – Pause

```

28 class Pause(pygame.sprite.Sprite):
29     def __init__(self, *groups: Tuple[pygame.sprite.Group]) -> None:
30         super().__init__(*groups)
31         self.rect = pygame.Rect(cfg.WINDOW.topleft, cfg.WINDOW.size)
32         self.image = pygame.Surface(self.rect.size).convert_alpha()
33         self.image.fill([120, 120, 120, 200])          # Transparent Grey

```

The help screen is implemented in an analogous way. The only difference is that an additional text is blitted onto the surface. The text is split into a left and a right column to improve readability.

Listing 4.34: Pong (Requirement 8) – Help

```

36 class Help(pygame.sprite.Sprite):
37     def __init__(self, *groups: Tuple[pygame.sprite.Group]) -> None:
38         super().__init__(*groups)
39         self.rect = pygame.Rect(cfg.WINDOW.topleft, cfg.WINDOW.size)
40         self.image = pygame.Surface(self.rect.size).convert_alpha()
41         self.image.fill([20, 20, 20, 200])          # Transparent Grey
42         font = pygame.font.Font(pygame.font.get_default_font(), 20)
43         text_l = "h\np\nESC\nnF2\nk\nl\nr\nn\nUP\nDOWN\nw\ns"
44         text_r = "- toggle help modus\n- toggle pause modus\n- quit\nn- toggle sound
                    modus\n"
45         text_r += "- toggle both paddles KI modus\n- toggle left paddle KI modus\n- toggle
                    right paddle KI modus\nn"
46         text_r += "- left paddle move up\n- left paddle move down\n- right paddle move up\n-
                    right paddle move down"
47         lines = font.render(text_l, True, "white")
48         self.image.blit(lines, (10, 10))
49         lines = font.render(text_r, True, "white")
50         self.image.blit(lines, (10 + 70, 10))

```

In the constructor of **Game**, two flags now need to be created to represent the respective modes (line 213 and line 214). After that, the two overlay objects are created and assigned to a **pygame.Group.Single** object.

Listing 4.35: Pong (Requirement 8) – Constructor of Game

```

199 class Game:
200     def __init__(self):
201         pygame.init()
202         self.window = pygame.Window(size=cfg.WINDOW.size, title="My Kind of Pong",
                                     position=pygame.WINDOWPOS_CENTERED)
203         self.screen = self.window.get_surface()
204         self.clock = pygame.time.Clock()
205         self.background = pygame.sprite.GroupSingle(Background())
206         self.all_sprites = pygame.sprite.Group()
207         self.paddle = {}

```

```

208     self.paddle["left"] = Paddle("left", self.all_sprites)
209     self.paddle["right"] = Paddle("right", self.all_sprites)
210     self.ball = Ball(self.all_sprites)
211     self.score = Score(self.all_sprites)
212     self.running = True
213     self.pausing = False                                # Pause Flag
214     self.helping = False                                # Help Flag
215     self.pause = pygame.sprite.GroupSingle(Pause())
216     self.help = pygame.sprite.GroupSingle(Help())

```

Once everything is prepared, the `update()` method is modified so that the actual game logic is only executed when neither pause mode nor help mode is active (line 231). If one of these modes is enabled, the game state is effectively frozen: positions, movements, and collisions are no longer updated, while the current screen remains visible.

Listing 4.36: Pong (Requirement 8) – `Game.update()`

```

230     def update(self):
231         if not (self.pausing or self.helping):           #
232             self.check_collision()
233             for i in cfg.KI.keys():
234                 if cfg.KI[i]:
235                     self.paddlecontroller(self.paddle[i])
236             self.all_sprites.update(action="move")

```

In `draw()`, the currently active mode is checked as well. If the game is paused or the help screen is active, the corresponding sprite is rendered on top of the game scene. Otherwise, only the current game state is rendered as usual.

Listing 4.37: Pong (Requirement 8) – `Game.draw()`

```

238     def draw(self):
239         self.background.draw(self.screen)
240         self.all_sprites.draw(self.screen)
241         if self.pausing:                                #
242             self.pause.draw(self.screen)
243         elif self.helping:                                #
244             self.help.draw(self.screen)
245         self.window.flip()

```

Pause vs. Help – what happens technically?

Both the pause mode and the help screen are based on the same fundamental idea: the game is still rendered visually, but the actual game simulation is stopped.

- **Pause:** All movement and state-changing calculations are suspended. The current game situation is frozen and merely covered by a semi-transparent overlay.
- **Help:** Technically identical to the pause mode, but extended by an additional text overlay. The player receives information about controls and gameplay while the game state itself remains unchanged.

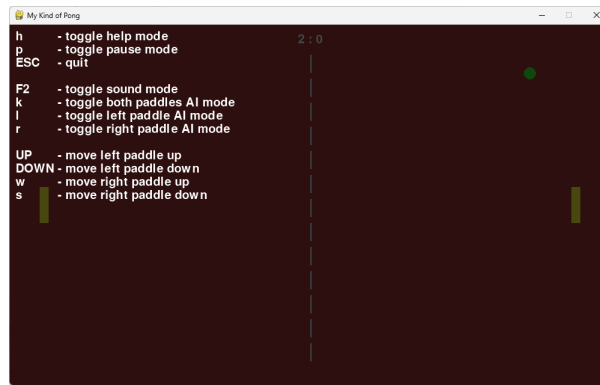


Figure 4.4: Pong: Help screen

The crucial point is that in both modes the method `update()` does not modify any game objects. Rendering continues, which keeps the game visually *alive* while it is logically paused.

This approach has several advantages

- 👉 No special-case logic inside individual sprite classes
- 👉 A clear separation between *game state* and *presentation*
- 👉 Easy to extend (e. g. for menus, dialogs, or settings)

Rule of thumb: Pausing does not mean drawing nothing – it means changing nothing.

4.2 Bubbles

In this chapter, the game *Bubbles* is discussed as an example. We will develop this game step by step in a systematic way. I will assume that the techniques introduced in chapter 1 are already familiar. I will deliberately omit docstring comments in the source code, since everything is explained in the text and the listings would otherwise become unnecessarily long. In the final version, however, these comments are included.

Thank you

I would like to point out right away that the idea for the game did not come from me. A student once presented it as a mobile version at an [Information Technology Assistants \(ITA\)](#) fair. Unfortunately, I can no longer remember the student's name, but I would like to take this opportunity to say a sincere *thank you*.

The game can be extended almost without limits: bubble popping animations, high score lists, and much more. But as is so often the case, the better is the enemy of the good. I hope you enjoy studying this example.

4.2.1 Requirement 1: Standards

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v01

Requirement 1 Standard functionality

1. *The window has an appropriate size.*
 2. *The background is either a suitable bitmap or a solid color.*
 3. *The game can be exited using `Esc` or by clicking the red “X”.*
 4. *All bitmaps are converted and scaled appropriately after loading.*
 5. *All bitmaps – except for the background – are transparent.*
 6. *All bitmaps are stored in `pygame.sprite.Group` or `pygame.sprite-GroupSingle` objects.*
 7. *The game has a frame-rate-independent execution speed.*
-

Requirement 1 does not only define specific requirements, but also general ones. For this reason, it will appear again in later implementations.

At this point, the preamble is presented once. I assume that you have sufficient Python knowledge to extend it as needed. The static configuration values of the game are stored, as usual, in the separate `config.py` file.

It is required that the window has an appropriate size. With $1220\text{ px} \times 1002\text{ px}$, the window is large enough to distribute the bubbles, yet small enough to allow quick mouse movement. Everything else has already been discussed in detail in previous chapters (e.g. FPS, DELTATIME, or PATH) and will therefore not be explained further here.



Figure 4.5: Bubbles: background image

Listing 4.38: Bubbles (requirement 1.1) – `config.py`

```

1 import os
2 from typing import Dict
3
4 import pygame
5
6 WINDOW = pygame.Rect(0, 0, 1220, 1002)
7 FPS = 60
8 DELTATIME = 1.0 / FPS
9 PATH: Dict[str, str] = {}
10 PATH["file"] = os.path.dirname(os.path.abspath(__file__))
11 PATH["image"] = os.path.join(PATH["file"], "images")
12 PATH["sound"] = os.path.join(PATH["file"], "sounds")
13 CAPTION = 'Bubbles'
14
15 def get_file(filename: str) -> str:
16     return os.path.join(PATH["file"], filename)
17
18 def get_image(filename: str) -> str:
19     return os.path.join(PATH["image"], filename)
20
21 def get_sound(filename: str) -> str:
22     return os.path.join(PATH["sound"], filename)

```

The `Background` class is a subclass of `Sprite`. It is only loaded and scaled to the appropriate size. Since the background never changes, there is no need to implement an `update()` method. Creating a dedicated subclass for this is somewhat like using a sledgehammer to crack a nut. We could just as well have implemented it directly as

a `Sprite` object. I chose this approach purely for the sake of clarity. The background image can be seen in figure 4.5 on the preceding page.

Listing 4.39: Bubbles (requirement 1.2) – Background

```

7 class Background(pygame.sprite.Sprite):
8     def __init__(self) -> None:
9         super().__init__()
10        imagename = cfg.get_image("aquarium.png")
11        self.image: pygame.Surface = pygame.image.load(imagename).convert()
12        self.image = pygame.transform.scale(self.image, cfg.WINDOW.size)
13        self.rect: pygame.Rect = self.image.get_rect()

```

In the `Game` class, the usual Pygame suspects are initialized or created in `__init__()`: `init()`, `Window()`, and `clock()`. The flag `running` for the main game loop is also initialized. The methods `run()`, `watch_for_events()`, `update()`, and `draw()` contain only basic functionality and therefore do not need to be explained further at this point.

Listing 4.40: Bubbles (requirement 1) – Game

```

16 class Game:
17
18     def __init__(self) -> None:
19         pygame.init()
20         self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.CAPTION)
21         self.screen = self.window.get_surface()
22         self.clock = pygame.time.Clock()
23         self.background = pygame.sprite.GroupSingle(Background())
24         self.running = True
25
26     def watch_for_events(self) -> None:
27         for event in pygame.event.get():
28             if event.type == pygame.QUIT:
29                 self.running = False
30             elif event.type == pygame.KEYDOWN:
31                 if event.key == pygame.K_ESCAPE:
32                     self.running = False
33
34     def draw(self) -> None:
35         self.background.draw(self.screen)
36         self.window.flip()
37
38     def update(self) -> None:
39         pass
40
41     def run(self) -> None:
42         time_previous = time()
43         self.running = True
44         while self.running:
45             self.watch_for_events()
46             self.update()
47             self.draw()
48             self.clock.tick(cfg.FPS)
49             time_current = time()
50             cfg.DELTATIME = time_current - time_previous
51             time_previous = time_current
52         pygame.quit()

```

However, these methods already define the overall flow of the game. All further properties of the game are merely extensions of this flow and no longer change it. Finally, the

call is made (see source code 4.41). With this, all subitems of requirement 1 on page 274 that apply here are fulfilled.

Listing 4.41: Bubbles (requirement 1) – invocation

```

55 def main():
56     Game().run()
57
58
59 if __name__ == "__main__":
60     main()

```

4.2.2 Requirement 2: Bubbles appear

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v02

Requirement 2 Bubbles appear

1. *A bubble appears at a random position.*
 2. *At the beginning, this happens every half second.*
 3. *It has an initial radius of 15 px.*
 4. *It keeps a minimum distance of 10 px from the edges.*
 5. *It keeps a minimum distance of 10 px from all other bubbles.*
-

For the bubble, we use the already transparent graphic from figure 4.6. The random position still needs to be restricted. The aquarium does not fill the entire screen (see figure 4.5 on page 275); instead, it sits inside something like a TV frame. So we have to define a playing area (*playground*). The bubbles should only appear inside this area.



Fig. 4.6: Bubble

The playing area is a rectangle with an offset from the left and top edges of the screen – `left` and `top` – and a width (`width`) and height (`height`). The corresponding values are defined in line 16. The distance to the border of the playing area and the minimum distance between bubbles are defined in line 15 as 10 px, in accordance with requirement 2.4. The initial radius – and therefore the minimum radius – is set to 15 px in line 14 because of requirement 2.3. While playing, I noticed that smaller initial radii are simply too hard to see.

Listing 4.42: Bubbles (requirement 2) – additions in config.py

```

14 RADIUS = {"min": 15}                                # Radius to start with
15 DISTANCE = 50                                       # Border-/Bubbledistance
16 PLAYGROUND = pygame.Rect(90, 90, 1055, 615) # Rect inside aquarium

```

The `Timer` class is exactly the one described above in chapter 2.9 on page 136; everything is explained there.

Listing 4.43: Bubbles (requirement 2) – Timer

```

9 class Timer:
10     def __init__(self, duration: int, with_start: bool = True) -> None:
11         self.duration = duration
12         if with_start:
13             self._next = pygame.time.get_ticks()
14         else:
15             self._next = pygame.time.get_ticks() + self.duration
16
17     def is_next_stop_reached(self) -> bool:
18         if pygame.time.get_ticks() > self._next:
19             self._next = pygame.time.get_ticks() + self.duration
20             return True
21         return False

```

Let us now take a look at the `Bubble` class. The constructor is self-explanatory; it only handles the usual suspects: `image`, `rect`, and `radius`. The `update()` method is currently empty, since no changes are required yet. However, the `randompos()` method is needed because of requirement 2.1. It calculates a new bubble center and assigns it to `rect`. If necessary, this method must be repeated until a free area is found (see requirement 2.4 and requirement 2.5).

Listing 4.44: Bubbles (requirement 2) – Bubble

```

33 class Bubble(pygame.sprite.Sprite):
34     def __init__(self) -> None:
35         super().__init__()
36         self.radius = cfg.RADIUS["min"]
37         imagename = cfg.get_image("bubble1.png")
38         self.image: pygame.Surface = pygame.image.load(imagename).convert_alpha()
39         self.image = pygame.transform.scale(self.image, (cfg.RADIUS["min"],
40             cfg.RADIUS["min"]))
41         self.rect: pygame.Rect = self.image.get_rect()
42
43     def update(self, *args: Any, **kwargs: Any) -> None:
44         pass
45
46     def randompos(self) -> None:
47         bubbledistance = cfg.DISTANCE + cfg.RADIUS["min"]
48         centerx = randint(cfg.PLAYGROUND.left + bubbledistance, cfg.PLAYGROUND.right -
49             bubbledistance)
50         centery = randint(cfg.PLAYGROUND.top + bubbledistance, cfg.PLAYGROUND.bottom -
51             bubbledistance)
52         self.rect.center = (centerx, centery)

```

The `Game` class now has to be extended accordingly. In line 58, the `Background` object is created. line 59 creates a `Timer` object with an interval of 500 ms, where no bubbles are generated during the first interval (see requirement 2.2).

Listing 4.45: Bubbles (requirement 2) – Constructor of Game

```

52 class Game:
53     def __init__(self) -> None:
54         pygame.init()
55         self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.CAPTION)
56         self.screen = self.window.get_surface()
57         self.clock = pygame.time.Clock()
58         self.background = pygame.sprite.GroupSingle(Background()) #
59         self.timer_bubble = Timer(500, False) # Timer 500ms
60         self.all_sprites = pygame.sprite.Group() # All bubbles
61         self.running = True

```

In the `draw()` method, only the `draw()` methods of the sprite groups are called. The `update()` method has also been adjusted; it now calls the `spawn_bubble()` method and thus delegates the task of creating new bubbles.

Listing 4.46: Bubbles (requirement 2) – `draw()` and `update()` of Game

```

71 def draw(self) -> None:
72     self.background.draw(self.screen)
73     self.all_sprites.draw(self.screen)
74     self.window.flip()
75
76 def update(self) -> None:
77     self.spawn_bubble()

```

The basic idea behind `spawn_bubble()` is to keep guessing a position for a new bubble until a free area is found.

Avoid endless loop

To avoid ending up in an [infinity loop](#), the number of attempts is limited to 100. If no free area is found, the bubble is not added to the sprite group – it is simply discarded.

For this purpose, the radius is temporarily increased (line 84) and then reduced back to its original value after the collision check (line 86).

This is an example showing that a method reference is passed to `pygame.sprite.spritecollide()` – in this case `pygame.sprite.collide_circle()` – and that the usual rectangle-based collision check is therefore not used.

`sprite-`
`collide()`

`collide_-`
`circle()`

Listing 4.47: Bubbles (requirement 2) – `spawn_bubble()` of Game

```

79 def spawn_bubble(self) -> None:
80     if self.timer_bubble.is_next_stop_reached():
81         b = Bubble()
82         for _ in range(100):
83             b.randompos()
84             b.radius += cfg.DISTANCE # Distance to other bubbles
85             collided = pygame.sprite.spritecollide(b, self.all_sprites, False,
86                 pygame.sprite.collide_circle)
87             b.radius -= cfg.DISTANCE # Old radius!
88             if not collided:

```

```

88         self.all_sprites.add(b)
89         break

```

The result can be seen in figure 4.7. The bubbles are evenly distributed across the playing area, and the required minimum distance to the edges and between the bubbles is maintained.



Figure 4.7: Bubbles: the bubbles have a minimum distance at the start

4.2.3 Requirement 3: Number of bubbles

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v03

Requirement 3 Number of bubbles

The maximum number of bubbles shall depend on the size of the playing area.

I want to define the maximum number in the `Game` class. Based on the available area, an upper limit is calculated:

Listing 4.48: Bubbles (requirement 3) – additions in `config.py`

```

17 MAX_BUBBLES = PLAYGROUND.height * PLAYGROUND.width // 10000 # A guess

```

This upper limit from line 17 is checked in line 81. A new bubble is only created if the maximum number has not yet been reached.

Listing 4.49: Bubbles (requirement 3) – additions in `Game.spawn_bubbles()`

```

79 def spawn_bubble(self) -> None:
80     if self.timer_bubble.is_next_stop_reached():
81         if len(self.all_sprites) <= cfg.MAX_BUBBLES: # Enough space?
82             b = Bubble()
83             for _ in range(100):
84                 b.randompos()
85                 b.radius += cfg.DISTANCE
86                 collided = pygame.sprite.spritecollide(b, self.all_sprites, False,
87                     pygame.sprite.collide_circle)
88                 b.radius -= cfg.DISTANCE
89                 if not collided:
90                     self.all_sprites.add(b)
91                     break

```

The rest of the program remains unchanged.

4.2.4 Requirement 4: Bubble growth

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v04

Requirement 4 Bubble growth

1. Bubbles of different sizes are managed in a container.
2. The maximum radius of a bubble is 240 px.

The purpose of requirement 4.1 is to save computing time. During the game, bubbles repeatedly start with a certain radius and then grow. Scaling the bitmap to the required size every single time would waste processing power – after all, the same bubble appears multiple times with identical radii. For this reason, it makes sense to scale the bubble once to all possible radii and store the results in a dictionary. The key used is the respective radius (see line 37).

scale()

The `get()` method then returns the appropriately scaled and ready-to-use image for a given radius. Before that, lines 40 and 41 check whether the radius lies within the valid range. If the radius is too large, the maximum value is used; if it is too small, the minimum value is applied instead.

Listing 4.50: Bubbles (requirement 4.1) – `BubbleContainer`

```

33 class BubbleContainer:
34     def __init__(self) -> None:
35         imagename = cfg.get_image("bubble1.png")
36         image: pygame.Surface = pygame.image.load(imagename).convert_alpha()

```

```

37         self.images = {i: pygame.transform.scale(image, (i * 2, i * 2)) for i in
38             range(cfg.RADIUS["min"], cfg.RADIUS["max"] + 1)} #
39
38     def get(self, radius: int) -> pygame.Surface:
39         radius = max(cfg.RADIUS["min"], radius) # Lower limit
40         radius = min(cfg.RADIUS["max"], radius) # Upper limit
41         return self.images[radius]
42

```

So far, only a start value – and thus a lower bound – for the bubble radius has been defined in `Game`. This definition is now extended in line 14 in accordance with requirement 4.2 by adding a maximum radius.

Listing 4.51: Bubbles (requirement 4.2) – extension of `config.py`

```

14 RADIUS = {"min": 15, "max": 240} #

```

The `BubbleContainer` is passed to the constructor of `Bubble`, allowing this class to retrieve images from it. A direct example of this can be found in line 50. The `image` attribute is set according to the current `radius`.

The `update()` method is no longer empty. Its main purpose is to make the bubble grow. To achieve this, the radius is continuously increased, which results in increasingly larger images being loaded from the `BubbleContainer` and displayed (line 62). The new radius is calculated in line 59. In the same line, this value is compared with the maximum radius from `config.py`, and the minimum of the two is selected. This logic prevents the radius from becoming too large.

Center-based scaling

But what is the purpose of lines 61 and 64? The reference point of an image in a sprite is its top-left corner. If the bubble grows, it would therefore expand to the right and downward; the left and top edges would remain fixed, which looks awkward. To avoid this, we store the old center point, load the new image, create the corresponding `Rect` object, and then move it back to the old center. This way, the bubble visually grows outward from its center in all directions.

Listing 4.52: Bubbles (requirement 4) – extension of `Bubble`

```

45 class Bubble(pygame.sprite.Sprite):
46     def __init__(self, bubble_container: BubbleContainer) -> None:
47         super().__init__()
48         self.bubble_container = bubble_container # Reference to container
49         self.radius = cfg.RADIUS["min"]
50         self.image = self.bubble_container.get(self.radius) # Get bubble
51         self.rect: pygame.Rect = self.image.get_rect()
52         self.fradius = float(self.radius)
53         self.speed = 100
54
55     def update(self, *args: Any, **kwargs: Any) -> None:
56         if "action" in kwargs.keys():
57             if kwargs["action"] == "grow":

```

```

58         self.fradius += self.speed * cfg.DELTATIME
59         self.fradius = min(self.fradius, cfg.RADIUS["max"]) # New radius
60         self.radius = round(self.fradius)
61         center = self.rect.center # Save center pos
62         self.image = self.bubble_container.get(self.radius) # New image
63         self.rect = self.image.get_rect()
64         self.rect.center = center # Restore center pos
65
66     def randompos(self) -> None:
67         bubbledistance = cfg.DISTANCE + cfg.RADIUS["min"]
68         centerx = randint(cfg.PLAYGROUND.left + bubbledistance, cfg.PLAYGROUND.right -
69             bubbledistance)
70         centery = randint(cfg.PLAYGROUND.top + bubbledistance, cfg.PLAYGROUND.bottom -
71             bubbledistance)
72         self.rect.center = (centerx, centery)

```

The `update()` method in `Game` only needs to be extended by calling all `update()` methods of the bubbles. This can be done very conveniently using the sprite group mechanism. Just like with `draw()`, `update()` can be called for the entire group in a single step (see line 99).

Listing 4.53: Bubbles (requirement 4) – extension of `Game.update()`

```

98     def update(self) -> None:
99         self.all_sprites.update(action="grow") # Bubbles growing
100         self.spawn_bubble()

```

Create the `BubbleContainer`

Listing 4.54: Bubbles (requirement 4) – extension of the constructor of `Game`

```

79     self.background = pygame.sprite.GroupSingle(Background())
80     self.bubble_container = BubbleContainer()
81     self.timer_bubble = Timer(500, False)

```

And in the `spawn_bubble()` method, the constructor call of `Bubble` is extended by passing the `BubbleContainer`.

Listing 4.55: Bubbles (requirement 4) – extension of `Game.spawn_bubble()`

```

102     def spawn_bubble(self) -> None:
103         if self.timer_bubble.is_next_stop_reached():
104             if len(self.all_sprites) <= cfg.MAX_BUBBLES:
105                 b = Bubble(self.bubble_container) #
106                 for _ in range(100):

```

The bubbles now grow outward from their center. The result might then look like the one shown in figure 4.8 on the following page.



Figure 4.8: Bubbles – the bubbles have grown and merged

4.2.5 Requirement 5: Mouse cursor

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v05

Requirement 5 Mouse cursor

If the mouse is inside a bubble, its appearance should change.

This requirement is intended to provide visual feedback to the player. It allows them to recognize more quickly whether they have already reached a bubble. Pygame itself does not provide a method or function to test whether a point lies inside a circle. However, figure 4.9 on the next page provides a simple approach to solving this problem.

The value d represents the distance in pixels between the center of the circle (x_1, y_1) and the point (x_2, y_2) . If $d \leq r$, the point lies inside the circle or touches it. However, we do not actually need the distance itself. Put simply, we only need to know whether the expression on the left side of the inequality is smaller than the one on the right side. We can therefore avoid the expensive square root operation and instead check $(x_2 - x_1)^2 + (y_2 - y_1)^2 \leq r^2$. We therefore extend `Bubble` with an appropriate method.

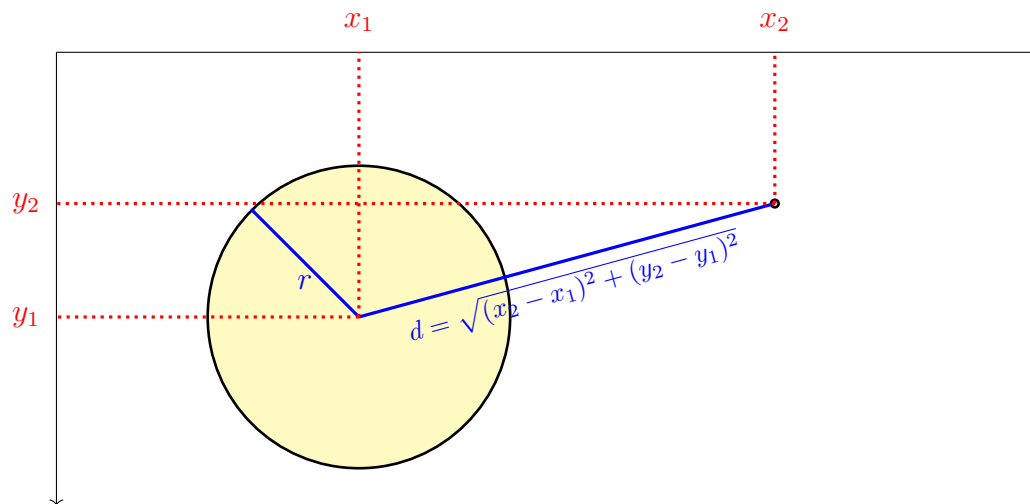


Figure 4.9: collision detection – point inside a circle (Pythagorean theorem)?

Listing 4.56: Bubbles (requirement 5) – `Game.collidepoint()`

```

117 def collidepoint(self, point: Tuple[int, int], sprite: pygame.sprite.Sprite) -> bool:
118     if hasattr(sprite, "radius"):
119         deltax = point[0] - sprite.rect.centerx
120         deltay = point[1] - sprite.rect.centery
121         return deltax * deltax + deltay * deltay <= pow(sprite.radius, 2)
122     return False

```

With the help of this method, the solution is no longer a problem. The variable `is_over` is a flag that keeps track of whether the mouse coordinates are inside a bubble or not. The normal case is that the mouse is not inside any bubble, so the variable is initialized with `False`.

After that, the current mouse position is obtained using `pygame.mouse.get_pos()`. This mouse position is passed to the `Bubble.collidepoint()` method in line 128. If a bubble is found that collides with the mouse, the flag is set to `True` and the loop is terminated using `break`. This saves some processing time, since not all remaining bubbles have to be checked. Depending on the flag, the mouse cursor is then set accordingly.

Listing 4.57: Bubbles (requirement 5) – `Game.set_mousecursor()`

```

124 def set_mousecursor(self) -> None:
125     is_over = False
126     pos = pygame.mouse.get_pos()
127     for b in self.all_sprites:
128         if self.collidepoint(pos, b):                # Mouse pos inside?
129             is_over = True
130             break
131     if is_over:
132         pygame.mouse.set_cursor(pygame.SYSTEM_CURSOR_CROSSHAIR)
133     else:
134         pygame.mouse.set_cursor(pygame.SYSTEM_CURSOR_HAND)

```

The `update()` method in `Game` still needs to be extended by adding the call to the collision check.

Listing 4.58: Bubbles (requirement 5) – `update()` in `Game`

```

99     def update(self) -> None:
100         self.all_sprites.update(action="grow")
101         self.spawn_bubble()
102         self.set_mousecursor() #

```

Try running the program. Place the mouse in a lower-left corner outside a bubble and wait until the growing bubble touches the mouse.

4.2.6 Requirement 6: Bubbles burst

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v06

Requirement 6 Bubbles burst

When a left mouse click occurs inside a bubble, the bubble should burst.

Most of the work required to implement this requirement has already been done with the implementation of the `Bubble.collidepoint()` method. We only need to use this method in a clever way – in fact, only a few remaining steps are necessary. In `watch_for_events()`, the left mouse click is detected first (line 92) and the current mouse position is passed to the newly created `sting()` method (line 94).

MOUSE-
BUTTON-
DOWN
`get_pos()`

`watch_for_events()` as a dispatcher

As a general rule, implement as little logic as possible in `watch_for_events()`. This method acts as a dispatcher; the actual processing should always be delegated to separate methods.

Listing 4.59: Bubbles (requirement 6) – `Game.watch_for_event()`

```

85     def watch_for_events(self) -> None:
86         for event in pygame.event.get():
87             if event.type == pygame.QUIT:
88                 self.running = False
89             elif event.type == pygame.KEYDOWN:
90                 if event.key == pygame.K_ESCAPE:
91                     self.running = False

```

```

92         elif event.type == pygame.MOUSEBUTTONDOWN: # Mouse clicked?
93             if event.button == 1: # left
94                 self.sting(pygame.mouse.get_pos()) #

```

The `sting()` method is now very simple. All `Bubble` objects are iterated over and checked to see whether the mouse position lies within their radius (line 140). If the answer is *yes*, the corresponding object is removed from the sprite group using `kill()`.

`kill()`

Listing 4.60: Bubbles (requirement 6) – `Game.sting()`

```

138 def sting(self, mousepos: Tuple[int, int]) -> None:
139     for bubble in self.all_sprites:
140         if self.collidepoint(mousepos, bubble): # Inside?
141             bubble.kill()

```

4.2.7 Requirement 7: Score



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v07

Requirement 7 Score

1. The game starts with 0 points.
2. When a bubble bursts, the score is increased proportionally to its radius.
3. The score is displayed in the lower part of the screen.

Popping bubbles should, of course, be rewarded with points. To do this, the score has to be calculated and displayed. The simplest way to keep track of the score is to use a variable in `config.py` or a global variable. I prefer option 1 (see source code 4.61).

Listing 4.61: Bubbles (requirement 7.1) – extension of `config.py`

```

18 POINTS = 0 # Score

```

Since popping a bubble no longer only makes it disappear but also updates the score, I added a new method to the `Bubble` class. In line 76, the radius of the bubble is simply added to the score.

Listing 4.62: Bubbles (requirement 7.2) – `Bubble.stung()`

```

74 def stung(self):
75     self.kill()
76     cfg.POINTS += self.radius # Increment points

```

The call to `stung()` is triggered by an adjusted `update()` method.

Listing 4.63: Bubbles (requirement 7.2) – `Bubble.update()`

```

55     def update(self, *args: Any, **kwargs: Any) -> None:
56         if "action" in kwargs.keys():
57             if kwargs["action"] == "grow":
58                 self.fradius += self.speed * cfg.DELTATIME
59                 self.fradius = min(self.fradius, cfg.RADIUS["max"])
60                 self.radius = round(self.fradius)
61                 center = self.rect.center
62                 self.image = self.bubble_container.get(self.radius)
63                 self.rect = self.image.get_rect()
64                 self.rect.center = center
65             elif kwargs["action"] == "sting":
66                 self.stung()

```

The `sting()` and `update()` methods in `Game` have to be adjusted accordingly (see line 162 and line 123).

Listing 4.64: Bubbles (requirement 7.2) – `Game.sting()`

```

159     def sting(self, mousepos: Tuple[int, int]) -> None:
160         for bubble in self.all_sprites:
161             if self.collidepoint(mousepos, bubble):
162                 bubble.update(action="sting") # Redesigned

```

Listing 4.65: Bubbles (requirement 7.2) – `Game.update()`

```

122     def update(self) -> None:
123         self.all_sprites.update(action="grow") #
124         self.spawn_bubble()
125         self.set_mousecursor()

```

This leaves requirement 7.3. Similar to the playing area, I want to define the dimensions of the lower section as an output box in `config.py`.

Rect

Listing 4.66: Bubbles (requirement 7.3) – extension of `config.py`

```

19         self._next = pygame.time.get_ticks() + self.duration

```

For displaying the score itself, I once again create a small class that encapsulates this task: `Points`. In the constructor, a `Font` object is created, which is then used in `update()` to render the score. The position of the text output is determined from the values defined in `config.py`. The rest is handled for me by the `Sprite` class.

Font

Listing 4.67: Bubbles (requirement 7.3) – Class `Points`

```

79     class Points(pygame.sprite.Sprite):
80         def __init__(self) -> None:
81             super().__init__()
82             self.font = pygame.font.Font(pygame.font.get_default_font(), 18)
83             self.oldpoints = -1

```

```

84
85 def update(self, *args: Any, **kwargs: Any) -> None:
86     if self.oldpoints != cfg.POINTS:
87         self.image = self.font.render(f"Points: {cfg.POINTS}", True, "red")
88         self.rect = self.image.get_rect()
89         self.rect.left = cfg.BOX.left
90         self.rect.top = cfg.BOX.top

```

A few extensions remain in `Game`. In the constructor, the `Points` object is added to the `Group` object.

Listing 4.68: Bubbles (requirement 7.3) – extension of the `Game` constructor

```

102 self.all_sprites = pygame.sprite.Group()
103 self.all_sprites.add(Points())
104 self.running = True

```

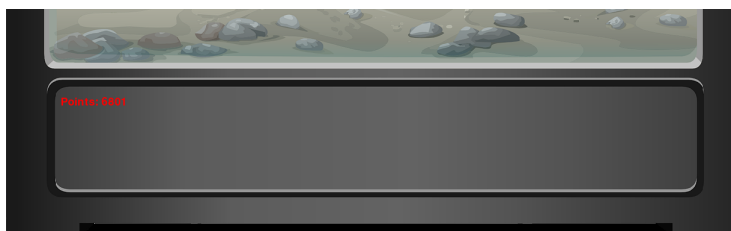


Figure 4.10: Bubbles – score display

In figure 4.10, you can see the score display in the lower part of the screen. This area could later also be used for a list of the top ten scores or other types of output.

4.2.8 Requirement 8: Game over

So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v08

Requirement 8 Game over

1. *If two bubbles touch, the game is lost.*
2. *If a bubble touches the edge, the game is lost.*

Note: To make the game playable, I set the growth speed of a bubble to 10.

Listing 4.69: Bubbles (requirement 8) – `Bubble.speed`

```
53      self.speed = 10
```

The basic structure of our game makes it fairly easy to implement this requirement by extending the `update()` method in `Game`.

Listing 4.70: Bubbles (requirement 8) – extension of `Game.update()`

```
125     def update(self) -> None:
126         if self.check_bubblecollision():                # Game over?
127             self.running = False
128         else:
129             self.all_sprites.update(action="grow")
130             self.spawn_bubble()
131             self.set_mousecursor()
```

In the new method `check_bubblecollision()`, it is checked whether bubbles touch each other or whether a bubble collides with the edge. This method is simply used as a decision maker (line 126) to determine whether the game should end. If the answer is *yes*, the flag of the main game loop is set; if the answer is *no*, the remaining game logic is executed as usual. The two nested `for`-loops starting at line 171 iterate over the bubble group twice and avoid two problems:

- A bubble must not be compared with itself. Therefore, the index of the inner loop always starts one position after the current index of the outer loop, and the outer loop index ends before the last element of the bubble group.
- If bubble 1 has already been compared with bubble 2, bubble 2 should not be compared again with bubble 1. This is also achieved by the shifted index.

sprites()

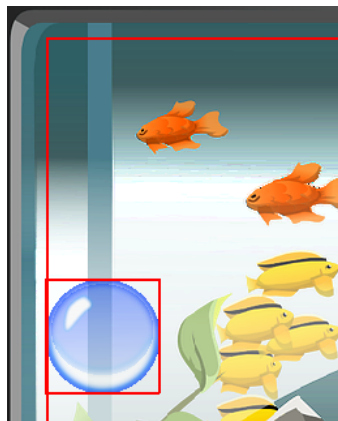


Figure 4.11: Bubbles – collision with the edge

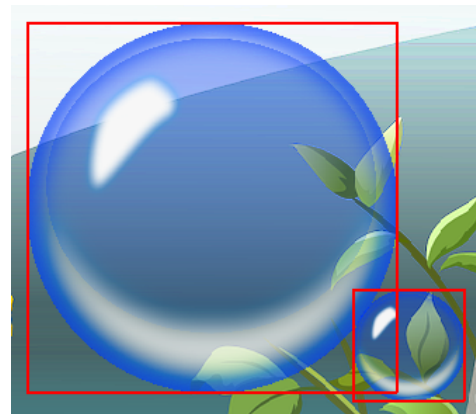


Figure 4.12: Bubbles – bubble collision

In line 176, requirement 8.1 is checked. For this purpose, circle-based collision detection using `collide_circle()` is applied. In line 178 and line 180, requirement 8.2 is implemented. This makes use of the fact that the playing area is a rectangle and that the

contains()

sprite itself also has a rectangular shape. The method `pygame.Rect.contains()` checks whether one rectangle is completely contained within another. If this is not the case – meaning the bubble leaves the playing area – a collision is detected.

Listing 4.71: Bubbles (requirement 8) – `Game.check_bubblecollision()`

```

170 def check_bubblecollision(self) -> bool:
171     for index1 in range(0, len(self.all_sprites) - 1): # Check bubbles
172         for index2 in range(index1 + 1, len(self.all_sprites)):
173             bubble1 = self.all_sprites.sprites()[index1]
174             bubble2 = self.all_sprites.sprites()[index2]
175             if type(bubble1).__name__ == "Bubble" and type(bubble2).__name__ == "Bubble":
176                 if pygame.sprite.collide_circle(bubble1, bubble2): # Bubbles?
177                     return True
178                 if not cfg.PLAYGROUND.contains(bubble1): # Bubble1 touches border
179                     return True
180                 if not cfg.PLAYGROUND.contains(bubble2): # Bubble2 touches border
181                     return True
182     return False

```

In figure 4.11 on the facing page, the collision of a bubble with the edge is shown. To make this easier to see, helper lines have been drawn. You can clearly see that the rectangle of the bubble is no longer contained within the rectangle of the playing area. figure 4.12 on the preceding page shows the collision of two bubbles. Here as well, helper lines are drawn. These helper lines are displayed when you remove the three comment characters in `Game.draw()`.

Listing 4.72: Bubbles (requirement 8) – helper lines in `Game`

```

117 def draw(self) -> None:
118     self.background.draw(self.screen)
119     self.all_sprites.draw(self.screen)
120     # pygame.draw.rect(self._screen, "red", Settings.PLAYGROUND, 2)
121     # for b in self._all_sprites:
122     #     pygame.draw.rect(self._screen, "red", b.rect, 2)
123     self.window.flip()

```

4.2.9 Requirement 9: Time-based adjustments



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v09

Requirement 9 Time-based adjustments

The bubbles should grow faster over time.

Since the bubbles are supposed to grow faster over time, I want to pass the growth speed to them as a constructor parameter. In line 54, this parameter is stored in an attribute.

Listing 4.73: Bubbles (requirement 9) – Bubble

```

46 class Bubble(pygame.sprite.Sprite):
47     def __init__(self, bubble_container: BubbleContainer, speed: int) -> None:
48         super().__init__()
49         self.bubble_container = bubble_container
50         self.radius = cfg.RADIUS["min"]
51         self.image = self.bubble_container.get(self.radius)
52         self.rect: pygame.Rect = self.image.get_rect()
53         self.fradius = float(self.radius)
54         self.speed = speed                                     # Rate of growing

```

Timer

These are all the required changes in `Bubble`; everything else happens in `Game`. In line 103, a timer is created that emits a signal every 1000ms. Below that, the initial growth speed of the bubbles is set to 10px.

Listing 4.74: Bubbles (requirement 9) – adjustment of the `Game` constructor

```

101     self.bubble_container = BubbleContainer()
102     self.timer_bubble = Timer(500, False)
103     self.timer_bubble_speed = Timer(1000, False) #
104     self.bubble_speed = 10
105     self.all_sprites = pygame.sprite.Group()

```

In `spawn_bubble()`, the timer is checked and, if necessary, the bubble growth speed is increased (line 134). The maximum growth speed is limited to 100px/s; anything faster does not seem playable to me. Each timer signal increases the speed by 5px/s. This is done in this method so that the new speed is available for newly created bubbles.

Listing 4.75: Bubbles (requirement 9) – `Game.spawn_bubble()`

```

133     def spawn_bubble(self) -> None:
134         if self.timer_bubble_speed.is_next_stop_reached(): #
135             if self.bubble_speed < 100:
136                 self.bubble_speed += 5
137         if self.timer_bubble.is_next_stop_reached():
138             if len(self.all_sprites) <= cfg.MAX_BUBBLES:
139                 b = Bubble(self.bubble_container, self.bubble_speed)
140                 for _ in range(100):
141                     b.randompos()
142                     b.radius += cfg.DISTANCE
143                     collided = pygame.sprite.spritecollide(b, self.all_sprites, False,
144                                                             pygame.sprite.collide_circle)
145                     b.radius -= cfg.DISTANCE
146                     if not collided:
147                         self.all_sprites.add(b)
148                         break

```

If you now try out the game, you will notice an easy start and a moderate increase in game difficulty.

4.2.10 Requirement 10: Display collision

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v10

Requirement 10 Display collision

If bubbles collide with the edge or with each other, they should change color and remain visible for 5 s before the application terminates.

So far, the game ends so quickly that I cannot really check whether I actually lost for a valid reason or whether the program is misbehaving. With this requirement, I want to be able to see the two colliding bubbles, or the bubble that touches the edge, in a different color. For this purpose, I colored the bubble red (see figure 4.13).

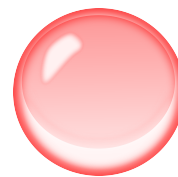


Fig. 4.13: Bubble 2

To achieve this, a second `BubbleContainer` with scaled red bubbles is required. To make access easier, these containers are stored in `Game` as a static dictionary.

In line 102, such a dictionary is created. Under a key, I can now store arbitrary `BubbleContainer` objects.

Listing 4.76: Bubbles (requirement 10) – extension of `Game`

```
101 class Game:
102     BUBBLE_CONTAINER: Dict[str, BubbleContainer] = {} # Now mor than one
```

The constructor of `BubbleContainer` now receives a filename as a parameter, allowing different graphics to be used as a basis.

Listing 4.77: Bubbles (requirement 10) – change to the constructor of `BubbleContainer`

```
34 class BubbleContainer:
35     def __init__(self, filename: str) -> None: # Now with filename
36         imagename = cfg.get_image(filename)
37         image: pygame.Surface = pygame.image.load(imagename).convert_alpha()
38         self.images = {i: pygame.transform.scale(image, (i * 2, i * 2)) for i in
            range(cfg.RADIUS["min"], cfg.RADIUS["max"] + 1)}
```

The constructor of `Game` now populates the static dictionary `BUBBLE_CONTAINER` (line 109 and line 110).

Listing 4.78: Bubbles (requirement 10) – change to the constructor of Game

```

108     self.clock = pygame.time.Clock()
109     Game.BUBBLE_CONTAINER["blue"] = BubbleContainer("bubble1.png")    # blue
110     Game.BUBBLE_CONTAINER["red"] = BubbleContainer("bubble2.png")    # red
111     self.background = pygame.sprite.GroupSingle(Background())

```

Several changes are now required in Bubble. The new attribute `mode` (line 49) determines the color of the bubble. Whenever an image is loaded from the `BubbleContainer`, this attribute controls which of the two `BubbleContainer` instances is used as the data source. As an example, line 63 in `update()` can be mentioned here.

Listing 4.79: Bubbles (requirement 10) – constructor of Bubble and `update()`

```

46 class Bubble(pygame.sprite.Sprite):
47     def __init__(self, speed: int) -> None:
48         super().__init__()
49         self.mode = "blue"                                # Color mode
50         self.radius = cfg.RADIUS["min"]
51         self.image = Game.BUBBLE_CONTAINER[self.mode].get(self.radius)
52         self.rect: pygame.Rect = self.image.get_rect()
53         self.fradius = float(self.radius)
54         self.speed = speed
55
56     def update(self, *args: Any, **kwargs: Any) -> None:
57         if "action" in kwargs.keys():
58             if kwargs["action"] == "grow":
59                 self.fradius += self.speed * cfg.DELTATIME
60                 self.fradius = min(self.fradius, cfg.RADIUS["max"])
61                 self.radius = round(self.fradius)
62                 center = self.rect.center
63                 self.image = Game.BUBBLE_CONTAINER[self.mode].get(self.radius) #
64                 self.rect = self.image.get_rect()
65                 self.rect.center = center
66             elif kwargs["action"] == "sting":
67                 self.stung()
68         elif "mode" in kwargs.keys():
69             self.set_mode(kwargs["mode"])

```

If the mode changes, the alternative color has to be reloaded. This is handled by the `set_mode()` method in Bubble.

Listing 4.80: Bubbles (requirement 10) – `Bubble.set_mode()`

```

71     def set_mode(self, mode: str) -> None:
72         if mode != self.mode:
73             self.mode = mode
74             self.image = Game.BUBBLE_CONTAINER[self.mode].get(self.radius)

```

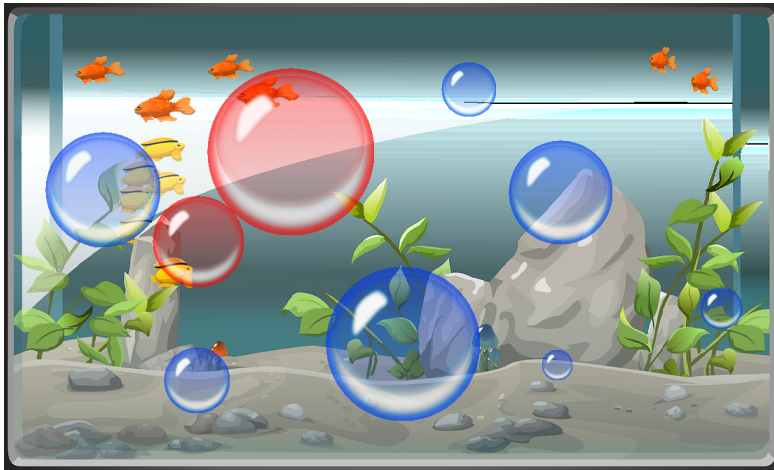


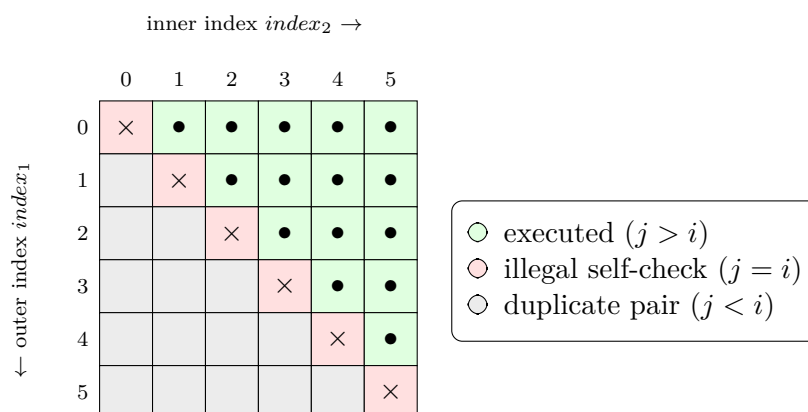
Fig. 4.14: Bubbles – displaying a collision

Now, in the case of a collision – that is, when the game ends – the mode simply needs to be changed.

In figure 4.14, you can see how the two colliding bubbles appear in red. An example of how this is implemented can be found in line 190. In a nested `for`-loop, all sprites are iterated over.

A naive approach would cause problems if both

loops compared all sprites from start to finish. First, because due to symmetry you would check $B = A$ even though you have already checked $A = B$. Second, because you would end up comparing each sprite with itself, which would lead to a false collision detection.

Figure 4.15: Why does the inner loop start at $index_1 + 1$?

In figure 4.15, the algorithm used is illustrated. For each outer index $i=index_1$, the inner loop runs $j=index_2$ from $i + 1$ to $N - 1$: `for  $index_2$  in range( $index_1 + 1$ , N)`

- Starting at $i + 1$ skips the diagonal ($j = i$), so a bubble is never compared with itself.
- It also skips everything left of the diagonal ($j < i$), which would repeat comparisons (e.g. (1, 2) and later (2, 1)).
- The outer loop stops at $N - 2$ (i.e. `range(0, N-1)`), because when $i = N - 1$ there is no valid $j > i$ left to compare.

After that, it is checked whether both sprites are of type `Bubble`. Only then is a collision check performed — before that, it would not be worthwhile. If a collision is detected, both bubbles are colored red and the function is exited.

But what if the two bubbles do not touch each other? In that case, there is no need to check whether either of them touches the edge. If one of them does, that bubble is also colored red and the function is exited.

Listing 4.81: Bubbles (requirement 10) – `Game.check_bubblecollision()`

```

183 def check_bubblecollision(self) -> bool:
184     for index1 in range(0, len(self.all_sprites) - 1):
185         for index2 in range(index1 + 1, len(self.all_sprites)):
186             bubble1 = self.all_sprites.sprites()[index1]
187             bubble2 = self.all_sprites.sprites()[index2]
188             if type(bubble1).__name__ == "Bubble" and type(bubble2).__name__ == "Bubble":
189                 if pygame.sprite.collide_circle(bubble1, bubble2):
190                     bubble1.update(mode="red") #
191                     bubble2.update(mode="red")
192                     return True
193             if not cfg.PLAYGROUND.contains(bubble1):
194                 bubble1.update(mode="red")
195                 return True
196             if not cfg.PLAYGROUND.contains(bubble2):
197                 bubble2.update(mode="red")
198                 return True
199     return False

```

To give me enough time to see the collision, I want to wait for 2s at the end. The method `pygame.time.wait()` pauses the application for the specified duration (line 212).

`wait()`

Listing 4.82: Bubbles (requirement 10) – waiting time in `run()`

```

201 def run(self) -> None:
202     time_previous = time()
203     self.running = True
204     while self.running:
205         self.watch_for_events()
206         self.update()
207         self.draw()
208         self.clock.tick(cfg.FPS)
209         time_current = time()
210         cfg.DELTATIME = time_current - time_previous
211         time_previous = time_current
212     pygame.time.wait(2000) # Wait a moment
213     pygame.quit()

```

4.2.11 Requirement 11: Pause

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v11

Requirement 11 Pause

The game enters or leaves pause mode by pressing the right mouse button or . The current game state is frozen and displayed in a grayed-out form.

The idea behind this requirement is that a necessary interruption should not automatically mean that the player loses the game. In figure 4.16, you can see what the pause screen should look like.

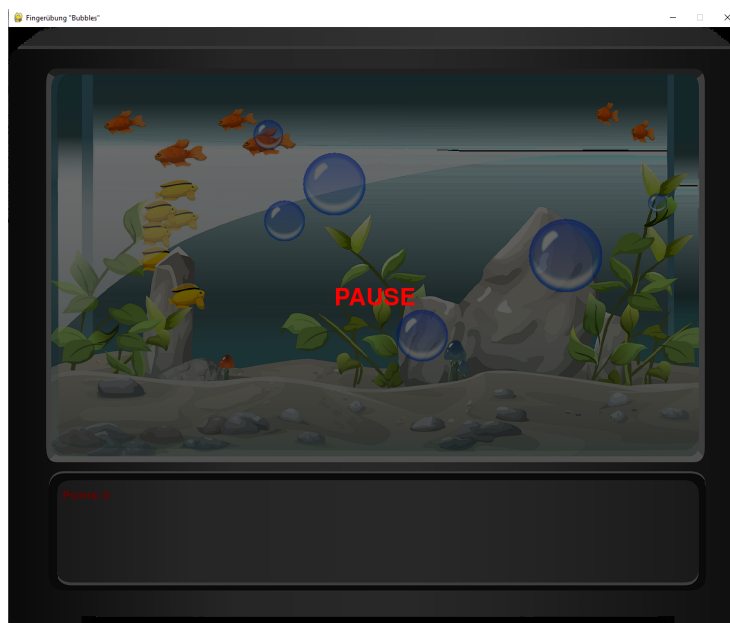


Figure 4.16: Bubbles – pause screen

In the constructor of `Game`, the flag `pausing` is defined. It later controls whether the game is currently in pause mode or not.

Listing 4.83: Bubbles (requirement 11) – constructor in `Game`

```

125         self.pausing = False                                     #

```

In `watch_for_events()`, it is now checked whether the P key (line 135) or the right mouse button (line 140) has been pressed. In both cases, the new `setpause()` method is called.

Listing 4.84: Bubbles (requirement 11) – `Game.watch_for_events()`

```

128     def watch_for_events(self) -> None:
129         for event in pygame.event.get():
130             if event.type == pygame.QUIT:
131                 self.running = False
132             elif event.type == pygame.KEYDOWN:
133                 if event.key == pygame.K_ESCAPE:
134                     self.running = False
135                 elif event.key == pygame.K_p:           #
136                     self.setpause()
137             elif event.type == pygame.MOUSEBUTTONDOWN:
138                 if event.button == 1:                   # left
139                     self.sting(pygame.mouse.get_pos())
140                 elif event.button == 3:                 # right
141                     self.setpause()

```

Für die Darstellung der Pause, habe ich die – vielleicht etwas überflüssige – Klasse `Pause` implementiert.

Listing 4.85: Bubbles (requirement 11) – Class `Pause`

```

45 class Pause(pygame.sprite.Sprite):
46     def __init__(self) -> None:
47         super().__init__()
48         imagename = cfg.get_image("pause.png")
49         self.image: pygame.Surface = pygame.image.load(imagename).convert_alpha()
50         self.rect = self.image.get_rect()

```

In the constructor of `Game`, an object of the `Pause` class is created so that it can be used in `draw()`.

Listing 4.86: Bubbles (requirement 11) – constructor in `Game`

```

126         self.msgpause = Pause()

```

However, the `setpause()` method still needs to be explained. This method either adds the `Pause` object to the list of sprites or removes it again, depending on whether the game is currently in pause mode or not. Afterwards, the boolean value of the flag is negated ([toggling](#)).

Listing 4.87: Bubbles (requirement 11) – `Game.setpause()`

```

157     def setpause(self):
158         if not self.pausing:
159             self.all_sprites.add(self.msgpause)

```

```

160         else:
161             self.msgpause.kill()
162             self.pausing = not self.pausing

```

Nothing more is required, since the rest is handled by the usual `update()` and `draw()` mechanisms.

4.2.12 Requirement 12: Restart



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v12

Requirement 12 Restart

At the end of the game, the player should be asked whether they want to restart the game or not.

The basic idea of the implementation is to define the state of the game using two flags. As with the pause feature, we need a flag that controls whether the semi-transparent foreground is placed over the game (**restarting**). This is always the case when the bubble collision check detects a collision.

flag

The other flag – `do_start` – indicates whether the player wants to restart. At the relevant points in `update()` and `draw()`, these flags are then evaluated.

The task of displaying a confirmation dialog in the foreground is essentially already solved by the `Pause` class. I can therefore generalize this class by renaming it to `Message` and passing the filename to the constructor as a string parameter (line 46).

Listing 4.88: Bubbles (requirement 12) – from `Pause` to `Message`

```

45 class Message(pygame.sprite.Sprite):
46     def __init__(self, filename: str) -> None:          #
47         super().__init__()
48         imagename = cfg.get_image(filename)
49         self.image: pygame.Surface = pygame.image.load(imagename).convert_alpha()
50         self.rect = self.image.get_rect()

```

In `Game`, starting at line 122, the adjustments required for restarting the game are implemented in the constructor. Essentially, two `Message` objects are created for pause and restart, and all attributes that need to be reset on a start or restart are handled in the new `restart()` method.

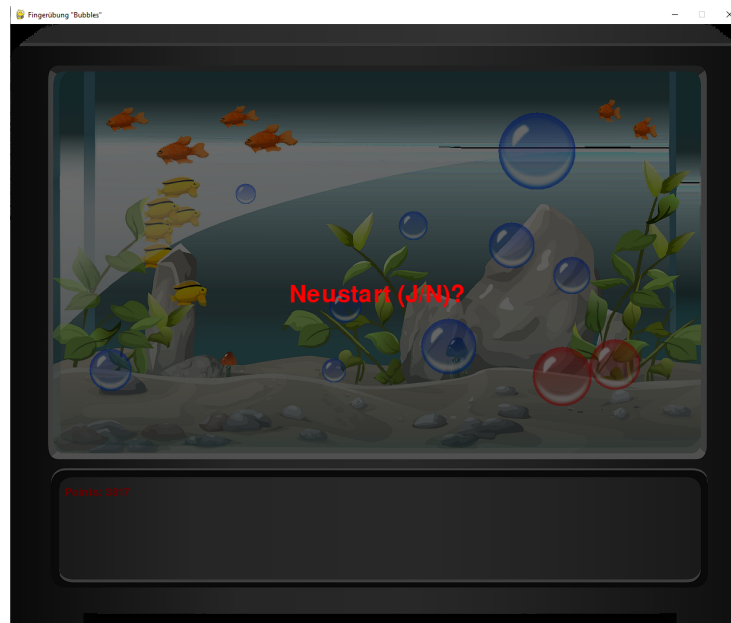


Figure 4.17: Bubbles – restart screen

Listing 4.89: Bubbles (requirement 12) – restructuring of the `Game` constructor

```

121     self.pausing = False
122     self.msg_pause = Message("pause.png")           #
123     self.msg_restart = Message("restart.png")
124     self.restart()

```

The score is reset, the sprite group containing the bubbles is cleared, the timers are reinitialized, the bubble growth speed is reset to its initial value, and the two flags described above are set to `False`.

Listing 4.90: Bubbles (requirement 12) – `Game.restart()`

```

163     def restart(self):
164         cfg.POINTS = 0
165         self.all_sprites.empty()
166         self.all_sprites.add(Points())
167         self.bubble_speed = 10
168         self.timer_bubble = Timer(500, False)
169         self.timer_bubble_speed = Timer(10000, False)
170         self.do_start = False
171         self.restarting = False

```

The method is called in `update()` when the corresponding flag `do_start` is set. In addition, `update()` inserts the restart screen into the sprite group and sets the flag `restarting` to `True` when a collision is detected.

Listing 4.91: Bubbles (requirement 12) – `Game.update()`

```

150 def update(self) -> None:
151     if self.do_start:                                # Restart?
152         self.restart()
153     if not self.pausing and self.running:
154         if self.check_bubblecollision():
155             if not self.restarting:
156                 self.all_sprites.add(self.msg_restart)
157                 self.restarting = True
158             else:
159                 self.all_sprites.update(action="grow")
160                 self.spawn_bubble()
161                 self.set_mousecursor()

```

The response to the restart screen is queried in `watch_for_events()` and translated into the corresponding flag values. If the player responds with `y` (line 135), the game has to be restarted, so `do_start` is set to `True`. If the player presses `N`, the game should end, which is why the flag of the main game loop is set to `False` (line 137).

Listing 4.92: Bubbles (requirement 12) – extension of `watch_for_events()`

```

135     elif event.key == pygame.K_j:                    #
136         self.do_start = True
137     elif event.key == pygame.K_n:                    #
138         self.running = False

```

Since we now display a semi-transparent foreground at the end of the game, there is no longer any need for a two-second pause to inspect the colliding bubbles (see figure 4.17 on the preceding page).

4.2.13 Requirement 13: Sound



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v13

Requirement 13 Sound

1. *The appearance of bubbles is accompanied by a sound.*
 2. *Popping a bubble is accompanied by a sound.*
 3. *Touching a bubble is accompanied by a sound.*
-

Finally, a small sound accompaniment is added. Similar to the bubble sprites, I do not want to lose performance by repeatedly loading sound files. Therefore, the sounds are stored in a static dictionary (line 110).

Listing 4.93: Bubbles (requirement 13) – SOUND_CONTAINER

```

109     BUBBLE_CONTAINER: Dict[str, BubbleContainer] = {}
110     SOUND_CONTAINER: Dict[str, pygame.mixer.Sound] = {} #

```

In the constructor of `Game`, the dictionary is populated with objects of the `Sound` class. The class used for sound effects is `pygame.mixer.Sound` (see line 119 ff.).

Sound

Listing 4.94: Bubbles (requirement 13) – populating SOUND_CONTAINER

```

113     pygame.init()
114     self.window = pygame.Window(size=cfg.WINDOW.size, title=cfg.CAPTION)
115     self.screen = self.window.get_surface()
116     self.clock = pygame.time.Clock()
117     Game.BUBBLE_CONTAINER["blue"] = BubbleContainer("bubble1.png")
118     Game.BUBBLE_CONTAINER["red"] = BubbleContainer("bubble2.png")
119     Game.SOUND_CONTAINER["bubble"] = pygame.mixer.Sound(cfg.get_sound("plopp1.mp3")) #
120     Game.SOUND_CONTAINER["burst"] = pygame.mixer.Sound(cfg.get_sound("burst.mp3"))
121     Game.SOUND_CONTAINER["clash"] = pygame.mixer.Sound(cfg.get_sound("glas.wav"))
122     self.background = pygame.sprite.GroupSingle(Background())

```

Now the sounds only need to be played at the appropriate places using `pygame.mixer.Sound.play()`. First, the sound that is played when a new bubble appears: in `spawn_bubble()` in line 200.

play()

Listing 4.95: Bubbles (requirement 13.1) – spawn_bubble()

```

198         if not collided:
199             self.all_sprites.add(b)
200             Game.SOUND_CONTAINER["bubble"].play() #
201             break

```

Then, when a bubble bursts in `sting()` (line 225):

Listing 4.96: Bubbles (requirement 13.2) – sting()

```

222     def sting(self, mousepos: Tuple[int, int]) -> None:
223         for bubble in self.all_sprites:
224             if self.collidepoint(mousepos, bubble):
225                 Game.SOUND_CONTAINER["burst"].play() #
226                 bubble.update(action="sting")

```

Finally, the sound is played when a collision with other bubbles or with the edge occurs in `update()`. Here, it must also be taken into account whether the game is currently displaying the restart prompt. If the answer is *yes*, the sound must not be played again; otherwise, the touch sound would be played continuously.

Listing 4.97: Bubbles (requirement 13.3) – update()

```

155     def update(self) -> None:
156         if self.do_start:
157             self.restart()
158         if not self.pausing and self.running:
159             if self.check_bubblecollision():

```

```

160         if not self.restarting:
161             Game.SOUND_CONTAINER["clash"].play() #
162             self.all_sprites.add(self.msgrestart)
163             self.restarting = True
164         else:
165             self.all_sprites.update(action="grow")
166             self.spawn_bubble()
167             self.set_mousecursor()

```

And that's it :-)

4.2.14 Or maybe not?

So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/02%20Bubbles/v14

Pause, restart, and game over are currently still implemented rather sloppily; for example, the end of the game is delayed by playing a sound instead of using a properly programmed delay. In one version (see source code 4.82 on page 296), I even froze the entire program for several seconds using `pygame.time.wait()` – brrr :-)

It is far more advantageous to control the game using states. What exactly this means should become clear when looking at the solution.

As a first step, let us think about which states the game can actually have:

- **PLAYING:** I am happily popping bubbles and collecting points.
- **PAUSED:**  has been pressed and the game should stop temporarily, but not terminate.
- **WAITING:** The game has ended, but remains active for a few more seconds, for example to display a farewell message.
- **GAME_OVER:** The game is exited. In our example, little or nothing happens here, but one might still want to close files or [sockets](#).

Let us simply add an enumeration for this:

Listing 4.98: Bubbles game state – `GameState(Enum)`

```

11 PLAYING = "playing"
12 PAUSED = "paused"
13 GAME_OVER = "game_over"
14 WAITING = "waiting"

```

Since we now have to manage more than just a single state indicating whether the game is paused or not, the attribute `self.state` is created in the constructor of `Game`. In addition, two attributes are introduced that will later be used to measure whether the game remains in a waiting state for 10s before it finally terminates.

Listing 4.99: Bubbles game state – Constructor of Game

```

134     self.msgpause = Message("pause.png")
135     self.msgrestart = Message("restart.png")
136     self.wait_start_time = None # Zeitstempel für die Wartezeit
137     self.wait_duration = 5000 # 5 Sekunden in Millisekunden

```

A real refactoring is now required for `watch_for_events()`. It no longer only checks for events, but also takes into account the current state of the game.

Consider pressing `p` as an example: if the game state is currently `PLAYING`, the game switches to pause mode by calling `set_pause()`. If the game is in the `PAUSED` state, calling `resume()` switches the game back to running mode.

Listing 4.100: Bubbles game state – `Game.watch_for_events()`

```

142     for event in pygame.event.get():
143         if event.type == pygame.QUIT:
144             self.running = False
145         elif event.type == pygame.KEYDOWN:
146             if event.key == pygame.K_ESCAPE:
147                 self.running = False
148             elif event.key == pygame.K_p:
149                 if self.state == GameState.PLAYING:
150                     self.set_pause()
151                 elif self.state == GameState.PAUSED:
152                     self.set_resume()
153             elif event.key == pygame.K_j:
154                 if self.state in [GameState.GAME_OVER, GameState.WAITING]:
155                     self.restart()
156             elif event.key == pygame.K_n:
157                 self.set_game_over()
158         elif event.type == pygame.MOUSEBUTTONDOWN:
159             if event.button == 1: # left
160                 if self.state == GameState.PLAYING:
161                     self.sting(pygame.mouse.get_pos())
162             elif event.button == 3: # right
163                 if self.state == GameState.PLAYING:
164                     self.set_pause()
165                 elif self.state == GameState.PAUSED:
166                     self.set_resume()

```

Let us now take a closer look at the three helper methods, each of which is tailored to a specific state: `set_pause()`, `set_resume()`, and `set_game_over()`. All three methods consist of two parts: in the first part, the state change itself is performed, and in the second part, an action required by this state change is executed. In some cases, a message screen is added to the sprite group; in others, it is removed again. In the third method, the time at which the 10 seconds waiting period begins is stored in `wait_start_time`.

Listing 4.101: Bubbles game state – `set_pause()`, `set_resume()`, and `set_game_over()` of Game

```

201     def set_pause(self) -> None:
202         self.state = GameState.PAUSED
203         self.all_sprites.add(self.msgpause)
204
205     def set_resume(self) -> None:

```

```

206         self.state = GameState.PLAYING
207         self.msgpause.kill()
208
209     def set_game_over(self) -> None:
210         self.state = GameState.WAITING
211         self.wait_start_time = pygame.time.get_ticks()

```

Just as the game states are taken into account in `watch_for_events()`, they must also be considered in `update()`.

Listing 4.102: Bubbles game state – `Game.update()`

```

174     if self.state == GameState.PLAYING:
175         if self.check_bubblecollision():
176             if not self.restarting:
177                 Game.SOUND_CONTAINER["clash"].play()
178                 self.all_sprites.add(self.msgrestart)
179                 self.restarting = True
180                 self.set_game_over()
181             else:
182                 self.all_sprites.update(action="grow")
183                 self.spawn_bubble()
184                 self.set_mousecursor()
185     elif self.state == GameState.WAITING:
186         self.check_waiting_timeout()
187     elif self.state == GameState.GAME_OVER:
188         self.running = False

```

What remains is the helper method that checks whether the 10 seconds have elapsed: `check_waiting_timeout()`. In this method, the elapsed time is compared with `wait_duration`. If the time has elapsed, the game state is set to `GAME_OVER`, so that the game can terminate cleanly.

Listing 4.103: Bubbles game state – `Game.check_waiting_timeout()`

```

274     if self.wait_start_time is not None:
275         elapsed_time = pygame.time.get_ticks() - self.wait_start_time
276         if elapsed_time >= self.wait_duration:
277             self.state = GameState.GAME_OVER
278             self.wait_start_time = None

```

In figure 4.18 on the next page, we can once again visually trace these state transitions.

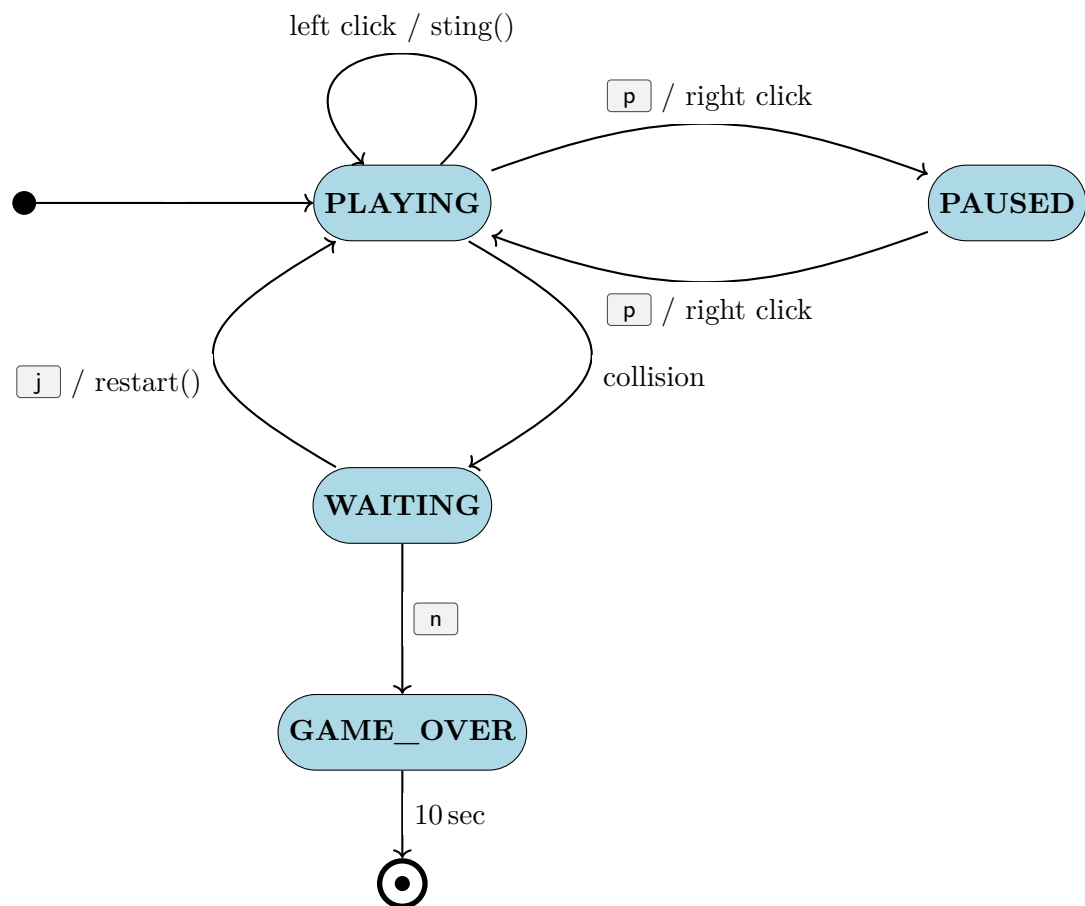


Figure 4.18: Bubbles – state diagram

4.3 Moonlander

In this chapter, we will build a Moon Lander game. For this project, I want to avoid pre-made sprites and create all graphics entirely from drawing primitives (see section 2.2 on page 28).

We will develop this game systematically, step by step, assuming that the techniques from chapter 2 are already familiar. I will omit docstring comments in the source code, since everything is explained in the text and the listings would otherwise become unnecessarily long. In the final version, they are included.

4.3.1 Requirement 1: Standards



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v01

Requirement 1 Standard functionality

1. *The window has a size of 600×800 px.*
 2. *The background is divided into a black sky, a blue Earth in the upper-right corner, and the lunar surface.*
 3. *The game can be exited using `[Esc]` or by clicking the red X.*
 4. *Pressing `[R]` triggers a restart.*
 5. *The game runs at a speed independent of the frame rate.*
-

Requirement 1.1 is already defined in the preamble. In addition, FPS and the associated DELTATIME are defined in `config.py`. The constant HORIZONT specifies where the lunar surface ends and the black night sky begins.

Listing 4.104: Moonlander (requirement 1.1) – `config.py`

```

1 import pygame
2
3 WINDOW = pygame.Rect(0, 0, 600, 800)
4 FPS = 60
5 DELTATIME = 1.0 / FPS
6 HORIZONT = 50

```

I implement requirement 1.2 using three classes: `Sky`, `Moon`, and `Earth`. Let us start with the `Sky` class. It has a fairly simple basic structure. In the constructor, a reference to the window is passed in and the size of the sky is stored as a `Rect` object. Space is left

at the bottom for the lunar surface. The `draw()` method then draws a black rectangle at the appropriate position.

Listing 4.105: Moonlander (requirement 1.2) – Class Sky

```

7 class Sky:
8     def __init__(self, screen:pygame.Surface) -> None:
9         top = 0
10        left = 0
11        width = cfg.WINDOW.width
12        height = cfg.WINDOW.height - cfg.HORIZONT
13        self.rect = pygame.Rect(top, left, width, height)
14        self.screen = screen
15
16    def draw(self) -> None:
17        pygame.draw.rect(self.screen, "black", self.rect)

```

The Moon class works in exactly the same way (see source code 4.106). The only differences are the different position and the different color – gray in this case.

Listing 4.106: Moonlander (requirement 1.2) – Class Moon

```

19 class Moon:
20     def __init__(self, screen:pygame.Surface) -> None:
21         top = 0
22         left = cfg.WINDOW.height - cfg.HORIZONT
23         width = cfg.WINDOW.width
24         height = cfg.HORIZONT
25         self.rect = pygame.Rect(top, left, width, height)
26         self.screen = screen
27
28
29    def draw(self) -> None:
30        pygame.draw.rect(self.screen, "gray", self.rect)

```

The Earth class draws a blue sphere in the upper-right corner of the screen (see source code 4.107).

Listing 4.107: Moonlander (requirement 1.2) – Class Earth

```

32 class Earth:
33     def __init__(self, screen:pygame.Surface) -> None:
34         self.radius = 80
35         left = cfg.WINDOW.right - 2*self.radius - 30
36         top = cfg.WINDOW.top + 15
37         width = 2*self.radius
38         height = 2*self.radius
39         self.rect = pygame.Rect(left, top, width, height)
40         self.screen = screen
41
42
43    def draw(self) -> None:
44        pygame.draw.circle(self.screen, "blue", self.rect.center, self.radius)

```

As usual, the game is encapsulated in its own class: **Game**. The three objects only need to be integrated into the standard structure of **Game**. This is also where quitting and restarting the game are implemented.

In source code 4.107 on the preceding page, Pygame is initialized in the constructor of `Game`, a window is created, the window's screen surface is obtained, and a `Clock` object is created for the delta-time logic (see section 2.4.3.1 on page 64).

Listing 4.108: Moonlander (requirement 1) – Constructor of `Game`

```

46 class Game:
47     def __init__(self) -> None:
48         pygame.init()
49         self.window = pygame.Window(size=cfg.WINDOW.size, title="MyMoonlander",
50                                     position=pygame.WINDOWPOS_CENTERED)
51         self.screen = self.window.get_surface()
52         self.clock = pygame.time.Clock()

```

The structure of the `run()` method follows the examples shown above. Its core consists of calling the event handler, updating the game objects, and drawing the game objects; in addition, the delta-time logic is applied.

Listing 4.109: Moonlander (requirement 1) – `Game.run()`

```

54 def run(self) -> None:
55     self.restart()
56     time_previous = time()
57     while self.running:
58         self.watch_for_events()
59         self.update()
60         self.draw()
61         self.clock.tick(cfg.FPS)
62         time_current = time()
63         cfg.DELTATIME = time_current - time_previous
64         time_previous = time_current
65     pygame.quit()

```

The event handler should no longer come as a surprise. Using `QUIT` or `Esc` ends the game, and pressing `r` triggers a restart.

Listing 4.110: Moonlander (requirement 1) – `Game.watch_for_events()`

```

67 def watch_for_events(self) -> None:
68     for event in pygame.event.get():
69         if event.type == pygame.QUIT:
70             self.running = False
71         elif event.type == pygame.KEYDOWN:
72             if event.key == pygame.K_ESCAPE:
73                 self.running = False
74             elif event.key == pygame.K_r:
75                 self.restart()

```

At the moment, the `update()` method serves only as a placeholder for functionality that will be added later.

Listing 4.111: Moonlander (requirement 1) – `Game.update()`

```

77 def update(self) -> None:
78     pass

```

In `draw()`, the drawing methods of the game objects are called, and the window buffer is flipped.

Listing 4.112: Moonlander (requirement 1) – `Game.draw()`

```

80     def draw(self) -> None:
81         self.background.draw()
82         self.moon.draw()
83         self.earth.draw()
84         self.window.flip()

```

The restart does not reset the state of the individual game objects. Instead, the objects are recreated entirely. This is the simplest way to implement a restart, but it is not suitable for every type of game.

Listing 4.113: Moonlander (requirement 1) – `Game.restart()`

```

86     def restart(self) -> None:
87         self.background = Sky(self.screen)
88         self.moon = Moon(self.screen)
89         self.earth = Earth(self.screen)
90         self.running = True

```

All that remains is the actual call that starts the game.

Listing 4.114: Moonlander (requirement 1) – `main()`

```

92     def main():
93         Game().run()
94
95     if __name__ == "__main__":
96         main()

```

After starting the program, a scene like the one shown in figure 4.19 appears.

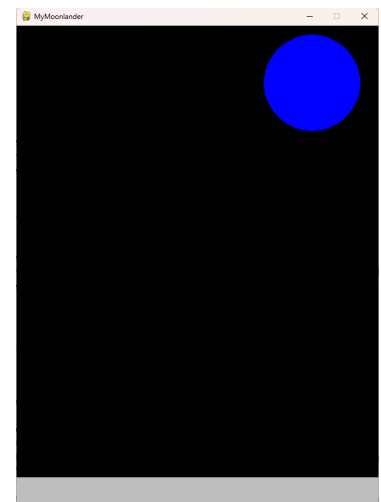


Fig. 4.19: Moonlander (1)

4.3.2 Requirement 2: Lunar surface

So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v02

So far, the lunar surface is just a gray rectangle. However, I want a gray mountainous landscape to enhance the visual appeal.

Requirement 2 Lunar surface

The lunar surface consists of consecutively arranged mountain ranges.

As a first step, I extend the constructor of `Moon` by adding the number of mountain ranges. Each mountain range is initially represented by a gray rectangle. The variations in height will be added later.

The actual lunar surface (landing area in line 25) remains a rectangle with a height of `HORIZONTAL`. In `self.layers`, the information for each mountain range is stored as a list.

Starting at line 28, the mountain ranges (the default is 5) are created. First, the color of each mountain range is defined (line 29). Starting from a base color value of 180, an amount depending on the layer index is subtracted. The larger the layer index, the more is subtracted from 180. In terms of color, this means that the mountain range becomes darker. The farther away a mountain range (the layer) is, the darker it appears.

The height of a mountain range (`y`) is calculated by moving upward at least 10px from the upper edge of the landing area. This value is then increased by a random number between 5 and 30, so that the heights of the mountain ranges are not always the same. To ensure that the mountain ranges in the background always stand out nicely, this value is additionally multiplied by the layer index. Finally, `draw()` is adjusted (source code 4.116 on the following page) so that the mountain ranges are drawn as rectangles. The lunar surface should look roughly like the one shown in figure 4.20.



Fig. 4.20: Moonlander (2)

Listing 4.115: Moonlander (requirement 2) – Constructor of `Moon`

```

20 class Moon:
21     def __init__(self, screen: pygame.Surface, layer_count: int=5):
22         self.screen = screen
23         top = cfg.WINDOW.height - cfg.HORIZONTAL
24         self.rect = pygame.Rect(0, top,
25                                 cfg.WINDOW.width, cfg.HORIZONTAL)    # Landing area
26
27         self.layers = []

```

```

28     for layer_index in range(layer_count): # Mountain range
29         mycolor = 180 - layer_index * 20 # Foreground darker, background lighter
30         self.layers.append({"y":self.rect.top - 10 - randint(5, 30)*layer_index,
31                             "color":(mycolor, mycolor, mycolor)})

```

Listing 4.116: Moonlander (requirement 2) – Moon.draw()

```

33     def draw(self):
34         pygame.draw.rect(self.screen, (230, 230, 230), self.rect)
35
36         for layer in reversed(self.layers):
37             r = pygame.Rect()
38             r.top = layer["y"]
39             r.left = self.rect.left
40             r.width = self.rect.width
41             r.height = self.rect.top - r.top
42             pygame.draw.rect(self.screen, layer["color"], r)

```

Now it is time to add the mountain peaks. The basic idea is to generate random height variations around the upper edge of each mountain range and subtract them from the height of that upper edge.

As a first step, the constructor of Moon is extended by the parameter `peaks`. In line 28, the distance between two height variations is calculated and stored in `dist`. This value could also be randomized further, but for some reason I did not feel like doing that here.

Now, within the loop, a peak or a valley is generated for each mountain range. The determination of the color (the shade of gray) remains unchanged. In `lofPeaks`, the peaks are stored as a list of points. The first point is always located at the far left on the upper edge of the landing area (line 33). This point serves as the starting point of our closed polygon.



Fig. 4.21: Moonlander (3)

After that, the list of peaks is extended with additional random points using a loop. In line 35, a height variation between -5 px and 10 px is chosen at random and subtracted from the upper edge. In the following line, the next peak is shifted to the right by adding `dist`. Once this inner `for`-loop has finished, the list of height points is complete and can be added to the corresponding layer in line 38. Before doing so, however, the final point of the polygon chain must still be generated and added.

Listing 4.117: Moonlander (requirement 2) – constructor of Moon with peaks

```

20     class Moon:
21         def __init__(self, screen: pygame.Surface, layer_count:int=5, peaks: int=35):
22             self.screen = screen
23             top = cfg.WINDOW.height - cfg.HORIZONT
24             self.rect = pygame.Rect(0, top,
25                                     cfg.WINDOW.width, cfg.HORIZONT) # Landing area
26
27             self._layers = []
28             dist = self.rect.width // peaks # Distance between height differences
29             for layer_index in range(layer_count): # Build mountain
30                 mycolor = 180 - layer_index * 20 # Foreground darker, background lighter

```

```

31     y = self.rect.top - 10 - randint(5, 30)*layer_index # Random starting height
32     x = self.rect.left                                # First peak starts at the left
33     lofPeaks = [(x, top)]                             # The first peak as a point
34     for i in range(peaks):                             # The other peaks of the layer are generated.
35         lofPeaks.append((x, y + randint(-5, 10))) # Random height difference
36         x += dist                                     # The next peak is further to the right
37     lofPeaks.append((self.rect.right, y))             # Last peak is at the right
38     lofPeaks.append((self.rect.right, top))           # Base of the mountain range
39     self._layers.append({"color":(mycolor, mycolor, mycolor),
40                         "peaks":lofPeaks})

```

The `draw()` method has now become pleasantly simple. For each mountain range, `draw.polygon()` is called; the actual work is done in the constructor. The result can be admired in figure 4.21 on the facing page.

Listing 4.118: Moonlander (requirement 2) – `Moon.draw()` with peaks

```

42     def draw(self):
43         pygame.draw.rect(self.screen, (230, 230, 230), self.rect)
44         for layer in reversed(self._layers):
45             pygame.draw.polygon(
46                 self.screen,
47                 layer["color"],
48                 layer["peaks"]
49             )

```

Finally, I want to give the mountains a bit more contour. To do this, the single polygon is split into many smaller ones, where each polygon spans from one peak to the next. First, it becomes apparent that the number of peaks now varies for each mountain range (line 29); this makes the ranges appear less like a checkerboard. Of course, the distance between the peaks then also has to be recalculated (line 30).



Fig. 4.22: Moonlander (4)

To make the source code easier to understand, I separated the generation of the peaks from the calculation of the corresponding polygons. Something genuinely new only happens starting at line 41. For each peak, four points are now determined: the starting peak, the peak to its right, the point directly below it down to the surface, and finally a point on the surface back to the left underneath the starting peak. In addition, a subtle shade of gray is chosen at random. These four points are stored as a polygon together with the corresponding color in the list `layers`.

Listing 4.119: Moonlander (requirement 2) – constructor of `Moon` with contour

```

20     class Moon:
21         def __init__(self, screen: pygame.Surface, layer_count:int=5, peaks: int=35):
22             self.screen = screen
23             top = cfg.WINDOW.height - cfg.HORIZONT
24             self.rect = pygame.Rect(0, top,
25                                     cfg.WINDOW.width, cfg.HORIZONT)
26
27             self._layers = []
28             for layer_index in range(layer_count):
29                 mypeaks = randint(peaks//2, peaks) # Number varies
30                 dist = self.rect.width // mypeaks # Distance between height differences

```

```

31         mycolor = 180 - layer_index * 20
32         y = self.rect.top - 10 - randint(5, 30)*layer_index
33         x = self.rect.left
34         lofPeaks = [(x, top)]
35         for i in range(mypeaks):
36             lofPeaks.append((x, y + randint(-5, 20)))
37             x += dist
38         lofPeaks.append((self.rect.right, y))
39         lofPeaks.append((self.rect.right, top))
40
41         poly = [] # A polygon path
42         for index in range(len(lofPeaks)-1):
43             p1 = lofPeaks[index]
44             p2 = lofPeaks[index+1]
45             p3 = (lofPeaks[index+1][0], self.rect.top)
46             p4 = (lofPeaks[index][0], self.rect.top)
47             r = randint(-5,5)
48             c = [mc + r for mc in (mycolor, mycolor, mycolor)]
49             poly.append({"points":(p1, p2, p3, p4), "color":c})
50         self._layers.append(poly)

```

After a few more minor and easy-to-understand changes in `draw()`, the Moon is complete (figure 4.22 on the previous page).

Listing 4.120: Moonlander (requirement 2) – Moon.draw() with contour

```

52     def draw(self):
53         pygame.draw.rect(self.screen, (230, 230, 230), self.rect)
54         for layer in reversed(self._layers):
55             for poly in layer:
56                 pygame.draw.polygon(
57                     self.screen,
58                     poly["color"],
59                     poly["points"])

```

Redrawing the mountain ranges from scratch every time is certainly a huge waste of computing time. A common technique this is to draw the image once onto a bitmap (`pygame.surface.Surface`) and then simply blit this bitmap each frame.

Note that `rect` is now needed for the entire bitmap and has therefore first been renamed to `landingarea`. It is also no longer necessary to keep `layers` and `landingarea` as attributes of the class, since this information is no longer required after the bitmap has been created.

Listing 4.121: Moonlander (requirement 2) – Moon as bitmap

```

20 class Moon:
21     def __init__(self, screen: pygame.Surface, layer_count:int=5, peaks: int=35):
22         self.screen = screen
23         self.surface = pygame.Surface((cfg.WINDOW.width,
24                                         cfg.HORIZONT + layer_count*30),
25                                         pygame.SRCALPHA)
26
27         self.rect = self.surface.get_rect()
28         self.rect.left = cfg.WINDOW.left
29         self.rect.bottom = cfg.WINDOW.bottom
30         landingarea = pygame.Rect(0, self.rect.height - cfg.HORIZONT,
31                                     cfg.WINDOW.width, cfg.HORIZONT)
32
33         layers = []

```

```

33     for layer_index in range(layer_count):
34         mypeaks = randint(peaks//2, peaks)
35         dist = landingarea.width // mypeaks
36         mycolor = 180 - layer_index * 20
37         y = landingarea.top - 10 - randint(5, 30)*layer_index
38         x = landingarea.left
39         lofPeaks = [(x, landingarea.top)]
40         for i in range(mypeaks):
41             lofPeaks.append((x, y + randint(-5, 20)))
42             x += dist
43         lofPeaks.append((landingarea.right, y))
44         lofPeaks.append((landingarea.right, landingarea.top))
45
46         poly = []
47         for index in range(len(lofPeaks)-1):
48             p1 = lofPeaks[index]
49             p2 = lofPeaks[index+1]
50             p3 = (lofPeaks[index+1][0], landingarea.top)
51             p4 = (lofPeaks[index][0], landingarea.top)
52             r = randint(-5,5)
53             c = [mc + r for mc in (mycolor, mycolor, mycolor)]
54             poly.append({"points":(p1, p2, p3, p4), "color":c})
55         layers.append(poly)
56
57     pygame.draw.rect(self.surface, (230, 230, 230), landingarea)
58     for layer in reversed(layers):
59         for poly in layer:
60             pygame.draw.polygon(
61                 self.surface,
62                 poly["color"],
63                 poly["points"])
64
65     def draw(self):
66         self.screen.blit(self.surface, self.rect.topleft)

```

4.3.3 Requirement 3: Earth



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v02

The Earth as a simple blue spot? That would be far too unattractive!

Requirement 3 Earth

1. *The Earth should have an atmospheric glow.*
 2. *Landmasses should be visible on the Earth.*
-

First, the Earth is also converted into a bitmap in order to improve performance. The procedure is analogous to source code 4.121 on the facing page.

Listing 4.122: Moonlander (requirement 3) – Earth as bitmap

```

68 class Earth:
69     def __init__(self, screen:pygame.Surface) -> None:
70         self.radius = 80
71         self.surface = pygame.Surface(
72             (2*self.radius, 2*self.radius),
73             pygame.SRCALPHA)
74         self.rect = self.surface.get_rect()
75         self.rect.left = cfg.WINDOW.right - 200
76         self.rect.top = cfg.WINDOW.top + 50
77         self.screen = screen
78
79         pygame.draw.circle(self.surface,
80                             (30, 144, 255),
81                             (self.radius, self.radius),
82                             self.radius)
83
84     def draw(self) -> None:
85         self.screen.blit(self.surface, self.rect.topleft)

```

Next, the atmospheric glow is created. The basic idea is to draw circles from the inside to the outside with increasing transparency. To achieve this, a loop counts down from 20 to 1. This counter is multiplied by 10 in line 81 and subtracted from 210, resulting in a sequence like (10, 20, ..., 200). Correspondingly, the radius of these circles increases steadily in line 83. Finally, the slightly reduced Earth itself is drawn (see figure 4.23 on the next page).

Listing 4.123: Moonlander (requirement 3.1) – Earth with atmospheric glow

```

79 for a in range(20, 1, -1):
80     pygame.draw.circle(self.surface,
81                         (135, 206, 250, 210-a*10), # Steigende Transparenz
82                         (self.radius, self.radius),
83                         self.radius-20+a)           # Wachsender Radius
84     pygame.draw.circle(self.surface, (30, 144, 255, 255),
85                         (self.radius, self.radius), self.radius-20)

```

I had the polygon data for the landmasses generated by ChatGPT (what a blessing!). For the sake of readability, this data has been moved to an external file (`continent_polygons.py`). It consists of a list of lists of points. The inner lists represent the landmasses as closed polygon paths. First, the polygon data is imported as a module:

Listing 4.124: Moonlander (requirement 3.2) – importing the polygon data

```

6 from continent_polygons import continent_polygons

```

The actual drawing is then fairly straightforward. The coordinates only need to be aligned with the center of the Earth and scaled to half the size so that they fit inside the circle (see figure 4.24 on the facing page).

Listing 4.125: Moonlander (requirement 3.2) – Earth with continents

```

90     for continent in continent_polygons:
91         poly = [(self.radius + (0.5*x), self.radius + (0.5*y)) for (x, y) in continent]
92         pygame.draw.polygon(self.surface, (181, 150, 116), poly)

```

That will be enough for the Earth. On to the next effect.

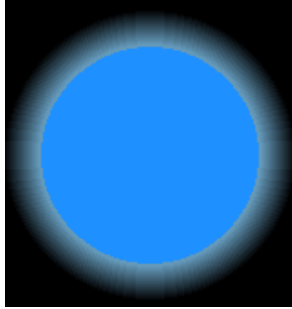


Figure 4.23: Moonlander (5) glowing



Figure 4.24: Moonlander (6) continent

4.3.4 Requirement 4: Stars

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v02

Outer space is neither black nor empty.

Requirement 4 Stars

1. Stars of different sizes should be visible in the background.
2. The stars should change in brightness and size. This should create a kind of twinkling effect.

First, the constructor of `Sky` is extended by a parameter specifying the number of stars; the default value is 200 stars. In line 18, a list for the stars is created. Afterwards, the loop fills this list with the corresponding number of entries. Position, size, and color are determined randomly.

Listing 4.126: Moonlander (requirement 4.1) – Constructor of `Sky`

```

9 class Sky:
10     def __init__(self, screen:pygame.Surface, star_count: int=200) -> None:
11         top = 0

```

```

12     left = 0
13     width = cfg.WINDOW.width
14     height = cfg.WINDOW.height - cfg.HORIZONT
15     self.rect = pygame.Rect(top, left, width, height)
16     self.screen = screen
17
18     self.stars = []                                # Sternenliste
19     for _ in range(star_count):
20         self.stars.append({"pos":(randint(2, self.rect.right-1),
21                                randint(2, self.rect.right-1)),
22                                "size":randint(1, 3),
23                                "color":randint(10, 255)})

```

In `draw()`, the entries of the list are used to render the stars.

Listing 4.127: Moonlander (requirement 4.1) – `Sky.draw()`

```

25     def draw(self) -> None:
26         pygame.draw.rect(self.screen, "black", self.rect)
27         for star in self.stars:
28             pygame.draw.circle(self.screen, (255,255,star["color"]), star["pos"],
                                   star["size"])

```

Creating the twinkling effect is a bit more interesting. As preparation, each star is assigned a random value in line 23 that specifies after how many frames a change in brightness should occur. At 60 fps, this corresponds to roughly 3.3 to 10 s.

Listing 4.128: Moonlander (requirement 4.2) – twinkling stars (1)

```

18     self.stars = []
19     for _ in range(star_count):
20         self.stars.append({"pos":(randint(2, self.rect.right-1),
21                                randint(2, self.rect.right-1)),
22                                "size":randint(1, 3),
23                                "duration": randint(200, 600), # Time in frames
24                                "counter":0,
25                                "color":randint(10, 255)})

```

Since the state of the game object now changes over time, the `update()` method is required. This method recalculates the brightness and size of the stars. In line 29, a counter is increased by 1 in every frame (that is, on each call). The value is then processed using the modulo operator. In this way, the counter always stays within a fixed range, preventing an overflow — that is, exceeding the valid range of an integer.

Within the loop, all stars are now processed. If the value of `counter` modulo `duration` is 0, exactly `duration` frames have passed and the color and size must be updated. The `draw()` method remains unchanged.

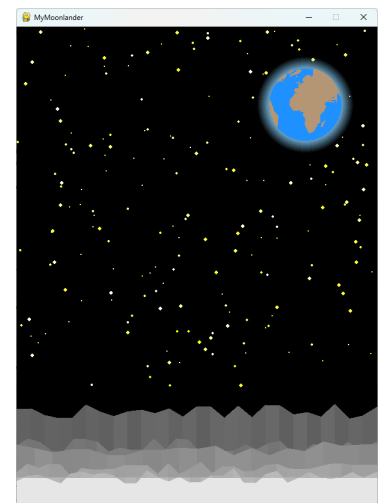


Fig. 4.25: Moonlander (7)

Listing 4.129: Moonlander (requirement 4.2) – twinkling stars (2)

```

27 def update(self) -> None:
28     for star in self.stars:
29         star["counter"] = (star["counter"] + 1) % (star["duration"] + 1) # Counter
30         if star["counter"] == 0:
31             star["color"] = (star["color"] + randint(0, 70)) % 256
32             star["size"] = (star["size"] + 1) % 4

```

Finally, the previously unused `update()` method in `Game` is extended with the corresponding method call, and everything should work as expected (see figure 4.25 on the preceding page).

Listing 4.130: Moonlander (requirement 4.2) – twinkling stars (3) `Game.update()`

```

147 def update(self) -> None:
148     self.background.update()

```

4.3.5 Requirement 5: Lander



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v03

Requirement 5 Lander

1. The lander consists of an antenna, a crew module, a base with connectors to the crew module, and landing legs with pads.
 2. Pressing the Space key displays a thrust exhaust.
 3. The lander starts roughly in the middle and fairly high up, but not directly at the top edge.
-

The first thing to notice is that I do not use just a single `Surface` object, but two. The idea behind this is to create one sprite for the lander with thrust and one without thrust. In `draw()`, the attribute `thrusting` (line 126) is then used to control which of the two sprites is blitted to the screen. For the sake of clarity, the drawing of the lander is encapsulated in the method `create_lander()` (line 125).

Listing 4.131: Moonlander (requirement 5.1) – Constructor of Lander

```

117 class Lander:
118     def __init__(self, screen: pygame.Surface) -> None:
119         self.screen = screen
120         self.surface = pygame.Surface((90,81), pygame.SRCALPHA)

```

```

121     self.surface_thrusting = pygame.Surface((90,81), pygame.SRCALPHA)
122     self.rect = self.surface.get_frect()
123     self.rect.centerx = cfg.WINDOW.centerx           # horizontal start position
124     self.rect.top = self.rect.height                 # vertical start position
125     self.create_lander()                             # Drawing is delegated
126     self.thrusting = False                           # Flag whether acceleration is active

```

Explaining each individual drawing step would certainly be somewhat tedious and would not provide much additional learning value. The easiest way to understand the source code is to change individual details and observe the effect. Nevertheless, I would like to address one specific aspect.

In the first step, all drawing operations are performed on the surface `surface`. This results in a lander without thrust. Starting at line 126, the surface with thrust is created. For this purpose, the lander without thrust is first copied onto `surface_thrusting` using `blit()`. After that, an additional thrust flame is drawn onto `surface_thrusting`. As a result, two `Surface` objects are available for rendering the lander. Both can be seen in figure 4.26 and figure 4.27 on the facing page.

Listing 4.132: Moonlander (requirement 5.1) – `Lander.create_lander()`

```

128     def create_lander(self) -> None:
129         # A few abbreviations
130         cx = self.rect.width // 2
131         cy = self.rect.height // 2
132         s = self.rect.width // 2
133         sur = self.surface
134
135         # Antenna
136         pygame.draw.line(sur, (220, 220, 220), (cx, cy - s//2), (cx, cy - s//1.2), 2)
137         pygame.draw.circle(sur, (255, 255, 255), (cx, cy - s//1.2), 3)
138
139         # Upper crew module (narrower)
140         pygame.draw.polygon(
141             sur, (160, 160, 160),
142             [(cx - s//4, cy - s//2),
143              (cx - s//6, cy - s//3),
144              (cx + s//6, cy - s//3),
145              (cx + s//4, cy - s//2)]
146         )
147
148         # Connector between base and crew module
149         conn_color = (160, 160, 160)
150         pygame.draw.line(sur, conn_color, (cx - s//3, cy), (cx - s//6, cy - s//3), 2)
151         pygame.draw.line(sur, conn_color, (cx, cy), (cx, cy - s//3), 2)
152         pygame.draw.line(sur, conn_color, (cx + s//3, cy), (cx + s//6, cy - s//3), 2)
153
154         # Module base (central capsule, light gray)
155         pygame.draw.polygon(
156             sur, (200, 200, 200),
157             [(cx - s//3, cy),
158              (cx - s//2, cy + s//2),
159              (cx + s//2, cy + s//2),
160              (cx + s//3, cy)]
161         )
162
163         # Windows in module
164         r = 5
165         window_color = (50, 50, 50)
166         pygame.draw.circle(sur, window_color, (cx, cy+(s//4)), r)

```

```

167     pygame.draw.circle(sur, window_color, (cx-(s//4), cy+(s//4)), r)
168     pygame.draw.circle(sur, window_color, (cx+(s//4), cy+(s//4)), r)
169
170     # Landing legs
171     leg_color = (100, 100, 100)
172     pygame.draw.line(sur, leg_color, (cx - s//2, cy + s//2), (cx - s, cy + s), 3)
173     pygame.draw.line(sur, leg_color, (cx + s//2, cy + s//2), (cx + s, cy + s), 3)
174     pygame.draw.line(sur, leg_color, (cx - s//4, cy + s//2), (cx - s//3, cy + s), 3)
175     pygame.draw.line(sur, leg_color, (cx + s//4, cy + s//2), (cx + s//3, cy + s), 3)
176
177     # Feet
178     feet_color = (150, 150, 150)
179     pygame.draw.circle(sur, feet_color, (cx - s + (r+2), cy + s - (r+2)), r-1)
180     pygame.draw.circle(sur, feet_color, (cx + s - (r+2), cy + s - (r+2)), r-1)
181     pygame.draw.circle(sur, feet_color, (cx - s//3 + (r-4), cy + s - (r+2)), r-1)
182     pygame.draw.circle(sur, feet_color, (cx + s//3 - (r-4), cy + s - (r+2)), r-1)
183
184     # Thruster exhaust
185     self.surface_thrusting.blit(sur, (0,0))
186     pygame.draw.polygon(self.surface_thrusting, (255, 140, 0), [
187         (cx - 5, cy + s//2),
188         (cx + 5, cy + s//2),
189         (cx, cy + s//2 + 20)
190     ])

```



Figure 4.26: Moonlander (8) – lander without thrust



Figure 4.27: Moonlander (9) – lander with thrust

In `update()`, the `thrusting` flag is controlled and set from outside the class.

Listing 4.133: Moonlander (requirement 5.1) – `Lander.update()`

```

192     def update(self, *args: Any, **kwargs: Any) -> None:
193         if "action" in kwargs.keys():
194             if kwargs["action"] == "thrust":
195                 self.thrusting = True
196             elif kwargs["action"] == "unthrust":
197                 self.thrusting = False

```

In `Game`, `watch_for_events()` must be adapted to handle the thrust control. Pressing activates the lander's *thrust* mode, and releasing the key deactivates it again.

Listing 4.134: Moonlander (requirement 5.2) – `Game.watch_for_events()`

```

225     def watch_for_events(self) -> None:
226         for event in pygame.event.get():
227             if event.type == pygame.QUIT:
228                 self.running = False
229             elif event.type == pygame.KEYDOWN:
230                 if event.key == pygame.K_ESCAPE:

```

```

231         self.running = False
232     elif event.key == pygame.K_r:
233         self.restart()
234     elif event.key == pygame.K_SPACE:
235         self.lander.update(action="thrust")
236 elif event.type == pygame.KEYUP:
237     if event.key == pygame.K_SPACE:
238         self.lander.update(action="unthrust")

```

In `Lander.draw()`, the output switches between the two surfaces depending on the current *thrust* mode.

Listing 4.135: Moonlander (requirement 5.2) – `Lander.draw()`

```

199 def draw(self) -> None:
200     if self.thrusting:
201         self.screen.blit(self.surface_thrusting, self.rect.topleft)
202     else:
203         self.screen.blit(self.surface, self.rect.topleft)

```

All that remains is to define the starting position, which is fairly simple. In the constructor of `Lander`, the appropriate position is determined (see line 123 and line 124). The result should look like the one shown in figure 4.28.

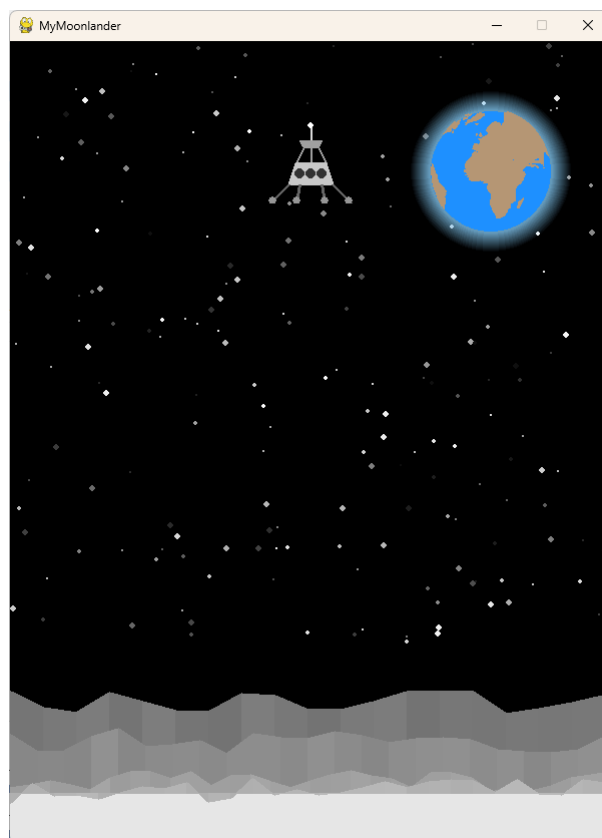


Figure 4.28: Moonlander (10) – the lander

4.3.6 Requirement 6: Gravitation and landing



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v04

Requirement 6 Gravitation and landing

1. *The lander is accelerated by the Moon's gravity at 1.62 m/s^2 .*
 2. *When the pads of the landing legs touch the lunar surface, the lander comes to a stop.*
-

For this purpose, several parameters are defined in `config.py`, starting at line 7. I included both lunar and Earth gravity. Of course, you are completely free to let the lander touch down on Venus or even Jupiter instead.

Listing 4.136: Moonlander (requirement 6.1) – physical constants

```

3 WINDOW = pygame.Rect(0, 0, 600, 800)
4 FPS = 60
5 DELTATIME = 1.0 / FPS
6 HORIZONT = 50
7 # Physical constants (Moon conditions)
8 MOON_GRAVITY = 1.62                # m/s²
9 EARTH_GRAVITY = 9.81              # m/s²
10 PIXELS_PER_METER = 10            # Scaling 1m = 10px
11 GRAVITY = MOON_GRAVITY * PIXELS_PER_METER # = 16.2 px/s²

```

In the constructor of the lander, its vertical velocity is defined in line 127. At the start of the game, this value is always set to 0. This is admittedly unrealistic, since the lander is already in the middle of its descent – but let's not worry about that for now.

Listing 4.137: Moonlander (requirement 6.1) – extension of the `Lander` constructor

```

127         self.velocity = 0          #

```

The `update()` method of `Lander` is extended by adding the action `move`. The actual calculation of the new position is encapsulated in the method `move()`.

Listing 4.138: Moonlander (requirement 6.1) – extension of `Lander.update()`

```

199         elif kwargs["action"] == "move":
200             self.move()

```

First, the new velocity is calculated based on gravity. After that, the change in position is computed using this velocity. If the lower boundary is crossed (the pads of the landing legs), the lander is aligned with the lunar surface and then remains there.

Listing 4.139: Moonlander (requirement 6.2) – `Lander.move()`

```

208 def move(self) -> None:
209     self.velocity += cfg.GRAVITY * cfg.DELTATIME
210     self.rect.top += self.velocity * cfg.DELTATIME
211     if self.rect.bottom >= cfg.WINDOW.bottom - cfg.HORIZONT:
212         self.rect.bottom = cfg.WINDOW.bottom - cfg.HORIZONT

```

Finally, `Game.update()` still needs to be adapted.

Listing 4.140: Moonlander (requirement 6) – `Game.update()`

```

249 def update(self) -> None:
250     self.background.update()
251     self.lander.update(action="move")      # Should move!

```

4.3.7 Requirement 7: Counter-thrust

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v05

Requirement 7 Counter-thrust

If counter-thrust is activated using , this thrust should affect the lander's descent speed. The counter-thrust should be -3 m/s^2 .

The counter-thrust is rather arbitrarily set to -3 m/s^2 . The negative sign is used because this thrust acts in the exact opposite direction of the Moon's gravity.

Listing 4.141: Moonlander (requirement 7) – magnitude of the counter-thrust

```

3 WINDOW = pygame.Rect(0, 0, 600, 800)
4 FPS = 60
5 DELTATIME = 1.0 / FPS
6 HORIZONT = 50
7 # Physical constants (Moon conditions)
8 MOON_GRAVITY = 1.62                # m/s²
9 EARTH_GRAVITY = 9.81              # m/s²
10 PIXELS_PER_METER = 10             # Scaling 1m = 10px
11 GRAVITY = MOON_GRAVITY * PIXELS_PER_METER # = 16.2 px/s²
12 THRUST = -3 * PIXELS_PER_METER     # = 30.0 px/s²

```

In `move()`, the counter-thrust is now included in the velocity calculation. To do this, it is first checked whether counter-thrust has been activated by pressing .

Listing 4.142: Moonlander (requirement 7) – adjustment of `Lander.move()`

```

208 def move(self) -> None:
209     if self.thrusting:
210         self.velocity += cfg.THRUST * cfg.DELTATIME
211     self.velocity += cfg.GRAVITY * cfg.DELTATIME
212     self.rect.top += self.velocity * cfg.DELTATIME
213     if self.rect.bottom >= cfg.WINDOW.bottom - cfg.HORIZONT:
214         self.rect.bottom = cfg.WINDOW.bottom - cfg.HORIZONT

```

With this in place, the player can now influence the descent speed of the lander using counter-thrust.

4.3.8 Requirement 8: Fuel



So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v06

Requirement 8 Fuel

1. *The lander has a limited fuel supply.*
2. *Depending on the difficulty level, different fuel amounts are available.*
3. *Fuel consumption is 20 units per second.*
4. *When the fuel supply is empty, no counter-thrust can be generated.*

First, the game constants are defined in `config.py`. `THRUST` represents the counter-thrust, but not in the unit m/s^2 ; instead, it is given in px/s^2 . The possible fuel supplies for requirement 8.2 are stored in the dictionary `LEVEL` in line 15.

Listing 4.143: Moonlander (requirement 8) – preparations in `config.py`

```

15 LEVEL = {"easy":sys.maxsize, "fair":500, "hard":450, "ai":380} #

```

In the constructor of `Lander`, the initial fuel supply is defined, and the current fuel level is stored in the attribute `fuel`, initialized with this starting value.

Listing 4.144: Moonlander (requirement 8) – adjustment in the constructor of `Lander`

```

128 self.fuel_initial = cfg.LEVEL["fair"]           # Starttreibstoff
129 self.fuel = self.fuel_initial                   # Aktueller Treibstoff
130 self.fuel_consumption = 20                      # Treibstoffverbrauch pro Sekunde

```

In line 219, it is now checked before calculating the counter-thrust whether there is still any fuel left in the tank, and in line 221 the consumed fuel is subtracted from the tank. If the tank is empty, the counter-thrust mode must be switched off and, to prevent a negative fuel value, the fuel level must be clamped to 0.

Listing 4.145: Moonlander (requirement 8) – `Lander.move()`

```

218 def move(self) -> None:
219     if self.thrusting and self.fuel > 0:                # Fuel remaining?
220         self.velocity += cfg.THRUST * cfg.DELTATIME
221         self.fuel -= self.fuel_consumption * cfg.DELTATIME    # Fuel consumption
222         if self.fuel < 0:
223             self.thrusting = False
224             self.fuel = 0
225     self.velocity += cfg.GRAVITY * cfg.DELTATIME
226     self.rect.top += self.velocity * cfg.DELTATIME
227     if self.rect.bottom >= cfg.WINDOW.bottom - cfg.HORIZONT:
228         self.rect.bottom = cfg.WINDOW.bottom - cfg.HORIZONT

```

To verify that the fuel supply starts correctly, is reduced properly when counter-thrust is applied, and that thrust is disabled once the tank is empty, I added a `print()` statement to `Lander.draw()` in line 216.

Listing 4.146: Moonlander (requirement 8) – `Lander.draw()`

```

211 def draw(self) -> None:
212     if self.thrusting:
213         self.screen.blit(self.surface_thrusting, self.rect.topleft)
214     else:
215         self.screen.blit(self.surface, self.rect.topleft)
216     print(self.fuel)                                     # Temporary only

```

Give it a try – it should work!

4.3.9 Requirement 9: Status display

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v07

Requirement 9 Status display

1. A separate status display is required for the lander.
 2. Velocity and altitude are displayed as text including their units.
 3. If counter-thrust is active, a colored bar is shown.
 4. The fuel supply is displayed as a progress bar.
-

All essential changes related to this feature take place in the `Lander` class. Since I want the position of the separate status display to depend on the position of the main window, the constructor signature has to be changed. Instead of passing a `Surface` object, a `Window` object is now passed in line 118.

Listing 4.147: Moonlander (requirement 9) – `Lander.draw()`

```

117 class Lander:
118     def __init__(self, window: pygame.window.Window) -> None: #
119         self.screen = window.get_surface()
120         self.surface = pygame.Surface((90,81), pygame.SRCALPHA)
121         self.surface_thrusting = pygame.Surface((90,81), pygame.SRCALPHA)
122         self.rect = self.surface.get_frect()
123         self.rect.centerx = cfg.WINDOW.centerx
124         self.rect.top = self.rect.height
125         self.create_lander()
126         self.thrusting = False
127         self.velocity = 0
128         self.fuel_initial = cfg.LEVEL["fair"]
129         self.fuel = self.fuel_initial
130         self.fuel_consumption = 20
131         self.create_status_window(window)

```

The separate window is created in `create_status_window()`. First, a window of appropriate size is created and the corresponding `Surface` object is obtained. I want the status window to be positioned to the right of the main window and aligned with its top edge. To achieve this, I take the top edge of the main window and assign this value to the top edge of the status window. Then I take the left edge of the main window, add the width of the main window to obtain its right edge, and finally add an additional 10 px of spacing.

Listing 4.148: Moonlander (requirement 9) – `Lander.create_status_window()`

```

197 def create_status_window(self, window: pygame.window.Window) -> None:
198     self.status_window = pygame.Window(size=(300, 100), title="Status")
199     self.status_screen = self.status_window.get_surface()
200     top = window.position[1]
201     left = window.position[0] + cfg.WINDOW.width + 10
202     self.status_window.position = (left, top)

```

In the final line, the `draw()` method is extended by a call to `draw_status()`. As a result, each time `draw()` is executed, not only the lander in the main window is redrawn, but the status window is updated as well.

Listing 4.149: Moonlander (requirement 9) – `Lander.draw()`

```

219 def draw(self) -> None:
220     if self.thrusting:
221         self.screen.blit(self.surface_thrusting, self.rect.topleft)
222     else:
223         self.screen.blit(self.surface, self.rect.topleft)
224     self.draw_status()

```

In `draw_status()`, the window is first filled with a black background. Starting at line 229, the status display for altitude and fuel is rendered as text. From line 240

onward, two bars are drawn. The first bar is only shown when the lander is currently applying counter-thrust. The second bar consists of two rectangles: a gray bar is drawn across the full width of the window, and a green bar is drawn from the left, proportionally scaled according to the remaining fuel supply.

Listing 4.150: Moonlander (requirement 9) – `Lander.draw_status()`

```

226 def draw_status(self) -> None:
227     self.status_screen.fill("black")
228
229     # Text output
230     font = pygame.font.SysFont("Consolas", 14)
231     labels = "Velocity (px/s):"
232     labels += "\nHeight (px):"
233     values = f"{self.velocity:>7.0f}"
234     values += f"\n{-1*(self.rect.bottom - (cfg.WINDOW.bottom - cfg.HORIZONT)):>7.0f}"
235     text_labels = font.render(labels, True, "white")
236     text_values = font.render(values, True, "white")
237     self.status_screen.blit(text_labels, (5, 10))
238     self.status_screen.blit(text_values, (230, 10))
239
240     # Bar display
241     if self.thrusting:
242         pygame.draw.rect(self.status_screen, (255, 140, 0), (5, 46, 290, 20))
243     pygame.draw.rect(self.status_screen, "grey", (5, 70, 290, 20))
244     ratio = int(290 * self.fuel / self.fuel_initial)
245     pygame.draw.rect(self.status_screen, "green", (5, 70, ratio, 20))
246
247     self.status_window.flip()

```

A few adjustments in `Game` are still required due to rendering output in multiple windows. One of these concerns event handling. When multiple windows are open in Pygame, the event `pygame.WINDOWCLOSE` must be processed (line 287). In this case, the flag of the main game loop has to be set to `False`, and the window associated with the event must be explicitly destroyed using `destroy()`.

Listing 4.151: Moonlander (requirement 9) – `Game.watch_for_events()`

```

283 def watch_for_events(self) -> None:
284     for event in pygame.event.get():
285         if event.type == pygame.QUIT:
286             self.running = False
287         elif event.type == pygame.WINDOWCLOSE: # New due to 2 windows!
288             self.running = False
289             event.window.destroy()
290         elif event.type == pygame.KEYDOWN:
291             if event.key == pygame.K_ESCAPE:
292                 self.running = False
293             elif event.key == pygame.K_r:
294                 self.restart()
295             elif event.key == pygame.K_SPACE:
296                 self.lander.update(action="thrust")
297         elif event.type == pygame.KEYUP:
298             if event.key == pygame.K_SPACE:
299                 self.lander.update(action="unthrust")

```

In `restart()`, the call to the constructor is also adjusted in line 316.

Listing 4.152: Moonlander (requirement 9) – `Game.restart()`

```
312 def restart(self) -> None:
313     self.background = Sky(self.screen)
314     self.moon = Moon(self.screen)
315     self.earth = Earth(self.screen)
316     self.lander = Lander(self.window)      # due to status window
317     self.running = True
```

The result then looks like the one shown in figure 4.29.

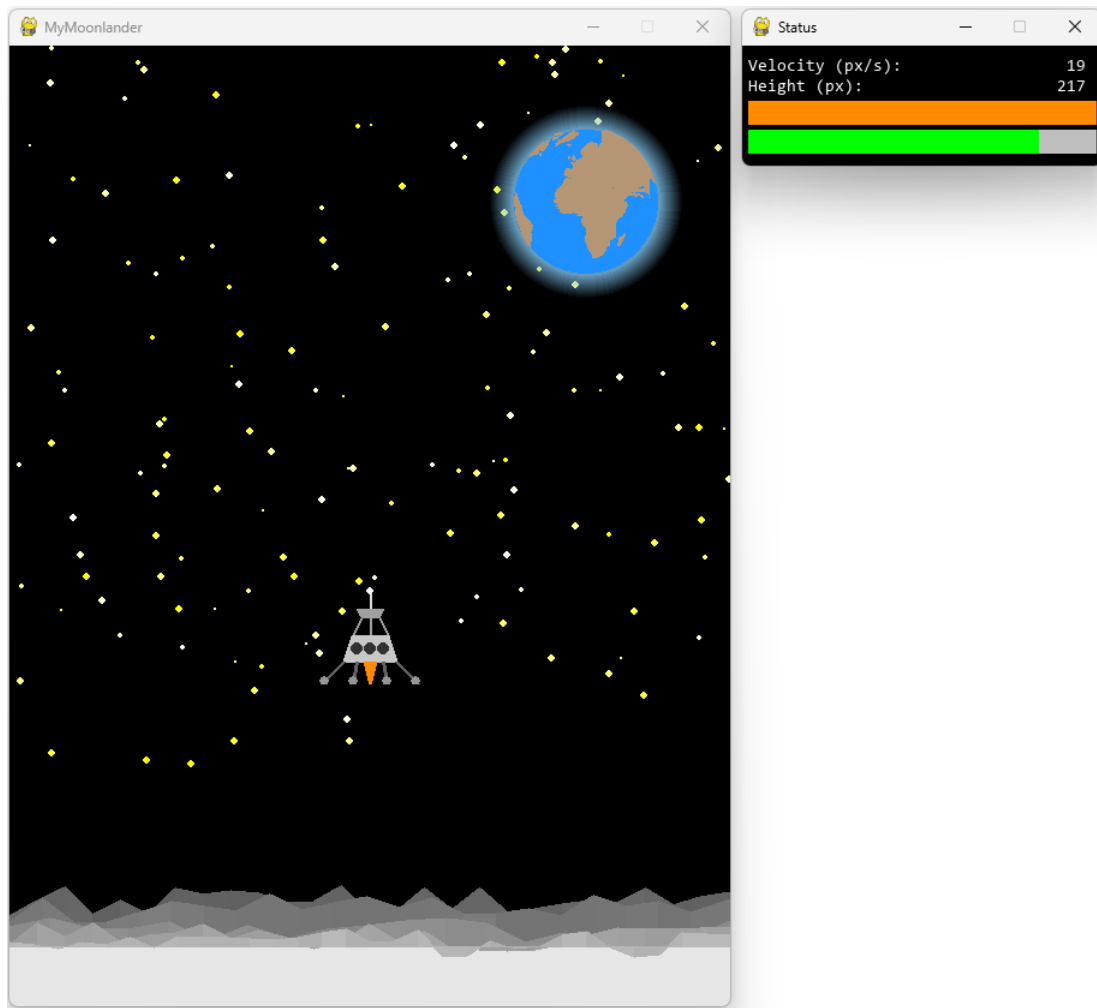


Figure 4.29: Moonlander (11) – now with status window

4.3.10 Requirement 10: Game over and restart

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v08

Requirement 10 Game over and restart

1. *If the lunar module lands with a velocity of < 5 px/s, the landing is considered safe.*
2. *If it lands at a higher velocity, it is considered destroyed.*
3. *The user is asked whether they want to quit the game with or restart it with .*

We prepare requirement 10.3 by encapsulating the display of the prompt in a simple class called `Question`. A `Surface` object containing the appropriate text is created and positioned accordingly. In `draw()`, this `Surface` object is then simply rendered on top of the lunar surface at the bottom of the screen.

Listing 4.153: Moonlander (requirement 10) – question

```

302 class Question:
303     def __init__(self, screen:pygame.Surface) -> None:
304         self.font = pygame.font.Font(None, 24)
305         self.screen = screen
306         self.surface = self.font.render("(Q)uit or (R)estart?", True, "red")
307         self.rect = self.surface.get_rect()
308         self.rect.centerx = cfg.WINDOW.centerx
309         self.rect.bottom = cfg.WINDOW.bottom - 10
310
311     def draw(self) -> None:
312         self.screen.blit(self.surface, self.rect.topleft)

```

How are quitting or restarting the game actually triggered? There are many possible approaches. In this case, I decided to use *events* created with `pygame.event.Event()`. The basic idea is that touching the lunar surface triggers an event: `LANDED` if the descent speed is low enough, otherwise `CRASHED`.

Listing 4.154: Moonlander (requirement 10) – MyEvents

```

11 LANDED = pygame.event.custom_type()
12 CRASHED = pygame.event.custom_type()

```

This requires `watch_for_events()` to be rewritten. In line 343 and line 346, the two events are intercepted. In both cases, the new flag `landing` is set to `False`. This allows

me to determine, for example, whether thrust may still be activated at all or whether the prompt for quitting or restarting the game should be displayed. In addition, an `update()` call is forwarded to the `Lander` so that it, too, is informed about its new state—for instance, to display an appropriate message in the status window.

For this reason, line 350 first checks whether the lander is still in the landing phase before allowing thrust to be activated.

The responses to the prompt are handled starting at line 354 and line 356. If `q` is pressed, the flag of the main program loop is simply set to `False`. If `r` is pressed, a restart is triggered by calling `restart()`.

Listing 4.155: Moonlander (requirement 10) – `Game.watch_for_events()`

```

336 def watch_for_events(self) -> None:
337     for event in pygame.event.get():
338         if event.type == pygame.QUIT:
339             self.running = False
340         elif event.type == pygame.WINDOWCLOSE:
341             self.running = False
342             event.window.destroy()
343         elif event.type == MyEvents.LANDED:           #
344             self.landing = False
345             self.lander.update(mode="landed", velocity=event.velocity)
346         elif event.type == MyEvents.CRASHED:         #
347             self.landing = False
348             self.lander.update(mode="crashed", velocity=event.velocity)
349         elif event.type == pygame.KEYDOWN:
350             if self.landing:                         #
351                 if event.key == pygame.K_SPACE:
352                     self.lander.update(action="thrust")
353             else:
354                 if event.key == pygame.K_q:          #
355                     self.running = False
356                 elif event.key == pygame.K_r:        #
357                     self.restart()
358                 if event.key == pygame.K_ESCAPE:
359                     self.running = False
360         elif event.type == pygame.KEYUP:
361             if event.key == pygame.K_SPACE:
362                 self.lander.update(action="unthrust")

```

In `Game`, the attribute `landing` is added to record whether the lander is still in the landing phase or has already touched the lunar surface.

Listing 4.156: Moonlander (requirement 10) – `Game.landing`

```

320     self.landing = True

```

Finally, `restart()` is extended in line 134 to reset the landing flag.

Listing 4.157: Moonlander (requirement 10) – `Game.restart()`

```

378 def restart(self) -> None:
379     self.landing = True                                     #
380     self.background = Sky(self.screen)
381     self.moon = Moon(self.screen)

```

```

382     self.earth = Earth(self.screen)
383     self.lander = Lander(self.window)
384     self.question = Question(self.screen)
385     self.running = True

```

Requirement 10.1 and requirement 10.2 are implemented in the new method `check_landing()` in `Lander`. When the lander reaches the lunar surface, its velocity is checked. If the descent speed is too high, the event `CRASHED` is triggered; otherwise, the event `LANDED` is emitted. The handling of these events itself has already been discussed above in `watch_for_events()` (source code 4.155 on the previous page).

Listing 4.158: Moonlander (requirement 10) – `Lander.check_landing()`

```

293     def check_landing(self) -> None:
294         if self.rect.bottom >= cfg.WINDOW.bottom - cfg.HORIZONT:
295             if self.velocity > 5:
296                 evt = pygame.event.Event(MyEvents.CRASHED, velocity=self.velocity)
297                 pygame.event.post(evt)
298             else:
299                 evt = pygame.event.Event(MyEvents.LANDED, velocity=self.velocity)
300                 pygame.event.post(evt)

```

Finally, `Game.update()` must be extended to include a call to `check_landing()`.

Listing 4.159: Moonlander (requirement 10) – `Game.update()`

```

364     def update(self) -> None:
365         self.background.update()
366         self.lander.update(action="move")
367         self.lander.check_landing()

```

A few adjustments in `Lander` are still required. First, in line 134, the attribute `mode` is introduced. It keeps track of which of the three states the lunar module is currently in: `landing`, `landed`, or `crashed`.

Listing 4.160: Moonlander (requirement 10) – `Lander.mode`

```

124 class Lander:
125     def __init__(self, window: pygame.window.Window) -> None: #
126         self.screen = window.get_surface()
127         self.surface = pygame.Surface((90,81), pygame.SRCALPHA)
128         self.surface.thrusting = pygame.Surface((90,81), pygame.SRCALPHA)
129         self.rect = self.surface.get_frect()
130         self.rect.centerx = cfg.WINDOW.centerx
131         self.rect.top = self.rect.height
132         self.create_lander()
133         self.create_lander_thrusting()
134         self.mode = "landing" # "landing", "landed" or "crashed"
135         self.thrusting = False
136         self.velocity = 0
137         self.fuel_initial = cfg.LEVEL["fair"]
138         self.fuel = self.fuel_initial
139         self.fuel_consumption = 20
140         self.create_status_window(window)

```


This attribute is set or updated in `update()`, starting at line 229. Once the ground has been touched – i.e. when the state is either `landed` or `crashed` – the thrust is switched off.

Listing 4.161: Moonlander (requirement 10) – `Lander.update()`

```

220 def update(self, *args: Any, **kwargs: Any) -> None:
221     if "action" in kwargs.keys():
222         if kwargs["action"] == "thrust":
223             self.thrust(True)
224         elif kwargs["action"] == "unthrust":
225             self.thrust(False)
226         elif kwargs["action"] == "move":
227             if self.mode == "landing":
228                 self.move()
229     if "mode" in kwargs.keys(): #
230         self.mode = kwargs["mode"]
231         if self.mode in ("landed", "crashed"):
232             self.thrust(False)

```

The status display is now extended to show the game-over state as well. Its appearance can be examined in figure 4.30 on the following page.

Listing 4.162: Moonlander (requirement 10) – `Lander.draw_status()`

```

247 def draw_status(self) -> None:
248     self.status_screen.fill("black")
249
250     # Text output
251     font = pygame.font.SysFont("Consolas", 14, bold=True)
252     labels = "Velocity (px/s):"
253     labels += "\nHeight (px):"
254     values = f"{self.velocity:>7.0f}"
255     values += f"\n{-1*(self.rect.bottom - (cfg.WINDOW.bottom - cfg.HORIZONT)):>7.0f}"
256     text_labels = font.render(labels, True, "white")
257     text_values = font.render(values, True, "white")
258     if self.mode == "landed":
259         text_mode = font.render(f"Status: {self.mode}", True, "green")
260     elif self.mode == "crashed":
261         text_mode = font.render(f"Status: {self.mode}", True, "red")
262     else:
263         text_mode = font.render(f"Status: {self.mode}", True, "white")
264     text_mode_rect = text_mode.get_rect(top=100)
265     text_mode_rect.left = self.status_screen.get_rect().centerx - text_mode_rect.centerx
266     self.status_screen.blit(text_labels, (5, 10))
267     self.status_screen.blit(text_values, (230, 10))
268     self.status_screen.blit(text_mode, text_mode_rect.topleft)
269
270     # Bar display
271     if self.thrusting:
272         pygame.draw.rect(self.status_screen, (255, 140, 0), (5, 46, 290, 20))
273         pygame.draw.rect(self.status_screen, "grey", (5, 70, 290, 20))
274         ratio = int(290 * self.fuel / self.fuel_initial)
275         pygame.draw.rect(self.status_screen, "green", (5, 70, ratio, 20))
276         self.status_window.flip()

```



Figure 4.30: Moonlander (12) – quit or restart?

4.3.11 Requirement 11: Autopilot

 So ... where are the files?

https://github.com/adamsralf/pygame_book/tree/main/src/02%20Examples/03%20Moonlander/v09

Requirement 11 Autopilot

The autopilot can be switched on or off using h.

First, something that actually has nothing to do with requirement 11. I want the physical values to be a bit closer to real-world figures. The thrust is set to -2.1 m/s^2 and the safe landing speed to 2.5 m/s . According to NASA documentation, Apollo 11 touched down at 0.7 m/s . The NASA target value was 1 m/s , and the acceptable range was between 0.5 and 2.5 m/s . The structural limit was reached at 3 m/s . A value below 0.5 m/s would have resulted in unnecessary fuel consumption.

Listing 4.163: Moonlander (requirement 11) – some constants

```
14 THRUST = -2.1 * PIXELS_PER_METER          # = 21.0 px/s²
15 SAVE_SPEED_LANDING = 2.5 * PIXELS_PER_METER # Safe landing velocity in px/s
```

Now back to requirement 11: In `watch_for_events()`, the key press h is detected and forwarded to the Lander.

Listing 4.164: Moonlander (requirement 11) – extension of `watch_for_events()`

```
366         if self.landing:
367             if event.key == pygame.K_SPACE:
368                 self.lander.update(action="thrust")
369             elif event.key == pygame.K_h:
370                 self.lander.update(action="toggle_ai")
```

In the constructor of `Lander`, the flag `ai` is introduced and initialized with `False` – although the term *ai* is admittedly a bit ambitious here ;-)

Listing 4.165: Moonlander (requirement 11) – extension of `Lander.__init__()`

```
136         self.fuel_consumption = 20
137         self.ai = False                # AI flag
138         self.create_status_window(window)
```

The `update()` method is extended as well. Starting at line 224, the `ai` flag is toggled on or off. When it is switched off, any thrust that may have been triggered by the autopilot must be stopped. In line 230, it is then checked whether the autopilot is active; if so, control is handed over to the autopilot.

Listing 4.166: Moonlander (requirement 11) – extension of `Lander.update()`

```

224         elif kwargs["action"] == "toggle_ai":           # AI on/off
225             self.ai = not self.ai
226             if not self.ai:
227                 self.thrust(False)
228         elif kwargs["action"] == "move":
229             if self.mode == "landing":
230                 if self.ai > 0:                           # AI active?
231                     self.controller()
232                 self.move()

```

Before diving into the actual implementation of the control logic, we first need to play around a bit with some physical formulas.

The formula for the final velocity in free fall is:

$$v = \sqrt{2 \cdot g \cdot h} \quad (4.1)$$

This equation gives us the final velocity v for a given gravitational acceleration g and a fall height h , assuming the initial velocity was 0 m/s. However, we are not actually interested in the final velocity. What we really care about is the height h : from which height do we have to start applying counter-thrust in order to reach our target velocity?

So we rearrange equation 4.1 to solve for h :

$$\begin{aligned}
 v &= \sqrt{2 \cdot g \cdot h} & \| x^2 \\
 v^2 &= 2 \cdot g \cdot h & \| : (2 \cdot g) \\
 \frac{v^2}{2 \cdot g} &= h &
 \end{aligned} \quad (4.2)$$

However, we are no longer dealing with lunar gravity alone; the counter-thrust of the lander also comes into play. In this case, the following applies:

$$acc = g_{Moon} + acc_{Lander} \quad (4.3)$$

Note that the sign of acc_{Lander} is opposite to that of the Moon's gravity g_{Moon} – that is, it is negative. We now substitute equation 4.3 into equation 4.2:

$$\begin{aligned}
 h &= \frac{v^2}{2 \cdot acc} & \| \leftarrow 4.3 \\
 h &= \frac{v^2}{2 \cdot (g_{Moon} + acc_{Lander})} &
 \end{aligned} \quad (4.4)$$

And equation 4.4 can already serve as the basis for our implementation. First, starting at line 239, we check whether the lander is still on its final approach. If not, all thrust is turned off and we are done, because there is nothing left to do.

In line 243, the net acceleration from equation 4.3 on the preceding page is computed, and then the target velocity is defined. This value is chosen to be far enough away from the maximum structural limit by using 50%. In line 245, we then check whether the current velocity is already below this safe velocity. If it is, there is nothing to do—except that thrust must be switched off.

Next, following equation 4.4 on the facing page, the distance to the ground at which counter-thrust must begin is calculated. In line 251, counter-thrust is then activated or deactivated accordingly. All clear?

With this, the lander is fully implemented for the purposes of this script. If the autopilot performs a safe landing, it should look like the scene shown in figure 4.31 on the next page.

Listing 4.167: Moonlander (requirement 11) – `Lander.controller()`

```

238 def controller(self):
239     if self.mode in ("landed", "crashed"):           # Landing finished?
240         self.thrust(False)
241         return
242
243     acc = -1 * (cfg.THRUST + cfg.GRAVITY)             # Net acceleration
244     v_save = cfg.SAVE_SPEED_LANDING * 0.5           # 50% buffer
245     if self.velocity <= v_save:                     # Already slow?
246         self.thrust(False)
247         return
248
249     brake_distance = self.velocity ** 2 / (2 * acc)
250     ground_distance = (cfg.WINDOW.height - 50) - self.rect.bottom
251     self.thrust(ground_distance <= brake_distance)  # Required braking distance >=
        remaining height?

```

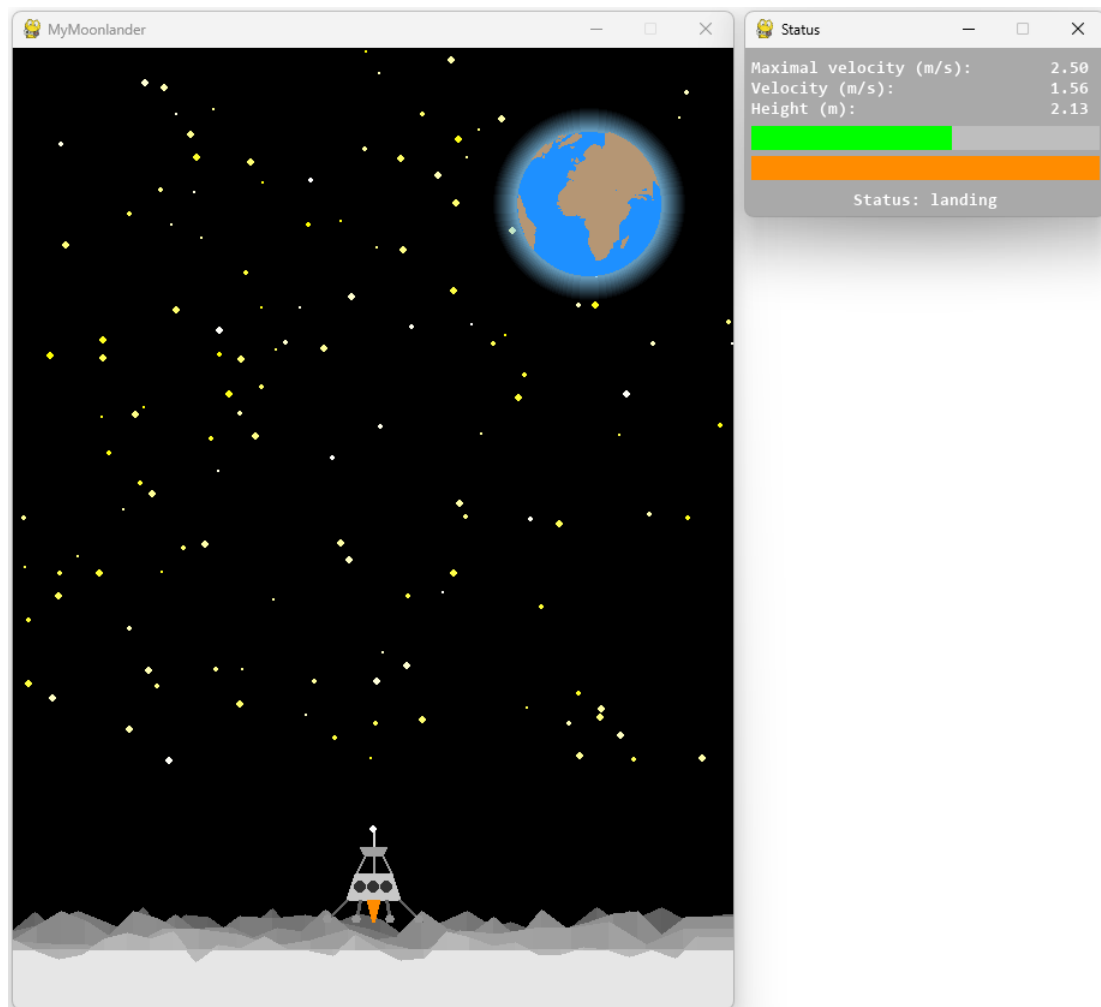


Figure 4.31: Moonlander (13) – autopilot

List of Figures

2.1	Pygame installation test	15
2.2	Playground	17
2.3	Resource usage without timing control	19
2.4	Resource usage with timing control	20
2.5	Multiple Windows	21
2.6	Infos about the graphical environment	23
2.7	Some graphic primitives	29
2.8	Not a particle swarm	30
2.9	Particle swarm Version 2	32
2.10	Particle swarm, Version 5: nearly finished	33
2.11	Example: Drawing a Landscape	37
2.12	Drawing a Landscape (2)	39
2.13	Drawing a Landscape (3)	39
2.14	Drawing a Landscape (4)	40
2.15	Drawing a Landscape (5)	40
2.16	Drawing a Moonlander	44
2.17	Load and blit bitmaps, Version 1.0	46
2.18	Sizes OK	47
2.19	α OK	48
2.20	Bitmaps positioning defender	49
2.21	Bitmaps positioning alien, Version 1	50
2.22	Bitmaps positioning alien, Version 2	51
2.23	Bitmaps positioning alien, Version 3	51
2.24	Tiles to build a forest	52
2.25	Messageboxes	53
2.26	Elements of a <code>Rect</code> -/ <code>FRect</code> -Object	59
2.27	Moving Bitmaps, Version 1.0	61
2.28	The Defender moves and bounces	63
2.29	Movement without normalization	66
2.30	Position Error of $1/fps$ and <code>pygame.clock.tick()</code>	70
2.31	Position Error of <code>Rect</code> and <code>FRect</code>	72
2.32	Position Error with Different Time Functions	73
2.33	Movement with normalization and $dt = 1/fps$	74
2.34	Movement with normalization and <code>pygame.clock.tick()</code>	74
2.35	Movement with normalization and <code>pygame.clock.tick()</code> (float)	75
2.36	Movement with normalization and <code>time.time()</code>	75
2.37	Typewriter	95
2.38	Simple text output using fonts	106
2.39	Text output using fonts	108
2.40	List of all installed fonts	111

2.41	Example of using a locally installed font	114
2.42	Example of a spritelib	116
2.43	Text output using bitmaps	116
2.44	Meaning of the variables in Spritelib	118
2.45	Clock	122
2.46	Collision detection with rectangles	123
2.47	Collision detection with circles	124
2.48	Collision detection using masks	124
2.49	From sprite to mask	125
2.50	Four sprites	126
2.51	Collision detection using rectangles (montage)	126
2.52	Collision detection using circles (montage)	126
2.53	Collision detection using masks (montage)	126
2.54	Tradeoff Accuracy vs. Costs	133
2.55	A simple collision game	134
2.56	Fireball using frame-based movement	136
2.57	Fireball using counter-based movement	140
2.58	Comparison of the 4 algorithms	144
2.59	Example of actions with a mouse	146
2.60	Example Stereo Sound	161
2.61	User-defined events	173
2.62	How the counter works	178
3.1	Animation of a cat: frame sprites	185
3.2	The exploding rock: frame sprites	191
3.3	Sprite library (original)	193
3.4	Sprite library (prepared)	193
3.5	Example of tile based playground	194
3.6	Forest playground (1)	198
3.7	Forest playground (2)	198
3.8	Forest playground (3)	201
3.9	Forest playground (4)	201
3.10	Tiles without borders	207
3.11	Tiles with borders	207
3.12	PlainWindow as a Viewport of the World	208
3.13	Performance without and with visibility culling	209
3.14	BirdEye (scaled)	212
3.15	Bird's-Eye View (simplified and with visibility indicator)	212
3.16	Performance without and with preprocessing	212
3.17	Bird's-Eye View: Green = Centered	216
3.18	Centered Camera – with border error	216
3.19	BirdEyeView: Grün=Centered	217
3.20	Centered Camera – without border error	217
3.21	Performance with element-based and world-based transformation	218

3.22	Bird's-Eye: Centered, Pagewise, InnerRect	220
3.23	Page / Edge Scrolling	220
3.24	Strategy pattern applied on <code>Camera</code> and scroll strategies in <code>cameraview.py</code>	222
3.25	Tic-Tac-Toe – Start	224
3.26	Tic-Tac-Toe – Playground	229
3.27	You've won	234
3.28	It's a draw	234
3.29	Tic-Tac-Toe – UML state machine diagram	238
3.30	Example of a simple State Machine	240
4.1	Pong: the background	253
4.2	Pong: paddles, ball, and score	264
4.3	Pong: paddle color indicates AI mode (left AI, right manual)	267
4.4	Pong: Help screen	273
4.5	Bubbles: background image	275
4.6	Bubble	277
4.7	Bubbles: the bubbles have a minimum distance at the start	280
4.8	Bubbles – the bubbles have grown and merged	284
4.9	collision detection – point inside a circle?	285
4.10	Bubbles – score display	289
4.11	Bubbles – collision with the edge	290
4.12	Bubbles – bubble collision	290
4.13	Bubble 2	293
4.14	Bubbles – displaying a collision	295
4.15	Why does the inner loop start at $index_1 + 1$?	295
4.16	Bubbles – pause screen	297
4.17	Bubbles – restart screen	300
4.18	Bubbles – state diagram	306
4.19	Moonlander (1)	310
4.20	Moonlander (2)	311
4.21	Moonlander (3)	312
4.22	Moonlander (4)	313
4.23	Moonlander (5) glowing	317
4.24	Moonlander (6) continent	317
4.25	Moonlander (7)	318
4.26	Moonlander (8) – lander without thrust	321
4.27	Moonlander (9) – lander with thrust	321
4.28	Moonlander (10) – the lander	322
4.29	Moonlander (11) – now with status window	329
4.30	Moonlander (12) – quit or restart?	334
4.31	Moonlander (13) – autopilot	338

List of Tables

2.1	Fields of <code>pygame.display.Info()</code>	22
2.2	Predefined Display Modes	26
2.3	Distance without normalized movement	66
2.4	Distance with normalized movement	67
2.5	Pixel coordinates with normalized speed	67
2.6	Error Propagation	70
2.7	Predefined Keyboard Constants	101
2.7	Predefined Keyboard Constants (continued)	102
2.7	Predefined Keyboard Constants (continued)	103
2.7	Predefined Keyboard Constants (continued)	104
2.8	Predefined Keyboard Modifier	104
2.8	Predefined Keyboard Modifier (continued)	105
2.9	List of mouse events	154
3.1	Computation of the row number (line 69 in listing 3.56)	229
3.2	Computation of the col number (line 70 in listing 3.56)	229
3.3	Evaluation of the STATUS approach	237
3.4	Decision table for designing scene handling	252

Glossary

Alpha Blending A technique used in computer graphics for transparency and color blending. In this process, the color values of a foreground pixel are combined with those of a background pixel based on an alpha value (α). The alpha value specifies how opaque a pixel is: $\alpha = 1$ means fully visible (opaque), $\alpha = 0$ means fully transparent. Formula:

$$C_{\text{new}} = \alpha \cdot C_{\text{foreground}} + (1 - \alpha) \cdot C_{\text{background}}$$

Pygame supports alpha blending through surfaces with an alpha channel (RGBA). This makes semi-transparent effects such as shadows, glow, or light effects possible. The quality and performance of alpha blending depend on the hardware and the SDL version. In the pygame-ce version, extended alpha operations and faster blit methods are available. [29](#)

Alpha Channel For each pixel of an image, color information is usually stored in the [RGB](#) format: the red channel, the green channel, and the blue channel. Using an additional piece of information, it is also possible to specify how transparent the pixel should be. This additional information is called the alpha channel. [29](#)

American Standard Code for Information Interchange A 7-bit character encoding standard that maps numbers (0-127) to letters, digits, punctuation, and control characters, enabling consistent text representation across different computer systems. [357](#)

Array A data structure that stores values under a unique index (usually a non-negative integer). In the strict sense, arrays contain only elements of the same data type. This restriction does not apply to languages such as PHP or Python. [120](#)

Binary AND Operation The binary AND operation combines two integer values by comparing their bits. For each bit position, the result is 1 if and only if both of the corresponding bits are 1; otherwise, it is 0. In Python, the binary AND operation is performed using the operator `&`. It is commonly used with bit masks, for example to test or combine modifier keys such as `↑`, `Ctrl`, or `Alt` in keyboard events. [92](#)

Bitmap The term bitmap has two levels of meaning in this context. In general, it refers to the color and transparency information of an image stored in a file. Typical examples are files in the formats [Joint Photographic Experts Group \(JPEG\)](#), [PNG](#), or [Windows Bitmap Format \(BMP\)](#). More specifically, it can also refer to the bitmap file format used for image storage (Windows Bitmap, BMP). [18](#)

Boss Key A boss key is a special key or key combination that immediately exits, hides, or pauses a game without asking for confirmation. Historically, it was used to make a game disappear instantly when a boss, teacher, or other authority figure

approached. The feature became popular in early PC games and is often mentioned with a touch of humor, but it can also serve as a practical shortcut for emergency exits or quick interruptions. [89](#)

broadcast system An event distribution mechanism in which incoming events are forwarded to all interested components. In games, this allows multiple systems to react to the same event. Unlike a state stack, where only the active state processes user input, a broadcast system delivers events to every registered listener. [251](#)

Clamp In programming, clamp (to *clamp* or *limit*) refers to a function that keeps a numerical value within a predefined range. If the value is below the minimum, the minimum is returned; if it is above the maximum, the maximum is returned. In games, clamping is commonly used to restrict camera positions, movements, or physical values (e.g. speed or zoom factor) to reasonable bounds—for example, to prevent the camera from scrolling outside the game world. [216](#)

Class A class describes the properties (attributes) and the methods (functions) of a logically self-contained programming unit. In practice, there are many different kinds of classes, but in principle a class defines which information belongs to it (for example, the brand, color, and year of manufacture of a car) and what can be done with an object of that class (for example, accelerating, buying, or refueling a car). The information is called *attributes*, and the possible actions are called *methods* or *member functions*. In Python, classes are defined using the keyword `class` and are initialized with `__init__()`. [17](#)

Collision Detection The process of checking whether two bitmaps *touch* each other in any way. In Pygame, we use three types of collision detection: checking whether the bounding rectangles of the bitmaps intersect, checking whether the inner circles of the bitmaps intersect, and checking whether non-transparent pixels of the bitmaps share the same coordinates. [57](#)

Constant A constant is a value that cannot be changed while a program is running. In many programming languages, variables can be declared as constants – that is, as unchangeable values – using keywords such as `const`. Direct numeric or string values written in the source code are also constants. Python does not have true constants at the language level; instead, a common convention is used to write constants in CAPITAL LETTERS (for example, `PI = 3.14159`). Although the value is technically still changeable, this naming style signals to other developers that the variable is intended to remain unchanged. [17](#)

CSV File A simple, text-based file format (*Comma-Separated Values*) used for storing tabular data. Values are separated by delimiters such as commas or semicolons. CSV files can be easily processed by programs such as spreadsheet applications, databases, or scripts. [193](#)

Degree (°) A unit for measuring angles. A full circle has $2\pi^\circ$. [150](#)

Dictionary A data structure that stores values under a unique key. Other common names are: lookup table, associative array, hash table. [118](#)

divide et impera A Latin expression meaning *divide and conquer*. The principle describes the strategy of decomposing a large and complicated problem into several smaller parts that can be handled independently. In the context of state machines, each state is responsible only for its own behavior, reducing the need for large conditional structures and improving readability and maintainability of the code. [238](#)

Don't ask – tell A design principle of object-oriented programming which states that an object should not ask another object about its internal state and then make decisions based on that information. Instead, an object should tell another object *what* to do, while the *how* remains fully encapsulated within the object itself. This improves encapsulation, substitutability, and maintainability, and reduces coupling between classes. [84](#)

Double Buffer This is a second memory area (back buffer) that has the same size as the screen memory (front buffer). When something is drawn onto the playfield, it is first drawn into the back buffer. Only after all game elements have been drawn in their new appearance is the front buffer swapped with the back buffer in a single step. With certain hardware or graphics configurations, it can happen that the screen memory is redrawn even though the game has not yet finished updating all states. This can lead to ugly artifacts or flickering. Double buffering is used to prevent this effect. [18](#)

DTP point Unit of measurement used in desktop publishing (DTP) and digital typography. A DTP point (also called a PostScript point) is defined as *one seventy-second of an inch* and therefore corresponds exactly to **1 pt = 0.013 888 888 888 888 89 inch** $\approx$ **0.3528 mm**. This definition was originally introduced by Adobe with the PostScript standard and has become the industry standard for layout, printing, and graphics software. In contrast, traditional typography sometimes uses slightly different point sizes (e.g. 1 pica point = 0.3515 mm). The DTP point is used to specify font sizes, spacing, and page margins in layout programs such as InDesign, Scribus, or L^AT_EX. Many graphics libraries (e.g. [Pygame-ce](#)) and GUI frameworks also use DTP points indirectly via pixel conversions for precise on-screen rendering. [357](#)

Equidistant The spacing between elements is always the same. For elements of equal size, this means that the space between them is always identical. For elements of different sizes, a reference point is required. For example, should the centers of the elements always have the same distance, or should the right edge of one element always have the same distance to the left edge of the next one? A distinction is also made between horizontal and vertical equidistance. [48](#)

Error A message that indicates a serious problem that prevents a program or part of a program from continuing correctly. Errors usually require immediate action, such

as fixing the code, correcting input data, or restarting the program. In many cases, an error causes the program to terminate. [53](#)

Event In software engineering, an event is used to control the flow of a program. The program is not executed in a strictly linear way; instead, specific event-handling routines (such as listeners, observers, or event handlers) are executed whenever a particular event occurs. Event-driven programming is considered part of parallel programming techniques and therefore shares their advantages and disadvantages (source: Wikipedia). [17](#), [171](#)

Fade Derived from the English verb *to fade*. In music and graphics, a distinction is made between *fade-in* and *fade-out*. During a fade-in, an image appears gradually or the volume is increased from zero to the target level. A fade-out does the opposite. [157](#)

Flag A variable that stores only a Boolean value (**True** or **False**) and is used to control the flow of a program. Flags are often used to represent states such as *program running*, *game finished*, or *input allowed*. By setting or resetting the flag, the behavior of a program can be controlled in a targeted way. Example in Python: `running = True`. [18](#)

Floating-point Number A floating-point number represents values as sums of powers of two, where the exponent can also be negative. Example: $6.75 = 2^2 + 2^1 + 2^{-1} + 2^{-2}$. Because storage space is limited, or because some numbers do not have a finite representation, this sum must be truncated at some point. The omitted terms lead to rounding errors. [69](#)

Font Refers to a complete, digitally stored set of characters that contains information about the graphical representation of letters, digits, and symbols. A font defines typographic properties such as style, stroke weight, spacing, and size. In Pygame, a font is represented by a Font object, which is used to render text into a bitmap or Surface. Fonts can be installed system-wide or loaded from separate files (e. g. `.ttf`, `.otf`). [108](#)

Frames Per Second Maximum number of images/frames per second. [19](#), [357](#)

Framework In computer science, a framework refers to a working environment that provides predefined structures and functionality. This can include individual classes, function libraries, or even complete development environments such as an [Integrated Development Environment \(IDE\)](#). [78](#)

Function In programming, a function is a block of instructions that has a name. It can take one or more parameters and can return results using `return`. In most cases, the principle applies that all values inside a function are local. In Python, a function definition starts with the keyword `def`. [17](#)

Function Pointer A function pointer refers to a variable or reference that points to a function and allows it to be treated like a value. Function pointers make it possible to pass functions as parameters, store them in data structures, or select

them dynamically. In languages such as C or C++, function pointers are used explicitly, while in Python functions are first-class objects and can therefore be used as function pointers without any special syntax. In Python, this is often referred to as a callable. [132](#)

Gadget An interactive or usable game element that provides a player character with special abilities, advantages, or tools. Gadgets may appear as equipment, aids, or technical devices and often influence gameplay, puzzle solving, or combat mechanics in a video game. [193](#)

Garbage Collector The garbage collector is a memory management component responsible for automatically freeing memory that is no longer in use. In Python, it detects objects that are no longer referenced by any part of the program and releases the memory they occupy. This helps prevent memory leaks and allows developers to focus on program logic rather than manual memory management. Python primarily uses reference counting, complemented by a garbage collection mechanism to detect and clean up cyclic references. [86](#)

Generative Pre-trained Transformer ChatGPT is a prototype of a chatbot—that is, a text-based dialog system used as a user interface—based on machine learning. The chatbot was developed by the U.S.-based company OpenAI and was released in November 2022. (Source: Wikipedia). [357](#)

Gravity A force that produces a constant acceleration acting on objects with mass. In game development, gravity is commonly implemented as a fixed acceleration vector applied each frame, rather than a full physical simulation. [32](#)

Infinite loop In computer science, an infinite loop is a sequence of instructions that repeats endlessly and has no defined termination condition. In most cases, infinite loops are unintended and therefore represent an error in an application. They often arise from incorrect loop conditions. However, infinite loops are sometimes used intentionally, for example: `while True:..` [279](#)

Infix An infix is a morphological element that is inserted inside a word stem to modify its grammatical or semantic meaning. In contrast to prefixes, which appear before the stem, and suffixes, which appear after the stem, an infix is placed within the stem itself. Infixes are common in many languages (for example Tagalog or Indonesian) but are rare in English and German. In English, infixes are mostly found in informal or expressive language, such as the emphatic insertion *-bloody-* in *abs**bloody**lutely*. [99](#)

Information A message that provides additional details or context about the current state of a program. Information messages do not indicate a problem and do not require immediate action. They are often used to inform the user about successful operations, current settings, or progress. [53](#)

Information Technology Assistants A state-recognized educational program at vocational colleges in North Rhine-Westphalia, Germany, providing school-based vocational training in the field of information technology. The program combines

theoretical foundations with practical content from computer science, programming, network technology, databases, software development, and IT systems. Depending on the specific program, students may also obtain the advanced technical college entrance qualification or the general higher education entrance qualification. Graduates are qualified for employment in IT-related professional fields or well prepared for further academic studies. [274](#), [357](#)

Integer An integer represents numbers as sums of powers of two with non-negative exponents. Example: $17 = 2^4 + 2^0$. The range of representable values is determined by the amount of memory allocated to the integer. With n bits available, unsigned integers can represent values in the range $[0, 2^n - 1]$, while signed integers can represent values in the range $[-2^{n-1}, 2^{n-1} - 1]$. [69](#)

Integrated Development Environment An integrated development environment. It is called *integrated* because it does not only include a compiler and linker, but also tools such as a code editor, debugger, profiler, and other utilities that support the entire development process. [348](#), [357](#)

Joint Photographic Experts Group A widely used raster image format for the compressed storage of digital images. JPEG uses lossy compression, in which fine details and color differences are partially removed or smoothed in order to significantly reduce file size. The compression level can be adjusted: Higher compression results in smaller files, but also in visible artifacts.

- Supports up to 24-bit color depth (true color)
- No transparency channel (alpha channel) as in [PNG](#)
- Ideal for photos, textures, and realistic graphics
- Less suitable for pixel art or UI elements with sharp edges

JPG files are commonly used in games, web applications, and desktop publishing (DTP) projects when storage space or loading time is more important than perfect image fidelity. In graphics libraries such as [Pygame-ce](#), JPG files can be loaded directly and used as *surfaces*. [345](#), [357](#)

Liskov Substitution Principle A principle of object-oriented programming stating that objects of a derived class must be usable anywhere an object of the base class is expected, without altering the correct behavior of the program. Formulated by Barbara Liskov in 1987. The LSP ensures that inheritance does not introduce unexpected side effects and that class hierarchies remain consistent. [84](#), [357](#)

Main Loop Every non-trivial program must decide whether it should continue running or whether processing can be finished. If processing cannot or should not be finished yet, the program must continue with user interaction or other program functions, and it must do so until the program can or should be terminated. This behavior is usually controlled by a main loop. Examples: An operating system runs until it is shut down. A Windows application runs until ALT+F4 is pressed. [18](#)

- Margin** Denotes the outer spacing of an element relative to other objects or to the boundary of a surrounding container. In *graphics* and *UI programming*, *margin* is used to specify how much empty space should exist outside a frame or surface. In contrast, [padding](#) describes the inner spacing between content and its border. In applications such as `CameraView`, *margin* can be used to define an outer safety area of the visible game region, determining how close objects or the camera are allowed to approach the edge. [352](#)
- Mask** A mask is a bitmap that makes it possible to distinguish important pixels from unimportant pixels of a sprite. For sprites with transparency, the mask can easily be created by treating all transparent pixels as unimportant. To save memory and computing time, masks are often not stored in common bitmap formats, but bit by bit. One byte can therefore encode the mask information for 8 pixels. [125](#)
- Message Queue** A queue provided by the operating system in which events and messages that are directed to applications or processes are stored. Such messages can include, for example, keyboard and mouse input, window events, or system signals. Programs regularly read from the message queue in order to react to current events. In event-driven applications such as graphical user interfaces or games, reading the message queue is a central part of the main program loop. [18](#)
- Milliseconds** One thousandth (1/1000) of a second. [140](#), [357](#)
- Modulo** A mathematical operator that returns the remainder of an integer division. The expression $a \bmod b$ yields the remainder that results when the number a is divided by b . The modulo operator is frequently used in computer science, for example for periodic processes, index calculations, or checking divisibility. [196](#)
- MP3** Short for *ISO MPEG Audio Layer 3*. An audio encoding and compression method for sound and music, developed largely by the German electrical engineer and mathematician Karlheinz Brandenburg. [159](#)
- Namespace** A namespace is a structural area in which identifiers such as variable names, function names, or class names are defined. Namespaces prevent name collisions by allowing a clear and unique mapping of names to objects. In Python, there are, among others, local, global, and module-level namespaces. Example: A variable in a module can be accessed using `modulename.variable`. [17](#)
- Object Oriented** The analysis, the design or the implementation based on objects – software entities that encapsulates data and functions. [357](#)
- OGG** An audio file encoding format. The name comes from the English verb *to ogg*. The goal was to provide a license-free, simple, and efficient audio encoding format. [159](#)
- Operating System** An operating system (OS) is the basic software that controls a computer and manages its hardware and software resources. It acts as an interface

between the user, application programs, and the computer hardware. The operating system is responsible for tasks such as process management, memory management, file access, and input/output control. Examples of operating systems include Windows, Linux, and macOS. [18](#), [357](#)

Padding Refers to additional spacing between an inner area and an outer boundary. In *graphics programming* and *UI layout*, *padding* specifies how much empty space exists inside a frame (e.g. a rectangle or surface) between the actual content and its border. In contrast, [Margin](#) denotes the outer spacing between an element and other objects. In games or camera implementations (e.g. in `CameraView`), *padding* can be used to define a safety zone around the player character or the visible area before a camera movement is triggered. [219](#), [351](#)

Pixel The smallest addressable unit of a digital image or screen display. A pixel typically represents a color that is composed of individual color channels (for example, red, green, and blue). The combination of many pixels forms a complete image. The more pixels a screen or image has, the higher the possible resolution and level of detail. [18](#), [357](#)

Polygon A closed [polyline](#). It is usually defined by a sequence of points, where the last point is connected to the first one. [29](#), [352](#)

Polyline A sequence of connected lines. It is usually defined by a sequence of points. A closed polyline is called a [polygon](#). [30](#), [352](#)

Portable Network Graphics A widely used raster image format that supports lossless compression and optionally an alpha channel for transparent pixels. PNG is especially well suited for graphics with sharp edges, text, or transparency effects and is often used in games and user interfaces. [46](#), [357](#)

Pygame-ce A free and open-source library written in Python for developing 2D games and multimedia applications. Pygame-ce is based on the C library SDL and provides functions for graphics rendering, event handling, audio playback, and input control via keyboard, mouse, and game controllers. Pygame was originally created by Pete Shinnars in the year 2000 and for many years was the standard framework for Python-based game development. However, since the original project did not receive updates for a long period of time, a fork called Pygame Community Edition (pygame-ce) was created in 2023. This version is actively developed by a community in order to support modern Python versions, improved graphics features (for example, alpha blending and better transformations), and higher performance. [9](#), [347](#), [350](#)

Pylance Pylance is the default Python extension for Visual Studio Code that supports Python development. Its main features are type checking and code completion. Pylance helps detect errors early by analyzing code statically. [149](#)

Pythagorean theorem In a right-angled triangle, the sum of the squares of the legs is equal to the square of the hypotenuse: $c^2 = a^2 + b^2$. The theorem is named after the mathematician *Pythagoras of Samos* (ca. 570 BC to ca. 510 BC). [285](#)

Python Python is a high-level interpreted programming language that supports both procedural and object-oriented paradigms. It was created in 1991 by Guido van Rossum and is currently one of the most popular programming languages. [9](#)

Radian (rad) A unit for measuring angles. A full circle has 2π rad. [150](#)

Random Access Memory RAM (Random Access Memory) is the main working memory of a computer. It stores data and program code that are currently being used so that the processor can access them quickly. The contents of RAM are volatile, meaning they are lost when the computer is powered off. [93](#), [357](#)

Red Green Blue An additive color model in which colors are represented by combining the three primary colors red, green, and blue. Each color channel typically has a value range from 0 to 255. By mixing different intensity levels, $256^3 = 16,777,216$ representable colors are created. The RGB model is used in digital displays, graphics programming, and image processing. [357](#)

Rendering Refers to the process of creating a concrete, displayable image from abstract data (e.g. text, vectors, sprites, or 3D models). In computer graphics and in frameworks such as Pygame, rendering typically means that an object computes its visual representation (a *Surface* or *Bitmap*) and prepares it for display on the screen. Rendering is often performed only when the content or state of an object has changed, in order to save computational resources. [106](#)

Rounding Error A rounding error occurs when a floating-point number cannot be represented exactly due to limited precision. Floating-point numbers are stored as sums of powers of two, and many decimal values (such as $\frac{1}{10}$ or $\frac{1}{60}$) do not have an exact finite representation in this system. As a result, values are stored as close approximations. When such approximations are used repeatedly in calculations, the small errors can accumulate and lead to noticeable deviations, especially in simulations, animations, and game loops. [69](#)

Semantics The meaning of a statement or specification. It is usually used in contrast to the [Syntax](#) of a statement. [29](#), [355](#)

Signature The signature of a function or method describes its formal properties that are visible from the outside. These include its visibility, return type, name, and parameters. The signature defines how a function or method can be called and how it interacts with other parts of the program. [84](#)

Simple Direct Media Layer SDL is a cross-platform multimedia library written in C. It provides low-level access to graphics, sound, input devices (keyboard, mouse, game controllers), and window management. SDL is widely used in game development and serves as the underlying technology for many frameworks and engines. In the context of Pygame-ce, SDL handles the communication with the operating system, while Pygame provides a more convenient Python interface on top of it. In short: SDL does the heavy lifting in the background, and Pygame makes it easy to use from Python. [357](#)

- Sine function** (`sin`) A trigonometric function that produces a smooth, periodic wave. For an angle x (usually in radians), it is defined as $y = \sin(x)$. The output is always between -1 and 1 , and the function repeats every 2π . In game programming, `sin` is often used for natural-looking oscillations such as floating, bobbing, and smooth back-and-forth motion. **Pygame tip:** A common pattern is `y = base_y + amplitude * math.sin(t * speed)` to animate smooth vertical bobbing. 41
- Single Responsibility Principle** Each class or function should have exactly one responsibility. It should focus on that single task and do it well. A solution to a specific problem should be encapsulated in one class or one method, and changes to that responsibility should affect only that part of the code. 82, 357
- Singleton** A design pattern that ensures that there is exactly one instance of a class. This instance is usually provided in a (semi-)public way. Due to its conceptual similarity to global variables, the Singleton pattern is considered controversial. 164
- Slicing** A technique that allows convenient extraction of subsets from strings or arrays. 120
- Socket** In computer science, a socket is an endpoint for communication between two programs over a network. It provides a standardized interface for sending and receiving data using protocols such as TCP or UDP. A socket is typically identified by an IP address and a port number and is commonly used in client-server architectures. 303
- Solid-State Drive** A mass storage technology that is not based on magnetic principles but on semiconductor technology. 357
- Sprite** A single two-dimensional graphical element that typically represents a game character, an object, or an animation. Sprites are rendered independently of the background and can be moved, rotated, or scaled. Many game engines and libraries such as Pygame provide special sprite classes for efficient management and updating of game objects. Other names include *movable object (MOB)* or *blitter object (BOB)*. 46, 354
- SpriteLib** A freely available collection of 2D game [sprites](#), often used for prototyping games and for educational purposes. SpriteLib provides graphics such as characters, objects, and animations that can be used in game engines or frameworks like Pygame. 115, 193
- Stack** A linear data structure that stores elements in a last-in, first-out (LIFO) order. Items are added with *push* and removed with *pop*. 248
- Stereo** Refers to a two-channel audio technique in which a sound signal is played back separately through a left and a right channel. Different volume levels or signals on the two channels create a spatial listening impression. In game development, stereo is often used to convey the position of a sound source in space, for example through stereo panning, where the volume of a sound is adjusted depending on the horizontal position of an object. 161

Stereo Panning An audio technique that places a sound in space by changing its volume on the left and right speakers. With stereo panning, the sound of a source is placed more strongly on the left or right channel depending on its horizontal position in space. In game development, for example with Pygame, stereo panning is used to make the position of an object audible, typically by controlling the left and right audio channels of a sound separately. [161](#), [164](#)

Strategy Pattern A **design pattern** from the category of **behavioral patterns**. It is used to make a specific behavior of an object **interchangeable** without modifying the object's class. The Strategy Pattern encapsulates a family of algorithms in separate classes that all implement a common interface. The context object delegates the execution of a particular task to the currently selected strategy. This allows the behavior to be changed dynamically at runtime. [222](#)

Superclass In object-oriented programming, a superclass is a class from which another class (the subclass) inherits attributes and methods. The superclass defines common behavior and interfaces that can be reused and extended by subclasses. In Python, the superclass can be accessed using the `super()` function, which allows a subclass to call methods of its superclass, such as the constructor `__init__()`. [85](#)

Syntax The form or grammar of a statement or specification. It is usually used in contrast to the [Semantics](#) of a statement. [353](#)

Tic-Tac-Toe A classic two-player game in which players alternate placing X and O symbols on a 3×3 board. The objective is to place three of one's own symbols in a horizontal, vertical, or diagonal row before the opponent. [224](#)

Tiled A free, cross-platform tile map editor for creating 2D tile-based maps for games. Tiled supports various map formats (e.g. orthogonal, isometric, hexagonal) as well as flexible tileset structures. The created maps can be exported in numerous formats and integrated into game frameworks such as Pygame, Godot, or Unity. More information can be found at <https://www.mapeditor.org/>. [193](#)

Toggling In computer science, this means that the value of a boolean variable switches from **True** to **False** or from **False** to **True**: *to toggle = to switch*. [298](#)

Trade-off A situation in which an advantage is gained at the expense of a disadvantage. In algorithmic and software-related contexts, a trade-off requires evaluating – based on the available data—whether the overall benefits outweigh the associated costs. For example, using indexes can dramatically speed up access to database contents (benefit). To achieve this speed-up, additional files must be created and maintained, which makes inserting, updating, and deleting data slower (cost). In software development, a trade-off often describes the deliberate balancing between competing factors such as accuracy and computational effort, memory usage and execution speed, or flexibility and complexity. Trade-offs are unavoidable and require decisions that depend on the specific situation. [133](#)

- TrueType Font** The font information is not stored in bitmap form, but in a vector-based format. This allows text to be rendered at *arbitrary* font sizes without losing quality. TrueType fonts are therefore well suited for scaling, high-resolution displays, and dynamic text rendering in applications and games. [357](#)
- Unicode** A way of coding characters and symbols. Most popular implementations are UTF-8, UTF-16, and UTF-32. [120](#)
- Universal Serial Bus** A bit-serial data transmission protocol used to connect peripheral devices to a computer. [357](#)
- User Interface** The part of a software application through which users interact with the program. It includes visual elements such as buttons, menus, icons, and text, as well as input methods like keyboard and mouse controls. [224](#), [357](#)
- Variable** A named reference to an object in memory. In Python, a variable is not a container that stores a fixed data type, but a flexible name that can refer to arbitrary objects. Unlike a constant, the value of a variable can be changed at any time by assigning a new object using assignment (=). Examples: `x = 10` or `name = "Alice"`. [18](#)
- Visual Studio Code** A free, cross-platform source code editor developed by Microsoft. It supports numerous programming languages and provides an integrated extension manager, debugging features, and a customizable interface. Due to its lightweight architecture, Git integration, and wide range of extensions, Visual Studio Code is suitable for both beginners and professional software development. [10](#)
- Warning** A message that indicates a potential problem or an unusual situation. A warning does not stop the program, but it signals that something may lead to errors or unexpected behavior if it is not addressed. Warnings are meant to draw attention to issues before they become serious problems. [53](#)
- Wayland** Wayland is a modern display server protocol for Linux and other Unix-like systems. It defines how graphical applications (clients) communicate with the display server (the compositor) to draw windows, handle input devices like keyboard and mouse, and manage screen updates. Unlike the older X11 system, Wayland is designed to be simpler, more secure, and more efficient. Many tasks that were previously handled by a separate window manager are integrated directly into the compositor, reducing complexity and latency. In practice, Wayland often results in smoother graphics, better support for modern hardware, and improved security, since applications cannot freely access or manipulate each other's windows. Popular desktop environments such as GNOME and KDE increasingly use Wayland as their default display system. [23](#)
- Windows Bitmap Format** Image information stored in the Windows Bitmap format. [345](#), [357](#)

Acronyms

- ASCII** American Standard Code for Information Interchange. *Glossar:* [American Standard Code for Information Interchange](#)
- BMP** Windows Bitmap Format. [345](#), *Glossar:* [Windows Bitmap Format](#)
- ChatGPT** Generative Pre-trained Transformer. *Glossar:* [Generative Pre-trained Transformer](#)
- FPS** Frames Per Second. [19](#), *Glossar:* [Frames Per Second](#)
- IDE** Integrated Development Environment. [348](#), *Glossar:* [Integrated Development Environment](#)
- ITA** Information Technology Assistants. [274](#), *Glossar:* [Information Technology Assistants](#)
- JPEG** Joint Photographic Experts Group. [345](#), *Glossar:* [Joint Photographic Experts Group](#)
- LSP** Liskov Substitution Principle. [84](#), *Glossar:* [Liskov Substitution Principle](#)
- ms** Milliseconds. [140](#), *Glossar:* [Milliseconds](#)
- OO** Object Oriented. [108](#), *Glossar:* [Object Oriented](#)
- OS** Operating System. [18](#), *Glossar:* [Operating System](#)
- PNG** Portable Network Graphics. [46](#), [345](#), [350](#), *Glossar:* [Portable Network Graphics](#)
- pt** DTP point. [108](#), *Glossar:* [DTP point](#)
- px** Pixel. [18](#), *Glossar:* [Pixel](#)
- RAM** Random Access Memory. [93](#), *Glossar:* [Random Access Memory](#)
- RGB** Red Green Blue. [19](#), [345](#), *Glossar:* [Red Green Blue](#)
- SDL** Simple Direct Media Layer. *Glossar:* [Simple Direct Media Layer](#)
- SRP** Single Responsibility Principle. [82](#), *Glossar:* [Single Responsibility Principle](#)
- SSD** Solid-State Drive. [18](#), *Glossar:* [Solid-State Drive](#)
- TTF** TrueType Font. [112](#), *Glossar:* [TrueType Font](#)
- UI** User Interface. [224](#), *Glossar:* [User Interface](#)
- USB** Universal Serial Bus. [18](#), *Glossar:* [Universal Serial Bus](#)

Index

- `__main__`, [83](#)
- Alpha blending, [29](#)
- Alpha channel, [29](#), [47](#), [271](#)
- Animation, [185](#)
- Auto Scrolling, [221](#)
- Autopilot, [335](#)
- associative array, [347](#)
- Ball, [258](#)
- BirdEyeView, [209](#)
- Bitmap, [45](#)
 - blit, [45](#)
 - load, [45](#)
 - moving, [59](#)
- Broadcast system, [251](#)
- Bubble growth, [281](#)
- Bubbles, [274](#)
- Bubbles appear, [277](#)
- Bubbles burst, [286](#)
- Button, [53](#)
- background music, [157](#)
- Circle, [28](#), [30](#)
- Clamp, [216](#)
- Collision, [123](#)
- Color
 - Alpha blending, [29](#)
 - Alpha channel, [29](#)
 - Information, [29](#)
 - Names, [29](#)
- Colors named, [43](#)
- Computer player, [265](#)
- Counter-thrust, [324](#)
- center-based scaling, [282](#)
- channel, [162](#)
- collision callback, [132](#)
- collision detection
 - circle, [123](#)
 - pixel, [124](#)
 - rectangle, [123](#)
- Dictionary, [89](#), [118](#)
- Direction, [61](#)
- Direction change, [62](#)
- Display
 - Mode, [26](#)
- Display collision, [293](#)
- Doublebuffer, [18](#)
- delta time, [138](#)
- deltatime, [64](#), [66](#), [71](#)
- dispatching hub, [149](#)
- Earth, [315](#)
- Edge Scrolling, [219](#)
- Endless Scrolling, [221](#)
- Event, [171](#)
- Externalize String Constants, [235](#)
- equidistant, [48](#)
- event, [90](#)
- event.mod, [92](#)
- Flag, [18](#)
- Font, [106](#)
 - color, [109](#)
 - size, [109](#)
- Fuel, [325](#)
- fade, [157](#)
- flag, [299](#)
- float, [69](#)
- function pointer, [132](#)
- Game over, [289](#)
- Game over and restart, [330](#)
- Graphic primitives, [28](#)
 - Circle, [28](#), [30](#)
 - Line, [28](#), [30](#)
 - Point, [28](#), [30](#)
 - Polygon, [29](#)
 - Rectangle, [29](#)

- Gravitation and landing, [323](#)
- Gravity, [32](#)
- hash table, [347](#)
- Index (Button), [54](#)
- Installation
 - Pygame-ce, [14](#)
 - Python, [13](#)
- int, [69](#)
- Keyboard
 - Constants, [101](#)
 - Modifier, [104](#)
- Lander, [319](#)
- Line, [28](#), [30](#)
- Lunar surface, [311](#)
- lookup table, [347](#)
- MessageBox, [53](#)
- Moonlander, [307](#)
- Mouse, [146](#)
- Mouse cursor, [284](#)
- main loop, [18](#)
- mask, [125](#)
- mouse
 - left button, [154](#)
 - middle button, [154](#)
 - right button, [154](#)
 - wheel, [154](#)
 - wheel down, [154](#)
 - wheel up, [154](#)
- mouse wheel, [147](#)
- mp3, [159](#)
- Number of bubbles, [280](#)
- Offset, [213](#)
- ogg, [159](#)
- Paddle hit, [264](#)
- Paddles, [255](#)
- Page Scrolling, [219](#)
- Pause, [297](#)
- Pause and help, [270](#)
- Playground, [226](#)
- Point, [28](#), [30](#)
- Polygon, [29](#)
- Pong, [253](#)
- Pushdown Automaton, [248](#)
- Pythagoras, Satz von, [285](#)
- project structure, [15](#)
- Rectangle, [29](#)
- Restart, [299](#)
- rectangle, [277](#)
- SRP, [263](#)
- Scenes, [224](#)
- Score, [287](#)
- Scoring, [261](#)
- Scrolling
 - Auto, [221](#)
 - Edge, [219](#)
 - Endless, [221](#)
 - Page, [219](#)
- Sound, [156](#), [268](#), [301](#)
- Speed, [62](#)
- Sprite
 - self.image, [79](#)
 - self.rect, [79](#)
- Spritelib, [193](#)
- Stack states, [248](#)
- Standard functionality, [224](#), [253](#), [274](#), [307](#)
- Stars, [317](#)
- State Machine, [224](#)
- State machine, [237](#)
- Status display, [326](#)
- screen, [18](#)
- self.mask, [127](#)
- self.radius, [127](#), [277](#)
- self.rect, [127](#), [277](#)
- sound
 - endless loops, [168](#)
 - event, [159](#)
 - fade-in, [168](#)
 - fade-out, [168](#)
 - music, [156](#)
- sound effects, [159](#)

- stereo panning, [269](#)
- sticky edge, [260](#)
- Tastatur, [89](#)
- Time-based, [136](#)
- Time-based adjustments, [291](#)
- Timer, [142](#), [278](#), [292](#)
- Transparency, [29](#), [47](#)
- time
 - accumulated, [144](#)
 - class Timer, [142](#)
 - cool down, [145](#)
 - start delay, [145](#)
 - time(), [71](#)
- Visibility culling, [207](#)
- Win or Draw?, [230](#), [233](#)

Index of the pygame Namespace

Color, [29](#), [43](#)
 DOUBLEBUF, [26](#)
 Event, [90](#)
 type, [172](#)
 FRect
 colliderect(), [208](#)
 move(), [79](#)
 move_ip(), [80](#)
 FULLSCREEN, [26](#)
 HIDDEN, [26](#)
 HWSURFACE, [26](#)
 KEYDOWN, [90](#), [101](#)
 KEYUP, [90](#), [91](#), [101](#)
 KEY, [100](#)
 get_just_pressed(), [100](#)
 get_just_released(), [100](#)
 get_pressed(), [100](#)
 key_code(), [101](#)
 KMOD_ALT, [104](#)
 KMOD_CAPS, [105](#)
 KMOD_CTRL, [104](#)
 KMOD_LALT, [104](#)
 KMOD_LCTRL, [104](#)
 KMOD_LMETA, [105](#)
 KMOD_LSHIFT, [92](#), [104](#)
 KMOD_META, [105](#)
 KMOD_MODE, [105](#)
 KMOD_NONE, [93](#), [104](#)
 KMOD_NUM, [105](#)
 KMOD_RALT, [104](#)
 KMOD_RCTRL, [104](#)
 KMOD_RMETA, [105](#)
 KMOD_RSHIFT, [92](#), [104](#)
 KMOD_SHIFT, [104](#)
 K_0, [101](#)
 K_1, [101](#)
 K_2, [101](#)
 K_3, [101](#)
 K_4, [101](#)
 K_5, [101](#)
 K_6, [101](#)
 K_7, [101](#)
 K_8, [101](#)
 K_9, [101](#)
 K_AC_BACK, [104](#)
 K_AMPERSAND, [101](#)
 K_ASTERISK, [101](#)
 K_AT, [102](#)
 K_BACKQUOTE, [102](#)
 K_BACKSLASH, [102](#)
 K_BACKSPACE, [101](#)
 K_BREAK, [104](#)
 K_CAPSLOCK, [104](#)
 K_CARET, [102](#)
 K_CLEAR, [101](#)
 K_COLON, [101](#)
 K_COMMA, [101](#)
 K_DELETE, [102](#)
 K_DOLLAR, [101](#)
 K_DOWN, [91](#), [103](#)
 K_END, [103](#)
 K_EQUALS, [102](#)
 K_ESCAPE, [91](#), [101](#)
 K_EURO, [104](#)
 K_EXCLAIM, [101](#)
 K_F10, [103](#)
 K_F11, [103](#)
 K_F12, [103](#)
 K_F13, [103](#)
 K_F14, [104](#)
 K_F15, [104](#)
 K_F1, [103](#)
 K_F2, [103](#)
 K_F3, [103](#)
 K_F4, [103](#)
 K_F5, [103](#)

[K_F6, 103](#)
[K_F7, 103](#)
[K_F8, 103](#)
[K_F9, 103](#)
[K_GREATER, 102](#)
[K_HASH, 101](#)
[K_HELP, 104](#)
[K_HOME, 103](#)
[K_INSERT, 103](#)
[K_KP0, 103](#)
[K_KP1, 103](#)
[K_KP2, 103](#)
[K_KP3, 103](#)
[K_KP4, 103](#)
[K_KP5, 103](#)
[K_KP6, 103](#)
[K_KP7, 103](#)
[K_KP8, 103](#)
[K_KP9, 103](#)
[K_KP_DIVIDE, 103](#)
[K_KP_ENTER, 103](#)
[K_KP_EQUALS, 103](#)
[K_KP_MINUS, 103](#)
[K_KP_MULTIPLY, 103](#)
[K_KP_PERIOD, 103](#)
[K_KP_PLUS, 103](#)
[K_LALT, 104](#)
[K_LCTRL, 104](#)
[K_LEFTBRACKET, 102](#)
[K_LEFTPAREN, 101](#)
[K_LEFT, 91, 103](#)
[K_LESS, 102](#)
[K_LMETA, 104](#)
[K_LSHIFT, 104](#)
[K_LSUPER, 104](#)
[K_MENU, 104](#)
[K_MINUS, 101](#)
[K_MODE, 104](#)
[K_NUMLOCK, 104](#)
[K_PAGEDOWN, 103](#)
[K_PAGEUP, 103](#)
[K_PAUSE, 101](#)
[K_PERIOD, 101](#)
[K_PLUS, 101](#)
[K_POWER, 104](#)
[K_PRINT, 104](#)
[K_QUESTION, 102](#)
[K_QUOTEDBL, 101](#)
[K_QUOTE, 101](#)
[K_RALT, 104](#)
[K_RCTRL, 104](#)
[K_RETURN, 101](#)
[K_RIGHTBRACKET, 102](#)
[K_RIGHTPAREN, 101](#)
[K_RIGHT, 91, 103](#)
[K_RMETA, 104](#)
[K_RSHIFT, 104](#)
[K_RSUPER, 104](#)
[K_SCROLLLOCK, 104](#)
[K_SEMICOLON, 102](#)
[K_SLASH, 101](#)
[K_SPACE, 101](#)
[K_SYSREQ, 104](#)
[K_TAB, 101](#)
[K_UNDERSCORE, 102](#)
[K_UP, 91, 103](#)
[K_a, 102](#)
[K_b, 102](#)
[K_c, 102](#)
[K_d, 102](#)
[K_e, 102](#)
[K_f, 102](#)
[K_g, 102](#)
[K_h, 102](#)
[K_i, 102](#)
[K_j, 102](#)
[K_k, 102](#)
[K_l, 102](#)
[K_m, 102](#)
[K_n, 102](#)
[K_o, 102](#)
[K_p, 102](#)
[K_q, 102](#)
[K_r, 102](#)
[K_s, 102](#)
[K_t, 102](#)
[K_u, 102](#)
[K_v, 102](#)

- K_w, [102](#)
- K_x, [102](#)
- K_y, [102](#)
- K_z, [102](#)
- MOUSEBUTTONDOWN, [147](#), [153](#), [286](#)
- MOUSEBUTTONUP, [147](#), [153](#)
- NOFRAME, [26](#)
- NUMEVENTS, [173](#), [183](#)
- OPENGL, [26](#)
- QUIT, [18](#), [25](#)
- RESIZABLE, [26](#)
- Rect, [29](#), [43](#), [288](#)
 - collidepoint(), [148](#), [154](#)
 - contains(), [291](#)
 - height, [277](#)
 - left, [277](#)
 - move(), [87](#)
 - move_ip(), [87](#)
 - top, [277](#)
 - width, [277](#)
- SCALED, [26](#)
- SHOWN, [26](#)
- SRCALPHA, [55](#), [58](#)
- SYSTEM_CURSOR_CROSSHAIR, [285](#)
- SYSTEM_CURSOR_HAND, [285](#)
- Surface(), [54](#), [57](#)
- Surface, [18](#), [254](#)
 - blit()
 - area, [52](#)
 - blit(), [46](#), [57](#), [61](#)
 - convert(), [47](#), [57](#)
 - convert_alpha(), [47](#), [57](#)
 - fill(), [19](#), [26](#)
 - get_rect(), [61](#), [76](#), [127](#)
 - set_at(), [30](#), [44](#)
 - set_colorkey(), [47](#), [57](#), [127](#)
 - subsurface(), [112](#), [119](#), [122](#)
- USEREVENT, [173](#), [181](#), [183](#)
- WINDOWCLOSE, [25](#)
- WINDOWPOS_CENTERED, [27](#)
- WINDOWPOS_UNDEFINED, [27](#)
- Window, [18](#), [26](#), [93](#), [151](#), [276](#)
 - destroy(), [26](#)
 - flip(), [18](#), [26](#)
 - get_surface(), [18](#), [26](#)
 - id, [93](#), [151](#)
 - position, [18](#), [26](#)
 - size, [33](#), [44](#)
 - title, [18](#), [26](#)
- clock
 - tick(), [69](#), [71](#)
- constants, [153](#)
- display
 - Info(), [22](#), [57](#)
 - flip(), [24](#), [25](#)
 - message_box(), [53](#), [57](#)
 - set_caption(), [24](#), [25](#)
 - set_mode(), [23](#), [25](#)
- draw
 - aacircle(), [44](#)
 - aaline(), [44](#)
 - circle(), [30](#), [43](#), [44](#)
 - flood_fill(), [44](#)
 - line(), [30](#), [43](#)
 - lines(), [30](#), [43](#)
 - polygon(), [29](#), [43](#)
 - rect(), [29](#), [43](#)
- event
 - Event(), [262](#)
 - Event, [174](#), [183](#), [330](#)
 - unicode, [120](#), [121](#)
 - button, [147](#), [154](#)
 - custom_type(), [173](#), [181](#), [183](#)
 - get(), [18](#), [25](#), [179](#), [180](#), [184](#)
 - key, [90](#)
 - poll(), [180](#), [184](#)
 - post(), [174](#), [184](#)
 - pos, [148](#)
 - type, [18](#), [25](#)
 - wait(), [180](#), [184](#)
- font
 - Font, [108](#), [121](#), [288](#)
 - render(), [109](#), [121](#)
 - SysFont(), [107](#), [121](#)
 - get_default_font(), [107](#), [108](#), [114](#), [121](#)
 - get_fonts(), [112](#), [121](#)
 - match_font(), [107](#), [112](#), [121](#)

- gfxdraw
 - pixel(), [30](#), [43](#)
- image, [57](#)
 - load(), [46](#), [57](#)
- init(), [18](#), [25](#), [64](#), [157](#), [276](#)
- key, [90](#)
 - get_just_pressed(), [99](#)
 - get_just_released(), [99](#)
 - get_pressed(), [99](#)
 - key_code(), [98](#)
- locals
 - WINDOWPOS_CENTERED, [184](#)
- mask
 - from_surface(), [127](#), [133](#)
- math
 - Vector2D, [89](#)
 - Vector2, [70](#), [77](#)
 - Vector3, [77](#)
 - clamp(), [52](#)
- mixer
 - Channel, [162](#), [169](#)
 - play(), [162](#), [169](#)
 - set_volume(), [163](#), [169](#)
 - Sound, [159](#), [162](#), [170](#), [302](#)
 - get_volume(), [159](#), [170](#)
 - play(), [160](#), [162](#), [170](#), [302](#)
 - set_volume(), [159](#), [164](#), [170](#)
 - find_channel(), [162](#), [170](#)
 - init(), [18](#), [157](#), [170](#)
 - music
 - fadeout(), [158](#), [170](#)
 - get_busy(), [170](#)
 - get_volume(), [170](#)
 - load(), [157](#), [170](#)
 - pause(), [158](#), [170](#)
 - play(), [157](#), [158](#), [170](#)
 - set_volume(), [157](#), [158](#), [164](#), [170](#)
 - unpause(), [158](#), [170](#)
 - set_num_channels(), [170](#)
- mouse
 - get_pos(), [31](#), [43](#), [148](#), [154](#), [285](#), [286](#)
 - get_pressed(), [31](#), [43](#), [153](#)
 - set_visible(), [148](#), [154](#)
- quit(), [19](#), [25](#)
- rect
 - FRect, [59](#), [70](#), [76](#)
 - bottomright, [59](#)
 - bottom, [59](#)
 - centerx, [59](#)
 - centery, [59](#)
 - center, [59](#)
 - clamp(), [89](#), [100](#)
 - clamp_ip(), [89](#), [100](#)
 - height, [59](#)
 - left, [59](#)
 - move(), [76](#)
 - right, [59](#)
 - size, [59](#)
 - topleft, [59](#)
 - top, [59](#)
 - width, [59](#)
 - Rect, [59](#), [76](#)
 - bottomright, [59](#)
 - bottom, [59](#)
 - centerx, [59](#)
 - centery, [59](#)
 - center, [59](#)
 - clamp(), [89](#), [100](#)
 - clamp_ip(), [89](#), [100](#)
 - height, [59](#)
 - left, [59](#)
 - move(), [63](#), [76](#)
 - right, [59](#)
 - size, [59](#)
 - topleft, [59](#)
 - top, [59](#)
 - width, [59](#)
- sprite
 - GroupSingle, [82](#), [87](#), [254](#)
 - sprite, [82](#), [87](#)
 - Group, [81](#), [87](#), [257](#), [289](#)
 - sprites(), [290](#)
 - Sprite, [78](#), [87](#)
 - kill(), [86](#), [87](#), [137](#), [145](#), [191](#), [287](#)
 - update(), [84](#)
 - collide_circle(), [130](#), [133](#), [279](#)
 - collide_mask(), [130](#), [133](#)

- collide_rect(), [80](#), [87](#), [130](#), [134](#),
[264](#)
- spritecollide(), [82](#), [87](#), [131](#), [134](#),
[279](#)
- surface
 - Surface
 - subsurface(), [195](#)
- time
 - Clock, [20](#), [25](#), [276](#)
 - tick(), [20](#), [25](#), [138](#)
 - tick_busy_loop(), [20](#), [26](#)
 - get_ticks(), [64](#), [76](#), [140](#), [145](#)
 - set_allowed(), [184](#)
 - set_timer(), [179](#), [181](#), [184](#)
 - wait(), [296](#), [303](#)
- transform
 - rotate(), [105](#), [150](#), [154](#)
 - scale(), [47](#), [58](#), [105](#), [281](#)
 - scale_by(), [209](#)